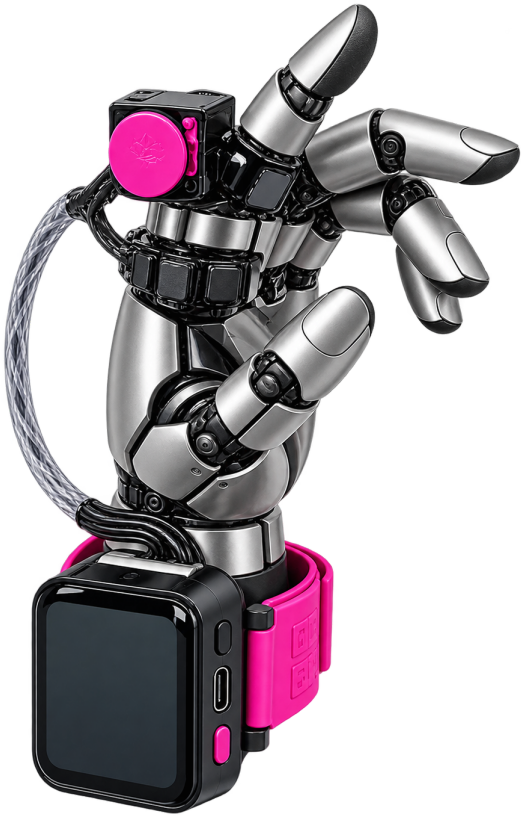
MITACS ACCELERATE · INTERNSHIP UNIT 1 · FINAL REPORT

360° hand tracking by sensor fusion

Optical hand tracking, forearm EMG and CyberFinger IMUs for expressive and professional social VR

Amirhossein Mohammadi

PhD student, Integrated Program in Neuroscience
McGill University · internship hosted at Mila



PROJECT	Development of 360° hand tracking by sensor fusion for social VR
PARTNER	SciCortex Technologies Corporation, Montréal
ACADEMIC SUPERVISOR	Prof. Majid Mohajerani, McGill University
PERIOD	Internship Unit 1 · 30 June – 29 October 2026
REPORT DATE	28 October 2026

Executive summary

Headset hand tracking only works while the headset's cameras can see the hand. When the hand leaves their view, or the fingers hide one another, tracking is lost. This Mitacs Accelerate project with SciCortex Technologies, *Development of 360° hand tracking by sensor-fusion for expressive and professional social virtual reality (VR)*, set out to keep the hand tracked in those moments. It fuses three sensors: the Meta Quest 3's optical hand tracking, a MindRove 8-channel forearm EMG armband (which records the electrical activity of the forearm muscles) and the CyberFinger's two motion sensors (IMUs) in the wrist module and on the back of the hand. Without the fused hand, SciCortex's product handles a hidden hand as its v2.1 manual describes: the hand "keeps turning with your wrist, and its position waits where the headset last saw it". This report covers the first internship unit, 30 June to 29 October 2026.

The research instrument is one Windows application, *bridge/cyberfinger_gui.py* (15,483 lines, 27 tabs). It records all three sensors on one clock and runs every model live in plain NumPy. Its tabs follow the research path:

- deciding when the camera can be trusted (**Gate**, **Occlusion**);
- online calibration with the camera as teacher (**Live Calib**, **Pose Hand**);
- decoding fingers from EMG;
- calibrating the IMUs against the camera;
- estimating the wrist's position out of view (six tabs, ending in **Hybrid Position**);
- recognising key postures and flagging posture changes (**Key Postures**, ⚡ **Transitions**).

The ✦ **Fusion Studio** tab brings all of this into one live view of the body, arm and whole hand. Against the proposal, ten of the twelve sub-tasks planned for this unit were completed. Two were only partly done: there was no participant study, and the buffer window-size study was not run. Estimating the hand's position out of view went beyond the original plan.

Headline results (single user):

- **Wrist position.** Two seconds after the camera lost the hand, Hybrid Position kept the wrist within 14.6 cm on average, against 36.0 cm for freezing it where it was last seen. Its mean over 0.5–8 s was 14.58 cm, against 22.62 cm for the earlier learned model and 36.95 cm for freezing (leave-one-clip-out, head turned away).
- **Orientation.** A camera-taught model of how each IMU sits on the body kept the knuckle sensor within a median 8.6–10.3° of the camera for 30 s out of view. The earlier one-sided offset was 49.8° off after 1 s.
- **Online calibration.** Without calibration gestures or retraining, Live Calib brought in-view finger-curl error from 0.455–0.547 down to 0.056–0.124. Camera self-teaching raised key-posture accuracy on hidden frames in a new session from 50% to 77%.
- **Posture changes.** A learned EMG detector flagged 143 of 190 posture changes about 57 ms before the fingers moved. With the optional soft hand and anticipation, the hidden hand's lag behind the camera's own hand fell from 230 ms to 45 ms.
- **Real-time quality.** Fusion Studio runs at 62 fps (was 35). Handovers between camera and prediction no longer cause jumps over 20 cm (was 25 per minute).

The work reached SciCortex's product:

- SciCortex merged the single-window Fusion Studio as pull request #1 on 25 September 2026 and shipped it in release v2.1.0 on 28 September.
- Two days after delivery, SciCortex's SteamVR driver adopted the Studio's two-sided IMU mounting model.
- I built the link that feeds the fused hand into the driver's FUSED source at 120 packets per second. In a simulated 90 Hz display, palm jitter fell from 10.2° to 1.45° RMS.
- When a SteamVR update on 21 October stopped the driver from receiving finger data, I fixed the driver, and SciCortex committed the same fix the next day.
- A Fusion Studio update with a Windows executable was submitted as pull request #7. It was open on 23 October; the executable's live-headset path is still untested.

The limits are clear and set the agenda for the second unit (1 November 2026 to 27 February 2027):

- All data come from one user, and no participant study was run.
- Most of the remaining position error comes from the upper arm, which no sensor measures. With the true upper-arm direction, the same framework reaches 8.13 cm.
- Without a magnetometer, the IMUs' horizontal heading can be trusted for only about a minute after the last camera fix.
- EMG patterns shift with arm position and from day to day.

The next steps are an upper-arm IMU, multi-day and (after an ethics check) multi-user recordings, a learned camera-reliability estimator, the transformer fusion with masked sensors planned for the second unit, and live trials of the features that still ship switched off.

Highlights

- Fusion Studio merged by SciCortex (PR #1, 25 Sep 2026) and shipped in release v2.1.0 (28 Sep 2026).
- Out of view, wrist error at 2 s was 14.6 cm, against 36.0 cm for freezing the hand where it was last seen.
- A camera-taught IMU mounting keeps hand orientation within a median 8.6–10.3° for 30 s out of view; SciCortex's driver adopted the model.
- Online calibration with no calibration gestures: in-view finger-curl error fell from 0.455–0.547 to 0.056–0.124.
- Camera self-teaching raised key-posture accuracy on hidden frames in a new session from 50% to 77%.
- A learned EMG detector flags 75% of posture changes about 57 ms before the fingers move.
- The fused hand streams into SteamVR at 120 packets/s, and Fusion Studio runs at 62 fps.
- SteamVR driver fix adopted by SciCortex; Fusion Studio update and Windows app submitted as PR #7.

Key results Headline quantitative results of the internship unit

Area	Result	Metric
Wrist position out of view (Hybrid Position)	14.58 cm mean over 0.5–8 s, against 22.62 cm (Learned Position) and 36.95 cm (freeze, the previous product behaviour); 14.6 vs 36.0 cm at 2 s	Mean 3-D wrist error, leave-one-clip-out over 5 clips, head turned away after the loss (look-away protocol)
Hand orientation out of view (camera-taught IMU mounting)	Median 8.6° at 1 s to 10.3° at 30 s; one-sided held offset 49.8° at 1 s and over 100° after 7 s	Knuckle-IMU rotation error vs camera, 241–281 simulated losses from the position clips
Live wrist bend in Fusion Studio	7.8° with the live knuckle mount, against 25.5° with the bend held, over 20 s out of view	Median hand forward-axis error, real Studio update, recorded clip, camera hidden
Online calibration (Live Calib)	In-view curl error 0.455–0.547 → 0.056–0.124; out-of-view posture identification 72% → 92%	Mean absolute curl error (0–1) vs camera in 3 recorded sessions; % correct on a real occlusion recording
Continual learning from the camera (Pose Hand)	Curl error 0.167 → 0.089 within about 1.7 s of camera view	Mean curl error (0–1), simulated re-donned armband
EMG finger decoding	Raw-EMG TCN 1.44 cm vs 1.80 cm for ridge; per-finger curl correlation 0.88–0.92 (error 0.099)	Mean joint error, contiguous split, clean labels; correlation of predicted vs camera curl
Key postures with the hand hidden	50% → 77% correct with camera self-teaching; next-session accuracy 45% → 86% offline	Frames the camera did not see, real-tick replay of second-session clips
Posture stability while the arm turns	Wrong posture switches 12.1 → 6.5 per minute	Leave-one-arm-position-out, held-posture clips
EMG transition flags (⚡ Transitions)	143 of 190 changes (75%) flagged about 57 ms before the fingers move, at 2.2 false flags/min; fixed threshold 63%	Leave-one-recording-out, 6 recordings
Out-of-view posture lag (soft hand + anticipation, off by default)	230 ms → 45 ms behind the camera's own hand	Median lag, real Studio tick with camera hidden, 190 posture changes
Camera ↔ prediction handovers	Position leaps over 20 cm: 25 → 0 per minute; anatomically wrong hidden EMG hands: 73% → 0%	Replay of a fast recorded session; real-session replay through the Studio
Camera trust (Gate)	Quest flags read valid/tracked on 100% of frames while hidden; tuned Gate labels 100% of open hands TRACKED and 96% of fists SELF-OCCLUDED	Labelled clips of 19 August 2026, replayed through the GUI rules
Real-time performance and SteamVR output	Fusion Studio 35 → 62 fps; fused hand sent at 120 packets/s with palm jitter 10.2° → 1.45° RMS (delay 15 → 30 ms)	Real program loop on a 60 Hz recorded session; fake driver with simulated 90 Hz display
IU1 plan	10 of 12 sub-tasks completed, 2 partial (participant study, window-size study)	Proposal sub-tasks 1.1–4.3 against delivered work

Contents

Executive summary 2

1 Project overview and objectives 14

1.1 The problem: a hand the headset cannot see . . . 14
1.2 Project, partner and team . . . 15
1.3 Objectives . . . 16
1.4 What was built . . . 17
1.5 Timeline of the internship . . . 19
1.6 Plan versus outcome . . . 21
1.7 Headline results . . . 21
1.8 Limitations and negative results . . . 23
1.9 How this report is organised . . . 24

2 The research application, tab by tab 25

2.1 Gloves . . . 27
2.2 Armband . . . 31
2.3 Gate . . . 34
2.4 Capture . . . 37
2.5 EMG Hand . . . 40
2.6 Encoder . . . 43
2.7 Train . . . 46
2.8 Fusion Hand . . . 48
2.9 Occlusion . . . 51
2.10 Live Calib . . . 54
2.11 Tracking Hand . . . 57
2.12 emg2pose (ref) . . . 60
2.13 Posture Library . . . 62
2.14 Posture Clusters . . . 65
2.15 Posture Hand . . . 68
2.16 Pose Hand . . . 71
2.17 Pose + Position . . . 74
2.18 Position Capture . . . 77

2.19 Learned Position . . . 80
2.20 Body View . . . 83
2.21 IMU FK Position . . . 86
2.22 Pose Hand (gravity) . . . 89
2.23 Hybrid Position . . . 92
2.24 ✦ Fusion Studio . . . 95
2.25 Key Postures . . . 100
2.26 ◈ Fusion Logic . . . 103
2.27 ⚡ Transitions . . . 106

3 System architecture, instrumentation and data pipeline 109

3.1 Hardware and sensors . . . 111
3.2 One program, one clock: the host software . . . 113
3.3 From sensors to VR applications: the SteamVR link . . . 115
3.4 Merging SciCortex release v2.1.0 into the research branch . . . 117
3.5 Gloves: connecting the CyberFingers . . . 117
3.6 Armband: forearm EMG in the same program . . . 119
3.7 Capture: synchronised, labelled clips . . . 120
3.8 Limits of the instrument . . . 122

4 Optical confidence, occlusion and online calibration 123

4.1 Why camera trust matters . . . 123
4.2 Starting point: the headset's own flags are whole-hand only . . . 124
4.3 Gate: why the camera can or cannot be trusted right now . . . 125
4.4 Occlusion: which fingers the camera really sees . . . 131
4.5 Live Calib: online calibration with the camera as teacher . . . 133
4.6 Status against the proposal and next steps . . . 136

5 EMG-based finger and posture decoding 138

5.1 Why EMG, and how this section is organised . . . 138
5.2 EMG Hand . . . 140
5.3 Encoder . . . 142
5.4 Train . . . 142
5.5 Fusion Hand . . . 143
5.6 Out-of-view EMG: the limb-position effect . . . 144

5.7 Tracking Hand . . . 146
5.8 emg2pose (ref) . . . 147
5.9 Posture Library . . . 148
5.10 Posture Clusters . . . 148
5.11 Posture Hand . . . 149
5.12 Pose Hand . . . 150
5.13 Pose Hand (gravity) . . . 151
5.14 Findings, limitations and next step . . . 152

6 IMU–optical orientation fusion and sensor-mount calibration 154

6.1 Why the CyberFinger's motion sensors needed calibrating . . . 154
6.2 Orientation work in the EMG Hand tab . . . 156
6.3 Orientation tests in the Capture tab (rigid_hold) . . . 159
6.4 Getting the coordinate conventions right . . . 160
6.5 Orientation work in ✦ Fusion Studio: camera-taught sensor-mount calibration . . . 160
6.6 Uptake in SciCortex's product . . . 166
6.7 Limitations and next steps . . . 167

7 Hand position when the camera cannot see the hand 168

7.1 The problem and the starting point . . . 168
7.2 Pose + Position . . . 169
7.3 Position Capture . . . 171
7.4 Learned Position . . . 172
7.5 Body View . . . 174
7.6 IMU FK Position . . . 174
7.7 The position lab benchmark . . . 176
7.8 Hybrid Position . . . 177
7.9 Where the work went, limitations and next step . . . 184

8 Key postures, transitions and anticipation 185

8.1 Why recognise a few key postures . . . 185
8.2 Key Postures . . . 186
8.3 ◈ Fusion Logic . . . 192
8.4 ⚡ Transitions . . . 194
8.5 Delivered to SciCortex . . . 199

8.6 Limitations and open work . . . 199

9 Fusion Studio: the integrated system 200

9.1 ✦ Fusion Studio . . . 200
9.2 How one frame is computed . . . 204
9.3 Development and measured results . . . 204
9.4 Sending the fused hand to SteamVR . . . 209
9.5 Delivery to SciCortex . . . 210
9.6 Status and limitations . . . 212

10 Contributions, deliverables and next steps 213

10.1 Overview of what was delivered . . . 213
10.2 IU1 objectives: planned versus delivered . . . 213
10.3 Contributions to SciCortex's product . . . 216
10.4 Engineering practice . . . 219
10.5 Data and code availability . . . 220
10.6 Limitations and lessons learned . . . 220
10.7 Next steps: plan for Internship Unit 2 . . . 222

Conclusion 223

Appendix A — Index of the GUI tabs 225

Appendix B — Glossary 226

Appendix C — References 227

A keepsake 228

List of figures

Figure 1.1 The CyberFinger (SciCortex product render) . . . 15
Figure 1.2 How the 27-tab application grew . . . 18
Figure 1.3 Mean wrist error after the camera loses the hand (leave-one-clip-out, head turned away... . . . 22
Figure 1.4 Fusion Studio with the hand outside the camera's view . . . 23
Figure 2.1 The Buttons + IMU sub-tab while the application replays the synchronised recording... . . . 27
Figure 2.2 The Skeleton sub-tab during the same replay of 'low_new3': the Quest 3 camera's recorded... . . . 28
Figure 2.3 The Armband tab running its built-in EMG Simulator source, not the real armband, captured 8... . . . 31
Figure 2.4 The Gate tab in the real GUI, replaying the synchronised session 'low_new3' of 19 September... . . . 34
Figure 2.5 The Capture tab at rest . . . 37
Figure 2.6 The EMG Hand tab while the real GUI replays the synchronised session 'low_new3' of 19... . . . 40
Figure 2.7 The Encoder tab in the real application, replaying the synchronised session 'low_new3' of... . . . 43
Figure 2.8 The Train tab in the real GUI, captured while the GUI's own update loop replayed the... . . . 46
Figure 2.9 Taken from the real application, replaying the synchronised session 'low_new3' (19... . . . 48
Figure 2.10 The Occlusion tab while the application replays the synchronised session 'low_new3' of 19... . . . 51
Figure 2.11 The Live Calib tab while the real GUI replays, through its own update loop, the... . . . 54
Figure 2.12 The Tracking Hand tab with the hand in view . . . 57
Figure 2.13 The emg2pose (ref) tab at frame 95 of 607 . . . 60
Figure 2.14 The Posture Library tab in the real application . . . 62
Figure 2.15 The Posture Clusters tab in Clusters mode . . . 65
Figure 2.16 The real GUI replaying the recorded synchronised session 'low_new3' of 19 September 2026... . . . 68
Figure 2.17 Pose Hand replaying the synchronised session 'low_new3' of 19 September 2026 (401 s of... . . . 71
Figure 2.18 The research GUI replaying the synchronised session 'low_new3' of 19 September 2026 (Quest... . . . 74
Figure 2.19 Position Capture in the real application while it replays the synchronised session... . . . 77
Figure 2.20 The real GUI replaying the synchronised recording 'low_new3' of 19 September 2026 (401 s of... . . . 80
Figure 2.21 Body View's FRONT (left) and TOP (right) views . . . 83
Figure 2.22 The real application replaying, through its own update loop, the synchronised recording... . . . 86
Figure 2.23 The Pose Hand (gravity) tab in the real GUI, replaying the synchronised 401-second session... . . . 89
Figure 2.24 The Hybrid Position tab while the GUI replays the recorded session low_new3 (19 September... . . . 92
Figure 2.25 The Studio during a replay of the synchronised session low_new3 from t = 120 s, run through... . . . 95
Figure 2.26 Another stretch of the low_new3 replay, from t = 60 s . . . 96
Figure 2.27 The Key Postures tab during a 25-second replay of the transition recording of 22 October... . . . 100
Figure 2.28 The ◈ Fusion Logic tab about 12 s after it was opened, during a replay of the recorded... . . . 103
Figure 2.29 The ⚡ Transitions tab, captured about 30 s after the tab was opened, during a replay of a... . . . 106

Figure 3.1 System overview . . . 110
Figure 3.2 The wearable sensor set used throughout the project (2026-08-25) . . . 111
Figure 3.3 The sensors run at very different rates, and the GUI tick is where they meet on one clock . . . 114
Figure 3.4 SciCortex's single-window bridge grew into a 27-tab research instrument . . . 115
Figure 3.5 A 3-way comparison at the level of classes and methods replaced git's line merge . . . 117
Figure 3.6 The research bridge on 2026-08-03, after the EMG merge . . . 118
Figure 3.7 SciCortex Bridge 2.1 layout, as shown in the v2.1 user manual . . . 119
Figure 3.8 The Armband tab's browser hand in use on 2026-08-03, with the MindRove armband on the... . . . 120
Figure 3.9 Size of each recording folder on 2026-10-28, 569.8 MB in total, recorded between 2026-08-18... . . . 122
Figure 4.1 Through every phase of a scripted occlusion test, the Quest reports the hand as tracked and... . . . 125
Figure 4.2 How well each cue predicts that a camera joint is wrong (error above 1.5 cm) in a 30-s... . . . 126
Figure 4.3 Gating cuts the error during occlusion from 23.4 to 16.0 cm at a small cost while the hand... . . . 127
Figure 4.4 The reference per-finger gate (fusion/gate.py) run offline on a real Quest capture (18... . . . 128
Figure 4.5 A negative result that shaped the live Gate tab . . . 128
Figure 4.6 The tuned live classifier is reliable for open hands and fists . . . 130
Figure 4.7 With one confidence number per hand, the camera can teach almost nothing but an open hand . . . 131
Figure 4.8 In the Occlusion module's self-test, the curled fingertips of a fist lose most of their... . . . 132
Figure 4.9 While the camera sees the hand, the calibrated output follows the camera far more closely... . . . 135
Figure 4.10 The two cheap changes built into Live Calib raised out-of-view posture identification from... . . . 135
Figure 5.1 All three sensors worn together on 25 August 2026 . . . 140
Figure 5.2 The EMG Hand tab on 25 August 2026, before any gesture had been recorded (fist, open and... . . . 141
Figure 5.3 Per-finger TCN (raw EMG + camera-aligned wrist orientation) . . . 144
Figure 5.4 Recomputed leave-self-out on the 2,554-frame matched session of 30 August . . . 145
Figure 5.5 Matched recording (2,554 frames, 49 s) . . . 145
Figure 5.6 Second run of the transfer probe (3,084 training frames) . . . 148
Figure 5.7 Logistic regression trained on two wrist orientations and tested on the held-out one... . . . 149
Figure 5.8 Posture recogniser accuracy . . . 150
Figure 5.9 Gravity tilt is best across sessions (better on 4 of 5 clips) . . . 152
Figure 6.1 The CyberFinger's two IMUs on 2026-08-25 . . . 155
Figure 6.2 The EMG Hand tab live on 2026-08-25, after the knuckle clip was tightened . . . 158
Figure 6.3 Offline fit residuals on clips recorded before the knuckle clip was tightened . . . 158
Figure 6.4 The failure that triggered this work (2026-09-25) . . . 161
Figure 6.5 The same failure at "PREDICTED 21.5 s": the fingers point back at the elbow . . . 161
Figure 6.6 Self-test of bridge/mount_calib.py . . . 163
Figure 6.7 The Fusion Studio's HAND panel at start-up . . . 164

Figure 6.8 Replay through the real Studio update with the camera hidden . . . 165
Figure 7.1 Mean wrist error after a simulated camera loss, replaying 5 real synchronised clips (first... . . . 170
Figure 7.2 FK wrist error on the 48-s blind log when the upper-arm direction comes from a simulated... . . . 176
Figure 7.3 Mean 3-D wrist error against time since the camera lost the hand . . . 179
Figure 7.4 Leave-one-clip-out, look-away protocol . . . 179
Figure 7.5 Mean 3-D error over each logged blind span, scored by the camera . . . 181
Figure 7.6 Hybrid Position tab, FRONT body view (left/right × up/down . . . 182
Figure 7.7 Hybrid Position tab, TOP body view (left/right × near/far, with “far / in front” at the top) . . . 183
Figure 8.1 Accuracy on held-out second-session clips (open, point, pinch . . . 187
Figure 8.2 Dropping the heading-dependent tilt shortcut and holding the posture while the arm turns . . . 189
Figure 8.3 Model trained 2026-10-23 11:18 . . . 191
Figure 8.4 ◈ Fusion Logic with the hand in a fist (2026-10-09, 5-posture prior) . . . 193
Figure 8.5 Transition map with REST (2026-10-23), from the real app, with rest, fist and open each as... . . . 194
Figure 8.6 ⚡ Transitions replaying recording tx_20261022_212403 through the real app with the learned... . . . 195
Figure 8.7 190 camera-seen changes from 6 recordings through the real Studio tick with the camera... . . . 197
Figure 8.8 The turn damping, needed against the flicker seen live, cost speed . . . 198
Figure 9.1 The Fusion Studio while the camera sees a fist . . . 201
Figure 9.2 FRONT view while the camera sees the hand (magenta = CAMERA) . . . 202
Figure 9.3 TOP view 0.9 s into a simulated loss of the camera (the FRONT view of the same moment is... . . . 203
Figure 9.4 POSITION panel during the same simulated loss . . . 203
Figure 9.5 IMUs panel . . . 205
Figure 9.6 Replay of the fast recorded session (commit e12d9b2) . . . 206
Figure 9.7 Real program loop with a recorded session streamed at 60 Hz (commit 2172b3e) . . . 207
Figure 9.8 Real Studio update on 40 held-posture clips (14,282 frames), with EMG models that never saw... . . . 208
Figure 9.9 A real session replayed through the real Studio update (commit 5027e7c) . . . 209
Figure 9.10 Simulated 90 Hz SteamVR display . . . 210
Figure 10.1 The October fusion logic as drawn in the Fusion Studio, here while the camera sees an open... . . . 216
Figure 10.2 Lines added by each pull request . . . 218
Figure 10.3 Commits by the intern from the first research commit (c7704d3, 17 Sep) onward . . . 220

List of tables

Key results Headline quantitative results of the internship unit . . . 4
Table 1.1 The project at a glance (from the Mitacs proposal, signed by the intern and the supervisor... . . . 15
Table 1.2 IU1 sub-objectives and tasks from the Mitacs proposal . . . 17

Table 1.3 Labelled recordings made with the research application . . . 19
Table 1.4 Headline results of IU1 . . . 22
Table 1.5 Map from the report's sections to the application's 27 tabs and the proposal's tasks . . . 24
Table 2.1 The 27 tabs at a glance (in the order they appear in the application) . . . 25
Table 3.1 Who built which part of the system . . . 110
Table 3.2 The two CyberFinger IMUs (source . . . 112
Table 3.3 Wire protocol v2 between the bridge, Fusion Studio and the SteamVR driver (source . . . 116
Table 3.4 The seven Capture presets . . . 121
Table 3.5 The synchronised clip format (from 2026-08-20) . . . 121
Table 4.1 Timeline of the confidence, occlusion and calibration work (sources . . . 124
Table 4.2 Training-label weights derived from the Gate verdict (ENC_LABEL_CONF in... . . . 129
Table 4.3 Live Calib results on synthetic data (module self-test) . . . 134
Table 4.4 Calibration engagement (coverage . . . 136
Table 4.5 Confidence, occlusion and calibration work mapped to the Mitacs proposal tasks . . . 137
Table 5.1 The EMG tabs at a glance (details and measurement conditions in the subsections below) . . . 139
Table 5.2 Continuous finger encoders . . . 143
Table 5.3 Out-of-view fixes tested on real data . . . 146
Table 5.4 Posture Hand design experiments . . . 149
Table 6.1 The CyberFinger's two inertial sensors as used in this work (docs/firmware-and-imu.md . . . 155
Table 6.2 Wrist-IMU calibration against the camera's wrist/palm frame . . . 157
Table 6.3 Dead-reckoning with a frozen offset on the rigid_hold clip (fusion/run_fusion_on_log.py,... . . . 160
Table 6.4 Knuckle hand frame against the camera after a simulated camera loss, pooled over the... . . . 163
Table 6.5 Results of the driver-level IMU fusion as measured and reported by SciCortex in... . . . 166
Table 7.1 The six position tabs in GUI order, with the date each was built and the commit that added... . . . 169
Table 7.2 The Position Capture dataset . . . 172
Table 7.3 Learned Position . . . 173
Table 7.4 IMU FK Position, first live blind log . . . 175
Table 7.5 low_new3 replayed through the Hybrid Position tab's real update code in blind mode . . . 180
Table 8.1 Every decision-rule family tried sits on the same speed–stability trade-off (simulation... . . . 190
Table 8.2 Key Postures milestones . . . 192
Table 8.3 Transition flags on recordings the detector did not train on . . . 196
Table 8.4 Real Studio tick with the camera hidden, 6 recordings, 190 changes (medians . . . 198
Table 9.1 The Fusion Studio's switches and their defaults (repo bridge/cyberfinger_gui.py,... . . . 202
Table 9.2 Fusion Studio deliveries to SciCortex . . . 212
Table 10.1 IU1 sub-tasks from proposal §2.4–2.5 compared with what was delivered (sources . . . 214

Table 10.2 Contributions to SciCortex's product (sources 216
Table 10.3 IU2 plan mapped to the proposal's tasks 5–8 (sources 222
Table B.1 Abbreviations and terms used in this report 226

1 Project overview and objectives

VR hand tracking from the headset's cameras stops working when the hand leaves the cameras' view or the fingers hide each other. This Mitacs Accelerate project with SciCortex Technologies set out to keep the hand tracked in those moments by fusing three sensors: the Meta Quest 3's cameras, a forearm EMG armband and the CyberFinger's motion sensors. During the first internship unit (30 June to 29 October 2026) I built a 27-tab research application that does this fusion, and its main result, Fusion Studio, shipped in SciCortex's release v2.1.0. This section states the objectives, summarises how the planned tasks were met, gives the headline results (for example, out-of-view wrist error of 14.6 cm at 2 s, against 36.0 cm for the product's earlier freeze) and states the limits that remain.

1.1 The problem: a hand the headset cannot see

VR headsets such as the Meta Quest track the user's hands with their own cameras (*optical hand tracking*). This works well when the hand is in front of the user and clearly visible. The project proposal identifies three structural limits of the approach:

- **Limited field of view.** The cameras face forward, so a hand at the side, behind the body or held low is lost.
- **Self-occlusion.** Fingers hide one another from the camera, for example in a closed fist.
- **False-positive gestures.** Ordinary hand movements are mistaken for commands.

SciCortex's CyberFinger is a hand-mounted VR controller, worn one per hand. Each has a finger module (joystick, trigger and a row of buttons) cabled to a wrist module (screen, buttons and a motion sensor). The CyberFinger does not replace the headset's hand tracking: the fingers still come from the headset, and the user's actions come from the CyberFinger's buttons and joystick. A deliberate button press signals intent, which solves the third limit. The first two limits were still open, and they are what this project addresses. In the proposal's words, "no existing system provides robust 360° hand tracking — all prior fusion work assumes the headset can see the hand."

SciCortex's own user manual (v2.1) shows the gap in the product. It says that when the hands are out of view, "with the joint IMU the hand keeps turning with your wrist, and its position waits where the headset last saw it". An IMU (inertial measurement unit) is a small accelerometer-plus-gyroscope motion sensor. On 29 September 2026 my supervisor described the driver-level "mini-fusion" as only a fallback "while we wait for your approach on the bridge". The bridge is the PC software in which this project's fusion runs.

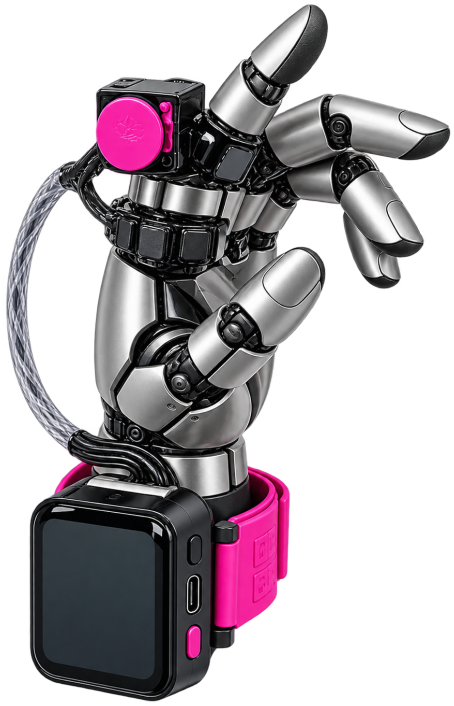


Figure 1.1 *The CyberFinger (SciCortex product render): the finger module with joystick and buttons on the index finger, cabled to the wrist module with its screen and buttons.*

1.2 Project, partner and team

Table 1.1 The project at a glance (from the Mitacs proposal, signed by the intern and the supervisor on 2026-05-05 and by the partner on 2026-05-08).

Item	Detail
Programme	Mitacs Accelerate; 8 months in 2 internship units (IU) of 4 months
Project title	Development of 360° hand tracking by sensor-fusion for expressive and professional social virtual reality (VR)
Partner	SciCortex Technologies Corporation, Montréal (Mila start-up, incorporated 2025-11-25)
Intern, IU1 (this report)	Amirhossein Mohammadi, PhD student, Integrated Program in Neuroscience, McGill University
Academic supervisor	Prof. Majid Mohajerani, McGill University (Centre de recherche Douglas), Department of Psychiatry
Partner contact	Ania Muller, CEO, SciCortex Technologies
Company-side guidance	Dr. Eilif Muller, SciCortex (named in the proposal to interact directly with the intern and guide the project)
Location	Hybrid; hosted at Mila, 6666 Rue Saint-Urbain, Montréal
IU1 dates	2026-06-30 to 2026-10-29
IU2 dates	2026-11-01 to 2027-02-27 (second PhD intern, to be named)
Funding	$20,000 in total: $10,000 from Mitacs and $10,000 from SciCortex ($5,000 per unit); all of it intern stipend, with no research expenses

The proposal planned a weekly one-hour meeting with the academic supervisor, plus email and ad hoc discussions. It listed Mila space, VR headsets, IMUs, EMG sensors and computing as resources. It declared no human-participant research. The deliverables it ticked were reports and presentations for the partner, patent applications and other intellectual-property claims, and peer-reviewed publications. On SciCortex's side, my code contributions were reviewed and merged through the company's GitHub account, DrSciCortex. Several design ideas in the later sections came from my supervisor during the project: the posture transition prior, the camera-versus-EMG weight w = f(m, d), the EMG-burst transition flag, the posture latch and the REST posture. Each is credited where it appears.

1.3 Objectives

The proposal states the overall goal as a multimodal sensor-fusion framework that extends optical hand tracking beyond its field-of-view and occlusion limits, giving continuous 360° hand and finger tracking. It fuses three signals:

- the headset's optical tracking;
- surface electromyography (sEMG or EMG): the electrical activity of the forearm muscles, recorded by an armband;
- a wrist IMU.

The intended result was a system that runs on the host PC and helps SciCortex's CyberFinger work reliably in professional and enterprise VR.

The central novelty was an **online calibration** that treats the camera as a teacher whenever it can see the hand. A rolling buffer stores pairs of (EMG features, optical hand pose) at moments when the camera is confident. When the camera loses the hand, a cross-attention lookup over that buffer corrects the EMG estimate for the current user and session. The user makes no explicit calibration gestures, and the model's weights are not updated at runtime. The proposal motivated this with an EMG accuracy drop of 20–30% between sessions (Buteau et al., 2024). It cited optical-plus-EMG fusion that improved finger angles by 5–15° under self-occlusion (Graf and Barthet, 2023).

The four gaps named in the proposal

1. No existing system provides robust 360° hand tracking; prior fusion work assumes the headset can see the hand.
2. Kalman-filter fusion cannot learn cross-modal representations or adapt to new users without hand-crafted reconfiguration.
3. No prior system uses optical tracking as a continuous, confidence-gated implicit calibration signal for the other sensors.
4. Combining a minimal intent-signalling controller (the CyberFinger) with continuous multimodal hand tracking had not been explored.

Planned approach

- Per-finger camera confidence from OpenXR's per-joint validity flags, plus geometric ray-casting.
- An EMG encoder built on a pretrained temporal convolutional network (TCN), initialised from public datasets (NinaPro or EMaGer).
- A Kalman filter fusing the IMU with the camera's hand orientation.

- The rolling-buffer, cross-attention online calibration described above.
- In IU2, a transformer fusion model trained with randomly masked sensors, so it stays robust when one sensor drops out.

Planned tasks for this internship unit

Table 1.2 IU1 sub-objectives and tasks from the Mitacs proposal. The proposal's week-by-week schedule could not be recovered from its text version, so planned week ranges are not quoted.

Sub-objective	Tasks
T1 Instrumentation and baselines	T1.1 synchronised data pipelines; T1.2 per-finger confidence; T1.3 performance benchmarks
T2 Modality encoders	T2.1 EMG encoder for finger regression; T2.2 IMU–optical fusion; T2.3 metrics and evaluation
T3 Attention-based online calibration	T3.1 rolling buffer of (optical, EMG) pairs; T3.2 cross-attention for session- and user-specific correction; T3.3 convergence and accuracy against window size, cross-session evaluation
T4 Fusion and integration	T4.1 confidence-gated fusion; T4.2 demo and evaluation with participants; T4.3 codebase, technical documentation, final report

IU2 (T5–T8) is planned to cover four things:

- a learned representation of variation between users and between IMU mountings, per-finger ray-cast occlusion and cross-session benchmarks (T5);
- the transformer fusion with masked-modality training (T6);
- calibration refinement, including dropping the buffer when residuals grow large (T7);
- systematic evaluation, a demo and a final report (T8).

1.4 What was built

Hardware

- **Meta Quest 3** hand tracking (a 26-joint hand skeleton plus the head pose), read in two ways: an OpenXR session started on demand, and SteamVR over Steam Link.
- **Two CyberFingers** (left and right). Each has a wrist ("body") IMU, and the lab units also have a second ("joint" or knuckle) IMU on the back of the hand. Both IMUs are 6-axis (accelerometer and gyroscope, with no magnetometer) and compute their own orientation on board. Their heading is arbitrary at power-on and drifts independently. The proposal's "SlimeVR IMU embedded in the CyberFinger" is this same on-board sensor.
- **MindRove 8-channel forearm armband**: dry stainless-steel electrodes, 500 Hz per channel, 24-bit, with a 6-axis IMU, streaming over its own Wi-Fi. The SiFi armband named as an alternative in the proposal was not used.
- **A Windows PC** running the research application in native Windows Python 3.12. Bluetooth LE to the CyberFinger needs Windows' own Bluetooth interface (WinRT), which the Linux subsystem (WSL2) cannot reach. Models were trained offline in WSL2 with PyTorch on an NVIDIA RTX 4090.

The research application

The research instrument is a single desktop application, *bridge/cyberfinger_gui.py*, built on SciCortex's CyberFinger Bridge GUI. At the end of the unit it was 15,483 lines long and had 27 tabs. Its first tab, **Gloves**, is SciCortex's original bridge window. The tabs grew under one rule that I kept throughout: every new idea went into a new tab with its own model file, and earlier tabs and their trained models were left untouched (later commits checked the old code byte for byte). The tab order therefore reads like a lab notebook, and the rest of this report follows it.

All live inference runs in plain numpy, with no deep-learning library on the Windows PC. Trained networks were exported to numpy and checked against PyTorch: the TCN matched to 3×10^{-8} and ran in 0.33 ms per frame. The application's update loop targets 60 Hz; during the six ⚡ Transitions recordings it measured 36–43 updates per second (Fusion Studio alone runs at 62 fps). Supporting numpy modules cover the occlusion gate, ray-cast occlusion, orientation fusion, online calibration, position models, key postures, finger fusion, transition events and the SteamVR link.

I also wrote four automated test files and a separate benchmark lab. The lab holds leave-one-clip-out and "look-away" evaluation protocols, plus harnesses that replay recordings through the application's real update loop. Leave-one-clip-out means each recording is scored by a model that never saw it in training. The July–August work entered version control on 17 September as one bulk commit, the first of my commits on the research branch. From 17 September to 23 October 2026 I made 55 commits on the research branch (20 in September, 35 in October), including the merge of release v2.1.0, and 3 commits for the two product submissions (490a312 on 25 September; b42142f and 2ceb348 on 23 October). The research-branch commits and 490a312 add 36,906 lines of Python (37,234 counting C++, Markdown and build files), including the generated 4,945-line single-window Fusion Studio.

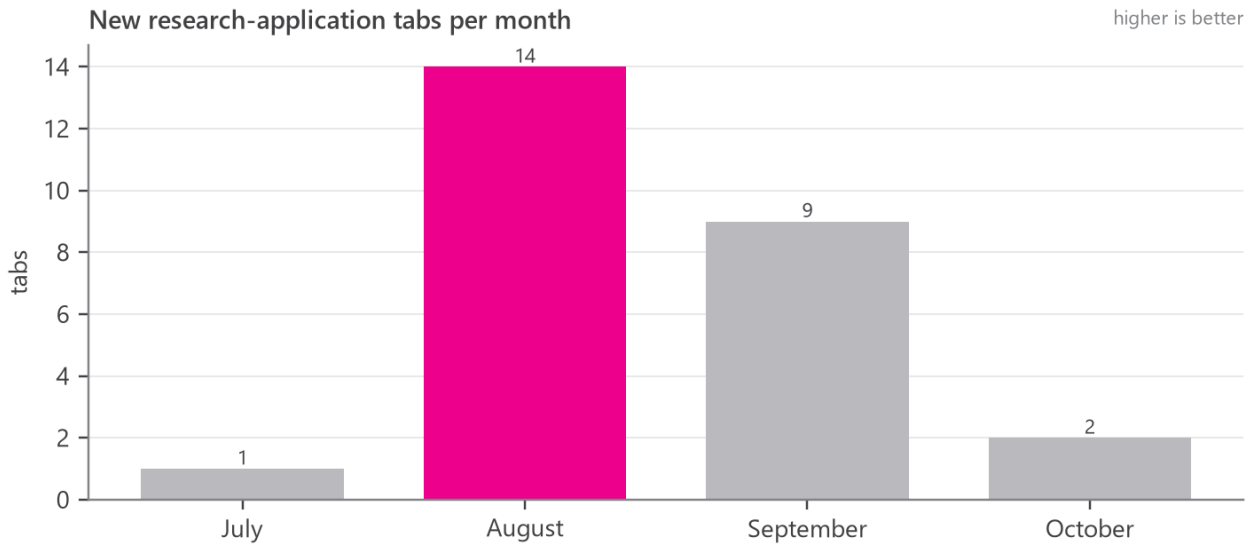


Figure 1.2 *How the 27-tab application grew; the first tab, Gloves, is SciCortex's original window and is not counted. The Occlusion tab was built on 28–30 August.*

Data collected

Table 1.3 Labelled recordings made with the research application. All were recorded by the intern on himself.

Dataset	Recorded with	Dates (2026)	Count
Gesture clips	Capture	Aug 19–26	13: 7 optical-only (143 columns) and 6 in the synchronised 175-column format (one without EMG); 2 of the 13 are aborted stubs
Encoder training pairs	Encoder	Aug 28–29	4 sessions (12,172; 13,027; 16,963; 4,113 pairs)
Online-calibration sessions	Live Calib	Aug 30	3 sessions (22:05, 23:11, 23:17)
Posture clips by wrist orientation	Posture Library	Aug 31	16
Camera-labelled wrist-position clips	Position Capture	Sep 17–21	8
Key-posture clips	Key Postures	Sep 24–25, Oct 23	46 in use (48 recorded)
Posture-change recordings	⚡ Transitions	Oct 22	6 (190 labelled changes)

1.5 Timeline of the internship

Live sessions with the application were logged on 11 days in August, 8 in September and 6 in October. Dates for July and August come from dated notes, files and recording names, because that work was committed in bulk later.

July: the armband and the first fusion question

- 7 July: wrote a 57-page developer guide for the MindRove armband.
- 7–24 July: built an EMG dashboard: an 8-channel signal scope, per-channel signal strength, an IMU panel, a connection light, a simulator, and a 3D hand recognising rest, open, fist, point and pinch.
- July: merged the dashboard into the CyberFinger bridge as the **Armband** tab. The original window became **Gloves**, so the CyberFinger (Bluetooth LE) and the armband (Wi-Fi) stream in one program.
- 26–27 July: studied whether OpenXR's per-joint tracking flags could mark hidden fingers. The specification defines such flags, but the study suggested that Quest's runtime keeps them set, and captures on 19 August confirmed this on Quest: the flags stayed set through a fist, a hand behind the other hand, out of view and behind an object. An on-demand OpenXR reader was added beside SteamVR's, which stays the default.
- In parallel, SciCortex added IMU rendering, SlimeVR tracker emulation and OpenVR hand-skeleton rendering (18–20 July), and an installer update (28 July).

August: instrumentation, orientation fusion, EMG decoding, online calibration

- 3 August: the CyberFinger IMU drove the browser 3D hand, made fully offline (the page grew from 624 to 846 KB) to work on the armband's internet-less Wi-Fi.
- 18 August: analysed the CyberFinger firmware and IMUs and documented them (docs/firmware-and-imu.md, docs/gate-spec.md).
- 19 August: **Gate** tab: five verdicts (TRACKED plus four occlusion types), tuned on labelled clips (open hand TRACKED on 100% of frames, fist SELF-OCCLUDED on 96%). Also the **Capture** tab.

- 20 August: synchronised three-sensor recording on one clock (T1.1).
- 24–26 August: **EMG Hand** and the T2.2 orientation fusion: camera–IMU residual median 10.6–12.6° (about 11°) on the two open-hand clips, and the IMU-only orientation stayed flat for at least 5 s on a rigid-hold clip. Added an automatic centre calibration for the right joystick's hardware offset.
- 28 August: **Encoder**, **Train** and **Fusion Hand** tabs. EMG plus IMU reached 1.30 cm joint error (EMG alone 1.45 cm), and a raw-EMG TCN 1.44 cm against 1.80 cm for a linear baseline. Showed that the Quest cannot label a closed fist.
- 29 August: per-finger curl model with mean absolute error 0.099 and per-finger correlation 0.88–0.92.
- 28–30 August: **Occlusion** tab: per-finger confidence by ray-casting from the headset to each joint.
- 30–31 August: **Live Calib** (the proposal's T3), **Tracking Hand**, **emg2pose (ref)** (Meta's published model reproduced at 11.0° angle error), **Posture Library**, **Posture Clusters**, **Posture Hand** and **Pose Hand**.

September: hand position out of view, and the first product release

- 17 September: **Pose + Position**, **Position Capture** and **Learned Position**. Showed that integrating accelerometer data gives no usable position: 150 cm of error after 1 s.
- 19–20 September: **Body View**, **IMU FK Position** and **Pose Hand (gravity)**.
- 21 September: **Hybrid Position**: forearm kinematics plus small learned models for elbow motion. This is the best out-of-view wrist estimate.
- 24–25 September: ✦ **Fusion Studio** (body, arm and whole hand in one view) and **Key Postures**. A camera-taught IMU mounting calibration kept the knuckle sensor within a median of 8.6–10.3° of the camera for 30 s.
- 25 September: submitted a single-window Fusion Studio to SciCortex, merged the same day (pull request #1).
- 26–28 September: SciCortex built SteamVR driver v2 on it and shipped release v2.1.0 (28 September). The driver gained a FUSED hand source ranked above the headset's own (PASSTHROUGH) hand; at release, nothing fed that source yet.
- 29–30 September: merged v2.1.0 back into the research branch and reviewed the v2.1 manual. Updated both CyberFingers to firmware 1.4.1-beta and adapted the code to the wrist IMU's new data slot.

October: real-time quality, SteamVR output, posture dynamics, second release

- 1 October: the v2.1 VR setup worked end to end. Camera-to-prediction handovers became smooth (single-frame jumps over 20 cm went from 25 to 0 per minute), Fusion Studio went from 35 to 62 frames per second, and the Studio began sending the fused hand to the driver's FUSED source (commit 374b22b; part of the update later submitted as pull request #7).
- 2 October: a test in Resonite (a social VR application) showed the fused hand moving less naturally than in the Studio. The stream was then filtered like the driver's own hand and sent at 120 packets per second.
- 8 October: an optional posture transition prior, and a per-finger camera-versus-EMG weight w = f(m, d), both proposed by my supervisor.
- 9 October: anatomical hand limits (anatomically wrong hidden EMG hands went from 73% to 0%) and the ◈ **Fusion Logic** tab.
- 21–22 October: a SteamVR update moved Steam Link to a newer driver interface (IVRDriverInput_005), and SciCortex's driver stopped receiving the hand's finger data. I fixed the driver; SciCortex committed the same fix on 23 October.

- 22 October: ⚡ **Transitions** tab, following my supervisor's idea: it flags the EMG burst that starts a posture change.
- 23 October: added REST (the relaxed hand between fist and open, suggested by my supervisor) as a sixth key posture, and submitted a second Fusion Studio update with a Windows executable to SciCortex as pull request #7 (open at the time of writing).

1.6 Plan versus outcome

Ten of the twelve IU1 sub-tasks were completed, several with a simpler or better-supported method than planned, and two were partial (no participant study; no buffer window-size study). The task-by-task comparison is in the final section.

Some IU2 items were started early. Per-finger ray-cast occlusion (T5.2) was done. Dropping the calibration buffer when residuals stay large (T7.2) is implemented and still live; it was refined so that a new, unseen pose does not wipe the buffer by mistake. The learned inter-user representation (T5.1), transformer fusion with masked-modality training (T6) and multi-participant evaluation (T8.1) were not started.

Work beyond the original task list:

- estimating the hand's **position** out of view (six tabs);
- key-posture recognition and early posture-change flags;
- the live link that feeds the fused hand into SteamVR;
- the driver fix after the SteamVR update;
- a Windows executable of Fusion Studio;
- firmware updates and a review of the v2.1 manual.

> **Fusion Studio, the project's integrated fused hand, was merged by SciCortex on 25 September 2026 and shipped in release v2.1.0 on 28 September. That release's SteamVR driver added a FUSED hand source ranked above the headset's own hand. From 1 October the Studio feeds that source; this link is part of the update submitted as PR #7.**

1.7 Headline results

Table 1.4 Headline results of IU1; single user throughout. Each is detailed in its tab's section.

Result	Value	How measured
Wrist position out of view (Hybrid Position)	11.3–17.8 cm at 0.5–8 s; freezing 25.9–51.4 cm	Leave-one-clip-out, head turned away
Hand orientation out of view (camera-taught mounting)	Median 8.6° at 1 s, 10.3° at 30 s; old held offset 50° at 1 s, 81° at 3 s	Simulated camera losses in 7 of the 8 position clips (241–281 per horizon)
Hand orientation under camera flips (camera–IMU cross-check)	Drawn error 150° → 4.7°	Replay with injected 150° camera flips
Position jumps over 20 cm at camera handovers	25 → 0 per minute	Replay of a fast session
Key postures with the hand hidden	77% of frames correct; 50% without camera teaching	Recorded sessions
Wrong posture switches while holding a posture	12.1 → 6.5 per minute	Leave-one-arm-position-out, still and turning arm together
Posture changes flagged from EMG	74–75% of 190; fixed threshold 60–63%	Leave-one-recording-out, 6 recordings
Out-of-view posture lag	230 ms (default gate) → 45 ms with the soft hand and anticipation (switched off by default until tried live)	Replay through Fusion Studio, camera hidden
Fusion Studio frame rate	35 → 62 fps	Replayed 60 Hz session
SteamVR stream	120 packets/s; palm jitter 10.2° → 1.45° RMS	Simulated 90 Hz display
EMG finger curls	Error 0.099; correlation 0.88–0.92 per finger	Held-out frames
Online calibration, no weight updates (Live Calib)	In-view curl error vs camera 0.455–0.547 → 0.056–0.124	3 recorded sessions, correction within a session
Online learning from the camera (Pose Hand)	Curl error 0.167 → 0.089 in about 1.7 s	Simulated re-worn armband

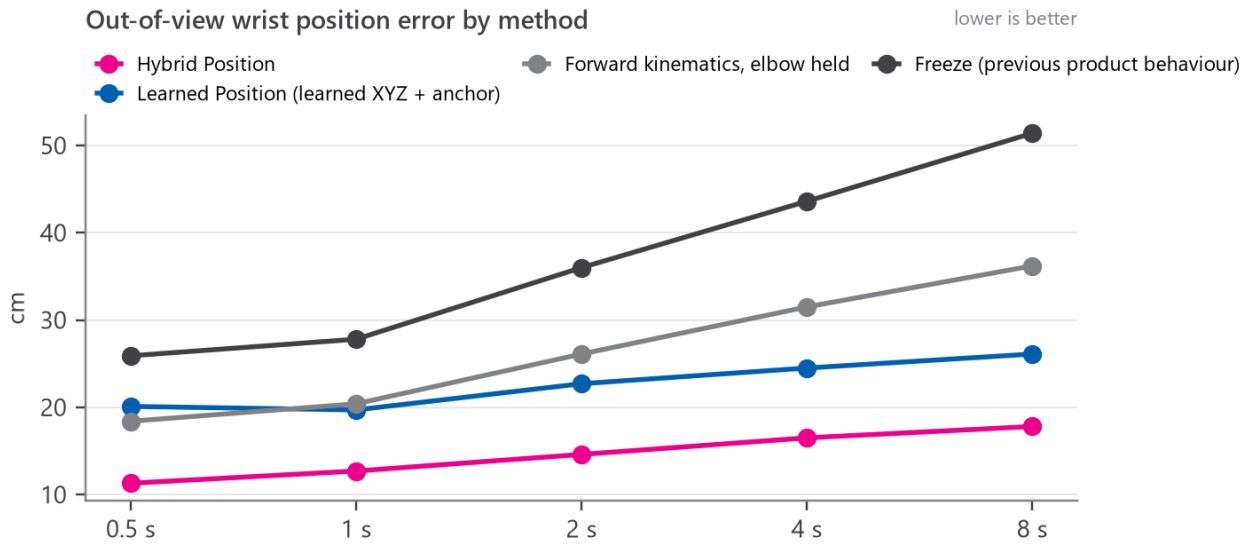


Figure 1.3 *Mean wrist error after the camera loses the hand (leave-one-clip-out, head turned away after the loss, single user). The error of freezing the last-seen position grows to 51.4 cm after 8 s; the hybrid method stays under 18 cm.*

Two seconds after the camera lost the hand, the Hybrid Position method kept the wrist within 14.6 cm on average, against 36.0 cm for freezing it where it was last seen (leave-one-clip-out, head turned away, single user).

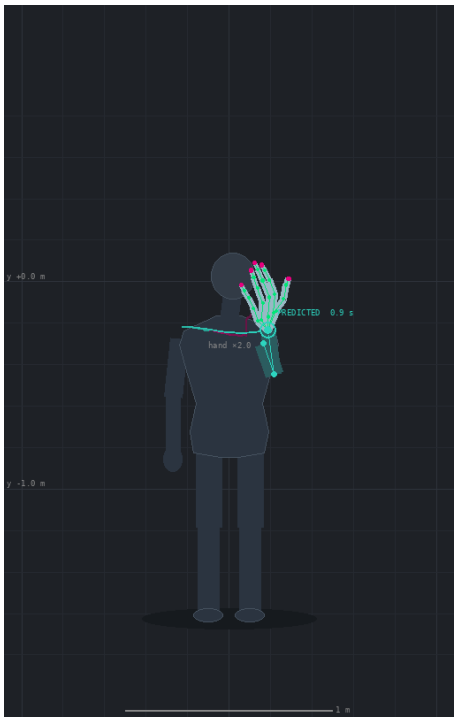


Figure 1.4 *Fusion Studio with the hand outside the camera's view: the hand is drawn at its predicted position ("PREDICTED 0.9 s"), with posture and orientation from the EMG armband and the CyberFinger IMUs.*

1.8 Limitations and negative results

Several findings were negative, and they shaped the design as much as the positive ones. They are stated here and detailed in the later sections.

- All accuracy figures use the Quest 3's own optical hand estimate as the reference. No motion-capture ground truth was used, and the camera cannot label a closed fist, so errors under self-occlusion are measured against a noisy reference.
- On Quest, OpenXR's per-joint flags carry no per-finger occlusion information, so camera confidence had to be computed geometrically.
- The Quest cannot label a closed fist: its estimate falls back to extended fingers, so the camera cannot teach the fist.
- Integrating IMU acceleration for position diverges (150 cm after 1 s). The CyberFinger also sends no gyroscope data, the firmware the units ran until 30 September sent no raw accelerometer data, and early logging ran at about 21 Hz, which rules out learned inertial odometry (estimating position from raw high-rate IMU data).
- The remaining position error comes from the upper arm, which no sensor observes. A simulated upper-arm sensor gives 5.1–9.4 cm.

- Without the camera, the 6-axis heading drifts (up to 92° in 5 minutes), so after minutes the estimate can swap left and right. The horizontal direction is trustworthy for about 1 minute after the last camera fix.
- EMG patterns shift with arm position and between days. Normalisation and distribution alignment (CORAL) made recognition worse (37–44%). The first Key Postures model collapsed across sessions, so the current one needs to see each posture once per session through the camera.
- Pretraining on Meta's emg2pose data gave no net gain: frozen pretrained features beat random ones by 14–18%, but fine-tuning tied training from scratch.
- EMG cannot predict wrist bend.
- An early tilt feature used a heading-dependent axis by mistake. It was corrected only in the new Pose Hand (gravity) tab, because earlier tabs were kept unchanged by design.
- Only the Studio's (right) hand is fused into SteamVR, and left-hand paths are untested live. The Windows executable's live headset path is untested.
- All data come from one user, and no participant study was run.

1.9 How this report is organised

Chapter 2 tours the application tab by tab: a screenshot of each of the 27 tabs, what it is for, what every part of its screen does and what was built in it. The chapters after it follow the research questions, grouping the tabs that answer each one. Each tab has its own heading, using the tab's name exactly as the application shows it.

Table 1.5 Map from the report's sections to the application's 27 tabs and the proposal's tasks.

Topic	Tabs	Proposal tasks
The research application, tab by tab	all 27 tabs, one illustrated entry each	—
System architecture, instrumentation and data pipeline	Gloves, Armband, Capture	T1.1, T4.3
Optical confidence, occlusion and online calibration	Gate, Occlusion, Live Calib	T1.2, T3
EMG-based finger and posture decoding	EMG Hand, Encoder, Train, Fusion Hand, Tracking Hand, emg2pose (ref), Posture Library, Posture Clusters, Posture Hand, Pose Hand, Pose Hand (gravity)	T2.1, T2.3
IMU–optical orientation fusion and sensor-mount calibration	orientation work in Gloves, EMG Hand, Capture and ✦ Fusion Studio	T2.2
Hand position when the camera cannot see the hand	Pose + Position, Position Capture, Learned Position, Body View, IMU FK Position, Hybrid Position	T1.3; beyond plan
Key postures, transitions and anticipation	Key Postures, ◈ Fusion Logic, ⚡ Transitions	T2.1, T4.1
Fusion Studio: the integrated system	✦ Fusion Studio	T4.1, T4.2
Contributions, deliverables and next steps	—	T4.3, T5–T8

2 The research application, tab by tab

A tour of all 27 tabs of the research application in the order they appear: for each one a screenshot from the real application replaying recorded data, what the tab is for, what every part of its screen does, and what was built and found in it, with a pointer to the detailed section.

This chapter is a guided tour of the research application, *bridge/cyberfinger_gui.py*: one entry per tab, in the order the tabs appear in the window. Each entry shows the tab, explains what it is for, walks through what is on the screen, and summarises what was built and learned in it, with a pointer to the section that describes the work in detail. The chapters that follow are organised by research question; this one is organised by what you see on screen.

How the screenshots were made. Every screenshot except the Body View figure comes from the real application running on Windows, but none shows live hardware. The application replayed recorded data through its own update loop. Most tabs replay a synchronised session of 19 September 2026 (headset camera, headset pose, both CyberFinger IMUs and forearm EMG); for the tabs that predict the hidden hand, the camera was deliberately hidden a few seconds before the screenshot. Key Postures, Fusion Logic and Transitions replay a recording of real posture changes with raw EMG, the Armband tab runs its built-in EMG simulator, and Body View is shown through its own drawing code. Each caption states which.

Table 2.1 The 27 tabs at a glance (in the order they appear in the application)

Tab	What it is for	Details
Gloves	The Gloves tab is SciCortex's original CyberFinger Bridge window, kept as the first tab of the research application.	§ 3.5
Armband	The Armband tab brings the MindRove 8-channel forearm armband into the same program that reads the CyberFinger.	§ 3.6
Gate	The Gate tab says, for each hand and in plain words, whether the Quest 3's camera can be trusted right now and, if not, why.	§ 4.3
Capture	The Capture tab records short, labelled clips of a hand doing one known thing (an open hand, a fist, a hand behind the other hand, and so on).	§ 3.7
EMG Hand	EMG Hand is where the project's first complete EMG pipeline was built.	§ 5.2
Encoder	The Encoder tab records the training data for the sEMG finger encoder.	§ 5.3
Train	The Train tab turns the clips recorded in the Encoder tab into a working finger model with one button press, inside the GUI.	§ 5.4
Fusion Hand	Fusion Hand is the end-to-end demonstration of Tasks 2.1 (the EMG + IMU hand encoder) and 2.2 (aligning the CyberFinger's IMU orientation with the camera) working together.	§ 5.5
Occlusion	The Occlusion tab shows, finger by finger, how well the Quest 3 camera can actually see the hand at this moment.	§ 4.4
Live Calib	Live Calib corrects the EMG finger encoder for the current session, user and electrode placement while the user simply moves.	§ 4.5

Tab	What it is for	Details
Tracking Hand	Tracking Hand tests one idea from the research proposal.	§ 5.7
emg2pose (ref)	This tab sets the published state of the art in decoding hand pose from muscle signals beside our own work.	§ 5.8
Posture Library	Posture Library is the tab for collecting posture data.	§ 5.9
Posture Clusters	Posture Clusters asks a direct question: can the forearm EMG features tell hand postures apart, and how much does the wrist's orientation get in the way?	§ 5.10
Posture Hand	Posture Hand answers one question without the camera: which of a small set of hand postures is the user holding?	§ 5.11
Pose Hand	Pose Hand predicts the hand's shape from the forearm muscles and the CyberFinger's motion sensors, so the fingers can still be drawn when the Quest 3 camera cannot see them.	§ 5.12
Pose + Position	Pose + Position is Pose Hand with a second question added.	§ 7.2
Position Capture	Position Capture records the data needed to learn where the hand is from the CyberFinger's IMUs when the headset camera cannot see it.	§ 7.3
Learned Position	The Quest 3 cameras only place the hand while the headset looks at it.	§ 7.4
Body View	Body View shows how far the hand is from the body, as well as its direction.	§ 7.5
IMU FK Position	IMU FK Position works out where the wrist is from geometry, without asking a learned model for its 3-D position (XYZ).	§ 7.6
Pose Hand (gravity)	Pose Hand (gravity) is a twin of the Pose Hand tab.	§ 5.13
Hybrid Position	Hybrid Position shows where the hand is when the Quest 3 camera cannot see it.	§ 7.8
✦ Fusion Studio	The ✦ Fusion Studio tab is the project's showcase.	§ 9.1
Key Postures	The Key Postures tab teaches the system to recognise a few important hand postures (open, fist, point, pinch, victory and rest) from forearm EMG alone.	§ 8.2
◈ Fusion Logic	The ◈ Fusion Logic tab draws, live, the two rules that drive the Fusion Studio's decisions, so a reviewer can see why the hidden hand looks the way it does.	§ 8.3
⚡ Transitions	The ⚡ Transitions tab watches the forearm EMG for the short burst that starts every posture change.	§ 8.4

2.1 Gloves

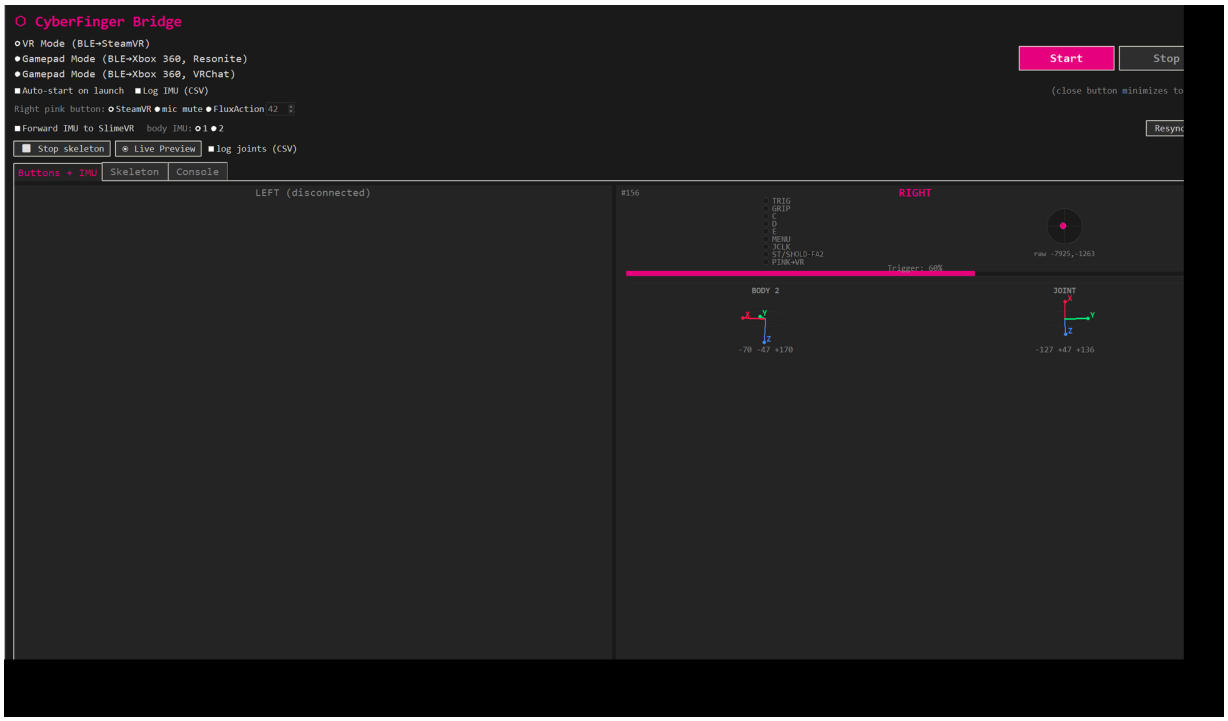


Figure 2.1 *The **Buttons + IMU** sub-tab while the application replays the synchronised recording 'low_new3' (19 September 2026) through its own update loop, from t = 60 s. Only the right CyberFinger was recorded, so the left panel reads disconnected. The BODY 2 and JOINT orientations are recorded data; the trigger, button lamps and stick are animated for illustration, because the recording holds no button data. The window's right edge, with the status text and the battery reading, is cut off in this capture.*

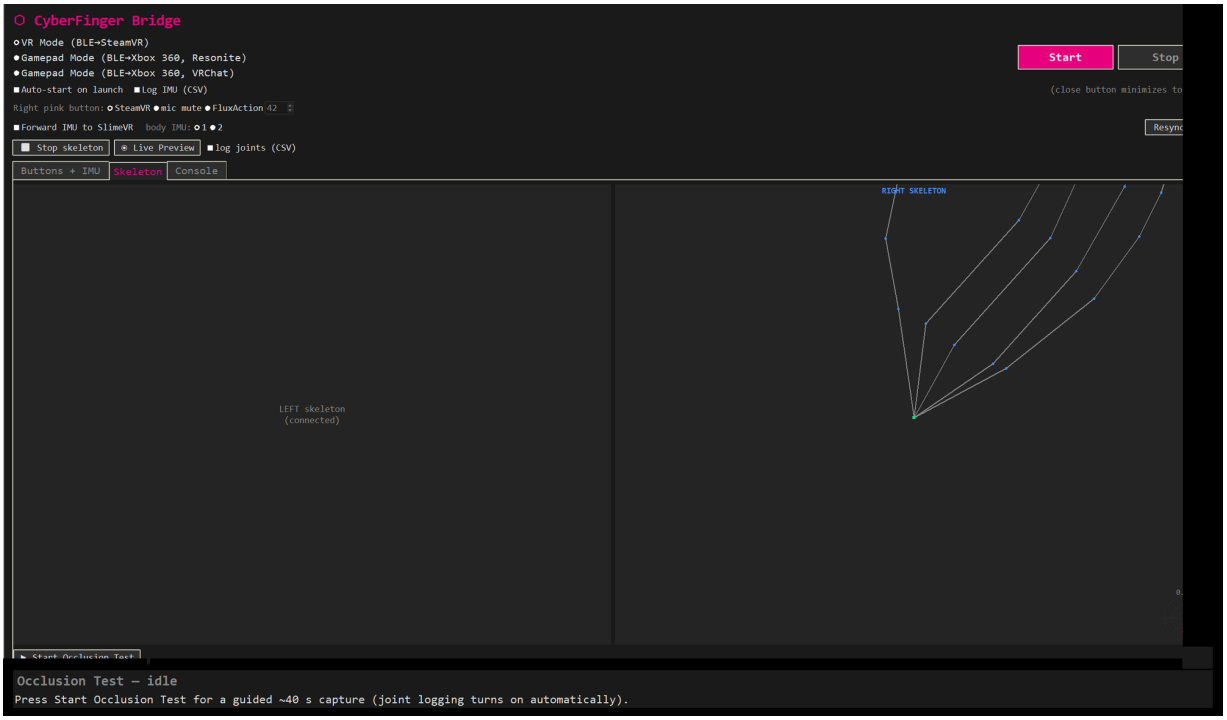


Figure 2.2 *The **Skeleton** sub-tab during the same replay of 'low_new3': the Quest 3 camera's recorded right-hand skeleton, with the idle Occlusion Test panel below. The hand is drawn so large that its fingertips run past the top of the panel, and the position dome sits at the cut-off right edge. At this moment the replay holds no left-hand joints, so the left panel shows only its status line.*

What it is for. The Gloves tab is SciCortex's original CyberFinger Bridge window, kept as the first tab of the research application. It connects both CyberFingers over Bluetooth Low Energy (BLE) and forwards them either to the CyberFinger SteamVR driver (**VR Mode**) or to a virtual Xbox 360 gamepad for Resonite or VRChat. It also shows each CyberFinger's buttons, stick, trigger, battery and IMU orientations (an IMU, or inertial measurement unit, is the motion sensor that reports how a part of the hand is turned). Many of the other tabs rely on the data this tab brings in, so it also gained the research controls that log the IMUs and bring in the headset's optical hand tracking.

On the screen

- ⬡ **CyberFinger Bridge** header and mode row: three radio buttons pick where the CyberFingers go: **VR Mode (BLE→SteamVR)**, **Gamepad Mode (BLE→Xbox 360, Resonite)** or **Gamepad Mode (BLE→Xbox 360, VRChat)**. **Start** connects both CyberFingers and begins forwarding in the chosen mode, and **Stop** disconnects them. A colour-coded status (Idle, Scanning, Connected or an error) sits at the right end of the title line.
- Options row: **Auto-start on launch** starts the last-used mode as soon as the window opens. **Log IMU (CSV)** records every IMU packet from both hands to a CSV file (a plain table that opens in a spreadsheet). The note **(close button minimizes to tray)** means closing the window hides it in the Windows system tray and leaves the bridge running.
- **Right pink button:** row: sets what the right CyberFinger's pink button does in VR Mode. **SteamVR** passes it to the app's own binding, **mic mute** mutes the Windows microphone, and **FluxAction** presses a numbered action in Resonite (through the MoreFluxActions mod), chosen in the box beside it (1–42, default 42).

- SlimeVR row: **Forward IMU to SlimeVR** also sends the IMUs to a SlimeVR server, a separate full-body tracking program, with the wrist IMU as the forearm and the knuckle IMU as the hand. **body IMU: 1 / 2** picks which of the two wrist-sensor slots is sent, falling back to the other if the chosen one is missing. **Resync IMU** (right) makes the SteamVR driver re-fit the IMU heading from the next views of the hand. It works in VR Mode only, and only on CyberFingers that have the knuckle (joint) IMU.
- Research row: ■ **Stop skeleton** (reads ▶ **Start skeleton** when stopped) starts or stops the default hand-skeleton reader. ◉ **Live Preview** (reads ■ **Stop Preview** while running) opens a separate OpenXR source that shows the Quest's optical hands without a VR world. OpenXR is the cross-vendor VR standard; the default reader uses SteamVR's OpenVR interface, which Live Preview leaves untouched. **log joints (CSV)** saves the raw camera joints to a file from the next **Start skeleton**, but only when the skeleton reader runs on OpenXR; the Occlusion Test turns joint logging on by itself.
- Sub-tab strip: **Buttons + IMU**, **Skeleton** and **Console**. The window remembers the last sub-tab used, and only the sub-tab on show is redrawn. **Console** holds the running text log (connections, firmware version, IMU-log and test messages).
- **Buttons + IMU**, one panel per hand (**LEFT** / **RIGHT**). Each panel shows its title, which is dimmed with **(disconnected)** when the CyberFinger is absent, plus a packet counter at the top left (here **#156**) and the battery level at the top right. Below these are lamps for **TRIG**, **GRIP**, **C**, **D**, **E**, **MENU**, **JCLK** and **ST/SE**. A **HOLD·FA2** pill beside **ST/SE** fills while the black button is held and lights when the hold counts as a long press. The pink-button lamp's label follows the setting above (here **PINK→VR**).
- Stick and trigger: the dot in the circle shows the stick after auto-centring and a deadzone, and the **raw** numbers underneath give the unprocessed counts the CyberFinger sends, before centring. A bar labelled **Trigger: n%** runs across the panel.
- IMU triads: one 3-D axis triad for each sensor slot the CyberFinger actually sends (**BODY 1** or **BODY 2** for the wrist module, **JOINT** for the knuckle sensor). Red, green and blue X/Y/Z arms are drawn against a horizon ring, with roll, pitch and yaw in degrees underneath. The panel shows **no IMU installed** when there is none. While the SteamVR driver link is active, a **HAPTIC** strip at the bottom shows the vibration the driver requests, and clicking it sends a test pulse.
- **Skeleton** sub-tab: one canvas per hand (**RIGHT SKELETON**) with grey bones, blue joints, pink fingertips and a green wrist square; a hand with no joints shows only its status (here **LEFT skeleton (connected)**). The hand is turned by its tracked rotation and drawn larger when it is closer to the head. A small dome in the bottom corner shows where the hand is as seen from the head, with rings at 30°, 60° and 90° off the gaze, a one-second trail, a velocity whisker, the other hand dimmed and the distance in metres. Underneath, the ▶ **Start Occlusion Test** panel shows **Occlusion Test — idle** and the instruction for a guided ~40 s capture. During a test it shows the phase number, a countdown, the instruction for each phase, and a banner that is green for VISIBLE phases and orange for occlusion phases.

What was done

- Adopted SciCortex's Bridge 2.1 layout at the 2026-09-29 merge and added a research row: skeleton start/stop, ◉ **Live Preview** on its own OpenXR source, and joint logging. The default skeleton backend deliberately stays OpenVR, and OpenXR runs only on demand. The path from VR Mode to the driver was verified end to end on 2026-10-01: both hands in PASSTHROUGH (the driver mode that passes on the headset's own hand tracking), 31 bones, ~360 Hz.

- Added IMU instrumentation. The firmware version is read over BLE (0x180A / 0x2A26) and logged at every connect. A per-packet IMU logger, fed from the Bluetooth thread, records host time, sequence number, hand, which IMU slots are present, the body and joint quaternions (each orientation as four numbers) and the raw acceleration at 2048 counts per g. The stream was measured at 99.4–100.0 reports/s per CyberFinger, with no lost packets in 121,104 (2026-08-18).
- Added a one-way BODY 2 → BODY 1 fallback for firmware 1.4.x. Before firmware 1.4 both lab units reported imu_present = 6 (BODY 2 + JOINT). After the 2026-09-30 flash the wrist sensor arrives as BODY 1, which the fallback handles.
- Fixed an off-centre stick with a joystick auto-centre. The RIGHT unit's stick rested at about (−16087, −15815), roughly −49% of full scale, while the LEFT read (0, 0). Low battery was ruled out, since both units were at 15–18%. The centre is now the median of the first 20 packets after every connect, followed by a 0.15 radial deadzone, and a log line is written when the offset exceeds 1,500 counts. After the fix (2026-08-26) the stick reads centred and the spurious gamepad input stopped. A stick that was actually broken would still need repair.
- Built the ▶ **Start Occlusion Test**: a 3 s countdown, then 5 scripted phases of 8 s each (VISIBLE, SELF OCCLUSION, VISIBLE, HAND BEHIND HAND, VISIBLE), with the skeleton and phase times written to CSV. The skeleton log folder (80.1 MB) holds 9 Quest skeleton captures from 2026-08-18 to 2026-08-29. Of these, 4 are phase-labelled Occlusion Test runs and 2 complete all five phases. The folder also holds 369 gate-event logs from 2026-08-18 to 2026-10-23.

→ In detail: Section 3.5 — Gloves: connecting the CyberFingers

2.2 Armband

Figure 2.3 *The Armband tab running its built-in EMG **Simulator** source, not the real armband, captured 8 s after **Start**. The purple **SIMULATED** status, the "SIMULATED DATA" watermark and the fixed 84% battery all come from the simulator. The traces on **CH1** to **CH8** and in the motion plots are generated test signals, not recorded muscle activity or movement.*

What it is for. The Armband tab brings the MindRove 8-channel forearm armband into the same program that reads the CyberFinger. The armband records EMG (electromyography: the electrical activity of the forearm muscles). Running in one program means muscle signals, IMU (inertial measurement unit: accelerometer and gyroscope) data and optical data share one clock. The tab shows the armband's signals live and serves a 3D hand to a web browser: a gesture recogniser poses the hand, and the CyberFinger's IMUs can turn it. It was built this way because only one program can own the armband's Wi-Fi session, so the armband's signals and the CyberFinger's have to be read by the same program. Every later EMG tab depends on it.

On the screen

- **Source: Armband** / **Synthetic** / **Simulator**: chooses where the signals come from. The options are the real armband over Wi-Fi, the MindRove software kit's built-in fake board, or a self-contained simulator that needs no hardware or driver software. With the real armband the connection runs in the background and retries every 2 s, so the window does not freeze while the armband is out of reach.
- **Start** / **Stop**: **Start** builds the dashboard for the chosen source and places its plots in the tab. Until then the area shows a short hint ("Pick a source and press Start."). If the plotting libraries are missing, pressing **Start** shows the packages to install instead. **Stop** ends the stream. Only one of the two buttons is active at a time; in the screenshot **Start** is greyed out and **Stop** is active.

- **Show 3D hand in browser** (top right): becomes active after Start. It opens a web page that the program itself serves on local port 8770 and that shows a 3D hand. The page asks for the latest pose about every 60 ms. The fingers take the gesture that the bundled recogniser names from the last 0.25 s of EMG. When a CyberFinger IMU is reporting, its orientation turns the hand. Dragging with the mouse rotates the view in either case, and with no IMU the hand also turns slowly by itself when left idle.
- **EMG DASHBOARD** header: a coloured status light and status line. **LIVE** is green, **SIMULATED** purple (shown for the **Simulator** source; the **Synthetic** board reports **LIVE**), **CONNECTING…** and **NO SIGNAL** amber, and **DISCONNECTED** red. Each state is followed by a plain hint, such as "can't open armband — check Wi-Fi & close other apps using it" or, as in the screenshot, "simulated data — device NOT connected". When the stream is live and there is no hint, the source name is shown instead.
- Status tiles **SOURCE**, **SAMPLE RATE**, **THROUGHPUT**, **SAMPLES**, **BATTERY**, **UPTIME**: in order, the data source, the nominal sample rate, the samples per second actually received (averaged since Start), the running sample count, the armband battery level and the time since Start. The battery level is green above 40%, amber above 15% and red below that.
- **EMG CHANNELS**: a scrolling view of the last 4 s (axis marks **-4s**, **-2s**, **now**) with one coloured lane per EMG channel, **CH1** to **CH8**. The note "band-pass 20–200 Hz · notch" gives the filtering: it keeps the 20–200 Hz muscle band, and a notch filter aimed at mains hum removes a narrow band at the power-line frequency (still set to the 50 Hz default; see the signal-chain note below). The number at the right of each lane is that channel's RMS (root mean square, a measure of signal strength) in µV over the window. For a fake source, a faint "SIMULATED DATA" watermark sits behind the traces.
- **ACTIVATION (RMS µV)**: one horizontal bar per channel showing the same RMS values, so the most active muscles stand out at a glance. The bar scale grows at once to fit the strongest channel and shrinks back slowly when activity drops.
- **ACCELEROMETER (g)** and **GYROSCOPE (°/s)**: the armband's own motion sensor over the same 4 s window. The left plot shows acceleration in g on a ±1.5 g scale; the right plot shows turning rate in degrees per second on a ±60°/s scale. Each has X, Y and Z traces in blue, green and orange.

What was done

- **Embedded rather than run separately.** I embedded my standalone MindRove dashboard instead of launching it beside the bridge or rewriting it, because only one program can own the armband's Wi-Fi session. It runs in embedded mode, its plot sits inside the tab, and the GUI's own update loop drives it. The two-tab GUI (**Gloves** + **Armband**) was running by 2026-08-03.
- **Updating that survives drawing errors.** Data are read on every tick even when the tab is hidden. The plots are redrawn every second tick, and only while the tab is visible. After 5 consecutive drawing failures, redrawing stops while the data and the browser hand keep working. This fallback handled the occasional FreeType drawing overflow seen in the WSL test environment.
- **Signal chain.** Data at 500 Hz pass through a 20 Hz high-pass filter, a 200 Hz low-pass filter and a mains notch into a 4 s rolling buffer. Later recordings confirmed the 500 Hz rate (497.3–499.5 samples/s delivered). The notch is still at the 50 Hz default, while mains in Canada is 60 Hz. The effect was not measured, but the notch should be set to 60 Hz.

- **Browser hand that works without internet.** The same process serves the hand on port 8770. A bundled linear model with 9 classes (fist, ok, open, peace, pinch, point, rest, rock, thumbs_up) on 40 features names the gesture. Hand position is deliberately not taken from the IMUs, because it would drift. On the armband's internet-less Wi-Fi, the page at first fell back to a cylinder. Embedding the 3D-model loader (GLTFLoader r128) and the hand mesh in the page fixed this on 2026-08-03: the page grew from 624 KB to 846 KB and is 849,093 bytes today. The mapping from the IMU axes to the hand's axes was never tuned on the hardware and is still the identity.
- **Lesson: the bundled gestures failed on live data.** On 2026-08-27 the bundled 9-class model failed on live features. A custom model that scored 100% in cross-validation on its own samples failed live about 20 minutes later. Both failures came from surface-EMG drift between sessions and electrode placements, the problem the later EMG tabs were built to handle.
- **Base for all the EMG work.** Every EMG-based tab, from EMG Hand through Transitions and Fusion Studio, reads its EMG from this embedded dashboard.

→ In detail: Section 3.6 — Armband: forearm EMG in the same program

2.3 Gate

Figure 2.4 *The Gate tab in the real GUI, replaying the synchronised session 'low_new3' of 19 September 2026 through the GUI's own update loop, from t = 60 s. The session is 401 s long and holds Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features. The box is this user's calibrated box (az ±87°, el −82° to +18°). The right hand is TRACKED inside the box: 0.47 m away, 18° right of and 28° below the gaze line, with a full green confidence bar. The left hand has no usable data at this moment, so it is reported OUT OF VIEW and its bar is empty.*

What it is for. The Gate tab says, for each hand and in plain words, whether the Quest 3's camera can be trusted right now and, if not, why. It gives one of five verdicts: TRACKED, OUT OF VIEW, SELF-OCCLUDED, OCCLUDED BY OTHER or BEHIND OBJECT. The tab was needed because the headset's own tracking flag cannot answer this question: on the labelled clips its *is_active* flag stayed at 1.00 even while the fingers were hidden in a fist. Every later fusion tab uses this verdict to decide when to switch from the camera to the CyberFinger's IMUs (inertial sensors) and the EMG armband (electromyography, the electrical activity of the forearm muscles), and training uses it to set how much weight each camera label gets.

On the screen

- ⬡ **Gate Radar — optical sensor view** (title and one-line guide): the guide says that the green box is where the Quest cameras actually track the hands, that a hand outside it is OUT OF VIEW, and that inside it each hand is labelled tracked, self-occluded or occluded by the other hand.
- ⊹ **Calibrate box** button and the message beside it: the button learns the tracking box from the directions in which the Quest actually tracked the hand during the session. It needs at least 300 tracked samples. It takes the 98th percentile of the absolute azimuth (the left-right angle from the gaze line) and the 2nd and 98th percentiles of the elevation (the up-down angle), widens each by 3°, clamps them to fixed limits and saves the box in the application's settings, so later sessions reuse it. With too few samples the message reads **move your hand to all edges during Live Preview first, then calibrate**. Otherwise it shows the new box and the number of samples used.

- **LEFT | RIGHT banner**: a bar split in two halves, each reading e.g. **LEFT: OUT OF VIEW** and coloured by that hand's verdict: green for **TRACKED**, red for **OUT OF VIEW**, orange for **SELF-OCCLUDED** or **OCCLUDED BY** (followed by the other hand's name), and purple for **BEHIND OBJECT**. Both hands show at once, even when their states differ.
- **Tracking box** (green rectangle, labelled e.g. **tracking box az±87° el[-82,18]**): the calibrated range of directions, seen from the headset, in which the Quest tracks hands. The view is scaled so that the box always fills about 82% of the canvas, centred on the box's own middle elevation. A wrist outside it is judged OUT OF VIEW.
- **Gaze crosshair** (small grey +): straight ahead of the headset (azimuth 0°, elevation 0°). Because the box is centred on its middle elevation, the crosshair's position shows the headset's tilt. With this user's box (−82° to +18°) it sits near the top, since the Quest tracks much further below the gaze line than above it.
- **Hand skeletons**: each hand's joints, converted to directions from the headset and drawn as the five finger chains from the wrist, with a square at the wrist and an L or R tag. A TRACKED hand is drawn in its own colour (left blue, right pink). With any other verdict the skeleton takes that verdict's colour. Without per-joint data a hand is shown as a single dot at the wrist. A hand that disappears leaves a ghost × at its last on-screen position, coloured by its verdict. When both hands are present, a line joins the wrists and turns red when either hand is occluded by the other. When neither hand is present, the centre shows the source status, e.g. **start ◉ Live Preview or ▶ Occlusion Test**.
- **Confidence bars** (one per hand, just above its status line): a geometric score equal to 1 minus the larger of two shares: the share of curled fingers and the share of joints hidden behind the other hand. It is red below 0.35, orange from 0.35 up to 0.70 and green from 0.70. It is not a probability. It also ignores the box and freeze tests, so a hand judged OUT OF VIEW while its pose is still reported can still show a full bar. It is empty when the hand has no data.
- **Status chips** (bottom left and bottom right): one line per hand with its short verdict (**tracked**, **OUT OF VIEW**, **SELF-OCC**, **OCC-BY-OTHER**, **BEHIND OBJECT**), in the hand's own colour when tracked and in the verdict's colour otherwise. For a present hand the line also gives the headset-to-wrist distance in metres and the wrist's azimuth and elevation.

What was done

- **Reference gate first (18 August)**: a written specification (docs/gate-spec.md) and a reference implementation (fusion/gate.py). Each joint's confidence is the product of four cues: the OpenXR whole-hand switch, a fixed field-of-view cone (half-angles of 55° horizontal and 45° vertical), a ray-cast occlusion test and a motion-plausibility test. On a 30-second synthetic benchmark (46,800 joint samples), the recommended gate separated reliable from unreliable joints with a ROC-AUC of 0.753. ROC-AUC measures how well a score separates two groups (0.5 = chance, 1 = perfect). For comparison, the OpenXR flags alone score 0.5 (a constant score by construction), and an occlusion test against the true geometry, the ceiling, reaches 0.879. Gating cut the joint error during occlusion from 23.4 cm to 16.0 cm, at the cost of 0.71 cm instead of 0.32 cm while the hand was visible. In this benchmark the fused estimate was an idealised stand-in, so these numbers show how the gate routes between sources, not end-to-end accuracy.
- **Negative result on real captures**: on scripted captures recorded with the headset worn on the forehead, the fixed cone judged tracked hands to be outside the view. The field-of-view cue was only 0.02–0.07 in the VISIBLE phases of the 18 August 17:35 capture. Ray-casting against the Quest's pose did not separate the phases either (visibility 0.94 while visible, 0.96 during self-occlusion), because the Quest returns a plausible pose that is not geometrically occluded. So the live tab does not run the reference gate. That gate is used offline.

- **Live classifier tuned on real clips**: the rules are checked in this order. OUT OF VIEW = the wrist is outside a calibrated tracking box (±87° azimuth, −82° to +18° elevation for this user). OCCLUDED BY OTHER = at least 15% of the joints lie inside the other hand's silhouette, as the camera sees it, and deeper than it. A hand that vanished where the other hand now sits also counts. SELF-OCCLUDED = at least 30% of the fingers are curled (knuckle-to-tip distance below 0.65 of the finger's length). A hand that vanished, or froze for at least 5 identical frames, is BEHIND OBJECT if it was at least 8° inside the box and OUT OF VIEW otherwise.
- **Results on labelled clips (19 August)**: the GUI's rules were replayed on the labelled right-hand clips that were also used to tune the thresholds. They labelled 100% of open-hand frames TRACKED and 96% of fist frames SELF-OCCLUDED, while the Quest's *is_active* flag stayed at 1.00. The hard cases got the expected verdict less often: wrist bend 63% of frames, out to the side 44%, behind an object 19% and behind the other hand 14%. A second fist clip of 21 frames was only 38% SELF-OCCLUDED. There are two causes. The Quest tracks very wide and keeps inferring plausible poses instead of reporting a loss. The hand-behind-hand clip was staged so that the hands barely overlapped in the camera's view.
- **The verdict feeds the rest of the system**: it is computed on every update, even when the tab is not open. Every change of verdict is written with a timestamp to skeleton_logs/gate_events_*.csv (369 logs from 25 days, 18 August to 23 October). From 20 August, every row of a synced Capture clip carries the verdict (gate_type) on the same clock as the optical, IMU and EMG data. In training, the verdict sets each camera label's weight: TRACKED 1.0, SELF-OCCLUDED 0.5, OCCLUDED BY OTHER 0.4, BEHIND OBJECT 0.3, OUT OF VIEW 0.2. Later recorders use the mean ray-cast finger confidence instead when it is available.
- **Known limits**: the confidence bar is a geometric score, not a probability. Box calibration collects samples only from frames already inside the current box and clamps the result, so one calibration widens the box by only about 3°. Without the headset pose the tab falls back to geometry alone and reports any visible hand as TRACKED. Geometry also misses some failures (the Quest sometimes flips the hand while the gate still says TRACKED), so Fusion Studio also cross-checks the camera against the knuckle IMU.

→ In detail: Section 4.3 — Gate: why the camera can or cannot be trusted right now

2.4 Capture

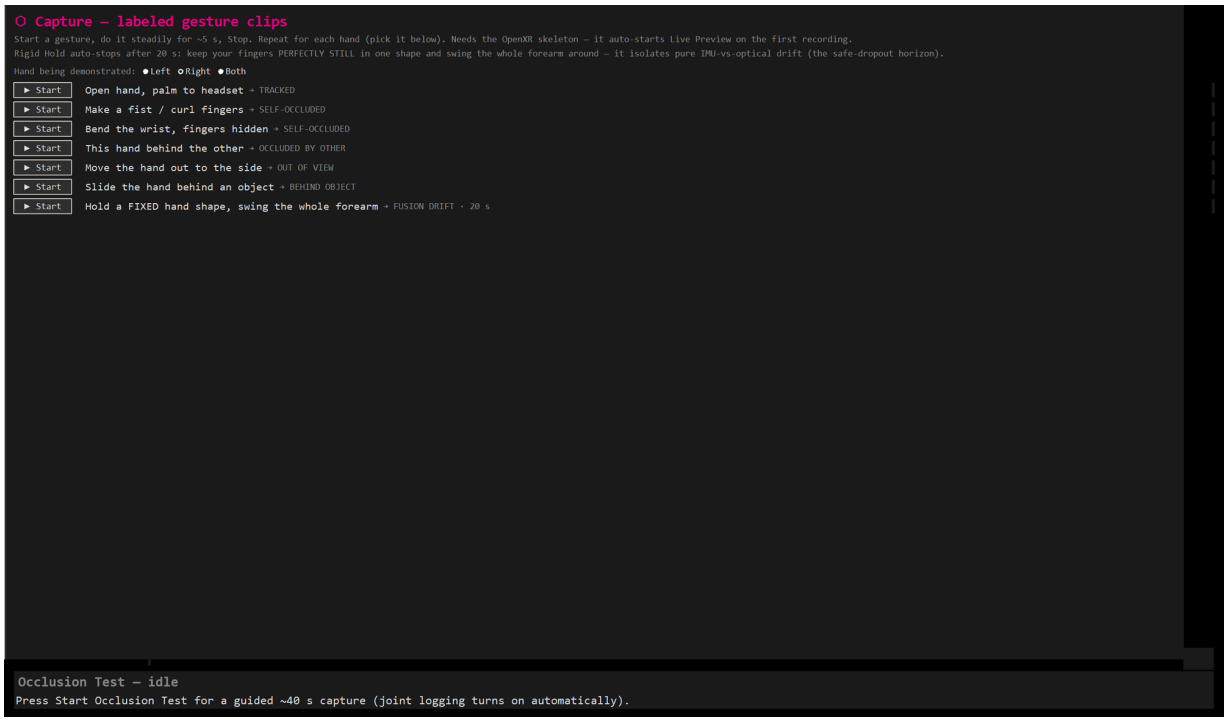


Figure 2.5 *The Capture tab at rest: the hand selector, then seven presets, each with its ▶ **Start** button and description, followed by the expected gate label (for the rigid hold, **FUSION DRIFT · 20 s**). Taken from the real GUI while it replayed the synchronised session 'low_new3' of 19 September 2026 (from t = 60 s) through its own update loop; this tab shows no live sensor data, so the replay leaves no visible trace here. No clip was being recorded, so the status readouts are empty, and the bottom strip belongs to the Gloves tab's Occlusion Test.*

What it is for. The Capture tab records short, labelled clips of a hand doing one known thing (an open hand, a fist, a hand behind the other hand, and so on). Each clip stores the headset's optical hand tracking, the CyberFinger's IMUs (motion sensors in the wrist module and the knuckle sensor) and the forearm EMG armband (electrodes that pick up muscle activity) in one file, on one shared timestamp. It was needed for two reasons. The occlusion gate, the logic that decides whether the camera's view of a hand can be trusted and, if not, why, had been tuned by guesswork. And fusing the three sensors needs data in which all three are recorded on the same clock. These clips were the project's first synchronised dataset, which delivered proposal Task 1.1.

On the screen

- ⬡ **Capture — labeled gesture clips**: the tab title.
- **Instruction lines**: "Start a gesture, do it steadily for ~5 s, Stop. Repeat for each hand…" explains the routine and notes that recording needs the OpenXR hand skeleton (the headset's per-joint hand data) and starts **Live Preview** on the first recording. The second line explains the **Rigid Hold** preset: it stops itself after 20 s, and the user keeps the fingers perfectly still and swings the whole forearm, so that any difference between IMU and camera comes from drift alone, not from finger movement.
- **Hand being demonstrated: Left / Right / Both**: chooses which hand the clip is named after (Right by default). It only sets the file name, because both hands are written to every clip.
- ▶ **Start** buttons, one per preset: a press opens a new clip file, bridge/gesture_logs/<gesture>_<hand>_<timestamp>.csv, and turns the button into ■ **Stop**, which

closes the file. The other six buttons are greyed out, so only one clip records at a time. If no OpenXR source is running, Live Preview is started automatically. If the OpenXR hand-skeleton source or the occlusion-gate module is not available, the row reports "needs OpenXR (pip install pyopenxr)" instead.

- **Preset descriptions** (white): the six occlusion scenarios the user acts out, namely "Open hand, palm to headset", "Make a fist / curl fingers", "Bend the wrist, fingers hidden", "This hand behind the other", "Move the hand out to the side" and "Slide the hand behind an object".
- **Expected gate labels** (grey, after the arrow): **TRACKED**, **SELF-OCCLUDED**, **OCCLUDED BY OTHER**, **OUT OF VIEW** and **BEHIND OBJECT** name the verdict the occlusion gate should give for that scenario. Every clip is therefore labelled by design (the preset's name is in the file name), while the gate's actual verdict at each tick is stored in every row (the gate_type column), so the two can be compared.
- **Hold a FIXED hand shape, swing the whole forearm → FUSION DRIFT · 20 s**: the rigid-hold preset. It stops itself after 20 s and gives a clean window for measuring how far the IMU orientation drifts from the optical one.
- **Status readout** at the right end of each row (empty in the screenshot): while a clip records it shows "recording… N frames" (for rigid hold, a countdown "recording… Ns left (N frames)"), and then "saved · N frames" once the clip is closed.
- **Occlusion Test — idle** strip at the very bottom: not part of this tab. It is the status panel of the separate guided Occlusion Test in the Gloves tab's Skeleton view, and it appears in this screenshot only as an artefact of how the window was captured.

What was done

- **A one-click labelled recorder.** The tab offers 7 presets, each with Start/Stop and the gate label it should produce. Rigid hold stops itself after 20 s, and recording starts Live Preview on its own. Inside the GUI's main update loop (the "tick"), the tab writes one row per hand per tick, and each row holds the latest value of every sensor stamped with the shared wall_time.
- **A synchronised clip format (from 2026-08-20).** Each row has 175 columns: a 13-column header (host, wall and host-nanosecond times, hand, an is_active flag and the headset pose), 130 optical columns (26 joints × x, y, z, flags, radius), 17 CyberFinger columns (gate label, imu_present, the BODY 1 / BODY 2 / JOINT orientation quaternions and BODY acceleration) and 15 armband columns (8 EMG RMS values, armband accelerometer and gyroscope, emg_present). The new columns were appended at the end of the row, so the older 143-column clips and the offline gate tools, which read columns by name, kept working.
- **Data collected.** Thirteen clips were recorded between 2026-08-19 and 2026-08-26: seven optical-only and six synchronised. Two were aborted stubs, which leaves 11 usable clips. Rows were written at about 21.5 per second per hand (median spacing 45.0–47.1 ms). Cross-correlating the IMU and optical rotation speeds peaks at lag 0 ($r = 0.645$, against 0.34 one frame later), so the streams were aligned to within one frame (<47 ms) and no fixed delay needed subtracting.
- **Gate tuning on real data.** The headset's own per-joint validity and tracking flags were set on 100% of frames in five of the six scenarios (79% in wrist bend), confirming that they carry no occlusion information. The clips also set the gate's finger-curl rule: open → CLEAR (tracked) on 100% of frames, fist → SELF-OCCLUDED on 96%, wrist bend → SELF-OCCLUDED on 63%.
- **First fusion numbers.** The first calibration between the IMU and the optical hand orientation left a median residual of 10.6–12.6° on the two open-hand clips of 2026-08-20. In the rigid-hold clip the arm swept 179°, and the wrist IMU alone, using the orientation filter of that time, held the orientation error between 13° and 19° from 0.5 to 5 s, with no growth over time. That gives a safe camera-dropout horizon of at least 5 s.

- **Limitations.** The EMG columns are an RMS (root-mean-square level) over the dashboard's whole 4 s buffer, about 33 ms stale. That measures how active the muscles are, but it is not fast enough to decode finger movements, so later recorders stored raw EMG instead (Encoder tab windows, 500 Hz Transitions logs). The clips use the old ~21.5 Hz tick, and recording needs an exclusive OpenXR session, so it cannot run alongside a VR application. One rigid-hold clip has no EMG because the armband was not streaming, and another contains rows without IMU data, recorded while the knuckle sensor was temporarily mounted on a finger.

→ In detail: Section 3.7 — Capture: synchronised, labelled clips

2.5 EMG Hand

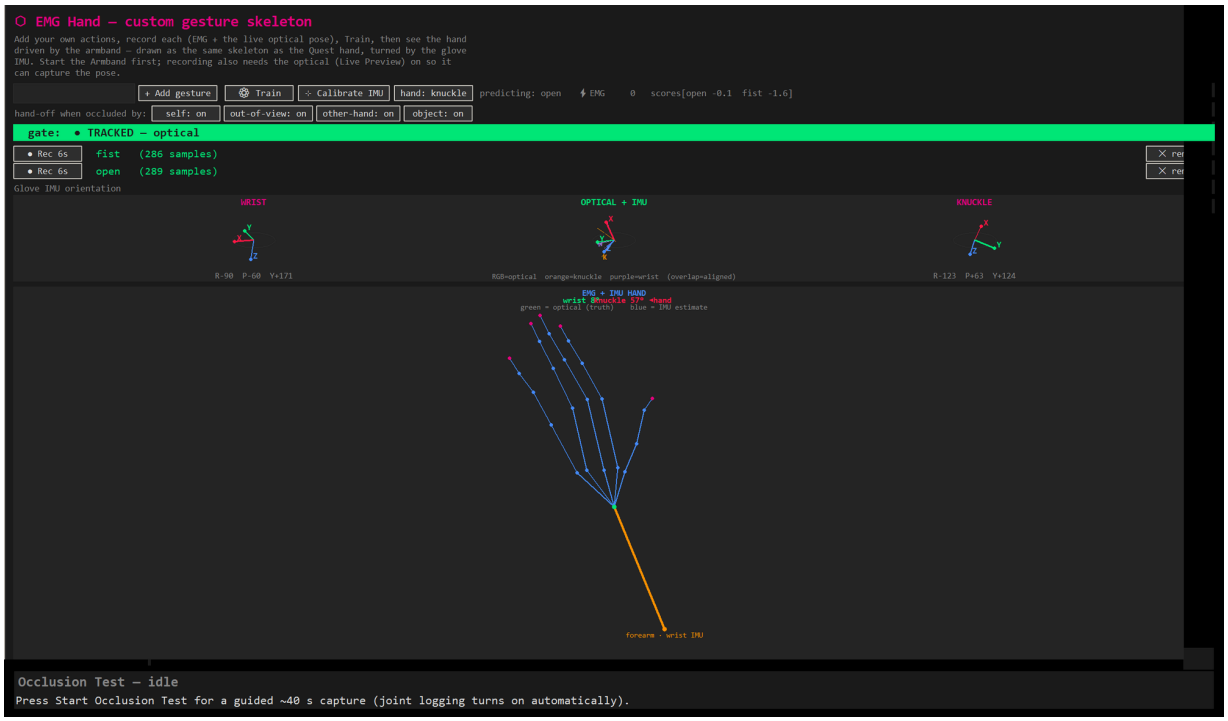


Figure 2.6 *The EMG Hand tab while the real GUI replays the synchronised session 'low_new3' of 19 September 2026 through its own update loop, captured after 6 s of replay starting at t = 60 s. The session is 401 s of Quest 3 camera hand joints, headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features. The replay feeds the session's recorded 40 EMG features and its recorded gate verdicts; the raw EMG signal is not replayed, so the status line's EMG level reads 0. The two gestures (fist and open) come from the gesture model the GUI had saved in an earlier session. At this moment the gate reads TRACKED, so the blue skeleton is the camera hand, and the orange forearm bone is turned by the wrist IMU. The Occlusion Test — idle strip at the bottom is a leftover of the window capture, not part of this tab.*

What it is for. EMG Hand is where the project's first complete EMG pipeline was built. EMG (electromyography) is the electrical activity of muscles, here the forearm muscles, picked up by the 8-channel MindRove armband. In this tab the user teaches the system a few gestures of their own, and the EMG then decides which gesture is being held. It was needed because the Quest 3 camera loses the hand whenever the hand is hidden or leaves the camera's view. When that happens, this tab keeps a hand on screen: the EMG supplies the finger shape and the CyberFinger's IMUs (inertial measurement units, motion sensors that report orientation) supply the hand's rotation. Everything is drawn with the same 26-joint skeleton as the camera hand, so the two can be compared directly.

On the screen

- **EMG Hand — custom gesture skeleton**: the title, followed by a short instruction. It says to add your own gestures, record each one (EMG plus the live camera pose), **Train**, and then watch the armband drive the hand. The armband must be started first, and recording also needs the camera hand (**Live Preview**) switched on.
- Name box, **+ Add gesture** and ⚙ **Train**: type a name (for example "fist") and press **+ Add gesture** or Enter to add a row for that gesture. ⚙ **Train** fits the gesture classifier on every gesture that has at least five samples, and at least two such gestures are needed. It then saves the model and reports "trained ✓" with the number of gestures. Otherwise it reports "need ≥2 gestures with ≥5 samples each".

- ⊹ **Calibrate IMU** and **hand: knuckle**: ⊹ **Calibrate IMU** restarts the orientation fusion. If the tab has stored at least 30 frames in which the camera tracked the hand, it works out the fixed rotation between each IMU (wrist and knuckle) and the camera's hand frame. It applies and saves that rotation, so it still holds with the camera off and after a restart, and shows each IMU's median fit error in degrees. With fewer frames it asks the user to show the palm and move the hand for about 5 s. Independently of the button, the tab keeps re-calibrating on every tracked frame and snaps back into line when the camera finds the hand again. **hand: knuckle** selects which IMU turns the hand while the camera has lost it. Each click cycles knuckle → wrist → auto; auto uses whichever IMU currently agrees better with the camera, switching only when the other one is better by more than 8°.
- Status readout (here **predicting: open ⚡EMG 0 scores[open -0.1 fist -1.6]**): once a model is trained, the classifier runs all the time, even while the camera hand is drawn. The line shows the gesture the EMG currently selects, the EMG activity level (its root-mean-square amplitude over the last 128 samples, about 0.26 s) and the two highest class scores, so it is clear whether the decision is clear-cut or close. While a gesture is being recorded, it shows "get ready…" and then "● REC", with the gesture name, the EMG level, whether the hand shape is being captured ("capturing shape ✓", "shape held" or "⚠ SHOW HAND TO CAMERA") and the sample count. The results of training, calibrating and recording are also reported here.
- **hand-off when occluded by:** with **self: on**, **out-of-view: on**, **other-hand: on** and **object: on**: one switch for each reason for camera loss that the occlusion gate reports (the Gate tab's classification of why the camera lost the hand). Switching a reason OFF (the button then reads, for example, **self: OFF**) makes the tab keep the camera hand in that case instead of handing over to EMG + IMU. For example, turning **self** off keeps the real fingers on screen during a fist, which the gate classifies as self-occlusion.
- **gate:** banner: a coloured bar that shows what the occlusion gate sees and which source draws the hand. Green "● TRACKED — optical" means the camera hand is drawn. Orange "◒ … — EMG + IMU" (for example "◒ out of view — EMG + IMU") means the hand is hidden and EMG and the IMUs have taken over. Pink with "(gate off) — optical" means a reason that was switched off is being ignored, and a plain dark bar with "— no hand" means the gate has no hand to judge. Any handoffs that are switched off are listed at the end of the bar.
- Gesture list (here **fist (286 samples)** and **open (289 samples)**): one row per gesture. ● **Rec 6s** records a take, and reads ● **recording…** meanwhile. It starts the camera preview and the armband if they are not running, and refuses with a warning if the armband still is not streaming. It discards the first 1.5 s, samples the next 6 s, and replaces the gesture's earlier samples. It also keeps the last camera skeleton seen while sampling as that gesture's finger shape, even when the gate calls the hand self-occluded, as a fist often is. Next come the gesture's name and sample count, which turns green at five samples. ✕ **remove**, at the right edge (cut off in this screenshot), deletes the gesture and clears the trained model, so ⚙ **Train** has to be pressed again.
- **Glove IMU orientation**: three sets of axes (triads). **WRIST** shows the raw orientation of the wrist-module IMU, with roll, pitch and yaw in degrees (here **R-90 P-60 Y+171**). **OPTICAL + IMU** shows the camera's hand orientation as red, green and blue axes. Each IMU's calibrated estimate is drawn over it as a thin triad, purple for the wrist (W) and orange for the knuckle (K); when they overlap, that IMU agrees with the camera. **KNUCKLE** shows the raw orientation of the knuckle IMU, also with roll, pitch and yaw.
- **EMG + IMU HAND**: the main view, drawn with the same 26-joint skeleton as the camera hand. The blue skeleton is the tab's output hand; pink dots mark the fingertips and a green dot the wrist. While the camera sees the hand, the blue skeleton is the camera hand itself, only lightly smoothed while the hand is almost still. When the hand is hidden, the finger shape is the stored skeleton of the gesture the EMG selects (before any training, the last camera finger shape), and the chosen IMU turns the hand onward from the last camera orientation, with a short cross-fade at each switch. While the hand is tracked, the

camera hand is also drawn as a thin green outline (*green = optical (truth), blue = IMU estimate*). The orange bone labelled **forearm · wrist IMU** is turned by the wrist-module IMU, so wrist bend shows as the angle between this bone and the hand. The **wrist** and **knuckle** chips at the top give each IMU's live disagreement with the camera in degrees: green below 25°, orange below 45°, red above. ◄**hand** marks the IMU that drives the hand.

- **Occlusion Test — idle** (bottom strip): this is not part of this tab. It is the guided occlusion-capture strip of the **Skeleton** sub-tab of the Gloves tab, left over by the window capture; it appears at the same place in other captures too.

What was done

- Built the project's first end-to-end EMG pipeline. It was working by 25 August 2026 and refined on 26–27 August. The user names gestures and records each one in a 7.5 s take while the camera watches; the first 1.5 s are discarded while the contraction settles, and the next 6 s are sampled. Training needs at least two gestures with at least five samples each. A shrinkage linear discriminant analysis (LDA) classifier, a standard linear classifier for EMG gestures, learns them with shrinkage 0.6 from 40 time-domain (TD) EMG features. At run time the EMG only decides *which* gesture is held, and the tab replays the camera skeleton stored for that gesture, so the EMG never invents finger angles.
- Settled the handoff between camera and EMG on 26 August. While the camera sees the hand, the skeleton *is* the optical hand (verified identical at the time, 0.0000 mm difference). When the hand is occluded, EMG supplies the finger shape and the IMUs rotate the hand from the last camera orientation. The rotation is applied relative to that orientation, so the hand does not jump. Cross-fades between sources adapt their length (about 40–200 ms) and blend bone directions rather than joint positions. A naive blend of joint positions shrank the index finger by 21% at mid-fade; blending bone directions kept bone lengths exact (0%).
- Aligned the IMUs with the camera so the orientation stays right through an occlusion. The IMU's rotation axes were about 28° off the camera's. An axis map fitted online cut the error of 1 s of dead reckoning (turning the hand by the IMU alone) through fast motion from 21° to 13°, and an offline fit cut it to 8°. The knuckle IMU under-reports rotation, measuring 0.64 times the true rotation. Combining both IMUs (knuckle 10°, wrist 11°, combined 10°) did not get below a floor of about 10°, which is attributed to the flexible mounts.
- Found the main lesson on 27 August. The gesture model scored 100% in cross-validation on its own recordings but failed live about 20 minutes later. A code audit found no bug, and no classifier setting fixed it: shrinkage from 0.2 to 0.99, a nearest-centroid classifier, a magnitude threshold and even the armband's own 9-class model all failed on the same live features. The cause was surface-EMG drift: the pattern across the 8 channels changes between recording and use. The immediate remedy was to record and use a model in the same session. The longer-term answer, learning continuously from the camera, became the Pose Hand tab.
- Kept four real-time improvements. Predictions with confidence below 0.3 are rejected. A 3-frame debounce (about 50 ms) removes 1–2-frame flickers. The EMG window was cut from 128 to 64 samples (about 0.13 s at 500 Hz), roughly halving the detection delay. The gesture is also held while the wrist turns faster than about 1.2° per frame (about 72°/s). This was a first guard against the limb-position effect, in which turning the forearm changes the EMG for the same hand shape.
- EMG Hand handles discrete gestures only. For continuous finger tracking it was superseded by the Encoder → Train → Fusion Hand chain, which reuses this tab's handoff and IMU-orientation logic.

→ In detail: Section 5.2 — EMG Hand

2.6 Encoder

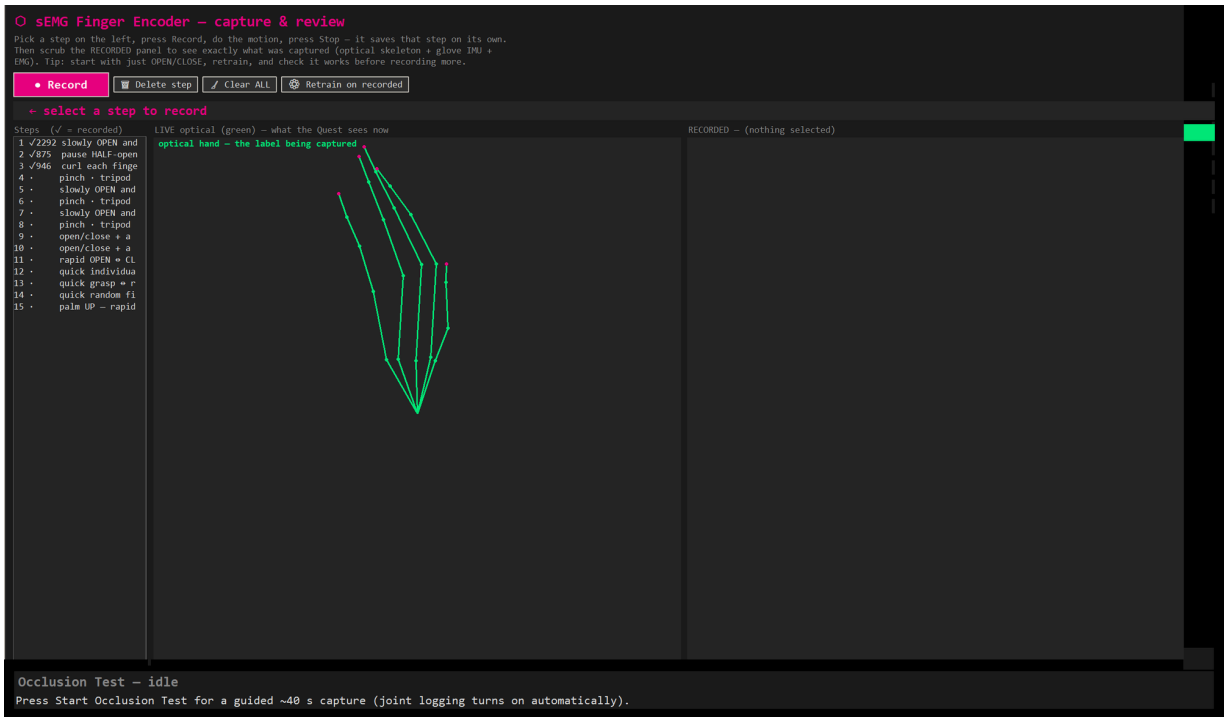


Figure 2.7 *The Encoder tab in the real application, replaying the synchronised session 'low_new3' of 19 September 2026 from t = 60 s through the application's own update loop. The session is 401 s of Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features. The LIVE panel shows the replayed camera hand and steps 1–3 show recordings already saved. No step is selected, so the RECORDED panel is empty. The 'Occlusion Test — idle' strip at the bottom was left over from capturing another view and is not part of the Encoder tab.*

What it is for. The Encoder tab records the training data for the *sEMG finger encoder*. This is a model that estimates continuous finger angles from the forearm's muscle signals (surface electromyography, sEMG, recorded by the MindRove armband), so the hand can still be animated when the Quest 3 camera cannot see the fingers. While the camera sees the hand, its 26-joint hand skeleton is the "answer" (the label), and the EMG and the CyberFinger's IMUs (motion sensors) are the inputs. The tab captures all of them together, frame by frame and one protocol step at a time, and lets the user replay each recorded step before training on it. It was needed because the earlier EMG work could only snap between a few fixed gestures, while continuous finger angles need large, synchronised and well-labelled sets of example pairs (Task 2.1).

On the screen

- **sEMG Finger Encoder — capture & review**: the title and a short instruction. Pick a step on the left, press Record, do the motion, press Stop, then scrub the RECORDED panel to check what was captured (optical skeleton + CyberFinger IMU + EMG). It ends with a tip: start with OPEN/CLOSE only, retrain, and check that it works before recording more.
- ● **Record**: starts capturing the step selected in the list (step 1 if none is selected), and the button turns into ■ **Stop**. While it runs, a frame is kept only when both the camera hand and the EMG are live, and the two IMU orientations are stored with it. On Stop, the step is saved to its own file in the encoder_data folder (a step with no frames is skipped) and loaded straight into the RECORDED panel. A status text to the right of the buttons reports the result, for example how many pairs were saved for that step.

- **Delete step** and **Clear ALL**: remove the selected step's recordings, or every recording, from the training set. The files are moved into a dated archive folder inside encoder_data rather than erased, so a deletion can be undone by moving them back.
- **Retrain on recorded**: a quick in-application check that the recorded data carries signal. It fits a linear (ridge) regression from [40 EMG features + wrist quaternion + knuckle quaternion] to the 26-joint pose, where the quaternions are the orientations of the two CyberFinger IMUs. It trains on a random 80% of the recorded frames and tests on the other 20%. The held-out joint error, for EMG only and for EMG with the IMU inputs, appears in the Train tab. The model is saved and loaded at once, and the Fusion Hand tab falls back to it when its offline-trained finger models are not available. The deeper per-finger model is trained offline.
- ← **select a step to record**: the guide banner. Once a step is chosen, it shows that step's full instruction, e.g. 'Step 1: PALM DOWN · slowly OPEN and CLOSE your whole hand ~10 times'.
- **Steps (✓ = recorded)**: the 15 steps of the guided protocol. To fit, each entry drops the wrist-orientation prefix (PALM DOWN, PALM UP, …) and shows only the first 15 characters, so the full instruction is read in the banner. A recorded step shows a tick and the frame count of its newest saved recording, and an unrecorded step shows '·'. In this screenshot, steps 1–3 already have recordings (2292, 875 and 946 frames).
- **LIVE optical (green) — what the Quest sees now**: the camera's 26-joint hand at this moment, centred on the wrist and drawn in its real orientation, with green bones and pink fingertips. Its caption reads 'optical hand — the label being captured', and it shows 'show your hand to the Quest' when no hand is tracked. Below the hand the code draws five per-finger flexion bars (thumb to pinky, 0 = straight, 1 = fully curled) and a status line 'pairs N EMG ✓ opt ✓ IMU ✓', which starts with '● REC' while recording. The line is green when all three sources are live and orange otherwise. This strip, like the RECORDED panel's slider and EMG trace, is not visible in this screenshot (see the last item).
- **RECORDED — (nothing selected)**: the review panel. When a recorded step is selected, its title names that step's newest file and its frame count. The stored skeleton is redrawn in pink (green fingertips) at the chosen frame, in the stored camera orientation when the file has one. Two small XYZ axis markers, labelled 'wrist IMU' and 'knuckle', show the two CyberFinger orientations at that frame. A readout gives the frame number and 'label conf', and adds '⚠ Quest guessing' below 0.9. A slider scrubs through the clip (starting at its middle), and an 'EMG amplitude over the clip' trace has an orange marker at the current frame. The panel is empty here because no step is selected.
- **Occlusion Test — idle**: the strip along the bottom of this screenshot is not part of the Encoder tab. In the code it belongs to the Occlusion Test panel of the Gloves tab's Skeleton view, and it appears here because it was left over from capturing that view. It sits over the bottom of the Encoder tab.

What was done

- Built a recorder of synchronised training pairs, the first step from snapping between a few gestures to continuous finger angles (Task 2.1). Each frame stores the 40 TD features (standard time-domain measures of the EMG signal's size and shape, computed for each of the 8 channels), the camera's 26-joint pose rotated into the hand's own frame so that only finger shape remains, and the wrist and knuckle IMU quaternions. It also stores the raw EMG window (8 channels × 256 samples, about 0.5 s) and a timestamp. From 29 August, it also stores the camera's hand rotation.

- Each frame also carries a label confidence from the Gate, the application's occlusion detector, which marks whether the camera sees the hand clearly (CLEAR) or the hand is hidden by itself (SELF), by the other hand (OTHER), by an object (OBJECT) or is out of view (OUT_OF_VIEW). The weights are CLEAR 1.0, SELF 0.5, OTHER 0.4, OBJECT 0.3 and OUT_OF_VIEW 0.2, so frames where the Quest is guessing count less in training.
- Designed a 15-step guided protocol covering the wrist orientations the model must handle: four palm-down steps, two palm-up, two sideways, arm raised, arm low and five fast-movement steps. Each step is recorded and saved on its own, and it can be replayed, re-recorded or archived without touching the others.
- Datasets collected: the first capture on 28 August was unguided (12,172 pairs). Two guided sessions followed the same evening: 10 clips with 13,027 pairs, and 15 clips with 16,963 pairs that also stored the raw EMG.
- The 29 August set (3 clips, 4,113 pairs) added a 'curl each finger alone' step. It is the only set that stores the camera rotation, so the later orientation-aligned models were trained on it.

→ In detail: Section 5.3 — Encoder

2.7 Train

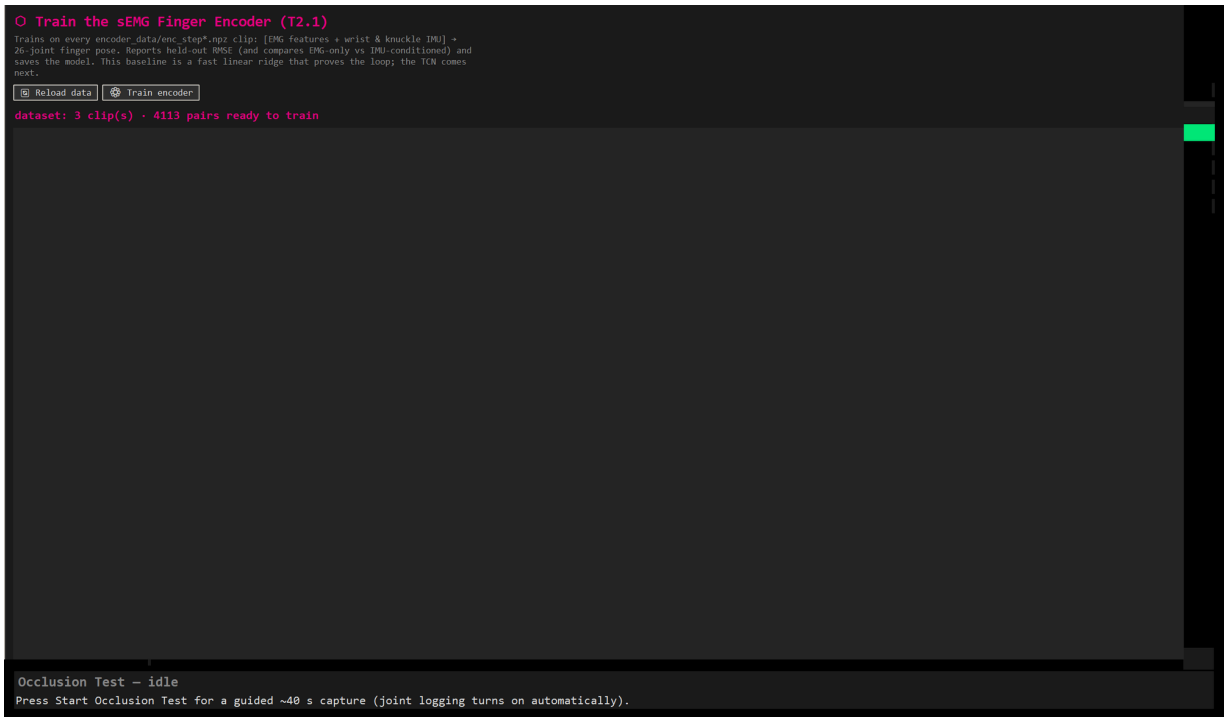


Figure 2.8 *The Train tab in the real GUI, captured while the GUI's own update loop replayed the synchronised session 'low_new3' of 19 September 2026 from t = 60 s. That 401 s session holds Quest 3 hand joints, headset pose, the CyberFinger's wrist-module and knuckle IMUs and MindRove EMG features. The tab does not use the replayed stream. Its summary line counts the clips in the encoder_data folder at capture time (3 clips, 4,113 pairs), not the 13,027-pair session behind the results reported below. The results panel is empty because* ***Train encoder*** *was not pressed for the capture. The "Occlusion Test — idle" strip at the bottom belongs to the Gloves tab's Skeleton page.*

What it is for. The Train tab turns the clips recorded in the **Encoder** tab into a working finger model with one button press, inside the GUI. The model is a *finger encoder*: it reads forearm sEMG (surface electromyography, the muscles' electrical activity picked up through the skin by the MindRove armband) together with the orientation of the CyberFinger's wrist module and knuckle sensor, and outputs the positions of the hand's 26 joints. The tab was needed to close the loop early. It shows that EMG predicts the hand at all, and tests whether also giving the model the hand's orientation helps, before more effort goes into a deeper model. The **Fusion Hand** tab reloads the saved model file as soon as training ends. Today it uses that model only as a fallback, behind the later TCN (temporal convolutional network, a neural network that learns directly from short windows of the raw EMG signal).

On the screen

- **Train the sEMG Finger Encoder (T2.1)**: the tab title. T2.1 is the project-plan task for the EMG finger encoder.
- **Description text**: says that the tab trains on every *encoder_data/enc_step*.npz* clip, mapping [EMG features + wrist & knuckle IMU] to the 26-joint finger pose. It reports the held-out error, compares EMG-only with IMU-conditioned input and saves the model. It also says that this linear baseline "proves the loop; the TCN comes next".
- **Reload data**: scans the *encoder_data* folder for the step clips recorded in the Encoder tab (or for any .npz file there if none are found). It opens each clip to count its training pairs and refreshes the summary line. A

pair is one frame: the EMG features and both IMU quaternions (a quaternion is a four-number description of a 3-D orientation), matched with the 26 joints the Quest camera saw at the same moment.

- **Train encoder**: loads every clip and shuffles the frames with a fixed random seed. It holds out 20% of them for testing, scales each input to zero mean and unit spread, and fits a ridge regression twice on the same split. Ridge regression is a linear least-squares fit with a penalty that keeps the weights small; here the penalty λ is 10. The first fit uses EMG only. The second adds the wrist and knuckle quaternions. The tab saves the second model to *encoder_model.npz* and immediately reloads it for the Fusion Hand tab.
- **Dataset summary line** (pink): filled when the GUI starts and again on each **Reload data**. It reads "dataset: N clip(s) · M pairs ready to train", or "no encoder data yet — record clips in the Encoder tab first" if there are no clips. In the screenshot it reads "dataset: 3 clip(s) · 4113 pairs ready to train".
- **Results panel** (the large dark text box): stays empty until training runs, then shows "training…". Afterwards it shows a report. It starts with "✓ trained on … pairs from … clip(s)". Under the heading "held-out joint RMSE (mean fingertip/joint error)" (RMSE: root-mean-square error) come two lines in centimetres, **EMG only** and **EMG + wrist+knuckle**, and the second is marked "IMU-conditioned (saved)". Below them are the saved file name, the input layout [EMG 40 + wrist quat 4 + knuckle quat 4] → 26-joint pose, and a closing note: the next step is to swap in the TCN. If there are no clips, the panel reads "No data. Record clips in the Encoder tab first." Errors appear as "train failed — …". The same comparison is also written to the GUI log.
- **Occlusion Test — idle** strip at the very bottom: not part of this tab. It is the guided-capture panel of the Gloves tab's Skeleton page, and it shows through at the bottom of this capture.

What was done

- Built a one-click trainer inside the Windows GUI. It is a ridge regression (λ = 10) from 40 EMG features plus the wrist and knuckle quaternions to the 26-joint pose (78 numbers: x, y, z per joint, relative to the wrist with the hand's overall rotation removed). The 40 EMG features are the standard time-domain (TD) set computed on each armband channel: mean absolute value, root mean square, waveform length, zero crossings and slope-sign changes. The trainer needs only NumPy. It saves *encoder_model.npz*, which Fusion Hand reloads straight away. The Encoder tab's **Retrain on recorded** button runs the same training.
- Every run includes a built-in A/B test: EMG only against EMG plus both IMUs, on the same split. On the 13,027 pairs of the first guided session, the held-out mean per-joint error was 1.45 cm with EMG alone and 1.30 cm with EMG plus both IMUs. Conditioning on hand orientation helps.
- These absolute numbers are optimistic. The tab splits frames randomly 80/20, so neighbouring frames of the same clip fall on both sides of the split. Also, the metric the tab labels "RMSE" is actually the mean Euclidean (straight-line) distance between predicted and true joints.
- A fairer offline test used the 15-clip session (16,963 pairs) with a contiguous train/test split and confidence-weighted training. It scored only frames with clean camera labels (confidence ≥ 0.9). Ridge regression reached 1.80 cm and the raw-EMG TCN 1.44 cm. Learning directly from the raw signal wins once fast movements are in the data, because the TD features average away the dynamics.
- The tab's announced next step, the TCN, was built offline. It has four 1-D convolution layers (8 → 32 → 32 → 48 → 48 channels, kernels 7/5/5/3). The first and third layers downsample by 2, and the second and fourth are dilated by 2. The output is mean-pooled, combined with the IMU quaternions and passed through a 128-unit layer. It was trained in PyTorch and exported to a pure-NumPy forward pass, so the Windows GUI needs no PyTorch. The export matches PyTorch to 3×10^{-8} and takes 0.33 ms per frame.

→ In detail: Section 5.4 — Train

2.8 Fusion Hand

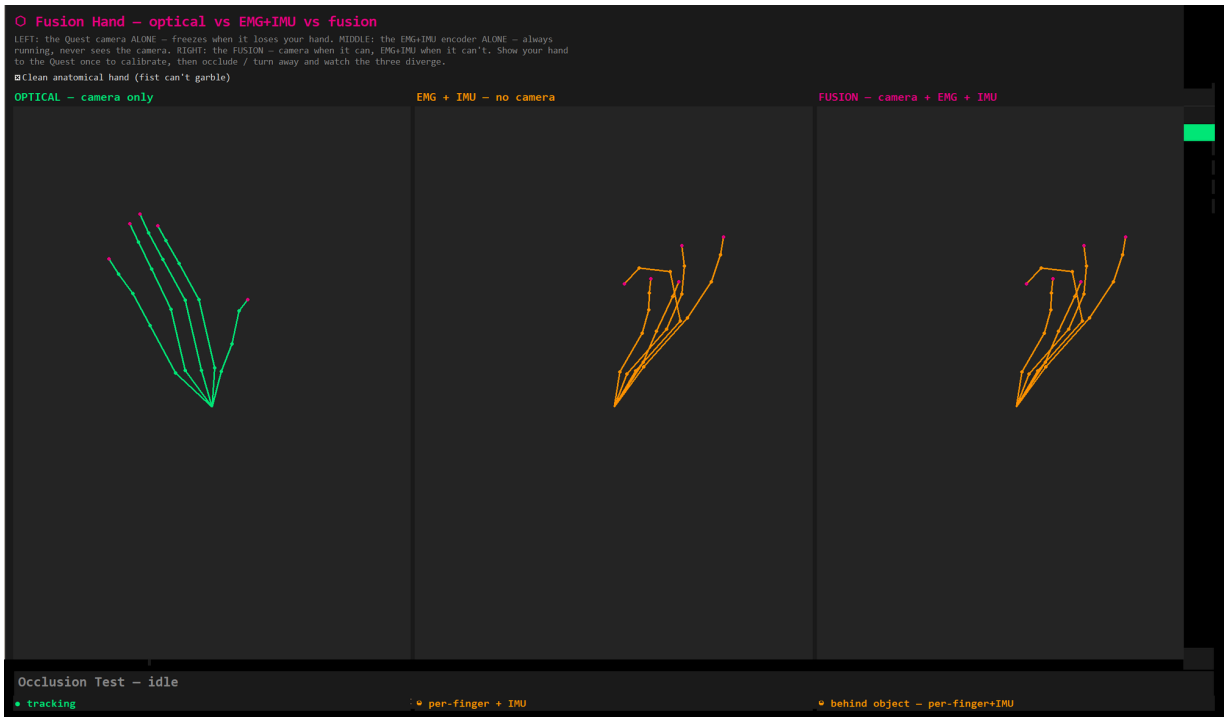


Figure 2.9 *Taken from the real application, replaying the synchronised session 'low_new3' (19 September 2026, 401 s) through its own update loop from t = 60 s. The session holds the Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features. Six seconds into the replay, the camera feed was frozen on purpose at its last frame (a forced camera loss), and the screenshot was taken 2 s later. The application's own gate saw a frozen hand well inside the camera's view and labelled the loss 'behind object'. The fusion panel has therefore handed over to the orange EMG + IMU per-finger hand ('◒ behind object — per-finger+IMU'). The left panel shows the frozen last frame yet reads '● tracking', because it only checks that camera joints arrive and the replay keeps sending that frame. The recording holds EMG features, not the raw EMG stream that the per-finger network reads. Here the network therefore ran on an empty (all-zero) raw-EMG window. The finger shape in the middle and right panels was not decoded from recorded muscle activity and shows only the layout and the handover. The grey 'Occlusion Test — idle' line near the bottom (from the Gloves tab's Skeleton page) and the dark strip with a green block at the right edge are artefacts of the window capture, not part of Fusion Hand.*

What it is for. Fusion Hand is the end-to-end demonstration of Tasks 2.1 (the EMG + IMU hand encoder) and 2.2 (aligning the CyberFinger's IMU orientation with the camera) working together. It puts three hands side by side: the Quest 3 camera's hand on its own, the hand estimated only from the MindRove EMG armband (electromyography, the electrical activity of the forearm muscles) and the CyberFinger's IMUs (inertial measurement units, which sense orientation), and the fused result. The tab was needed because the camera tracks fingers well while it can see them, but it freezes the hand when the hand is hidden or out of view. The fused hand keeps moving and keeps turning with the wrist IMU through those gaps. Building the tab also showed that the camera cannot label a closed fist, which led to the anatomical hand model used here.

On the screen

- ⬡ **Fusion Hand — optical vs EMG+IMU vs fusion** — the tab title.

- **Instruction line** (under the title) — explains the three panels. LEFT is the Quest camera alone, which freezes when it loses the hand. MIDDLE is the EMG + IMU encoder alone, which is always running. RIGHT is the fusion: the camera when it can see the hand, EMG + IMU when it can't. The user shows the hand to the Quest once so the wrist IMU can be calibrated against the camera, then hides the hand or turns away and watches the three panels diverge.
- **Clean anatomical hand (fist can't garble)** — a checkbox, ticked by default, that chooses how the EMG + IMU hand is drawn. When it is ticked, a clean anatomical hand model is bent by five per-finger curls (0 = straight, 1 = fully bent) from the per-finger curl network. The curls are smoothed by a One-Euro filter, an adaptive filter that holds steady when the hand is still and follows quickly when it moves fast. If the per-finger network is missing or cannot run, a single overall closure taken from the raw-EMG network's output bends all the fingers together. When the box is unticked, or the anatomical hand file is missing, the panel draws the raw-EMG network's own 26-joint output instead (or, if that network is absent, the output of the earlier ridge-regression encoder). That output is lightly smoothed and its bones are rebuilt to reference lengths.
- **OPTICAL — camera only** (left, green) — the Quest 3's 26-joint hand skeleton, centred on the wrist and turned by the camera's own hand orientation. The panel does not consult the reliability gate. Its status line reads ● *tracking* whenever camera joints arrive. When the camera stops sending the hand, the last frame stays on screen in grey with ✕ *LOST — camera can't see (frozen)*. That frozen hand is what a camera-only system gives the user.
- **EMG + IMU — no camera** (middle, orange) — the encoder on its own, always running. The finger shape comes from the armband's EMG and is never copied from the camera. The per-finger network does take the wrist orientation as an input: the camera's measurement while it sees the hand, and the IMU's otherwise. The hand is turned by the CyberFinger's wrist-module IMU through the calibrated orientation fuser, which keeps estimating while the camera is blind. Until the fuser has calibrated, the panel keeps the last orientation shown. The status line names the model in use: ◒ *per-finger + IMU*, *anatomical* with a closure percentage, *TCN + IMU* or *ridge + IMU*.
- **FUSION — camera + EMG + IMU** (right, pink title) — the fused hand. The reliability gate is the Gate tab's per-frame verdict on the camera hand. While it rates the hand CLEAR, or names an occlusion reason that has been switched off in the EMG Hand tab, this panel shows the camera's fingers and orientation in green, marked ● *optical*. Otherwise the panel switches to the EMG + IMU fingers turned by the wrist IMU, in orange. Its status line then gives the gate's reason and the model, for example ◒ *behind object — per-finger+IMU*; the other reasons are *self-occluded*, *out of view* and *other-hand*. Each switch between the two sources is a 0.15 s cross-fade of both finger shape and hand orientation, so the hand does not jump when the camera drops out or comes back.
- **show / hide your hand** — the message a panel shows when it has nothing to draw, for example the left panel before the camera has seen the hand. All three panels show it until at least one encoder model or the anatomical hand model is available. Otherwise each panel draws the hand as a skeleton scaled to the panel: each finger is a chain of bones from the wrist, with a dot at each joint and a pink dot at the fingertip.

What was done

- Built the three-panel demonstration of Tasks 2.1 and 2.2 end to end. The fused hand shows the camera's fingers when the hand is visible and the encoder's fingers, turned by the IMU, when it is not. The tab can run either the raw-EMG TCN or a per-finger curl TCN. A TCN (temporal convolutional network) is a neural network that reads a short window of raw EMG. The per-finger network maps raw EMG (8 × 128 samples) plus camera-aligned wrist orientation to five finger curls, which are smoothed with a One-Euro filter (min cutoff 1.5, beta 0.8). Rebuilding the raw-EMG network's bones to reference lengths removed a 0.39 cm bone stretch.
- Found a limit of the sensor itself: the Quest cannot label a closed fist. In the 200 most-curled frames (up to 66° mean bend), the middle, ring and little fingers are drawn extended and crossing through each other, and every fist frame carries Gate confidence 0.5 (self-occlusion). In the later posture data, all 3,497 fist frames are at 0.5 and none reaches 0.9. A model trained on these labels faithfully reproduces the errors: this is a data limitation, not a model bug.
- Replaced the garbled fist with an anatomical hand prior. It takes a clean open skeleton from confident frames plus a flexion axis per finger and curls the fingers procedurally. A 72° thumb-opposition swing wraps the thumb over the fingers. The TCN's overall openness drives the closure, calibrated so that an open hand maps to 0.02 and a fist to 0.95. Raw EMG amplitude could not do this job because flexors and extensors both fire (amplitude 54 for open against 52 for fist). An early sign error made the fingers curl backwards. It was fixed by aligning each finger's axis with the open-to-fist direction found in the data (fingertips move from −1.8 cm to +2.2 cm towards the palm side).
- Showed that the training recipe mattered as much as the architecture. An 80-epoch recipe kept the fingers responsive: its open-versus-fist output gap was about 2.4 cm, against 2.64 cm in the labels. A 200-epoch recipe with a cosine schedule and weight decay froze at 0.19 cm, predicting the average hand. A perturbation test showed that wrist yaw was not what garbled the fist: a 90° yaw change moved the fist error only from 1.76 to 1.70 cm.
- Added per-finger control. In the first protocol steps the fingers moved together (finger-to-finger curl correlation 0.78–0.98), so any model could only open and close the whole hand. A new 'curl each finger alone' step was recorded palm to camera with 100% CLEAR labels and weighted 2.5 times in training. It brought that correlation down to about zero. The per-finger model then reached a mean curl error of 0.099 (previously 0.20), with per-finger correlations of 0.88–0.92. In recordings without isolation clips, the middle, ring and little-finger curls still correlate 0.93–0.99, so these three fingers are not decoded independently in normal use.
- Day-to-day drift appeared again: a model trained the day before read that day's fist as open, while a same-session model reached a curl error of 0.147 (open → 0.14, fist → 0.85). There was also a process lesson. The original tcn_model.npz was overwritten and lost during this work, so model files are now backed up before retraining.

→ In detail: Section 5.5 — Fusion Hand

2.9 Occlusion

Figure 2.10 *The Occlusion tab while the application replays the synchronised session 'low_new3' of 19 September 2026 from t = 60 s through its own update loop. The session holds the Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and the MindRove EMG features; no live hardware was used. The hand is open and in view, so every joint is green, the gate reads CLEAR and all five fingers are reliable (mean confidence 1.00). The right edge of the capture, including the bar values, is cut off, and the strip along the bottom belongs to other tabs.*

What it is for. The Occlusion tab shows, finger by finger, how well the Quest 3 camera can actually see the hand at this moment. Before it existed, camera reliability came as one verdict for the whole hand. That was too blunt: any posture with two or more curled fingers is labelled *self-occluded* (hidden behind the hand's own parts), so a camera "teacher" that is trusted only at high confidence could teach almost nothing but an open hand. Per-finger visibility works as a quality filter: the fingers the camera really sees can keep teaching the EMG and IMU models, and the hidden ones are left out. The tab covers proposal Task 1.2 and brings forward the ray-casting task T5.2 from Internship 2.

On the screen

- ⬡ **Per-finger confidence — ray-cast occlusion (T1.2)** — the tab title and a short explanation. A *ray-cast* draws a straight line from the headset camera to each hand joint and checks whether the hand's own geometry blocks it. Green means the camera's reading of that part can be trusted, red means it cannot. T1.2 is the proposal task the tab serves.
- **gate: CLEAR 5/5 fingers reliable mean confidence 1.00** — the status banner. *gate* repeats the whole-hand reliability gate from the Gate tab (CLEAR, SELF, OBJECT, OTHER or OUT_OF_VIEW). The count is the number of fingers whose confidence is 0.6 or more, the threshold for "reliable", and the last figure is the mean of the five finger confidences. The banner turns green when at least four fingers are reliable and orange otherwise. Without a tracked hand and a valid headset pose, it shows the gate state or the prompt "show your hand to the Quest (needs HMD + hand)" instead.
- **● green = camera sees it ● red = self-occluded** — the colour key in the top-left corner of the hand view.

- Hand view (the large central panel) — the Quest's 26-joint skeleton of the tracked hand, drawn around the wrist in its real-world orientation. This is the right hand, the one that wears the CyberFinger and the armband. Each joint dot is coloured on a green–amber–red scale by its smoothed visibility, from 1 (fully seen) to 0 (hidden). Each bone takes the colour of its less visible end, and fingertips are drawn as larger dots. In the screenshot every joint is green.
- **per-finger confidence** (cut off as "per-finger conf" at the right edge of the screenshot) — five horizontal bars labelled **thumb**, **index**, **middle**, **ring** and **pinky**. Each bar's length and colour give that finger's confidence: a weighted mean of its joints' visibility, in which the fingertip counts twice as much as the base. The numeric value is printed at the right end of each bar, which also falls outside this capture. Here all five bars are full.
- No buttons — the tab is a pure display. It is computed only while the tab is open, from the same headset and hand stream (live or replayed) that the rest of the application uses.
- Bottom strip (**Occlusion Test — idle**, **tracking**, **per-finger + IMU**, **behind object — per-finger+IMU**) — not part of this tab, which has no such labels. These belong to other tabs and show through in this capture: the status line of the guided Occlusion Test on the Gloves tab's Skeleton page, and the panel captions of the Fusion Hand tab.

What was done

- **Why it was needed.** With one confidence number for the whole hand, the share of frames that reached the 0.9 trust level was 100% for open, 1% for point, 0% for ok and 0% for fist. Any posture with two or more curled fingers is labelled SELF-OCCLUDED (label weight 0.5), so it never reaches that level.
- **Per-finger visibility from geometry.** The module fusion/occlusion.py follows Graf & Barthet (2023). It casts a ray from the headset camera to each joint and tests it against the hand's own bones, modelled as capsules: 1.0 cm radius for the fingers and 1.8 cm for the palm bones, plus cross-links along the knuckle ridge. A joint counts as occluded in proportion to how deeply its ray passes through a bone in front of it, and the joint's own bones are never counted. Each finger's confidence is a weighted mean of its joints, with the weight rising from 1 at the base to 2 at the fingertip.
- **Wired into the application.** The GUI multiplies the ray-cast by a per-joint field-of-view test against the camera box calibrated with ⊹ **Calibrate box** on the Gate tab (10° soft edge). It sets everything to 0 when the gate says OUT OF VIEW and scales it by 0.35 for BEHIND OBJECT or OCCLUDED BY OTHER, because the ray-cast cannot see an external blocker. Finally it smooths the result over time (EMA 0.6/0.4). A finger counts as reliable at 0.6 or more.
- **Self-test.** The test uses a synthetic right hand seen from the back-of-hand side. Every finger of the open hand scores 1.00. In a fist, the finger confidences fall to 0.76 (thumb), 0.45, 0.39, 0.45 and 0.50 (pinky), and the fingertips average 0.10 because they sit behind the knuckle ridge.
- **Speed and reuse.** On 1 October the ray-cast was vectorised, cutting it from 4.8 to 0.18 ms per frame (27 times faster) with identical results on 3,200 recorded frames. Together with lighter drawing, this took Fusion Studio from 35 to 62 frames per second. The same ray-cast feeds Live Calib, Pose Hand, Fusion Studio and Fusion Logic. In Pose Hand, the visible index finger of a point was learned even though the whole hand had low confidence: the predicted index curl moved from 0.60 to 0.08, close to the camera's 0.16. In Fusion Studio and Fusion Logic, when camera and EMG disagree, the choice between them flips at a visibility of 0.7; placing the flip at 0.5 instead cost partly hidden fingers one point of posture accuracy (93.2% → 92.1%).
- **Limits found.** On real Quest output the separation is modest, because the Quest's inferred fist is not geometrically closed. In the recorded Live Calib sessions, mean finger confidence was 0.88–0.89 for SELF-OCCLUDED frames against 0.96–1.00 for TRACKED frames, so many self-occluded frames still pass

the 0.6 threshold (these values come from Live Calib's own ray-cast, without the field-of-view taper and smoothing). On the scripted captures there was no useful separation: visible and self-occluded frames both scored 0.96 on 18 August, and on 19 August visible frames scored 0.99 against 1.00 for self-occluded ones. Fingers hidden inside a posture, such as the curled fingers of a point, stay at about 0.6 and cannot be taught by the camera.

→ In detail: Section 4.4 — Occlusion: which fingers the camera really sees

2.10 Live Calib

Figure 2.11 *The Live Calib tab while the real GUI replays, through its own update loop, the synchronised recording 'low_new3' of 19 September 2026 (Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features), captured about 10 s after the replay started at t = 60 s. The tab collects pairs only while it is open, so its 285 buffered pairs were gathered in those seconds. The camera sees the hand (CLEAR), the drift box is unticked, and all five fingers in the FUSION panel come from the camera (green). The replay supplies the recorded EMG features that the buffer lookup uses, but leaves empty the raw EMG window that the base encoder reads, so the* ***ENCODER alone (base)*** *panel and the banner's error figures do not show a real encoder reading. The strip along the bottom edge (****Occlusion Test — idle*** *and three status words) shows labels from other tabs, the Gloves tab's Skeleton page and the Fusion Hand tab, that appear in the window capture; it is not part of Live Calib.*

What it is for. Live Calib corrects the EMG finger encoder (the model that turns the MindRove armband's forearm muscle signals into finger curls, 0 = straight and 1 = fully curled) for the current session, user and electrode placement while the user simply moves. It needs no calibration gestures and no retraining, and it is the proposal's central contribution (Task 3). It was needed because surface EMG drifts between sessions and electrode placements: the literature reports 20–30% accuracy drops between sessions, and a model trained the previous day read the next day's fist as open. The camera acts as the teacher: whenever it sees the fingers clearly, its finger pose is stored with the matching EMG, and those stored pairs correct the encoder, much as GPS corrects inertial dead reckoning.

On the screen

- ⬡ **Online calibration — cross-attention (T3, the novelty)**: the tab title (T3 is the proposal's Task 3). The grey text below it sums up the method: a rolling buffer of (EMG → optical finger pose) pairs that the camera clearly saw, admitted using the per-finger camera confidence from Task 1.2, corrects the encoder live without retraining. It also names the three panels (LEFT, MIDDLE, RIGHT) and tells the user to fill the buffer first and then tick 'Simulate drift'.

- ↺ **Reset calibration**: empties the buffer of stored pairs and resets the drift detector, the coverage smoothing, the finger smoothing filters, the smoothed EMG query and the last remembered camera label, for example after the armband has been re-seated.
- **Simulate electrode drift (biases the encoder)**: a checkbox that adds a fixed bias of ±0.40–0.45 to each finger of the base encoder's output (kept within 0–1). The EMG and the buffer stay clean. It stands in for an electrode shift, so the calibration's recovery can be shown on demand.
- ● **Record session**: starts a per-frame log while the tab is open, and the button changes to ■ **Stop & save**. On stop, the log is written to calib_logs/calib_session_<date>_<time>.npz with the gate state, coverage, EMG and wrist-orientation similarity, base, calibrated and camera curls, wrist tilt and EMG features for every frame, plus a snapshot of the buffer.
- Status banner: **buffer … pairs** is the number of stored pairs (at most 800). **cov** is the coverage, 0–1 and smoothed over time: how closely the current EMG and wrist angle match stored pairs, and therefore how much of the output comes from the buffer lookup rather than the encoder. **base err … → calib …** is a running average of the finger-curl error of the encoder alone and of the calibrated output against the camera, updated only on frames where the camera reports the hand; the signed percentage after it is the reduction. The bracket repeats the occlusion state (for example CLEAR) and the coverage, and says where the FUSION hand comes from: *optical*, *calibrated*, *encoder(OOD)* (out of distribution: hand not tracked and coverage 0.4 or below), or *per-finger n/5 optical* (how many fingers the camera drives). The banner turns green when calibration cuts the error by more than 5%, and turns orange and adds ⚠ **drift → rebuilding buffer** when an electrode shift is detected.
- **ENCODER alone (base)**, left panel, orange: a clean anatomical hand curled by the base encoder's five finger curls (including the simulated bias when that box is ticked), smoothed to remove jitter. Fingertips are marked in pink. When there is no data the panel shows *show your hand + armband on*.
- **+ ONLINE CALIBRATION**, middle panel, green: the same hand curled by the calibrated curls. These blend the encoder output with the buffer lookup, weighted by coverage, and are then steadied by an adaptive (One-Euro) low-pass filter. In the left and middle panels, the hand is oriented by the camera while it tracks cleanly. Otherwise it is oriented by the CyberFinger's wrist-module IMU, so it keeps turning through occlusion instead of freezing.
- **FUSION — camera + EMG + IMU**, right panel: the full result, blended finger by finger. Each finger's camera weight ramps up from finger confidence 0.30 to 0.70 and is smoothed over time. Each calibrated finger is shifted so that its base joint in the palm (the metacarpal joint) sits on the camera hand's, so it cannot detach from the hand. A finger is drawn green when the camera drives it and orange when the calibrated EMG drives it. When the camera reports no hand at all, the whole calibrated hand is drawn in orange and turned by the IMU.

What was done

- Built online calibration with no retraining and no runtime gradients. Frames are admitted when the mean per-finger camera confidence is 0.6 or more, and the buffer holds up to 800 pairs: 40 time-domain EMG features plus the wrist's gravity tilt as the key, and the camera's five finger curls as the value. The current EMG, smoothed over about 0.7 s, queries the buffer by cross-attention, a similarity-weighted lookup (EMG temperature 0.12, orientation temperature 0.08, the 96 best matches averaged by similarity). The result is blended with the encoder according to a smoothed coverage score.
- Added safeguards. Each frame is predicted from past pairs before it is stored. When the buffer is full, density-based eviction removes a pair from the largest cluster, so every posture stays represented. Electrode-shift detection clears and rebuilds the buffer when the smoothed residual exceeds 0.22 with more than 40 pairs stored. Gated camera-teacher labels carry the last camera pose onto the EMG through

short occlusions, at most once every 0.15 s and only if that label is at most 1 s old, the wrist tilt cosine is above 0.9 and the live estimate agrees to within 0.30.

- Fixed four problems found by testing on recorded sessions on 30–31 August. Standardising (z-scoring) the EMG features replaced a cosine similarity of about 0.8 for every input with 0.98 for familiar patterns against 0.33 for unfamiliar ones. Density-based eviction kept fist, point and open balanced (about 270 of 800 pairs each; 5 of 5 postures retained), top-k attention (attending only to the best matches) let match quality decide, and after a real remapping the index finger re-learned from 0 to 0.76 over 600 frames. Lowering the start of the coverage ramp from similarity 0.70 to 0.58 raised coverage while random unfamiliar inputs still get 0, and smoothing the coverage cut engage/disengage flicker by 78%. Adaptive normalisation, AdaBN and CORAL, the fixes ranked best in the literature, all made out-of-view posture identification worse; two simple changes did help and were built in: query smoothing and gated camera-teacher labels.
- While the camera saw the hand, the finger-curl error against the camera fell from 0.455–0.547 with the encoder trained the previous day to 0.056–0.124 with calibration, in three recorded sessions on 30 August. This shows correction within a session, not generalisation to new sessions. In the module's synthetic self-test, drift recovery cut the curl error from 0.203 to 0.063 (69% lower), and the unsmoothed coverage was 0.55 for a wrist angle between angles already seen against 0.00 for a never-seen angle, where the output defers fully to the encoder.
- Out-of-view posture recognition from EMG on a real occlusion recording (2,554 frames) rose from 72% to 92%, and for the fist from 47% to 83%, with the two changes built into Live Calib. These figures are recorded in the project notes and a code comment; the analysis scripts were not found in the repository.
- Limitations: the limb-position effect (the same gesture produces a different EMG pattern when the arm is held differently) caps engagement out of view. EMG similarity fell to 0.65–0.77, so the output was roughly 35–70% calibrated, depending on the session. Live Calib can correct only postures and wrist angles the camera has labelled. The occluded fist's index finger stays soft (about 0.69 against 0.95 for a full fist). Correction holds only within a session. The design interpolates camera poses instead of applying the proposal's affine correction of EMG latent vectors, and the formal Task 3.3 benchmark remains future work.

→ In detail: Section 4.5 — Live Calib: online calibration with the camera as teacher

2.11 Tracking Hand

Figure 2.12 *The Tracking Hand tab with the hand in view. The camera is the anchor (status anchored (camera)), so the right panel shows the green camera hand and the middle panel shows the camera's finger curls on the model hand. The screenshot was taken by replaying the synchronised session 'low_new3' of 19 September 2026 (Quest 3 hand joints, headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features) through the application's own update loop from t = 60 s, with no camera loss forced. The replay carries EMG features, not the raw EMG window this tab reads (that buffer stays at zero), so **openness** shows '—'. The strips along the bottom and right edges are left over from other tabs in the window capture and are not part of this tab.*

What it is for. Tracking Hand tests one idea from the research proposal. The idea is that the forearm muscle signal (sEMG, surface electromyography, read by the MindRove armband) is best used to measure how the fingers are *changing*, not where they are. The camera would then set the hand pose and the EMG would carry it forward while the camera cannot see the hand, much like GPS-corrected dead reckoning. The tab was built because the earlier decoders, which estimate the pose directly, let a held fist blur into an ambiguous pose out of view. The change-based 'Tracking' approach of Meta's emg2pose (an open benchmark and model family for decoding hand pose from sEMG) promised to keep it sharp. The tab shows the camera hand, the change-tracked hand and the fused result side by side, and it records sessions so the method can be checked against real out-of-view movement.

On the screen

- **Velocity tracking — hold & integrate (vemg2pose Tracking, #6)**: the tab title. It names the method under test, the 'Tracking' formulation of emg2pose, whose vemg2pose model predicts the change in pose rather than the pose itself. The '#6' is this idea's number in a ranked list of fixes drawn up from the literature on 30 August, not a task number from the Mitacs proposal. The grey text below states the original premise. The hand is anchored to the camera whenever the camera sees it, and the EMG-predicted change is added up through occlusion so that a held fist stays sharp out of view. The panels show, from left to right, camera, velocity-tracked and fusion. The final out-of-view behaviour differs from this premise, as the results explain.
- **Re-anchor**: discards the held hand pose, the last finger shape the camera saw and the open/fist calibration of the EMG amplitude, and resets the display smoothing. The held hand restarts from the camera's current finger curls (or from the network's output when the camera has none). It locks to the camera again on the next frame in which the camera is a trustworthy anchor. The user has to show an open hand and a fist again before the amplitude openness returns.
- **Record session**: starts a timestamped per-frame log. Each frame holds the raw 128-sample, 8-channel EMG window and its mean amplitude, the network's position and velocity outputs, and the tab's held finger curls (before display smoothing). It also holds the camera's finger curls, the gate verdict and whether the camera was anchoring, the wrist rotation rate and the wrist IMU's tilt vector. While recording, the button reads **Stop & save**, and pressing it writes calib_logs/track_session_<date>_<time>.npz. The 32-second out-of-view test recording described below was made with this button.
- Status line (here *CLEAR openness — rot 0.0 → anchored (camera)*): the first field is the occlusion gate's verdict for the hand. It reads CLEAR when the camera sees the hand unobstructed, otherwise SELF (fingers hidden behind the hand itself), OUT_OF_VIEW (outside the camera's view), OTHER (blocked by the other hand) or OBJECT (blocked by an object). **openness** is the EMG amplitude estimate, from 0 (open) to 1 (fist), and shows '—' until it is calibrated. **rot** is how fast the wrist is turning, computed from the frame-to-frame change in orientation of the IMU (inertial measurement unit: accelerometer and gyroscope) in the CyberFinger's wrist module. After the arrow comes the state: *anchored (camera)*, *calibrating — show OPEN & FIST to camera*, *wrist rotating*, *OPEN*, *FIST*, *mid* or *MODEL IDLE — connect EMG + IMU*.
- **CAMERA (optical)** (green, left): the 26-joint hand reported by the Quest 3 cameras, redrawn around the wrist with its fingertips marked in pink. It is turned to the hand's current orientation, which is the camera's own while the camera anchors and the IMU's otherwise. The camera counts as a trustworthy anchor only when the gate reads CLEAR and the mean per-finger confidence is at least 0.6. That confidence is computed from how visible each joint is from the headset's position. With no hand, the panel reads 'show your hand + armband on'.
- **VELOCITY-TRACKED (EMG hold + integrate)** (orange, middle): the tab's own estimate, which is five finger-curl values (0 = straight, 1 = fully curled) applied to a clean anatomical model hand and smoothed. While the camera anchors, the curls are copied from the camera hand, so the panel follows the left one, drawn with the model hand's fixed proportions. Whenever the camera is not a trustworthy anchor (out of view, or in view but hidden or low-confidence), the curls follow the EMG amplitude openness and keep the last finger shape the camera saw. This needs an open hand and a fist to have been seen first. Before that calibration, the curls come from adding up the network's predicted curl velocities, with a slow pull towards its absolute curl output.
- **FUSION — camera + velocity EMG/IMU** (pink title, right): the combined output. It shows the green camera hand while the camera anchors, as in this screenshot, and otherwise the orange velocity-tracked hand. In that case the hand's orientation comes from the CyberFinger's wrist-module IMU, which the shared IMU fuser aligns to the camera's frame and carries through occlusion. Before the fuser has been aligned, the IMU's raw orientation is used.

What was done

- Built on 30–31 August to test the proposal's premise (Section 2.5) that sEMG is inherently differential, following emg2pose's 'Tracking' formulation, which predicts the change in pose rather than the pose itself. The hand holds the last camera pose, adds up the EMG-predicted changes while hidden and snaps back to the camera once the camera's confidence reaches 0.6.
- A two-head TCN (temporal convolutional network) reads a 128-sample raw EMG window plus seven conditioning numbers (the camera-aligned wrist quaternion and the wrist tilt), and outputs five curls and five curl velocities. It was trained for 120 epochs (Adam, learning rate 2×10^{-3}, loss = position + 0.5 × velocity) on the 4,113 samples of the 29 August set, 3,195 of which have velocity labels. It runs in NumPy inside the application, matching the trained network to about 10^{-7}. Its position error of 0.105 is a training-set figure, because the training script has no held-out split.
- Adding up the predicted changes alone drifted: curl error 0.34 and openness correlation 0.45. A complementary filter adds the integrated velocity (gain 1.2), pulls slowly (0.06 per step) towards the network's absolute output and switches the pull off while the wrist rotates. It reached 0.09 and 0.86. Both pairs of numbers come from in-view recordings the model was trained on, and they did not capture the out-of-view behaviour. A self-tuning version of the filter was worse both in and out of view and was reverted.
- A real out-of-view recording (1,315 frames, 32 s) exposed the failure. On its 645 out-of-view frames, the absolute output sat near 'fist' (mean 0.85), and the velocity output read all muscle activity as closing (+1.31 per second).
- The fix was simpler. The 8-channel pattern of log-RMS amplitude (the logarithm of each channel's root-mean-square level) is projected onto a fist-minus-open direction learned live from the camera and re-centred adaptively (blend 0.18, suppressed while the wrist rotates). This separated open from fist in 77% of in-view frames, and the share of out-of-view frames drawn as open rose from 0% to 38%. In the final code, once both an open hand and a fist have been seen, this amplitude openness alone drives the out-of-view hand, and velocity integration runs only before that calibration.
- Outcome: with this hardware, a clear negative result for the differential premise. Adding up predicted changes drifted, and out of view a simple amplitude feature was more robust than the orientation-conditioned network. Two limitations remain. Opening out of view is perceptibly late (the user's verdict: 'works but with delay'), and the user must show both an open hand and a fist to the camera before the feature is calibrated.

→ In detail: Section 5.7 — Tracking Hand

2.12 emg2pose (ref)

Figure 2.13 *The emg2pose (ref) tab at frame 95 of 607. Motion-capture truth is on the left (green), the emg2pose prediction from sEMG in the middle (orange) and the two overlaid on the right. The banner shows 8.6° error for this frame and 11.0° for the whole window. The image was captured from the GUI's own window while the GUI replayed the recorded session 'low_new3' (19 September 2026) through its update loop. This tab ignores that stream and plays Meta's stored offline result. The strip along the bottom edge ('Occlusion Test — idle' and the three status words under it) is left over from other tabs (the Gloves tab's Skeleton page and the Fusion Hand tab) and is not part of this tab.*

What it is for. This tab sets the published state of the art in decoding hand pose from muscle signals beside our own work. It replays Meta's pretrained emg2pose model (NeurIPS 2024) as it reconstructs finger joint angles from surface EMG (sEMG, the muscles' electrical activity recorded through the skin). The prediction is shown next to motion-capture ground truth from Meta's open dataset. We needed it as a yardstick, because it shows the accuracy a large model trained on a large dataset reaches. The decoders we build on our own hardware (for example in the **Tracking Hand** tab) can be judged against it. It is only a reference: the model was built for a 16-channel wrist band and cannot run on our 8-channel MindRove forearm armband.

On the screen

- ⬡ **emg2pose — Meta SOTA reference (NeurIPS 2024, 16-ch wrist sEMG)**: the tab title. SOTA means state of the art. The title names the source model and the kind of signal it was built for.
- **Description line** ("Meta's pretrained vemg2pose 'Tracking' model decoding hand pose from REAL sEMG…"): says the model uses a 16-channel, 2 kHz wrist band, so it cannot drive our hardware and serves only as a benchmark against the **Tracking Hand** tab. It also explains the three panels below. The on-screen wording calls our 8-channel forearm signal "the CyberFinger's". That signal actually comes from the MindRove armband, which is worn together with the CyberFinger.
- **Status banner** (pink): shows the dataset session name (cut to 32 characters), the current **frame** out of 607 and this frame's **joint-angle error** in degrees. The joint-angle error is the mean absolute difference between the 20 predicted and true joint angles. The banner also shows the **overall MAE** (mean absolute

error) for the whole 20-s window. If the stored result file is missing, the banner shows how to generate it instead.

- **GROUND TRUTH (mocap)**: the left panel. It shows a green hand skeleton. Meta's standard hand model converts the motion-capture joint angles into 21 three-dimensional points. These are drawn as five chains, each running from the wrist through one finger's joints to its fingertip. The drawing is a flat projection of the 3-D hand, centred and scaled to fit the panel.
- **emg2pose PREDICTION (sEMG)**: the middle panel. It shows an orange skeleton of the same hand, posed from the joint angles the model decoded from the EMG alone. Like the left panel, it is scaled on its own.
- **OVERLAY — truth vs prediction**: the right panel. It draws the green truth and the orange prediction at one shared scale and centre, so you can see where, and by how much, the prediction departs from the truth at that instant.
- **Playback** (no on-screen control): the tab has no buttons. It plays the stored result in a loop at 30 frames per second and starts again from the first frame each time the tab is opened. It runs only while the tab is visible and changes nothing in any other tab.

What was done

- Built on 31 August as a reference benchmark. I ran Meta's pretrained vemg2pose *Tracking* checkpoint offline on the emg2pose mini dataset and stored the result for the tab to replay. Over a 20-s window of one session, it reached 11.0° mean joint-angle error. The tab replays the window's 607 frames (30 fps) as ground truth, prediction and overlay, with the error for each frame.
- Why it stays a reference: emg2pose uses 16 channels at 2 kHz on the wrist, while our band has 8 channels at 500 Hz on the forearm, so the model cannot run on our hardware. The dataset's licence (CC-BY-NC-SA 4.0) allows research use only, so the model is not a default for SciCortex's product.
- A cross-montage probe tested whether Meta's learned features still help on our hardware. I replaced emg2pose's input layer with an 8-channel one and trained the network on our data with a clip-wise 75/25 split (whole clips go to one side or the other), giving 3,084 training and 1,029 validation frames. With the rest of the network frozen, pretrained features beat random ones by 18.4% in a first run and by 13.9% in a second, so the features do transfer to the forearm.
- In the second run, the validation finger-curl error (on a 0–1 scale) was 0.1134 for frozen random features and 0.0976 for frozen pretrained features. A fine-tune started from the pretrained weights reached 0.0926, against 0.0915 for a network trained from scratch, so it only tied. I kept the from-scratch decoder, because at this data volume more recordings on our own hardware are the bigger lever.
- Scope: this probe is comparable to, but not the same as, the proposal's plan to pretrain on the NinaPro or EMaGer datasets. No NinaPro or EMaGer test was run.

→ In detail: Section 5.8 — emg2pose (ref)

2.13 Posture Library

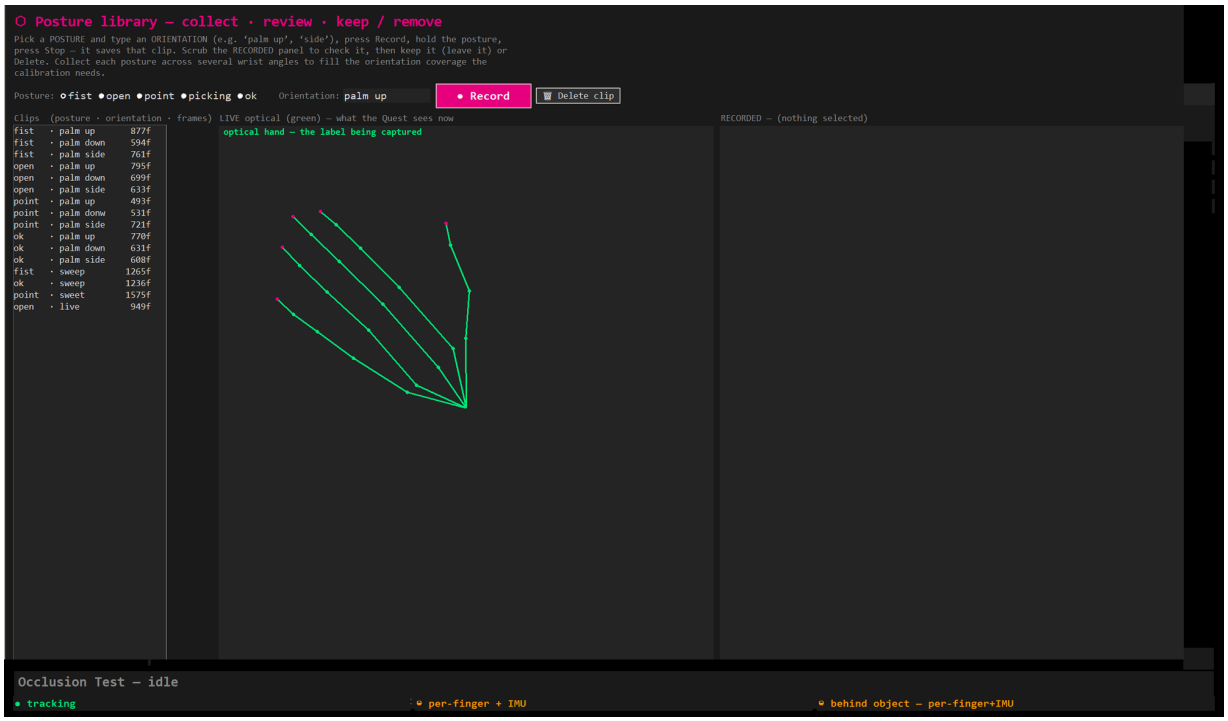


Figure 2.14 *The Posture Library tab in the real application. It is replaying the synchronised session 'low_new3' of 19 September 2026 (401 s: Quest 3 hand joints, headset pose, the CyberFinger's wrist-module and knuckle IMUs, MindRove EMG features) from t = 60 s through its own update loop. Image size 2400×1398 px. The green hand is a replayed camera frame, the list holds the 16 clips saved on 31 August, and the RECORDED panel is empty because no clip is selected. The band along the bottom ('Occlusion Test — idle' and the captions under it) is left over from other tabs in the capture. It hides this tab's flexion bars, scrub slider and EMG strip.*

What it is for. Posture Library is the tab for collecting posture data. The user picks a hand posture (fist, open, point, picking or ok), types the wrist orientation it is held in, and records a short clip. Each clip can then be reviewed frame by frame and either kept or removed. The tab was needed because from 31 August the work moved away from predicting every finger freely and towards a small set of postures, whose labels are more reliable. Every clip is labelled with its posture and wrist orientation because the armband's signals for one posture can change as the wrist turns (the *limb-position effect*), so each posture has to be collected on purpose at several angles. The clips use the same file format as the Encoder tab. They are the data that the next tabs analyse (Posture Clusters) and train on (Posture Hand, Pose Hand).

On the screen

- ⬡ **Posture library — collect · review · keep / remove**: the title. The instruction line below it gives the workflow: pick a POSTURE, type an ORIENTATION, press Record, hold the posture, press Stop, scrub the RECORDED panel to check the clip, then keep it or delete it, and collect each posture at several wrist angles.
- **Posture:** and **Orientation:**: radio buttons choose the label (**fist**, **open**, **point**, **picking**, **ok**). A free-text box names the wrist angle and starts at 'palm up'. Whatever is typed there becomes the clip's orientation label and, with spaces and symbols replaced, part of its file name. That is why two entries in the list show typing slips ('palm donw', 'sweet').

- ● **Record**: starts a clip, and the button changes to ■ **Stop**. A frame is stored only when the Quest sees the hand and the armband is streaming with a full raw-EMG window. Each frame holds the EMG features, the raw EMG window, the 26 camera joints, the camera's hand orientation, the CyberFinger's wrist-module and knuckle IMU orientations, a label confidence and a timestamp. On **Stop** the clip is saved to its own file, the list is refreshed, and the newest clip is selected and loaded for review. A status note next to the buttons reports 'saved … N frames → file', or that the clip was skipped because no frames qualified ('0 pairs (skipped …)').
- 🗑 **Delete clip**: moves the selected clip into an archive folder inside the data folder (posture_data/_archived_…), so the file is not erased and a removal can be undone. The status note confirms the move. With no clip selected it asks for one.
- **Clips (posture · orientation · frames)**: every saved clip, oldest first, showing posture, orientation and frame count. Clicking a row loads that clip into the RECORDED panel. Clips recorded with the record button in the Posture Hand tab also appear here, with the orientation 'live'. The screenshot shows the 31 August collection: fist, open, point and ok at palm up, palm down and palm side, then sweeps for fist, ok and point ('sweet' is a typing slip for sweep), and one open clip labelled 'live'.
- **LIVE optical (green) — what the Quest sees now**: the camera's current hand, drawn as a 26-joint skeleton in green with pink fingertips and turned to the hand's measured orientation. It is captioned *optical hand — the label being captured*, and shows 'show your hand to the Quest' when no hand is seen. In the screenshot it is a replayed camera frame. Below it (hidden in this capture by the band at the bottom) are flexion bars for thumb, index, middle, ring and pinky (0 = straight, 1 = fully curled) and a line reading 'pairs N EMG ✓/✗ opt ✓/✗ IMU ✓/✗'. That line is prefixed '● REC' while recording, and is green when all three sources are live and orange when any is missing.
- **RECORDED — (nothing selected)**: the review panel. For a selected clip it draws the recorded skeleton in pink, with green fingertips, at the chosen frame. The skeleton is re-turned to the orientation the camera measured, so it matches the live view. Two small red/green/blue axis markers show the *wrist IMU* and *knuckle* orientations at that frame. A line reads 'frame k/n label conf x.xx' and is flagged '⚠ Quest guessing' when the confidence is below 0.9. It is empty in the screenshot because no clip is selected.
- Scrub slider and *EMG amplitude over the clip* strip (under the RECORDED panel, hidden in this capture by the band at the bottom): the slider starts at the middle of the clip and moves through its frames. The strip plots the average EMG amplitude of every frame across the whole clip, with an orange marker at the frame being viewed.
- Band along the bottom (**Occlusion Test — idle**, and under it '● tracking', 'per-finger + IMU' and 'behind object — per-finger+IMU'): not part of this tab. It is left over from other tabs in the screen capture. The first line is the status strip of the occlusion-test panel in the Gloves tab's Skeleton page. The second holds the panel captions of the Fusion Hand tab.

What was done

- Built a capture tab whose clips are labelled by posture and wrist orientation. Its recording buffers are separate from the Encoder tab's, and each clip is saved as its own file in the Encoder's format plus the posture and orientation labels. Every frame holds EMG features, raw EMG, the 26 camera joints, the camera's hand orientation and both CyberFinger IMU orientations (wrist module and knuckle), so the later posture tabs can train directly from these files.
- Clips are reviewed before they are kept. After **Stop** the new clip loads at once, and scrubbing shows its skeleton, both IMU orientations, the label confidence and the EMG amplitude. 🗑 **Delete clip** moves a clip to an archive folder instead of erasing it, so a deletion can be undone.

- Each frame records how far its camera label can be trusted, taken from the verdict of the reliability gate (the Gate tab) at that moment. The confidence is highest when the hand is clearly in view, 0.5 when the hand hides its own fingers, and lower for other kinds of occlusion. Doubtful frames are kept but marked: the review panel flags them, and later training can leave them out.
- On 31 August the tab collected 16 clips with 13,138 frames: four clips each of fist (3,497 frames), ok (3,245), open (3,076) and point (3,320). Each posture was held palm up, palm down and sideways. The fourth clip was a sweep between orientations for fist, ok and point, and a 'live' clip for open, recorded from the Posture Hand tab. This is the list shown in the screenshot.
- In this data, 'ok' is a thumbs-up shape (thumb out, fingers curled). Its canonical curls are 0.21, 1.00, 0.97, 1.00 and 0.96 from thumb to little finger (0 = straight, 1 = fully curled). Only the open clips and 32 point frames carry confident camera labels, and every fist and ok frame has confidence 0.5. So even the canonical thumbs-up shape is based on frames the camera was guessing.

→ In detail: Section 5.9 — Posture Library

2.14 Posture Clusters

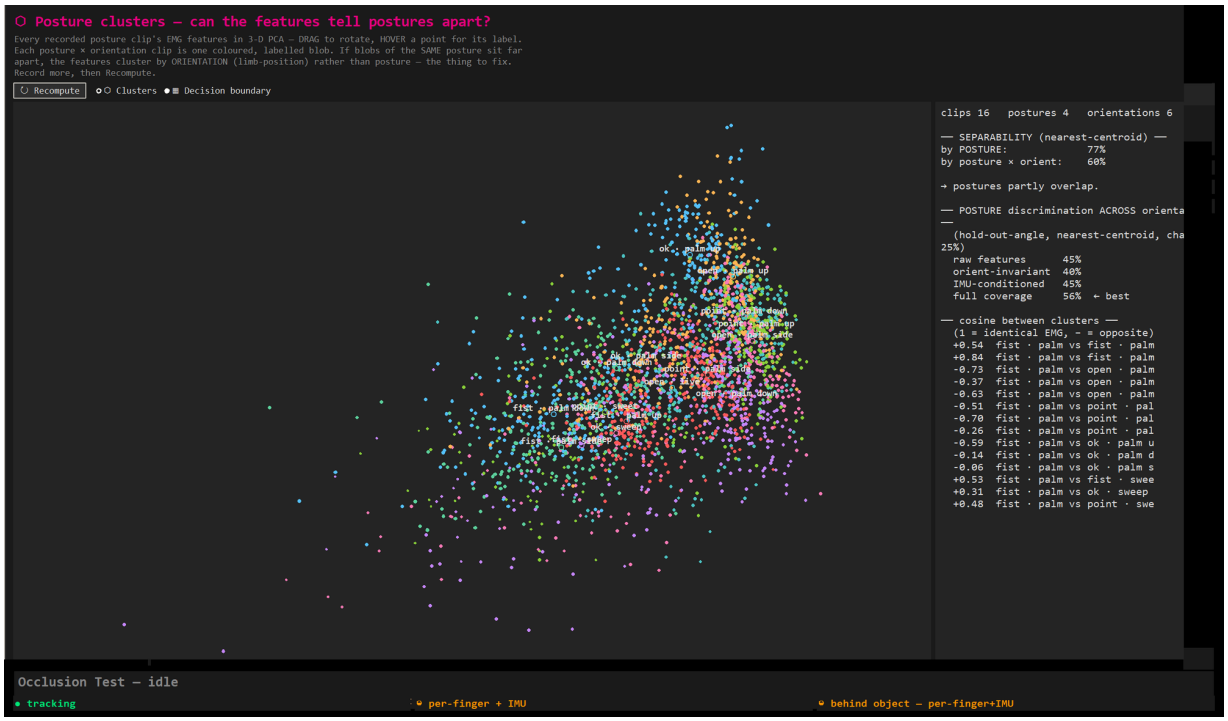


Figure 2.15 *The Posture Clusters tab in* ***Clusters*** *mode. The EMG features of every stored Posture Library clip (here 16 clips: 4 postures at 6 orientations) form a 3-D PCA cloud with a labelled centre for each clip. The labels overlap in the dense middle. The separability, held-out-orientation and cluster-cosine readouts are on the right, cut off at the window's right edge. Captured from the real GUI while it replayed the synchronised session 'low_new3' of 19 September 2026 from t = 60 s through its own update loop. The tab analyses previously recorded clips, so the replay does not change what it shows. The strip along the bottom edge (Occlusion Test — idle) belongs to a panel of another tab and is not part of Posture Clusters.*

What it is for. Posture Clusters asks a direct question: can the forearm EMG features (the 40 numbers computed from the MindRove armband's 8 channels, five standard time-domain measures per channel) tell hand postures apart, and how much does the wrist's orientation get in the way? This matters because the same posture held at a different wrist angle produces different EMG. This is the *limb-position effect*, and it threatens any EMG posture recogniser that has to work wherever the arm is pointing. The tab works offline on the clips recorded in the Posture Library tab, each labelled with a posture and a wrist orientation. It shows how those clips group, measures how separable they are, and compares ways of coping with orientation.

On the screen

- **Title and instructions**: the heading **Posture clusters — can the features tell postures apart?** and a short guide. Every recorded clip's EMG features are shown in 3-D: drag to rotate, hover over a point for its label. If clips of the same posture sit far apart, the features are grouping by orientation (limb position) rather than by posture.
- **Recompute**: re-reads every clip saved by the Posture Library tab and redoes the whole analysis. The analysis also runs on its own each time the tab is opened, so newly recorded clips are always included.
- **Clusters** / **Decision boundary**: two radio buttons that switch the large 3-D view between the cluster cloud (shown in the screenshot) and the classifier's decision-boundary view.
- **3-D view, Clusters mode**: the features are first put on a common scale, then projected onto their first three principal components by principal-component analysis (PCA). These are the three directions in which

the features vary most. Each dot is one recorded frame. Long clips are thinned to every n-th frame, so each clip shows at most about 300 dots. Dots are coloured by clip (eight colours, reused when there are more clips), and nearer dots are drawn larger to give depth. Each clip's centre has a ring and a label such as *fist · palm up*. Dragging rotates the cloud, and hovering shows a dot's clip name. A footer line, not visible in the screenshot, gives the share of variance carried by each of the three axes.

- **3-D view, Decision boundary mode** (not shown in the screenshot): the frames are projected instead onto linear-discriminant-analysis (LDA) axes, the directions that best separate the postures (up to three, one fewer than the number of postures). A small linear classifier of the same type as the posture encoder behind the Posture Hand tab is fitted in that space, with each posture given equal total weight however many frames it has. Its boundaries between postures appear as pale flat sheets, and the data appear as white-ringed dots coloured by posture. A colour legend sits at the top left, and the footer gives the number of discriminant axes and the classifier's accuracy in this space.
- **clips** / **postures** / **orientations** (top line of the right panel): how many clips, distinct postures and distinct orientations are in the analysis.
- **SEPARABILITY (nearest-centroid)**: each frame is assigned to the clip whose average feature vector (its centroid) is closest. The centroids are built from the same frames that are scored. **by POSTURE** is the share of frames that land on a clip of the right posture. **by posture × orient** is the share that land on exactly the right clip. A short verdict follows. It reads *postures separate cleanly* when more than 80% of frames land on the right posture, and *postures partly overlap* otherwise (as in the screenshot). With only one posture recorded, it instead explains that the separate blobs are the limb-position effect.
- **POSTURE discrimination ACROSS orientation**: a quick head-to-head of four approaches. Each orientation is held out in turn, the model is built on the others, a nearest-centroid test is scored on the held-out frames, and the scores are averaged. The line under the heading gives chance (one divided by the number of postures). **raw features** uses the EMG features as recorded. **orient-invariant** first removes the directions along which the orientation averages differ. **IMU-conditioned** adds three numbers describing the wrist's orientation, read from the CyberFinger's wrist module. **full coverage** instead trains and tests on a random 70/30 split of all frames, so every orientation is seen in training. The highest score is marked ← **best**. These scores cover all recorded postures at once, so they cannot be compared directly with the report's pairwise logistic-regression results.
- **cosine between clusters**: the cosine between the average feature vectors of two clips. A value of +1 means the same EMG pattern, and negative values mean opposite patterns, so the table shows how differently the same posture can look at different wrist angles. Only the first 14 pairs in list order are printed, which in practice compares the oldest clip with the next 14. Clip names are cut to 11 characters, so the orientation is often truncated: every row in the screenshot starts with *fist · palm*.

What was done

- Built an offline analysis tab over every Posture Library clip. It has a rotatable 3-D PCA view, separability scores by posture and by posture × orientation, a cosine table between clusters, and a head-to-head of approaches with one wrist orientation held out.
- Made the limb-position effect visible. A fist held palm up, palm down and sideways (877, 594 and 761 frames) formed three separate clusters, and the fist's orientation could be identified from the EMG 76% of the time. The cluster centres pointed in nearly opposite directions (cosine −0.83 for up against down, −0.72 for up against side), and all 40 features shifted. Orientation dominates the EMG variance.

- Showed that postures stay separable across orientations. In the full pairwise analysis (logistic regression trained on two angles and tested on the third, chance 50%), fist vs open reached 98% on raw features. The 'orientation-invariant' transforms backfired: projecting onto an invariant subspace gave 92%, and regressing out the wrist tilt only 24%.
- On the subtle fist vs point pair, which differ only in the index finger, adding the wrist IMU helped (92% → 95%). Recording all orientations gave 99–100% on every pair (fist / open, open / point and fist / point).
- Added a decision-boundary view that projects the features onto three LDA dimensions, keeping 86% of the separation, and draws the boundaries of a linear classifier of the deployed type as planes. The largest shared boundary was between fist and ok, the pair later found hardest to separate.
- Resulting guidance for the posture recognisers: record full orientation coverage, use IMU conditioning for unseen angles, and never blindly remove orientation.

→ In detail: Section 5.10 — Posture Clusters

2.15 Posture Hand

Figure 2.16 *The real GUI replaying the recorded synchronised session 'low_new3' of 19 September 2026 (401 s of Quest 3 camera hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features) from t = 60 s through its own update loop; no hardware was live. The camera (left) shows an open hand, and the recogniser reads OPEN at 95% and draws the open posture's canonical hand (right). The strip along the bottom edge ("Occlusion Test — idle" and three status labels) and the dark slivers at the right edge are left-over pixels from other tabs in the window capture, not part of Posture Hand.*

What it is for. Posture Hand answers one question without the camera: which of a small set of hand postures is the user holding? It reads the forearm muscle signals (EMG, electromyography: the electrical activity of the forearm muscles, picked up by the MindRove 8-channel armband) and the tilt of the CyberFinger's wrist module, measured by its IMU (inertial measurement unit), and draws the recognised posture's typical hand next to the Quest 3 camera's hand so the two can be compared directly. The tab exists because the camera loses the fingers when the hand leaves its view or hides them, as in a closed fist, while the armband and the CyberFinger keep sensing, so naming the posture still gives a plausible hand shape in those moments. The recogniser keeps its own model file, so retraining it here never changes the other tabs' models.

On the screen

- ⬡ **Posture Hand — IMU + EMG encoder vs camera** — the tab title. The text below it explains the layout (camera on the left, recogniser on the right) and how to fix failures: pick the posture, press ● **Record** and perform it in view, with the camera as ground truth; ⚙ **Fine-tune** then folds those hard cases into the model.

- ⚙ **Fine-tune (retrain on posture_data)** — retrains the recogniser inside the application on every clip in the posture data folder (clips recorded in the Posture Library tab and with this tab's ● **Record**), adding synthetic variants of each clip's recorded raw EMG for robustness. It saves the result to the tab's own model file and writes to the application log how many real and synthetic frames were used, which classes were found and the training accuracy; the same line appears briefly in the banner until the next update replaces it with the live prediction. If fewer than two postures have been recorded, it stops and asks for recordings in the Posture Library first.
- **collect failures** → with the radio buttons **fist**, **open**, **point**, **picking**, **ok** — chooses which posture the next recording will be labelled as (**fist** is the default). The list is the shared posture vocabulary of the Posture Library tab, so it includes **picking**, which is not one of the current model's classes.
- ● **Record** (it changes to ■ **Stop & save** while recording) — while on, it keeps only the frames in which the camera clearly sees the hand and both EMG and wrist-IMU data are present, and labels them all with the chosen posture. For each frame it saves the camera skeleton, the EMG features, the raw EMG window and the CyberFinger's wrist and knuckle orientations. On stop it writes the clip into the posture data folder and refreshes the Posture Library list, so ⚙ **Fine-tune** can learn the hard cases.
- Orange recording status line (empty in the screenshot because nothing is being recorded) — during a recording it shows the posture being recorded, the frame count, how often the recogniser agreed with the chosen posture and which wrong postures it picked most. After saving it reports the frames saved and the recogniser's agreement during the clip, and prompts the user to press ⚙ **Fine-tune**.
- Prediction banner (*predicted: OPEN (95%) fist 0% ok 4% open 95% point 1%*) — the posture the recogniser is showing, in capitals with its probability, followed by the probability of every class in the model. The probabilities are smoothed: they react quickly to a real change, stay steady while a posture is held and change only slowly while the wrist is turning. The shown posture changes only through a persistence gate, which requires a new posture to win clearly for a short time and blocks changes while the wrist is turning fast. If no model exists, the banner asks for training; if EMG or the IMU is missing, it asks to connect them.
- **CAMERA (optical)** (left, green) — the Quest 3 camera's hand skeleton, the reference the recogniser is judged against.
- **IMU + EMG encoder** (right, pink) — the recogniser's hand. The recognised posture's canonical finger curls (the median camera-measured curls of that posture's recordings) bend a clean anatomical hand model, and a smoothing filter makes the hand morph smoothly when the posture changes. The hand takes its rotation from the camera while the camera sees it clearly; otherwise it is turned by the CyberFinger's wrist-module IMU, through the calibrated IMU–camera fusion when that calibration is available. It therefore stays drawn when the hand is out of view.

What was done

- Built a separate EMG + IMU posture recogniser with its own model file. It is a class-balanced softmax classifier (a linear model that gives a probability for each posture, trained so that every posture counts equally however many frames it has) on 43 inputs: 40 time-domain EMG features (simple statistics of each armband channel's signal, such as its mean amplitude) plus a 3-axis tilt of the wrist module (meant as its tilt relative to gravity, but later found to be a horizontal axis that turns with the IMU's heading; see Pose Hand (gravity)). It identifies the held posture and draws that posture's canonical hand (the median camera-measured finger curls of its clips), including when the hand is out of view. The current model has four classes: fist, ok, open and point.
- Accuracy: on 6,104 frames (fist, open and point at three wrist orientations) it scored 99.9% on the training data and 89% with one orientation held out. With class balancing, point recall (the share of real point frames recognised as point) reached 98–100%; without it, point was slow to detect.

- Design experiments: five classifier families (logistic regression, linear and quadratic discriminant analysis, k-nearest neighbours and a multilayer-perceptron neural network) all scored 74–76% on 4 classes at an unseen wrist angle, so the limit lies in the features, not the classifier. Adding the knuckle-IMU tilt, and then the wrist-to-knuckle rotation, lowered accuracy from 75% to 63% and then 50%; fist vs ok fell from 74% to 53%, against 76% for EMG alone. These inputs were rejected because the knuckle sensor sits on the metacarpals (the long bones of the hand between the wrist and the knuckles) and cannot see the thumb.
- Synthetic EMG augmentation based on physical effects: each clip's raw EMG is rotated by one electrode channel (as when the band is put back on slightly turned), its gain and contraction intensity are varied and noise is added, the tilt is jittered, and the synthetic frames are weighted 0.4 against the real ones. Clean accuracy went from 95% to 90%, while accuracy on a simulated re-donned band (a one-channel rotation plus gain and noise) rose from 46% to 75%. For context, the literature cited in the proposal reports 20–30% accuracy drops between sessions (Buteau et al., 2024).
- Persistence gate: a switch needs a top probability above 0.55 and a lead of more than 0.15 over the posture currently shown. With a fixed 0.15 s hold it made 0 switches in 10 s of noisy holding and followed a real change in 133 ms. The committed gate adapts to confidence: the hold shortens for confident predictions (hold = 0.15 s × (1 − 0.85 × strength), where strength runs from 0 for a barely clear win to 1 for a very confident one), down to about 23 ms.
- Added in-app fine-tuning and an active-learning recorder (a recorder that collects new training examples where the model fails). The user picks a posture and performs it in the camera's view; every clearly seen frame is saved with that label, and the status line shows how often the model disagreed. ⚙ **Fine-tune** then folds those clips into the model.

→ In detail: Section 5.11 — Posture Hand

2.16 Pose Hand

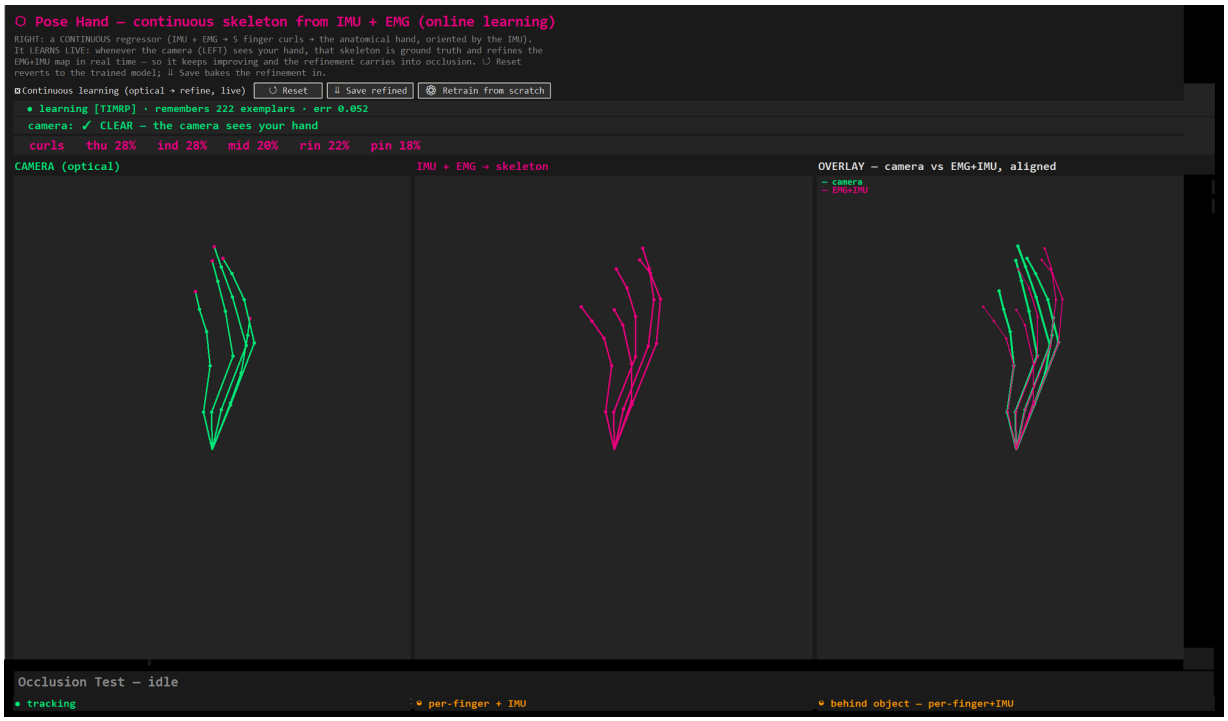


Figure 2.17 *Pose Hand replaying the synchronised session 'low_new3' of 19 September 2026 (401 s of Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features), fed from t = 60 s through the GUI's own update loop rather than captured live. The camera sees the hand (CLEAR), all five fingers are being taught ([TIMRP]) and the rehearsal buffer holds 222 samples. In the overlay the predicted hand (pink) and the camera skeleton (green) share the wrist and palm; the fingers differ mainly in how far they are spread, which the five curls do not control. The strip along the bottom edge is not part of Pose Hand: it is a capture artefact showing labels from other tabs (the Gloves tab's Occlusion Test panel and the Fusion Hand tab's panel captions).*

What it is for. Pose Hand predicts the hand's shape from the forearm muscles and the CyberFinger's motion sensors, so the fingers can still be drawn when the Quest 3 camera cannot see them. The earlier **Posture Hand** tab snaps between a few fixed shapes. Pose Hand is its continuous alternative. It estimates five finger *curls* (0 = straight, 1 = fully curled), so it can draw any in-between hand. It also keeps learning from the camera while in use: every moment the camera sees the hand gives it free corrections, and these corrections carry over into the moments when the hand is hidden.

On the screen

- ⬡ **Pose Hand — continuous skeleton from IMU + EMG (online learning)** — the title. The grey text under it explains the idea: a continuous regressor (a model that outputs numbers rather than picking a category) maps IMU + EMG to 5 finger curls, which bend an anatomical hand turned by the IMU. Whenever the camera sees the hand, the camera skeleton is treated as the truth and refines that map live. The text calls the prediction “RIGHT” and the camera “LEFT”; on screen they are the middle and left panels. IMU means the motion sensors; EMG means the electrical activity of the forearm muscles, read by the MindRove armband.
- **Continuous learning (optical → refine, live)** — a checkbox, on by default. While it is ticked, and the armband and the CyberFinger are streaming, every frame in which the camera sees at least one finger well enough makes one small correction to the model's weights. The correction uses that frame plus 8 samples

replayed from a memory of past frames (the *rehearsal buffer*), and the frame is added to that memory. Unticked, the model only predicts.

- ↺ **Reset** and ⇊ **Save refined** — ↺ **Reset** throws away the live refinement: it goes back to the weights loaded from the model file, empties the rehearsal buffer, and the status line reads **online learning armed — show your hand to the camera to refine**. ⇊ **Save refined** writes the live, camera-refined weights into the model file (posture_pose_model.npz), so the refinement survives a restart. The status line reports how many in-view frames of refinement were saved, and learning then restarts from the saved model with an empty buffer.
- ⚙ **Retrain from scratch** — rebuilds the model inside the app from the recorded posture clips (the posture_data folder). It is fitted on camera finger curls from frames where the camera is confident, and on a fixed fist shape for fist frames the camera is not confident about; other frames without a confident camera view are left out. Synthetic variations of the EMG (shifted channels, changed gains, added noise) are added at a lower weight to make it more robust when the armband is put back on. The result overwrites posture_pose_model.npz, a file several other tabs also read, and the live refinement restarts from it.
- Learning status line (green) — e.g. **● learning [TIMRP] · remembers 222 exemplars · err 0.052**, refreshed every sixth learning frame. The letters (Thumb, Index, Middle, Ring, Pinky) are the fingers the camera currently sees well enough to teach at more than half weight. *exemplars* is how full the rehearsal buffer is (up to 300 samples). *err* is a running average of the gap between the camera's curls and the model's curls on the fingers being taught. Once at least one correction has been made, the line reads **occluded — predicting from EMG+IMU (refined on N frames)** when the camera gives no hand skeleton to learn from, or when learning is off and no finger is seen; N is the number of frames learned from so far.
- **camera:** banner — the occlusion state of the tracked hand: ✓ **CLEAR — the camera sees your hand** (green), ◐ **SELF-OCCLUSION — fingers hidden behind the hand (fist / point)** (orange), ✕ **OCCLUDED — blocked by something in front** or ✕ **OCCLUDED — blocked by an object** (orange), or ⦸ **OUT OF VIEW — hand not in the camera frame** (red). With no camera data it reads **— no optical —**.
- Curls banner (pink) — **curls thu … ind … mid … rin … pin …**: the five predicted curls as percentages, after a light smoothing filter that steadies the hand but still follows fast movements. If no model exists yet, it reads **pose regressor not trained — press ⚙ Retrain**. If the armband or the CyberFinger is missing, it reads **connect EMG + IMU and show your hand**.
- **CAMERA (optical)** panel — the Quest 3 hand skeleton, centred on the wrist. Each finger is coloured by its own visibility estimate (0 to 1): green at 0.7 or more (seen), orange from 0.4 (partly seen) and red below that (hidden); fingertip dots are always pink. The estimate casts a ray from the headset to each joint and checks whether the hand's own bones block it, and it removes joints outside the camera's field of view. It is lowered when the hand is blocked by something in front or by an object, and set to zero when the hand is out of view.
- **IMU + EMG → skeleton** panel — the model's output: a clean 26-joint anatomical hand, curled by the five predicted curls. Strong curls are folded a little further so that a fist closes fully, and the thumb wraps further so that pinch tips meet. While the camera sees the hand (CLEAR), the hand is turned to the camera's orientation. Otherwise it follows the wrist-module IMU, corrected by the camera-learned mounting offset once one is available.
- **OVERLAY — camera vs EMG+IMU, aligned** panel — both hands drawn from the same wrist point with the same orientation: the camera hand as thick green lines underneath and the prediction as thin pink lines on top (legend **— camera** / **— EMG+IMU**). Wherever the two hands separate, you are looking at the live prediction error.

What was done

- Built on 31 August as the continuous alternative to Posture Hand. A *weighted ridge regression* (λ = 5) maps the same 43 EMG and IMU inputs as Posture Hand to five finger curls. A ridge regression is a linear fit with a penalty that keeps the weights small and stable. The curls drive the anatomical hand, which is turned by the IMU.
- Training labels come free from the camera, but only selectively. Camera curls are used only where the camera is confident (3,108 frames: 3,076 open and 32 point). A fixed fist shape (0.90, 0.95, 0.95, 0.95, 0.90) is used for the occluded fists (3,497 frames). Nothing is used for the remaining 6,533 *ok* and *point* frames. On a random 80/20 split the per-finger correlation was 0.90–0.91, but because the trusted targets are essentially two shapes, this mostly measures how well open and fist are told apart. A point came out too soft: about 0.57 on every finger, instead of 0.13 for the index and about 0.9 for the others, because these recordings had no clips that isolate single fingers. The offline model is weak on its own.
- Online learning does most of the work. Every frame in which the camera sees the hand applies one *normalised least-mean-squares* (NLMS) update, a small correction step scaled to the size of the input. On a simulated session in which the armband was taken off and put back on, the curl error fell from 0.167 to 0.089 after about 50 in-view frames (about 1.7 s), and to 0.075 at convergence.
- Fixed **starved learning**. The whole-hand camera confidence is high almost only for an open hand (confident frames: open 100%, point 1%, ok 0%, fist 0%). Learning is therefore gated per finger by a ray-cast visibility estimate, weight = clip((confidence − 0.5) / 0.4), which is the per-finger ray-casting idea of the proposal (Task 5.2). The visible index finger of a point is now taught: its predicted curl moved from 0.60 to 0.08, against about 0.16 from the camera.
- Fixed **catastrophic forgetting**, where a model loses what it learned about postures it has not seen for a while. After a spell of open-hand-only frames, the thumb curl of a point drifted from 0.75 to 0.25. A 300-slot rehearsal buffer now replays 8 stored samples at every update and keeps variety by evicting its most redundant samples. With the buffer, the thumb stayed at 0.72.
- Pose Hand's model file (posture_pose_model.npz) supplies the hidden fingers in Fusion Studio and in SciCortex's Fusion Studio release. In the current release, a recognised Key Posture takes priority and Pose Hand is the fallback. ⇊ **Save refined** deliberately writes the online-refined weights back into that shared file.

→ In detail: Section 5.12 — Pose Hand

2.17 Pose + Position

Figure 2.18 *The research GUI replaying the synchronised session 'low_new3' of 19 September 2026 (Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features) through its own update loop from t = 60 s. The camera was cut on purpose 2.5 s before the capture. After the cut, the replay keeps repeating the last camera-seen joints, as the Quest does. The gate classifies this as "✕ OCCLUDED — blocked by an object", the camera hand is drawn red and the position panel holds the last camera-seen wrist as a grey dot with a dashed uncertainty ring. At this window width the panel's right edge is clipped and the last readout lines under the dome overlap. The strip along the bottom edge does not belong to this tab.*

What it is for. Pose + Position is Pose Hand with a second question added. Pose Hand shows what shape the hand has; this tab also shows where the hand is. It reuses Pose Hand's three skeleton panels and its posture model unchanged (EMG and wrist-module IMU features mapped to five finger curls, refined live whenever the camera sees the hand) and adds a fourth panel that places the wrist relative to the head. It was built for the first position question of the project: what should the hand's position do when the Quest camera loses the hand, given that an IMU senses rotation but not translation?

On the screen

- ⬡ **Pose + Position — the hand's shape AND where it is**: the title and a short description. The tab is Pose Hand plus the hand's position relative to the head. Pink means the camera is placing the wrist. Grey means the dot holds where the hand was last seen, because an IMU senses rotation, not translation, and cannot follow position the way the camera does. Ticking "IMU-carry" shows the experimental fixed-elbow estimate in orange instead.

- **Continuous learning (optical → refine, live)** (ticked by default), ↺ **Reset**, ⇊ **Save refined**, ⚙ **Retrain from scratch**: these controls act on the posture model this tab shares with Pose Hand. While the box is ticked and EMG and IMU data are live, every frame in which the camera sees at least one finger well refines the model, each finger weighted by how well it is seen. ↺ **Reset** discards the live refinement and its memory of examples, and also clears this tab's position memory. ⇊ **Save refined** writes the refined weights into the model file, so Pose Hand uses them too. ⚙ **Retrain from scratch** rebuilds the model from the recorded posture data.
- **Learning status line** (green), here "● learning [TIMRP] · remembers 300 exemplars · err 0.029". The letters are the fingers (thumb, index, middle, ring, pinky) the camera saw clearly in the latest learning step. "Exemplars" is a memory of up to 300 past examples that is replayed during learning. A new example replaces the most similar stored one, so rare postures are not forgotten. "err" is the running average of the curl error against the camera, on the 0-to-1 curl scale. The line switches to "occluded — predicting from EMG+IMU (refined on … frames)" only when the camera delivers no hand at all, or when learning is off and no finger is seen. In the screenshot the camera still delivers its frozen hand, so the line keeps the value it had before the camera loss.
- **camera:** banner: the verdict of the occlusion gate, the module that decides frame by frame whether the camera's hand data can be trusted. It reads "✓ CLEAR — the camera sees your hand", "◐ SELF-OCCLUSION — fingers hidden behind the hand (fist / point)", "✕ OCCLUDED — blocked by something in front", "✕ OCCLUDED — blocked by an object" or "⦸ OUT OF VIEW — hand not in the camera frame". Only "blocked by an object" and "OUT OF VIEW" make this tab treat the hand's position as lost. The gate is needed because the Quest keeps repeating the last pose of a hand it can no longer see.
- **curls** banner: the five finger curls predicted from EMG and IMU (thu, ind, mid, rin, pin), smoothed and shown as percentages (0% = straight, 100% = fully curled).
- **CAMERA (optical)**: the Quest's hand skeleton, with each finger coloured by an estimate of how well the camera really sees it. The estimate is a geometric check for fingers hidden behind the hand or outside the camera's field of view, and it is lowered when the gate reports an occlusion. A finger is green when clearly seen, orange when partly seen and red when not seen. In the screenshot the whole hand is red: it is the frozen last pose, and the model learns from none of its fingers.
- **IMU + EMG → skeleton**: the hand rebuilt from the five predicted curls by bending an anatomical 26-joint hand model. While the gate reports CLEAR, it takes the camera's hand orientation. Otherwise it is oriented by the CyberFinger's wrist-module IMU, corrected by the IMU's camera calibration when one exists.
- **OVERLAY — camera vs EMG+IMU, aligned**: both hands drawn from the same wrist point with the same orientation. The camera hand is thick green ("— camera") and the prediction is thin pink ("— EMG+IMU"). While the camera sees the hand, the gap between the two is the live prediction error.
- **HAND POSITION — relative to your head**, with the checkbox **IMU-carry out of view (experimental)**, off by default. When it is ticked, a lost wrist is placed at a learned elbow pivot plus the forearm direction from the IMU. The panel is a dome radar of directions around the head: "centre = straight ahead · rim = beside / behind", with rings at 30°, 60° and 90° from the gaze. The wrist's distance in metres is printed above the dome (here 0.33 m). The dot is drawn larger when the hand is closer and hollow when the hand is more than 90° from the gaze. It leaves a 1-second fading trail, and while the hand moves, a short line points to where it will be 0.15 s later. The dot is pink when the camera places it, orange when the IMU carries it and grey when it is held. A held dot gets a dashed ring that widens the longer the hand stays unseen.
- **Readout under the dome**: a source label, here "· HELD — where the camera last saw it" (the other two are "● CAMERA — the Quest is placing your wrist" and "○ IMU-CARRIED — experimental: assumes a FIXED elbow"). Then comes "dist … az … el …" (the distance, plus the left/right and up/down angles from the

gaze), then the offset in metres as “right … up … fwd …”. Below these is a note, here “last seen … s ago — true position unknown until the camera re-acquires it (no sensor here measures translation)”. The held position is stored in world coordinates and re-projected through the current headset pose every frame, so the dot still moves correctly when the head turns.

What was done

- Built as a purely additive tab. It is the Pose Hand tab with the same model, the same live learning and the same three panels, plus a fourth panel for hand position on the same dome radar as the Gloves tab. Pink marks a camera-seen position, grey a held position and orange an IMU-carried estimate. The occlusion gate decides when the camera has lost the hand.
- The first idea treated the elbow as a fixed pivot: wrist ≈ pivot + L × forearm direction. The pivot is learned while the hand is in view, and the wrist IMU then swings the hand around it. The first check used a simulation that assumed a fixed elbow, so it tested the code, not the idea.
- On five real synchronised clips (optical wrist as ground truth; the first 35% of each clip treated as seen, the rest as lost), the pivot lost to simply holding the last position. Mean wrist errors were 3.8 cm for freeze, 6.8 cm for the pivot and 8.8 cm for a learned ridge regression from orientation to position. Even with a perfect orientation sensor the pivot gave 15.2 cm, so the model was the problem, not the IMU. The elbow implied by the data wandered 47 cm while the wrist moved 13 cm, and forearm orientation explained only 41–58% of the wrist position even in-sample. One caveat: the clips held only 11–17 cm of wrist travel, which favours freezing.
- Dead reckoning with the armband's accelerometer (integrating acceleration twice from the last camera fix) failed within a fraction of a second. Mean errors were 10.6 cm after 0.25 s, 38.8 cm after 0.5 s, 150 cm after 1 s and 524 cm after 2 s, against 1.4–3.6 cm for freeze. The cause is gravity leakage: a 15–20° orientation error leaves about 2.5 m/s^2 of false acceleration, and even 1° produces 8.6 cm of drift in one second.
- A literature check found two IMU-only routes to position. The first is a kinematic chain with an IMU on every arm segment from a fixed root. Nie et al. (2021) used upper-arm and forearm 9-axis IMUs and report a mean signed reach error of 0.09 ± 1.81 cm against Vicon motion capture, but for a one-dimensional reach distance on two healthy participants. The second is learned inertial odometry anchored by the camera. HOOV (Streli et al., CHI 2023) uses a raw 427 Hz wrist IMU and reports a mean error of 7.77 cm offline, and a median of 15.11 cm (mean 16.97 cm) in its real-time study. The CyberFinger setup supported neither, because it has no upper-arm IMU and no raw, high-rate inertial data.
- Decision: by default the tab **holds** the last camera-seen position in world coordinates, re-projected every frame, and draws it as a grey dot with a widening dashed ring. The pivot stays available as the opt-in checkbox “IMU-carry out of view (experimental)”, off by default. The lasting lesson was about method: validate on recordings, never on a simulation that builds in the model's own assumption.

→ In detail: Section 7.2 — Pose + Position

2.18 Position Capture

Figure 2.19 *Position Capture in the real application while it replays the synchronised session 'low_new3' of 19 September 2026 from t = 60 s through the tab's own update loop. The session holds Quest 3 camera hand joints, headset pose, the CyberFinger's wrist-module and knuckle IMUs and MindRove EMG features. The sensor data come from that recording, not from live devices. Nothing is being recorded and no clip is selected, so the FRONT and TOP coverage maps show all eight saved clips combined. ✗* ***forward*** *appears because no body heading was set during the replay, so no live hand dot is drawn. The palm-direction bars are covered by a strip from the Skeleton tab (**Occlusion Test — idle**).*

What it is for. Position Capture records the data needed to *learn* where the hand is from the CyberFinger's IMUs when the headset camera cannot see it. Every frame in which the Quest 3 camera truly sees the wrist becomes a free training label: it pairs the IMU orientation (and EMG) with the wrist's position around the body, so the camera acts as the teacher. Holding the last seen position out of view is honest but static, and the existing clips had too little hand travel to train a position model. This tab was built to collect pairs that cover the whole reachable space and every palm direction, and to show live which parts are still missing.

On the screen

- ⬡ **Position Capture — teach the IMUs where the hand is (camera = teacher)**: the title and a three-line summary. It says what is recorded (camera wrist position, headset pose, both of the CyberFinger's IMUs and optional EMG) and that only frames the camera truly sees count as labels. It also tells the user to fill the two maps by sweeping high, low, left, right, near and far and turning the palm at each spot.
- **clip label** and **● Record**: the typed label (default **sweep**) becomes part of the saved file name, position_data/pos_<label>_<timestamp>.npz. **● Record** starts the optical source if it is not already running (camera hand joints plus headset pose), clears the live maps and then stores one synchronised frame per update, even if the user switches to another tab. While recording, the button reads **■ Stop & save**. Stopping saves the clip and selects it in the list. A clip with fewer than 60 camera-labelled frames is discarded instead, with a message saying why.

- ⌖ **Set forward**: takes the body's heading from the headset's current horizontal viewing direction. If the user is looking too far up or down, it asks them to look straight ahead and press it again. The first recording sets the heading automatically if it has not been set. The heading defines the body frame in which positions are stored. The origin is a neck pivot 10 cm below and 8 cm behind the headset, one axis is room-up, and the forward axis stays at this fixed heading. The user can therefore turn their head to look at the hand without the frame turning.
- **Readiness checks** (here ✓ **camera** ✓ **headset** ✓ **IMU (calibrated)** ✓ **EMG** ✗ **forward**): one tick or cross per source. **IMU** is ticked when the wrist module's orientation is arriving. It reads **(not calibrated yet)** until the camera has had a clear view of the hand, which lets the wrist module's orientation be aligned to the camera. That calibrated forearm orientation is also saved with each frame. EMG is optional and shows a dash instead of a cross when the armband is off. The line turns green only when camera, headset, IMU and forward are all ready, and is orange otherwise. Here it is orange because no heading has been set.
- **Status line**: before recording it shows the prompt **press ● Record, face forward, and start sweeping**. It also reports messages such as a saved or discarded clip. While recording, it shows ● **REC** with the elapsed seconds and the labelled and total frame counts. When the current frame cannot be a label, it adds **NOT SEEN** and the reason. The reason is **frozen pose** (the Quest repeating a pose it cannot see), the verdict of the occlusion gate (the application's check on whether the camera's hand estimate can be trusted, for example out of view or behind an object) or **no hand**. Above 1.2 m/s it adds **SLOW DOWN**. While recording, the line is pink when the current frame is a label and orange when it is not.
- **View caption** and **show ALL clips combined**: the pink caption says what the maps show. During a recording it reads **SHOWING: this recording only (starts empty)**. After a row is clicked in the list, it reads **SHOWING: clip** followed by that clip's label and length. Otherwise it shows all saved clips together (here **SHOWING: all 8 saved clip(s) combined — click a clip to see it alone**). Ticking the checkbox forces the combined view and, during a recording, adds the live recording on top (**+ this recording**).
- **FRONT VIEW — you, seen from the front** and **TOP VIEW — you, seen from above**: coverage heat-maps in the body frame with 10 cm cells. The front map spans 1.6 m side to side and runs from 1 m below the neck to 0.5 m above it. The top map has the same width and runs from 0.3 m behind the neck to 0.9 m in front of it. A cell turns greener with each camera-labelled wrist position that falls in it and is full green at 20 frames. Empty cells are drawn as outlines only. The **head** ring marks the neck-pivot origin, and the edges are labelled up/down/left/right and far/near. The live hand appears as a pink dot when the camera sees it and as a grey ring at its last labelled position when it does not. No dot is drawn until a heading has been set, which is why none appears here.
- **PALM FACING — vary it at every spot (a model that only saw one palm direction won't generalise)** and the stats line: six bars (**up**, **down**, **left**, **right**, **away**, **toward**), each with its count, sort the labelled frames by the palm's main direction in the body frame. A bar is green when it has at least a quarter of the largest count, otherwise orange. In this screenshot the bars are covered by the Skeleton tab's **Occlusion Test — idle** strip, which is not part of this tab. The line below shows **labelled frames**, counting only the frames that fall inside the front map, which is why it reads 46201 here while the clip list adds up to 46,203. It also shows **cells covered front … top …** (cells with at least 20 frames), the **reach** (the wide × high × deep span of the positions, in cm) and the reminder **a cell turns full green at 20 frames**.

- **HOW TO RECORD**: the six-step recording protocol. Wear everything, stand or sit as in the demo, face forward and press ● **Record**, which sets the heading if it is not yet set. Keep the body facing that way while the head stays free. Keep the hand where the camera sees it (pink dot, not grey). Sweep slowly through high, low, left, right, near and far, turning the palm at each spot. For hip, side and lap positions, look at the hand so that these normally out-of-view postures get labelled. Record 1–2 minute clips, three of them, so that one can be held out to test the model honestly. A closing line repeats that the user should move slowly, because a fast hand blurs the camera's estimate.
- **CLIPS (label · length · labelled frames · span)** and 🗑 **Archive selected clip**: the list has one row per saved clip, oldest first. Each row gives the clip's label, length in seconds, number of camera-labelled frames and spatial span (wide × high × deep, in cm). Clicking a row shows that clip's coverage alone on the maps. The archive button, below the list and not visible in this screenshot, moves the selected file into a dated _archived_ folder inside position_data instead of deleting it, so the action can be undone. It does nothing during a recording.

What was done

- Built on 17 September to record pairs of IMU orientation and wrist position that cover the whole space around the body, with the camera as the teacher, because the existing clips had too little hand travel to learn position from.
- Body frame: the origin is a neck pivot 10 cm below and 8 cm behind the headset, one axis is room-up, and the body heading is captured once with ⌖ **Set forward** rather than following the head. I checked this through the tab's real update code: with the hand fixed, turning the head ±60° in yaw and −55° in pitch changed the stored position by 0.03 cm.
- A frame counts as camera-labelled only if four conditions hold: the hand joints are present, the headset pose is valid, the pose is not frozen, and the occlusion gate does not report out of view, behind an object or blocked by something in front. Live aids: 10 cm FRONT and TOP coverage cells (full at 20 frames), six palm-direction bars and a SLOW DOWN warning above 1.2 m/s. Clips with fewer than 60 labelled frames are discarded. Each saved clip holds the hand joints and the headset pose. It also holds three IMU quaternion slots: BODY1 (empty on the lab CyberFinger), the wrist module and the knuckle. The remaining fields are the CyberFinger's accelerometer readings, EMG, armband acceleration, the camera-calibrated forearm orientation, the body-frame wrist position and the palm normal. Cross-correlating the angular speeds of the IMU and the camera showed a lag of at most 10 ms between them.
- The first real clip (114 s, 93% labelled) revealed a bias in the data. Waist, hip and lap coverage was 0%, and the lowest point reached was chest height (about 37 cm below the neck). There were no far reaches (more than 45 cm forward), and the user turned their body about 25% of the time. Because the camera labels only what the head points at, the data was skewed toward the gaze direction. This led to targeted low and far recordings, the heading-free frames used later and the position lab's look-away test.
- The resulting dataset has eight clips recorded between 17 and 21 September, with one user and the right hand: 46,203 camera-labelled frames in total. It includes the first hip and lap data, down to −0.83 m (low_new), and the main held-out clip low_new3 (401.2 s, 92.9% labelled). It is the training and evaluation data for all later position work and the training set of the deployed Hybrid Position weights.

→ In detail: Section 7.3 — Position Capture

2.19 Learned Position

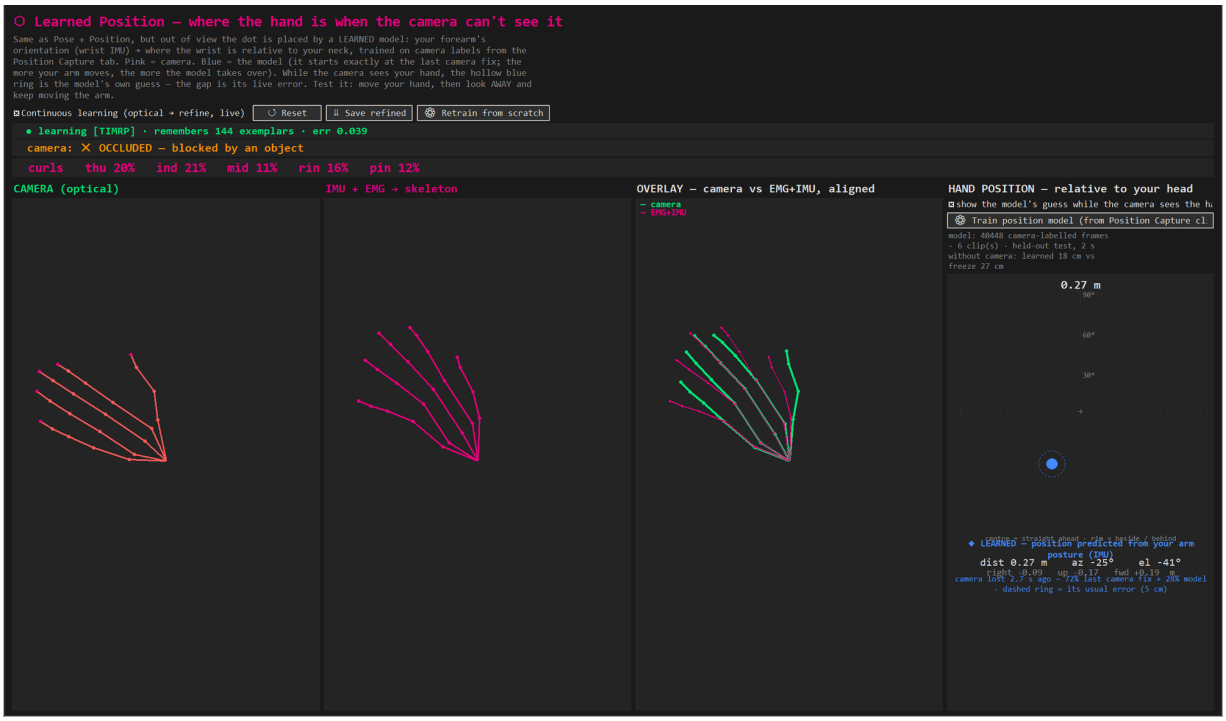


Figure 2.20 *The real GUI replaying the synchronised recording 'low_new3' of 19 September 2026 (401 s of Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features) through its own update loop, from t = 60 s. The camera was hidden on purpose shortly before the capture (a forced loss; the readout says 'camera lost 2.7 s ago'), so the camera line reads ✕* ***OCCLUDED — blocked by an object*** *and every finger in the CAMERA panel is drawn red. The blue dot on the position dome is the model's out-of-view estimate, still 72% last camera fix and 28% model, with a dashed ring for its usual error (5 cm). The loaded model (40,448 frames, 6 clips, per its grey info line) was trained with this recording included, so the picture shows how the tab displays its estimate, not a held-out accuracy test.*

What it is for. The Quest 3 cameras only place the hand while the headset looks at it. Once the hand leaves the view, the earlier Pose + Position tab by default holds the dot where the hand was last seen, because an IMU (inertial measurement unit) senses rotation, not movement through space. Learned Position tests a different answer: a model trained on camera-labelled recordings from the Position Capture tab estimates the wrist position from the forearm's orientation, measured by the CyberFinger's wrist-module IMU, and starts from the last camera fix. While the camera still sees the hand, the model's own guess is drawn as a hollow blue ring next to the camera's pink dot, so its error is always visible.

On the screen

- ⬡ **Learned Position — where the hand is when the camera can't see it** — the title, followed by a short explanation. Pink means the camera is placing the wrist. Blue means the learned model is placing it, starting exactly at the last camera fix and taking over more as the arm moves. The text ends with a suggested test: move the hand, look away, keep moving the arm.
- **Continuous learning (optical → refine, live)**, ↺ **Reset**, ⇊ **Save refined**, ⚙ **Retrain from scratch** — controls for the finger-shape model, which this tab shares with Pose Hand. With the box ticked, every frame in which the camera sees at least one finger well becomes a teaching example, weighted finger by finger, that refines the EMG + IMU → finger-bend model live (EMG is the muscle-electrical signal from the MindRove armband). ↺ **Reset** discards that refinement, going back to the trained model, and also clears

this tab's position anchor and running error. ⇊ **Save refined** writes the refinement into the shared model file. ⚙ **Retrain from scratch** retrains the finger model from the recorded posture data.

- Status lines (green learning line, **camera:** line, **curls** line) — the green line reports live learning: which fingers the camera is currently teaching (T, I, M, R, P for thumb to little finger), how many examples the replay memory holds, and the running error. When the camera delivers no hand at all, or learning is off and no finger is seen, it reads **occluded — predicting from EMG+IMU** instead. The **camera:** line gives the occlusion state: ✓ **CLEAR**, ◐ **SELF-OCCLUSION** (fingers hidden behind the hand), ✕ **OCCLUDED** (blocked by something in front, or by an object) or ⦸ **OUT OF VIEW**. The **curls** line gives the predicted bend of each finger as a percentage.
- **CAMERA (optical)** — the hand skeleton as the Quest 3 reports it. Each finger is coloured by how well the camera sees it: green for well seen, orange for partly seen, red for poorly seen.
- **IMU + EMG → skeleton** — the hand rebuilt without the camera. EMG features plus a tilt feature from the wrist-module IMU give five finger-bend values, which drive an anatomical 26-joint hand. The hand is oriented by the camera when it sees the hand clearly, and by the calibrated wrist-module IMU otherwise.
- **OVERLAY — camera vs EMG+IMU, aligned** — both hands drawn on the same wrist and orientation: the camera in thick green, the EMG + IMU prediction in thin pink. Where the two separate is the live finger-shape error.
- **HAND POSITION — relative to your head** (header controls) — the checkbox **show the model's guess while the camera sees the hand** turns the hollow blue ring on or off, and the button ⚙ **Train position model (from Position Capture clips)** retrains the position model inside the application, using the same code as the offline trainer. Below them, a grey line states what the loaded model was trained on (camera-labelled frames, number of clips) and its held-out error 2 s after losing the camera, compared with freezing.
- Position dome — a map of directions around the head: the centre is straight ahead of the gaze, rings mark 30°, 60° and 90°, and the rim means beside or behind. The distance from the head appears above it in metres. The dot is pink (camera), blue (learned model, used once the **camera:** line reports the hand occluded or out of view) or grey (waiting, holding the last fix). It grows as the hand comes closer and is drawn hollow when the hand is behind. A fading 1-second trail shows where the hand has been, and a short whisker shows where it is heading (0.15 s ahead). In view, a hollow blue ring joined to the dot by a dashed line marks the model's own guess. Out of view, a dashed blue ring around the dot shows the model's usual error.
- Readout under the dome — the source label (e.g. ◆ **LEARNED — position predicted from your arm posture (IMU)**), then distance, azimuth (left/right angle) and elevation (up/down angle), then the right/up/forward offsets in metres. A last note line changes with the state. In view it reads ‘blue ring = the model's guess’ with the current and average error. Out of view it reads ‘camera lost … s ago’ with the share still taken from the last camera fix, the share taken from the model and the size of the dashed ring. If something is missing, it says what to do: train a model, start Live Preview for the headset pose, or show the hand so the wrist IMU can align.

What was done

- Built the tab as a purely additive subclass of Pose + Position. Its position model takes the forearm orientation, expressed in a frame centred at the neck, aligned with gravity and turned with the head's heading, and outputs the wrist's offset from the neck. It uses 500 random ReLU features followed by ridge regression, a regularised linear fit ($\lambda = 30$). Two augmentations encode physical facts: random yaw copies (6 over ±180°) stop it from using gaze direction as a shortcut, and random forearm twists stop it from reading palm-up versus palm-down as movement. The trainer and the GUI share one file

(position_model.py), so they cannot drift apart. Adding the knuckle orientation made it worse (17.4 vs 14.9 cm), so only the forearm is used.

- Replaced the first anchor rule, which faded the last camera fix out over time ($\tau \approx 0.5$ s), with a **motion-based** fade: est = model(now) + exp(−|model(now) − model(then)| / 0.5 m) × (fix − model(then)). With the arm held still for 4.5 s out of view, the anchor weight stayed at 1.000 and the drift was 0.00 cm.
- Found an evaluation trap. The recorded forearm orientation is camera-corrected on every clear frame, but live and out of view the calibration is held at its last value. On one test segment, stored inputs therefore gave 14.3/15.5/15.3/15.1 cm against 14.7/16.7/17.2/18.4 cm with held inputs, at 0.5/1/2/4 s; the held input drifts by about 6° after 1 s and 13° after 4 s. Every later evaluation used held-calibration inputs.
- Results. On the held-out clip low_new3, a model trained on the other five clips (28,240 frames) gave 11.1/15.4/18.2/21.1 cm at 0.5/1/2/4 s, against 13.1/20.5/26.9/36.8 cm for freezing. The shipped model was retrained on all six clips (40,448 frames). In a leave-one-clip-out test over the five long clips, the motion anchor gave 18.1/18.3/20.5/22.5/23.8 cm at 0.5/1/2/4/8 s, against 24.5/27.5/35.7/43.3/50.0 cm for freezing. It was never worse than freezing, while the timer anchor was up to 5.1 cm worse. Replaying low_new3 through the tab's real update code gave 10.6/16.9/20.7/22.6 cm (motion anchor), 16.8/23.3/28.1/29.1 cm (timer) and 12.9/20.3/27.4/37.5 cm (freeze). On the stricter look-away benchmark it scored 22.62 cm.
- Limits found. With arm-posture input alone, the error across sessions did not fall below about 20–26 cm, and with no camera fix at all the held-out error was 27.7 cm. Depth is weak: on forward/back reaches of 28–46 cm the model moved only 61–71% of the way, with 16–18 cm of sideways or vertical drift. Twist augmentation cut the shift caused by a synthetic ±90° forearm twist from about 37 cm to about 11 cm. An armband-accelerometer input lowered the error (e.g. 24.0 to 20.0 cm), almost entirely in the vertical axis, but was not added to the tab.

→ In detail: Section 7.4 — Learned Position

2.20 Body View

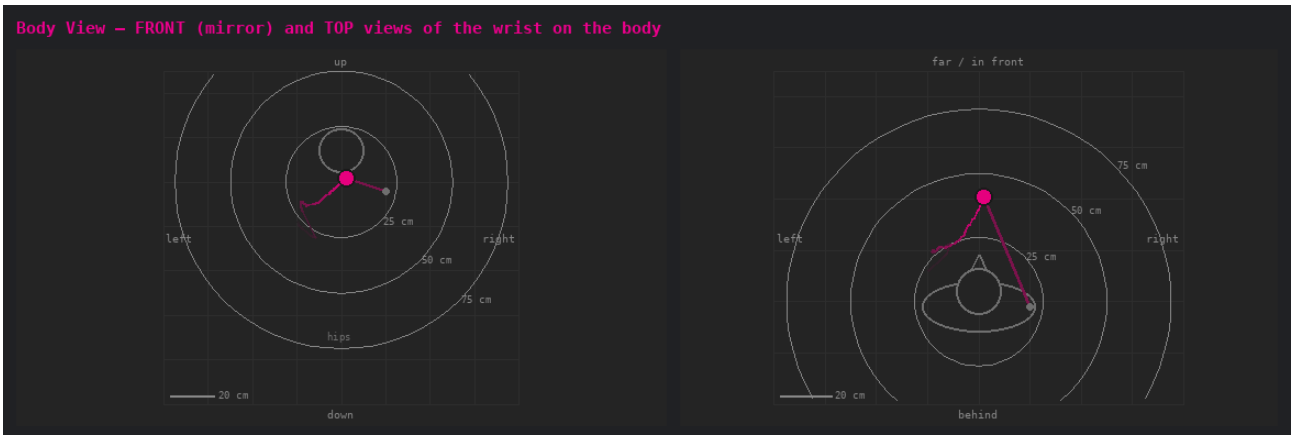


Figure 2.21 *Body View's FRONT (left) and TOP (right) views. This is not a screen capture: the tab's own drawing code was run under Linux on 6 s of the wrist positions stored with the recorded session low_new3 (19 September 2026), which are the positions the tab computes from the camera, and the two view panels were then copied into this image. A live or full-window capture was not possible because showing the tab during this replay froze the Windows application while it was drawing. The heading above the views was added for the figure. The copy shows the dashed distance rings as solid lines and leaves out the torso outline that the FRONT view draws below the head. The camera is placing the right wrist (pink dot and its fading 3-second trail) level with the neck and just inside the 50 cm ring in front of it, and the line to the grey dot is the schematic arm from the right shoulder.*

What it is for. Body View shows *how far* the hand is from the body, as well as its direction. The earlier dome radar (Pose + Position tab) places the hand only by its direction as seen from the eyes, so a hand 20 cm away and a hand 70 cm away land on the same spot there. Body View draws the wrist position in metres in two views of the body, one from the front and one from above. It shows reach, sideways movement and movement beside or behind the body. It also shows at a glance whether the position comes from the Quest camera or, out of view, from the Learned Position model.

On the screen

- ⬡ **Body View — your hand's shape, and where it is around your body**: the tab title. A short explanation under it gives the colour code: pink = camera, blue = learned model out of view, hollow blue ring = the model's guess while the camera can still check it. It also asks the user to face forward once and press ⌖ **Set forward**.
- Control bar: **Continuous learning (optical → refine, live)**, ↺ **Reset**, ⇊ **Save refined** and ⚙ **Retrain from scratch**. These control the finger-pose model that this tab shares with Pose Hand. While the camera sees the hand, it refines that model live. The buttons undo the refinement, save it into the model file, or retrain the model from the recorded posture data. In this tab, ↺ **Reset** also clears the tab's position memory (the last camera fix, the camera-vs-model offset and the average model error).
- Status lines: a learning line (for example, which fingers are being learned and how many examples are stored) and a **camera:** line. The camera line comes from the occlusion gate (✓ **CLEAR**, ◐ **SELF-OCCLUSION**, ✕ **OCCLUDED** or ⦸ **OUT OF VIEW**). Below them, a banner gives the five predicted finger curls in percent, or says what is missing (for example, connect EMG + IMU).
- Three hand panels in a shorter top row, the same as in Pose Hand. **CAMERA (optical)** shows the Quest skeleton, with each finger coloured green, orange or red by how clearly the camera sees it. **IMU + EMG → skeleton** shows the hand rebuilt from the armband's muscle signals (EMG) and the CyberFinger's motion sensors (IMU). **OVERLAY — camera vs EMG+IMU, aligned** draws one on top of the other, so any gap between them is the live error.

- **FRONT VIEW — mirror: your right is on the right** (left-hand view in the figure): the wrist in left/right and up/down, in metres. The origin is a neck point placed just below and behind the headset. The view shows a head, shoulders and torso outline, with **hips** marked below.
- **TOP VIEW — from above, forward is up** (right-hand view in the figure): the wrist in left/right and near/far, seen from above. The outline shows the shoulders, the head and a nose mark pointing forward. The axis labels **far / in front** and **behind** show when the hand goes beside or behind the body.
- Marks drawn in both views: a 20 cm grid with a **20 cm** scale bar, and dashed rings at **25 cm**, **50 cm** and **75 cm** from the neck. A schematic arm line runs from the shoulder point (grey dot) to the wrist dot. The dot is pink when the camera places the wrist, blue when the Learned Position model does, and a hollow grey ring when the tab is holding the last camera position. A fading 3-second trail takes the same colour. While the camera sees the hand, a hollow blue ring joined by a dashed line marks the model's own guess. When the camera has lost the hand, a dashed blue ring around the dot shows the model's usual error at true scale. A position outside the view is drawn as a hollow ring on the edge.
- **WHERE THE HAND IS** column, ⌖ **Set forward** button: takes the headset's current horizontal heading as the direction the body faces. The views are then fixed to the body, so turning only the head does not swing the hand across the display. The tab sets this automatically from the first headset pose. If the user is looking up or down, it asks them to look straight ahead and press again; if there is no headset pose yet, it asks them to press ◉ **Live Preview** in the Gloves tab.
- Checkboxes **show the model's guess (hollow ring)** and **show the 3-second trail**: turn the ghost ring and the trail on or off.
- Position readout: first the source line, ● **CAMERA — the Quest is placing your wrist**, ◆ **LEARNED — position predicted from your arm posture (IMU)** or · **WAITING — holding the last camera position**. Then the distance right or left, above or below the neck, in front or BEHIND, and the total distance from the neck, in cm. A final note gives either the model's error now and on average, or, out of view, how long ago the camera was lost and the mix of last camera fix and model. When something is missing, the note says so instead (no headset pose, no position model, or wrist IMU not yet calibrated); the messages of ⌖ **Set forward** also appear here.

What was done

- Added on 19 September as a separate tab that changes no other tab. It brings reach into the display: metric FRONT (right × up) and TOP (right × forward) views around a body outline, with a 20 cm grid and rings at 25, 50 and 75 cm from the neck.
- Uses Position Capture's body frame (neck origin, heading set with ⌖ **Set forward**), so turning the head does not move the hand on the display. Without a headset pose, it falls back to a head frame and says so.
- Shows which source is placing the wrist: camera (pink), the Learned Position model out of view (blue), or holding the last camera position (grey). A 3-second trail is tinted by source. While the camera sees the hand, the model's guess appears as a hollow blue ghost, and the readout gives its error now and on average. Out of view, the model's position is corrected by the last camera-vs-model offset, which fades as the arm moves away from where the camera lost it. A dashed ring drawn to true scale shows the model's usual error.
- The camera is trusted only when there is a headset pose, the occlusion gate does not report the hand out of view or blocked, and the Quest is not repeating a frozen pose. The tab reads the Learned Position model without changing it, shares the Pose Hand finger model, and keeps its own position state.
- Verified: the recorded low_new3 clip was replayed through the tab's real update code, and the camera frames landed on the clip's stored body-frame positions to 0.0000 cm. A smoke test of all display states in the real application raised no exceptions.

- Body View's layout later became the base of the Hybrid Position tab, which reuses its hand panels and FRONT and TOP views.

→ In detail: Section 7.5 — Body View

2.21 IMU FK Position

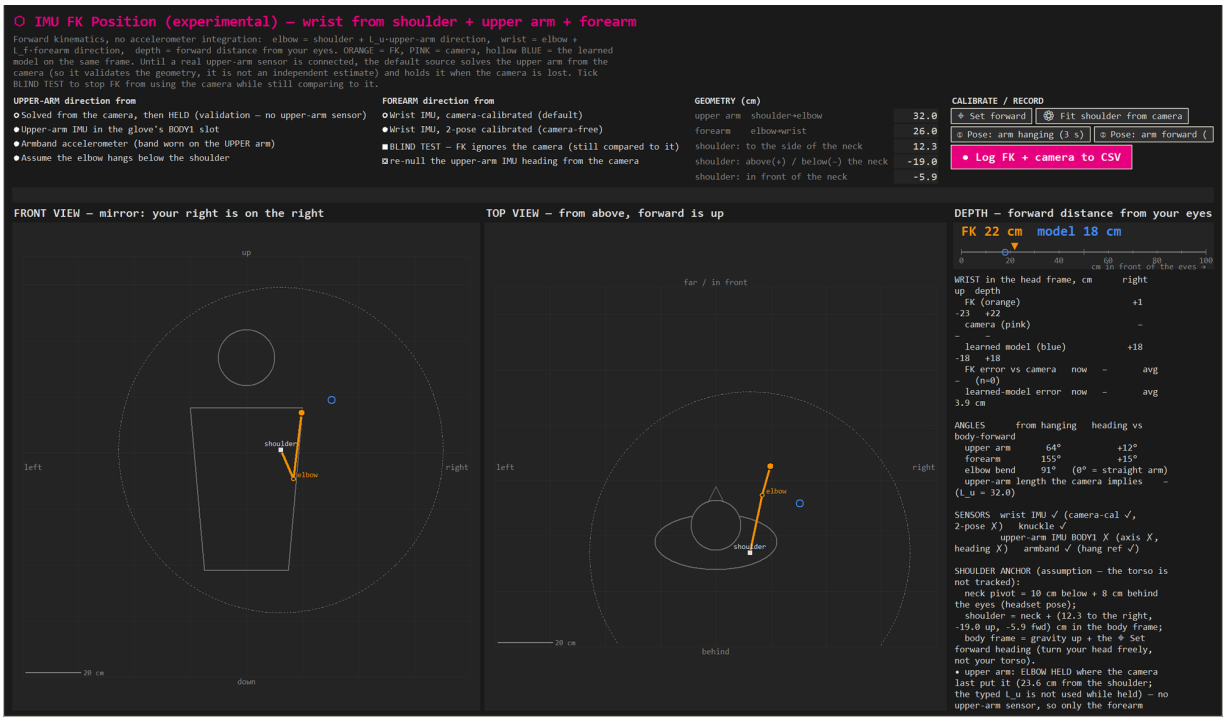


Figure 2.22 *The real application replaying, through its own update loop, the synchronised recording 'low_new3' of 19 September 2026 from t = 60 s. The 401-s recording holds Quest 3 hand joints, the headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG. The camera was hidden (a forced loss) 2.5 s before the capture, so no pink camera point appears and the camera rows read "–". The orange FK arm keeps the elbow held where the camera last put it, while the wrist-module IMU moves the forearm. The status line gives the held elbow as 23.6 cm from the shoulder, so the typed 32.0 cm is not used. The hollow blue ring is the learned model's estimate. The GEOMETRY fields show the lab's saved calibration, not the defaults.*

What it is for. IMU FK Position works out where the wrist is from geometry, without asking a learned model for its 3-D position (XYZ). It uses forward kinematics (FK): starting at the shoulder, it adds the upper arm and then the forearm, each as a length times a direction, so wrist = shoulder + L_u × (upper-arm direction) + L_f × (forearm direction), where L_u and L_f are the upper-arm and forearm lengths. No acceleration is integrated, so the error does not keep growing with time. The tab was built to test whether this geometry tracks reach and depth better than the learned position model when the Quest camera loses the hand. It also prepares the pipeline for an upper-arm sensor, which the lab CyberFinger does not have (its BODY1 sensor slot is empty).

On the screen

- ⬡ **IMU FK Position (experimental) — wrist from shoulder + upper arm + forearm**: the title and a short explanation. The explanation gives the two FK equations and defines depth as the forward distance from the eyes. It sets the colour code used everywhere on the tab: **ORANGE** = FK, **PINK** = camera, **hollow BLUE** = the learned model on the same frame. It also warns that the default upper-arm source uses the camera, so in that mode FK checks the geometry and is not an independent estimate.
- **UPPER-ARM direction from**: four radio buttons choose where the upper-arm direction comes from. **Solved from the camera, then HELD (validation — no upper-arm sensor)** is the default. While the camera sees the hand, the upper arm is solved from it. When the hand is lost, the elbow is held where the camera last put it. **Upper-arm IMU in the glove's BODY1 slot** reads a spare sensor input on the CyberFinger, which is empty on the lab unit, and needs the two calibration poses described below.

Armband accelerometer (band worn on the UPPER arm) measures how far the arm is lifted by comparing gravity now with gravity in the hanging pose. It assumes the arm points the same horizontal way as the forearm. **Assume the elbow hangs below the shoulder** fixes the upper arm straight down.

- **FOREARM direction from**: **Wrist IMU, camera-calibrated (default)** uses the wrist-module IMU, aligned to the camera by the shared wrist fuser. **Wrist IMU, 2-pose calibrated (camera-free)** uses the same IMU, calibrated by the two poses instead. Below them are two checkboxes. **BLIND TEST — FK ignores the camera (still compared to it)** stops FK and the wrist fuser from using the camera, while the camera keeps scoring FK. **re-null the upper-arm IMU heading from the camera** is on by default. In BODY1 mode it slowly corrects the drifting heading of the upper-arm IMU whenever the camera sees the hand.
- **GEOMETRY (cm)**: five editable fields. **upper arm shoulder→elbow** and **forearm elbow→wrist** are the two lengths. The other three place the shoulder relative to the neck: **shoulder: to the side of the neck**, **shoulder: above(+) / below(−) the neck** and **shoulder: in front of the neck**. Out-of-range values are clamped (the upper arm, for example, to 10–50 cm). The values are saved to the tab's calibration file when Enter is pressed or the cursor leaves a field.
- **CALIBRATE / RECORD**: ⌖ **Set forward** takes the headset's current horizontal heading as body-forward. It is also set automatically when the first headset pose arrives. The shoulder anchor and both views are fixed to this direction, so the head can turn freely. ⚙ **Fit shoulder from camera** needs about 200 camera frames with the hand moving widely. It refits the shoulder position, with the lengths held fixed, so that every elbow implied by the camera lies one upper-arm length from the shoulder. It rejects implausible fits. ① **Pose: arm hanging (3 s)** captures each IMU's long axis and the armband's gravity reference 3 s after the click. ② **Pose: arm forward (3 s)** then captures each IMU's heading, and is rejected if the arm is not roughly horizontal. Together the two poses are the camera-free calibration. ● **Log FK + camera to CSV** records every frame to a CSV file and starts the headset preview if it is not running. While recording, the button reads ■ **Stop logging**.
- Message strip (the grey bar under the controls): short prompts and confirmations, such as the countdown while a calibration pose is captured, the result of the shoulder fit and the number of rows logged so far. It is empty in the screenshot.
- **FRONT VIEW — mirror: your right is on the right**: a front view, fixed to the body, with an outline of the head and torso, a 20 cm grid and a scale bar. It shows the shoulder (white square) and the orange FK chain from the shoulder to the elbow (hollow orange circle) and on to the FK wrist (orange dot). An orange trail behind the wrist fades over 3 seconds. When the camera sees the hand, its wrist appears as a pink dot, joined to the FK wrist by a dotted line. The hollow blue ring is the learned model's wrist. A dashed circle of radius L_u + L_f around the shoulder marks how far the arm can reach.
- **TOP VIEW — from above, forward is up**: the same items seen from above, running from **far / in front** at the top to **behind** at the bottom. The head is drawn with a nose marker, and the shoulders as an ellipse. This view shows depth, meaning how far forward the hand reaches, which the front view cannot show.
- **DEPTH — forward distance from your eyes**: a 0–100 cm ruler. It marks the FK wrist with an orange triangle, the camera wrist with a pink dot and the learned model with a blue ring. Their values are printed above it in large type, for example **FK 22 cm** and **model 18 cm**. When both FK and the camera are available it also shows **FK − camera = … cm**.
- Readout panel: **WRIST in the head frame, cm** gives right/up/depth for FK, the camera and the learned model. Below that are **FK error vs camera** and **learned-model error**, each shown now and as an average. The FK error reads n/a while the upper arm is being solved from the camera. **ANGLES** lists each segment's angle from hanging and its heading relative to body-forward, the elbow bend and the upper-arm length the camera implies. **SENSORS** shows a tick or a cross for the wrist IMU, the knuckle sensor, the BODY1 slot and the armband, and for their calibration steps. **SHOULDER ANCHOR** states the assumption behind the

shoulder position: because the torso is not tracked, the shoulder is fixed relative to a neck point 10 cm below and 8 cm behind the eyes. Status lines follow, for example **ELBOW HELD where the camera last put it**, and then the camera state.

What was done

- Built an experimental FK wrist estimator that integrates no acceleration. The upper-arm source can be switched between four options (solved from the camera then held, the BODY1 IMU, the armband on the upper arm, or a hanging arm), so a real upper-arm sensor can be added without a redesign. The tab also has two forearm sources, a BLIND TEST mode, a shoulder fit and CSV logging. The default geometry comes from a sphere fit on the recorded clips: shoulder 16.5 cm right, 19.0 cm down and 5.0 cm forward of the neck, upper arm 30 cm, forearm 26 cm. Letting the fit choose the upper-arm length freely gave an unphysical 43 cm.
- Measured the frame conventions on recordings instead of assuming them. The CyberFinger IMU world is z-up: gravity matched only its z axis, with a residual of 5–12° against 44–50° for x and y. Each sensor has its own arbitrary heading, which drifted by about 1° per minute in those recordings. The OpenXR world is y-up, and the headset looks along −z: the wrist seen by the camera was on the −z side in 100% of frames. A side finding: the “gravity tilt” input of the Pose Hand and Posture Hand models was really a horizontal axis. This led to the separate Pose Hand (gravity) tab, and the original models were left untouched.
- Offline results on the recorded clips: with a perfect upper-arm direction taken from the camera, FK reached a median wrist error of 5.2–5.6 cm (3.9–4.8 cm in depth). With 10° or 20° of upper-arm error it reached 7.5 or 12 cm, against 20–26 cm for the learned model. With the upper arm held (no new hardware), the real update code gave 8.6/14.6/21.1/20.5 cm at 0.5/1/2/4 s. Freeze, which holds the last camera position, gave 11.0/21.4/33.1/31.8 cm.
- First live blind log: 55 s long, 48 s of it blind, pure forward/back reaches, no upper-arm sensor. FK followed depth (correlation 0.97, 61% of each depth move), while the learned model moved the wrong way (−20%). The median 3-D error was 14.3 cm for FK, 31.7 cm for the learned model and 11.3 cm for freeze. Freeze scored lowest because the reaches were purely forward/back (sideways spread 2 cm), so it erred only in depth. FK's extra error had two causes. First, the upper arm was set too long: 32 cm, against the 25.9 cm the camera implied. Recomputing with 25.9 cm gives 11.4 cm, level with freeze, and the depth bias falls from +6.1 to +0.3 cm. Second, FK slides sideways on an arc around the held elbow.
- Held-elbow fix: the tab now holds the elbow where the camera last implied it, instead of using the typed length. This cut the error at the moment of loss from 4.7–7.8 cm to 0.6 cm. The error fell from 7.1 to 4.3 cm at 0.5 s and from 9.3 to 7.0 cm at 1 s, and was unchanged from 2 s on. A simulated BODY1 upper-arm IMU run through the real FK code gave 4.0 cm (2.6 cm in depth). This confirms the pipeline and the axis conventions, not real-world accuracy. With real hardware, the expected error is about 7–12 cm.
- A sensor-free “synergy” model, which predicts the upper-arm swing from the forearm, only reached about the learned model's level. The tab therefore stays experimental until a real upper-arm sensor is available. Its forearm term, with the elbow held, became the backbone of the Hybrid Position tab.

→ In detail: Section 7.6 — IMU FK Position

2.22 Pose Hand (gravity)

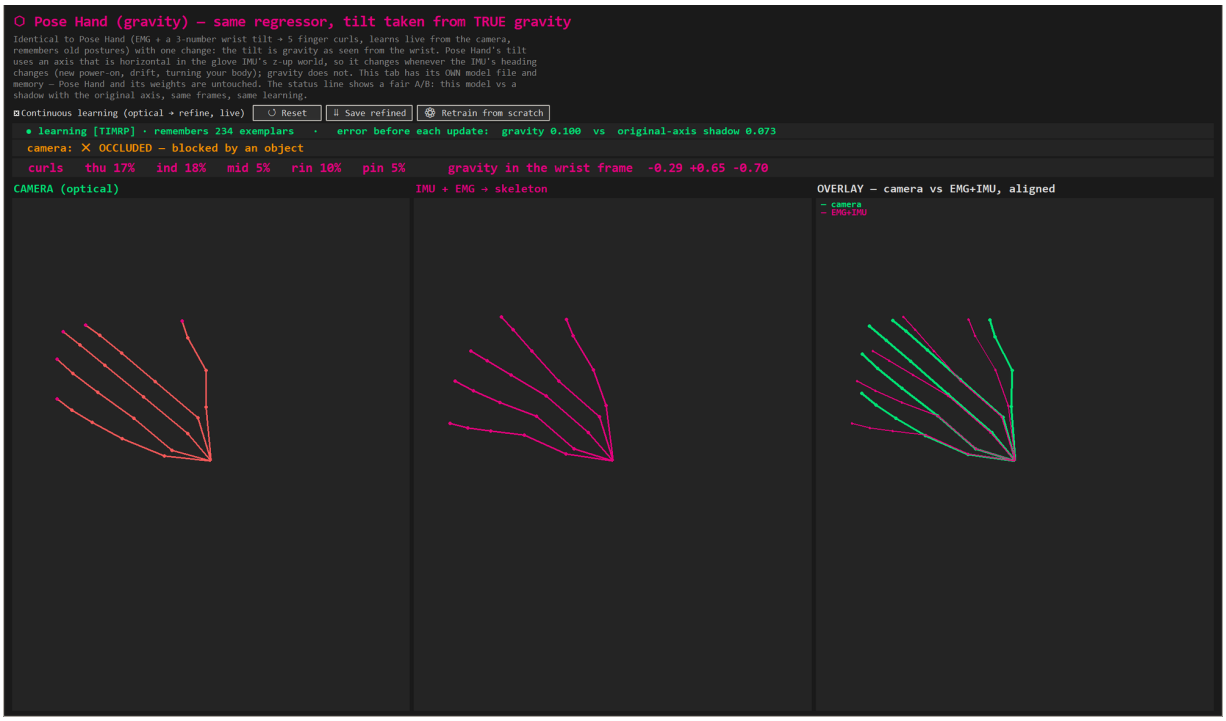


Figure 2.23 *The Pose Hand (gravity) tab in the real GUI, replaying the synchronised 401-second session 'low_new3' of 19 September 2026 from t = 60 s through the GUI's own update loop. The recording holds the Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, and the MindRove EMG features. At this moment the camera reports the hand blocked by an object, so the camera skeleton is drawn red (low confidence), while the EMG + gravity-tilt model goes on predicting the five finger curls. The green line still shows the last learning message from earlier in the replay: at that point the original-axis shadow's running error (0.073) was below the gravity model's (0.100).*

What it is for. Pose Hand (gravity) is a twin of the Pose Hand tab. It predicts the five finger curls continuously from the forearm EMG plus a three-number wrist tilt taken from the CyberFinger's wrist IMU, and it refines itself live whenever the Quest camera sees the hand. The tab was needed because the original tilt input turned out not to be gravity. In the CyberFinger IMU's z-up world it is a horizontal axis that turns with the IMU's *heading* (its compass direction around the vertical), which an IMU without a magnetometer cannot fix and which drifts. Pose Hand was already in use, so instead of changing it, this tab takes the tilt from true gravity and has its own model and learning state. It also runs the original axis as a shadow model on the same frames, so the two can be compared fairly while the system runs.

On the screen

- ⬡ **Pose Hand (gravity) — same regressor, tilt taken from TRUE gravity**: the title, with a short explanation below it. The tab is identical to Pose Hand (EMG + a 3-number wrist tilt → 5 finger curls, learning live from the camera) with one change: the tilt is gravity as seen from the wrist. It has its own model file and memory, and the status line shows the A/B comparison.

- **Continuous learning (optical → refine, live)** (ticked by default): when ticked, every frame in which the camera sees a finger is used as ground truth to refine the model. Each finger learns only in proportion to the camera's confidence that it can see that finger, starting above 0.5 and with full weight from 0.9 upward. Each update is a normalised least-mean-squares step on the current frame plus up to 8 frames replayed from a 300-sample memory. Once the memory is full, a new sample replaces the most similar stored one, so learning a new posture does not erase older ones.
- ↺ **Reset**, ⇊ **Save refined**, ⚙ **Retrain from scratch**: **Reset** returns both the gravity model and its shadow to the trained weights loaded from the model file and clears their memories and error averages. **Save refined** writes the live-refined gravity weights into this tab's own file, posture_pose_gravity_model.npz, and never into Pose Hand's file. **Retrain from scratch** runs this tab's own trainer on the recorded posture data. The trainer fits the gravity-tilt model and the shadow's starting weights with the old axis.
- **Learning status line** (green): while learning it reads, for example, '● learning [TIMRP] · remembers 234 exemplars · error before each update: gravity … vs original-axis shadow …'. The letters name the fingers being taught with more than half weight (thumb, index, middle, ring, pinky), and the count is the size of the replay memory. The two numbers are running averages of each model's curl error, measured on every frame *before* the model learns from it. Both models see the same frames and use the same learning rule, which keeps the A/B comparison fair. The line keeps its last message until a new one is written. Other messages report that learning is armed, that the hand is occluded and the curls come from EMG + gravity tilt, or that a save has finished.
- **camera:** banner: the camera's verdict on the hand, in one of five states: '✓ CLEAR', '◐ SELF-OCCLUSION' (fingers hidden behind the hand), '✕ OCCLUDED — blocked by something in front', '✕ OCCLUDED — blocked by an object' (shown in the screenshot) or '⦸ OUT OF VIEW'; it reads '— no optical —' when there is no camera verdict at all.
- **curls** banner: the five predicted finger curls in percent (thu, ind, mid, rin, pin; 0% = straight, 100% = fully curled), after a smoothing filter. To the right, **gravity in the wrist frame** gives the three numbers of the tilt input: the direction of gravity as seen from the wrist IMU, which depends only on how the forearm is tilted and never on heading.
- **CAMERA (optical)** panel: the Quest 3 hand skeleton, centred on the wrist. Each finger is coloured by how confident the camera is that it can see it: green when confidence is at least 0.7, orange from 0.4 to 0.7 and red below 0.4. A geometric self-occlusion and field-of-view check computes this confidence, and the camera's occlusion verdict scales it down (to zero when the hand is out of view).
- **IMU + EMG → skeleton** panel (pink): the predicted curls, slightly boosted for display, bend a clean anatomical 26-joint hand model. The hand is oriented by the camera when the camera has a clear view, otherwise by the wrist IMU (corrected by its camera calibration once one exists), so it keeps moving with the forearm when the camera loses the hand.
- **OVERLAY — camera vs EMG+IMU, aligned**: both hands drawn from the same wrist point with the same orientation. The camera hand is thick green underneath and the EMG + IMU prediction is thin pink on top, so while the camera sees the hand clearly, any gap between them is the live tracking error.

What was done

- **The problem.** The IMU FK Position work established that the CyberFinger IMU's world frame is z-up. Pose Hand's 'gravity tilt', computed as $R(q)^T[0, -1, 0]$, is therefore not gravity: it is a horizontal axis that turns with the IMU's drifting heading.

- **Built additively (20 September).** A new tab with its own trainer, model file and online-learning state, which computes the tilt from true gravity, $R(q)^T[0, 0, -1]$. A shadow model with the old axis learns from the same frames with the same rule, which gives a live, fair A/B comparison. The change added 212 lines and removed none, and Pose Hand's model file was verified byte-identical (commit a118f35).
- **Heading no longer leaks in.** With the hand unchanged, turning the heading by 30°, 90° or 180° swings the old feature by exactly that angle and the gravity feature by 0.00°. In the real update loop with learning off, a 120° heading turn changed the gravity model's curls by 0.0000, against 0.156 on average (0.40 at most) for the old axis.
- **Better across sessions.** The models were trained on 31 August and tested on clips from 17 and 19 September. Mean curl error, on a 0–1 scale where 1 is a full curl, was 0.319 for the old axis, 0.284 for gravity and 0.343 with no tilt. Gravity was better on 4 of 5 clips.
- **Not one-sided.** Within a single day, with one orientation held out, using no tilt at all was marginally best: 0.242, against 0.253 for gravity and 0.277 for the old axis. With online learning, both versions re-converge while the camera sees the hand (0.035 old against 0.033 gravity after a 120° heading jump). The difference matters out of view, where nothing can re-teach a heading-dependent feature.
- **What followed.** The later Key Postures recogniser dropped the tilt input entirely after measuring that it acted as a shortcut. With one arm position held out, accuracy rose from 88.7% to 91.4% and wrong switches fell from 12.1 to 9.0 per minute; a true-gravity tilt was worse (15 per minute). The Fusion Studio release still uses Pose Hand with the original axis.

→ In detail: Section 5.13 — Pose Hand (gravity)

2.23 Hybrid Position

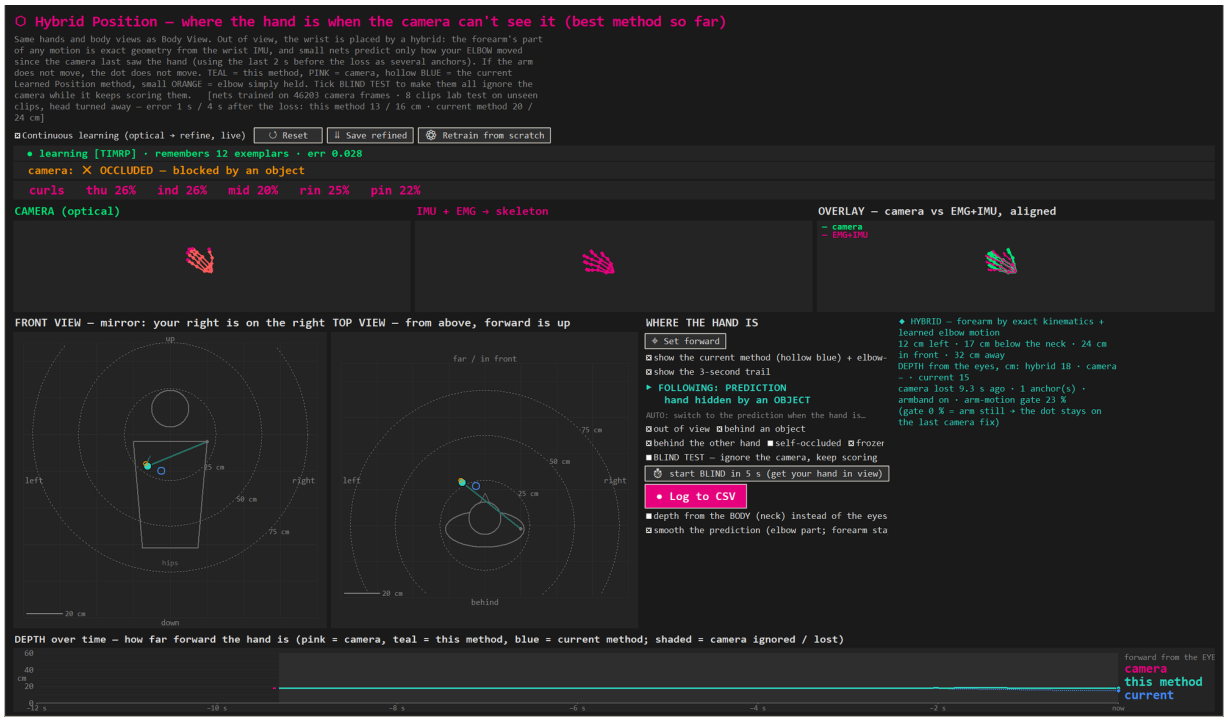


Figure 2.24 *The Hybrid Position tab while the GUI replays the recorded session low_new3 (19 September 2026: Quest 3 hand joints and headset pose, the CyberFinger's wrist-module and knuckle IMUs, MindRove EMG) through its own update loop from t = 60 s; the image is 2400 × 1398 px. The camera was hidden on purpose (a forced loss), so the gate reads blocked by an object, the tab shows ▶ **FOLLOWING: PREDICTION** and the readout reports that the camera lost the hand 9.3 s earlier. The teal dot is the tab's out-of-view prediction, shown with the Learned Position (hollow blue) and elbow-held (orange) markers; this is a replay, not live hardware use, and blind-test mode is off, so no error table is shown.*

What it is for. Hybrid Position shows where the hand is when the Quest 3 camera cannot see it. Optical tracking loses the hand whenever it leaves the camera's view or is hidden, and the shipped behaviour then freezes the position. This tab brings the best method from the position lab into the research GUI. The CyberFinger's wrist IMU (inertial measurement unit) gives the forearm's motion exactly through forward kinematics (FK, geometry from joint angles and segment lengths). Small learned networks estimate only what the sensors cannot see: how the elbow has moved since the camera last saw the hand. The tab switches between the camera and the prediction automatically, and a blind test lets the camera score every estimator live.

On the screen

- **Title and description**: the heading ⬡ **Hybrid Position — where the hand is when the camera can't see it (best method so far)**, followed by a short explanation and the colour code (TEAL = this method, PINK = camera, hollow BLUE = the current Learned Position method, small ORANGE = elbow simply held). The bracketed text at the end comes from the loaded weights file: *nets trained on 46203 camera frames · 8 clips*, then the lab test on unseen clips with the head turned away, giving the error 1 s / 4 s after the loss (this method 13 / 16 cm, current method 20 / 24 cm).
- **Finger-pose bar and status strips**: these come from Pose Hand and use the same finger model. **Continuous learning (optical → refine, live)** uses the camera skeleton, whenever the hand is visible, to refine the EMG + IMU → finger-curl map. ↺ **Reset** discards that refinement and also restarts the position

tracker. ⇊ **Save refined** saves the refinement into the model, and ⚙ **Retrain from scratch** retrains it from the recorded posture data. Below the bar are three strips: the learning status (which fingers are being learned, the number of stored examples, the error), the camera gate (here **camera: ✕ OCCLUDED — blocked by an object**) and the five predicted finger curls in percent.

- **Hand panels**: **CAMERA (optical)** draws the Quest skeleton with each finger coloured by visibility (green, orange or red). **IMU + EMG → skeleton** draws the hand predicted from the armband and the IMU, oriented by the camera while it sees the hand clearly and by the wrist IMU otherwise. **OVERLAY — camera vs EMG+IMU, aligned** lays both hands on the same wrist and orientation, so any gap between them is the live pose error.
- **FRONT VIEW — mirror: your right is on the right**: a metric left/right × up/down view of the hand around a body outline (head and torso, with the hips labelled). The origin is the neck, with a 20 cm grid, rings at 25 / 50 / 75 cm and a 20 cm scale bar. The large dot is the wrist, coloured by its source, and a line from the shoulder draws a schematic arm. A hollow blue ring marks the Learned Position estimate and a small orange ring marks FK with the elbow held. When the prediction places the dot while the camera can still see the wrist (as in a blind test), a pink dot linked by a dashed line shows where the camera actually sees it. A fading 3-second trail shows recent motion.
- **TOP VIEW — from above, forward is up**: the same hand seen from above (left/right × near/far, labelled **far / in front** and **behind**), with the shoulders, head and nose drawn as an outline. This view shows how far the hand reaches (depth), which the FRONT view cannot show. It uses the same colours, rings and markers.
- **WHERE THE HAND IS**: ⌖ **Set forward** takes the way the headset faces as the body's forward direction and restarts the tracker (it is also set automatically when the first headset pose arrives). After that the views are fixed to the body, so turning only the head does not swing the hand around. Two checkboxes turn on or off the comparison markers (**show the current method (hollow blue) + elbow-held (orange)**) and the trail (**show the 3-second trail**).
- **Follow line and AUTO switches**: ▶ **FOLLOWING: CAMERA** or ▶ **FOLLOWING: PREDICTION**, with the reason (here *hand hidden by an OBJECT*). Under **AUTO: switch to the prediction when the hand is…**, checkboxes choose which camera states hand the dot to the prediction: **out of view**, **behind an object**, **behind the other hand**, **self-occluded** (off by default) and **frozen** (the Quest repeating its last pose). The camera gets the dot back only after three trusted frames in a row, and the dot glides back rather than jumping.
- **Test and logging controls**: **BLIND TEST — ignore the camera, keep scoring** makes every estimator run as if the hand were lost, while the camera still measures their errors. ⏱ **start BLIND in 5 s (get your hand in view)** first turns blind mode off, counts down so that the hand can settle in view, then turns it on. ● **Log to CSV** (it becomes ■ **Stop logging**) writes every frame to a file under hybrid_logs for later analysis. **depth from the BODY (neck) instead of the eyes** switches the DEPTH strip below from the distance in front of the eyes to the distance in front of the neck in the body frame, which does not change when only the head turns. **smooth the prediction (elbow part; forearm stays instant)** turns the elbow smoothing filter on or off (it is on by default).
- **Readout (right)**: the current source (◆ **HYBRID — forearm by exact kinematics + learned elbow motion**), then the hand's position from the neck (cm left or right, above or below, in front or behind, total distance) and the depth from the eyes for the hybrid, the camera and the current method. While the tab is predicting, it adds the time since the camera lost the hand, the number of anchors used, whether the armband is on, and the **arm-motion gate** percentage (0% means the arm is still, so the dot stays on the last camera fix). In blind-test mode it adds a table of the current and running-average error against the camera for this method, Learned Position, elbow held and freeze.

- **DEPTH over time**: a strip chart of the last 12 s of how far forward the hand is, with the camera in pink, this method in teal and the current method as a dashed blue line. Shaded spans mark time when the camera was ignored or lost. A legend on the right gives each value now and the difference *this − camera*. In the screenshot the shaded span covers the last 9 s or so: the pink camera trace stops where it begins, the teal line runs through it, and the dashed blue line appears only in the last 2 s; the legend's values are cut off at the right edge at this window width.

What was done

- Built the tab on the Body View layout, with a streaming tracker that runs in pure numpy, so the GUI needs no deep-learning library. The forearm comes from FK: the camera wrist at the moment of loss plus L_f × the change in forearm direction, with L_f = 26 cm. Twelve small multilayer perceptrons (MLPs, small fully connected networks) predict only the change of the elbow offset. The last camera fix and the camera frames 0.25, 0.5, 1 and 2 s before the loss act as several anchors whose estimates are averaged, and a still-arm gate keeps the output exactly on the last fix when the forearm does not move. The deployed weights were trained on all eight Position Capture clips (46,203 camera-labelled frames, 85,200 training pairs).
- On the position-lab look-away benchmark, the mean wrist error over 0.5–8 s after the loss was 14.58 cm, against 22.62 cm for Learned Position and 36.95 cm for freezing (the shipped behaviour). That is 8.0 cm better than the previous best and at least 5.2 cm better than freezing in every clip and at every horizon.
- Checked in the real GUI code. The streaming tracker matched the lab estimator exactly (maximum difference 0.0000 cm over 1,600 episodes). Replaying the unseen clip low_new3 in blind mode gave 8.3 / 12.4 / 14.1 / 16.3 / 17.6 cm at 0.5 / 1 / 2 / 4 / 8 s, against 12.6–51.3 cm for freezing. With the arm still for 6 s and the sensors frozen, drift was 0.000 cm.
- In live blind tests, starting from a proper camera fix, the hybrid scored 10.8 cm, against 22.5 cm for Learned Position and 14.3 cm for freezing; FK with the elbow held scored 9.6 cm. The hybrid followed 71% of the depth of each reach (correlation 0.83). One test started blind mode with the hand resting on the mouse, which most likely gave a poor anchor, so the ⏱ **start BLIND in 5 s (get your hand in view)** button was added. After 2.5–3 minutes without the camera, freezing did better (30.8 against 32.1 cm). The cause is heading drift: without a magnetometer, the wrist IMU's heading (rotation about the vertical) is not tied to anything once the camera is gone. The working rule from these tests is that the horizontal direction can be trusted for about one minute after the last camera fix.
- AUTO switching was tested on 18 simulated out-of-view spans in a replay of low_new3: the prediction took over at the first frame and the camera returned after 2–3 frames. The errors were 8.3 / 12.8 / 16.9 cm for losses of 1 / 3 / 6 s, against 12.4 / 20.7 / 36.1 cm for freezing. With the hand near the face (two clips tested with weights trained only on the six older clips), the hybrid was best where the elbow slides (8.5 cm, against 13.6 cm for Learned Position and 26.8 cm for freezing). Where the forearm itself carries the reach, plain FK with the elbow held was slightly better (7.9 against 9.7 cm).
- Smoothing was measured before anything was added: the raw output was already smoother than the camera's own wrist track (jitter 0.57 against 0.90 cm). A One-Euro filter, an adaptive low-pass filter, is applied to the elbow part only. It cuts the jitter to 0.31 cm for a 0.16 cm accuracy cost (14.48 to 14.64 cm on 98 unseen 12-s blind spans). Caching the static drawing layers cut redraw time from 21 to 10 ms. The depth strip and CSV logging were added, and the logs are analysed with analyze_hybrid_log.py.

→ In detail: Section 7.8 — Hybrid Position

2.24 ✦ Fusion Studio

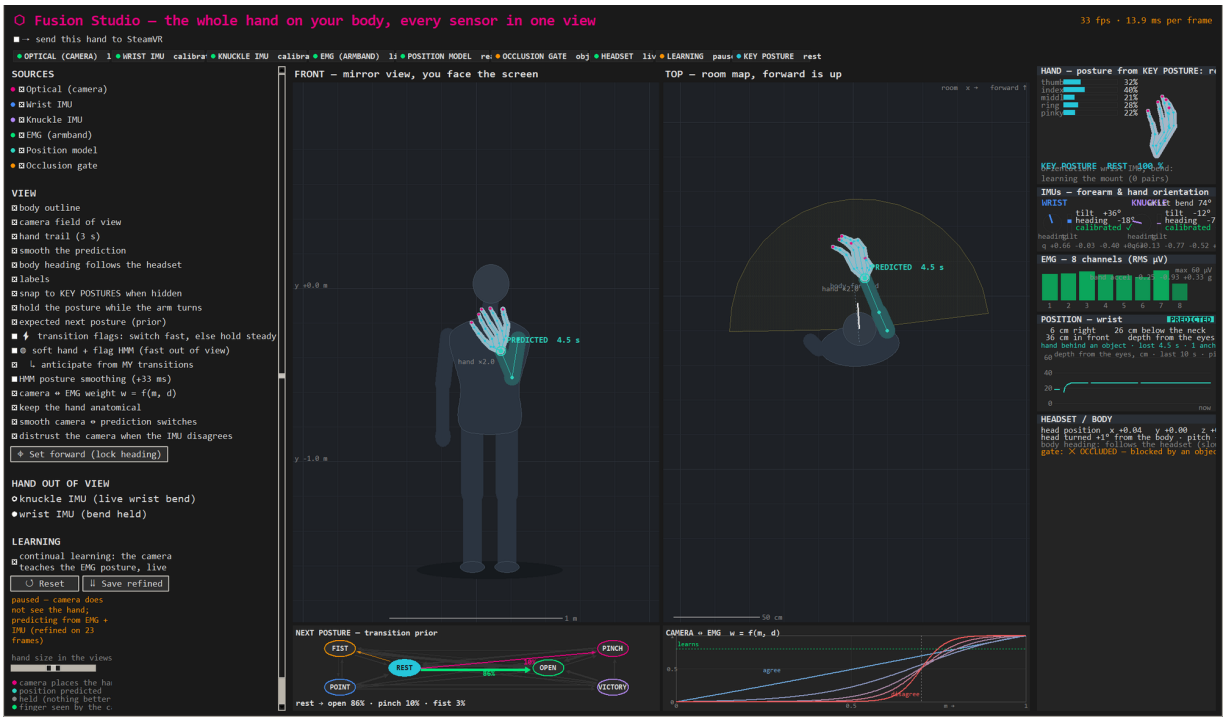


Figure 2.25 *The Studio during a replay of the synchronised session low_new3 from t = 120 s, run through the tab's own update loop. The session was recorded on 19 September 2026 and holds the Quest 3 hand joints, the headset pose, the CyberFinger's wrist-module and knuckle IMUs, and MindRove EMG features. The camera still reports the hand inside its field of view (yellow wedge), but the occlusion gate rates it "blocked by an object". The wrist is therefore drawn as PREDICTED (4.5 s since the last trusted camera position), and the fingers are snapped to the REST key posture.*

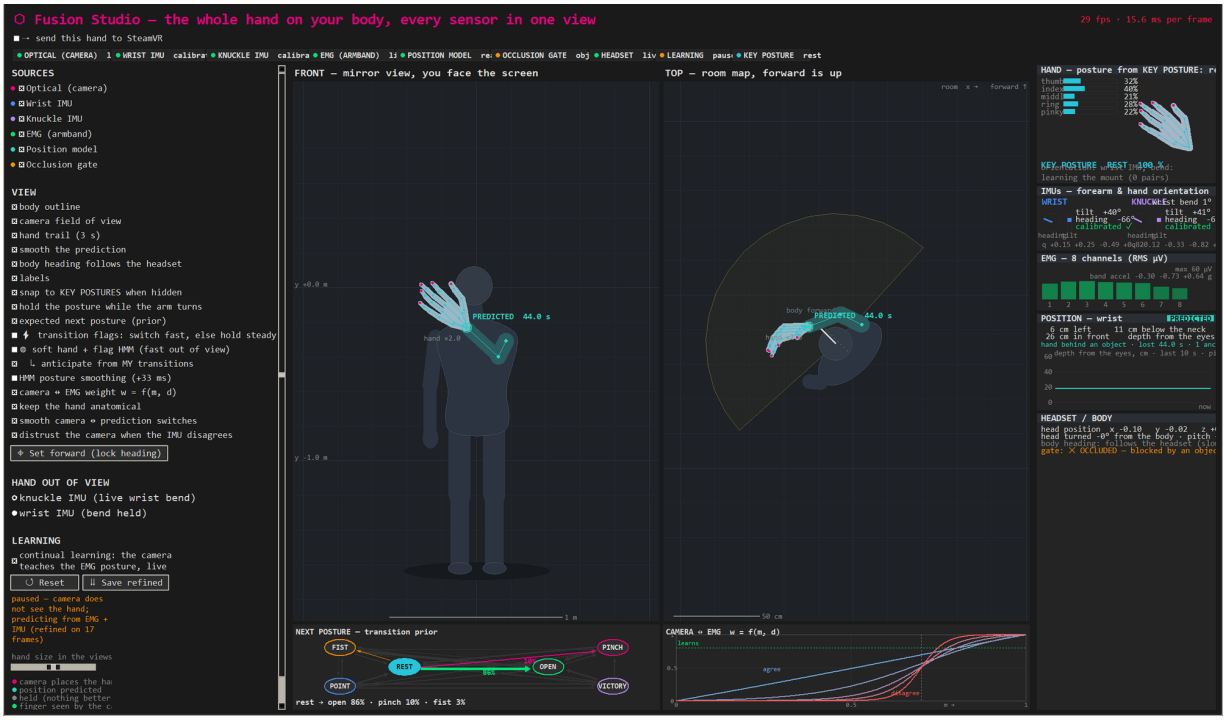


Figure 2.26 *Another stretch of the low_new3 replay, from t = 60 s. The camera was hidden on purpose (a forced loss) 2.5 s before the screenshot, so the tab shows its out-of-view prediction. The wrist is PREDICTED 6 cm left, 11 cm below the neck and 26 cm in front. The label's 44.0 s counts from the last camera position the Studio trusted, not from the forced loss. The CyberFinger's wrist module turns the hand, because the knuckle sensor's mounting is still being learned ("learning the mount (0 pairs)"). The fingers stay in the REST key posture, and continual learning is paused.*

What it is for. The ✦ Fusion Studio tab is the project's showcase. It is one screen that draws the user's body, placed and turned by the headset, together with the tracked arm and the whole 26-joint hand at its estimated position, orientation and finger posture, using every sensor at once. Its job is to keep the hand moving when the Meta Quest 3 camera loses it: the wrist position then comes from the Hybrid Position model, the hand orientation and wrist bend come from the CyberFinger's IMUs, and the finger posture comes from the MindRove EMG armband. It was needed because each earlier tab answered only one of these questions, and because out of view the CyberFinger driver only keeps turning the hand, while its position waits where the headset last saw it.

On the screen

- **Header**: the title ⬡ **Fusion Studio — the whole hand on your body, every sensor in one view** and, at the right, a live readout of "fps · ms per frame" (green at 50 fps or more, orange from 30, red below). Under it is the switch ⇢ **send this hand to SteamVR** (off by default). It sends the Studio's fused hand to the CyberFinger driver, which needs VR Mode and ▶ Start on the Gloves tab, and its status line shows the driver's mode and the send rate. Short orange messages, such as a missing headset pose, appear at the right of the same row.
- **Status chips**: one chip for each part of the system: OPTICAL (CAMERA), WRIST IMU, KNUCKLE IMU, EMG (ARMBAND), POSITION MODEL, OCCLUSION GATE, HEADSET, LEARNING and KEY POSTURE. Each chip has a coloured dot and a short state, for example live, calibrated or uncalibrated, missing, off, distrusted, ready, the gate's verdict (such as object), paused, or the recognised posture. In the screenshots, which were taken with enlarged text, the end of each chip's state is hidden under the next chip.

- **SOURCES**: six switches, each with its colour dot, all on by default: **Optical (camera)**, **Wrist IMU**, **Knuckle IMU**, **EMG (armband)**, **Position model** and **Occlusion gate**. Switching a source off leaves it out of the fused hand, so each source's contribution can be shown on its own. With the gate off, the camera's hand is no longer checked for occlusion.
- **VIEW**: first come six switches, all on by default. **body outline**, **camera field of view**, **hand trail (3 s)** and **labels** choose what is drawn. **smooth the prediction** smooths the predicted wrist position, and **body heading follows the headset** lets the body turn slowly with the head. The fusion switches follow. These are on by default: **snap to KEY POSTURES when hidden**, **hold the posture while the arm turns**, **expected next posture (prior)**, ↳ **anticipate from MY transitions**, **camera ↔ EMG weight w = f(m, d)**, **keep the hand anatomical**, **smooth camera ↔ prediction switches** and **distrust the camera when the IMU disagrees**. Three experimental options that still await live trials are off: ⚡ **transition flags: switch fast, else hold steady**, ◍ **soft hand + flag HMM (fast out of view)** and **HMM posture smoothing (+33 ms)**. An HMM (hidden Markov model) is a model of how postures follow one another over time. The button ⌖ **Set forward (lock heading)** locks the body heading to the direction the user faces at that moment and switches off **body heading follows the headset**.
- **HAND OUT OF VIEW** and **LEARNING**: two radio buttons choose what turns the hand when the camera cannot see it. With the default, **knuckle IMU (live wrist bend)**, the knuckle sensor turns the hand, so the wrist bend stays live. This needs the camera to have taught the knuckle sensor's mounting first, and until then the wrist module turns the hand. With **wrist IMU (bend held)**, the wrist module turns the hand and the last wrist bend the camera saw is kept. The switch **continual learning: the camera teaches the EMG posture, live** (on by default) lets every frame in which the camera clearly sees a finger refine the EMG finger-curl model, which is shared with the Pose Hand tab. ↺ **Reset** returns to the trained model and ⇊ **Save refined** saves the refinement into it. A status line says whether learning is running or paused and how many frames the model has been refined on. Below are the **hand size in the views** slider (1–3×, default 2×) and a colour legend: pink = the camera places the hand, teal = position predicted, grey = held, and green, orange or red for a finger that is seen, partly hidden or hidden (and so taken from the EMG).
- **FRONT — mirror view, you face the screen**: the user's body silhouette at real-world scale, with height lines every 20 cm labelled in metres and a 1 m scale bar. The headset's space puts y = 0 near eye level. The neck is placed from the headset. The view shows the tracked arm from shoulder to elbow to wrist and the 26-joint hand, enlarged about the wrist ("hand ×2.0") so the posture stays readable. The untracked arm hangs at the side. The arm and the wrist ring are coloured by where the position comes from, and a label reads CAMERA, PREDICTED (with the seconds since the camera was lost) or HELD. A fading 3-s trail shows the recent path.
- **TOP — room map, forward is up**: the same scene from above, on a 20 cm room-fixed grid with a 50 cm scale bar. It shows the body outline, the head with a line for where the headset looks, a dashed "body forward" line, the camera's field of view as a yellow wedge, and the arm and hand at their estimated position.
- **NEXT POSTURE — transition prior** and **CAMERA ↔ EMG w = f(m, d)**: small versions of the Fusion Logic tab's diagrams, under the two stages. The first is a map of the key postures (FIST, REST, OPEN, PINCH, POINT, VICTORY). The more likely a transition is, the thicker its arrow. These chances come from the public emg2pose dataset and the user's own sessions. The arrows leaving the current posture are coloured, and the three most likely next postures are written underneath (for example "rest → open 86% · pinch 10% · fist 3%"). The second plots w, the weight given to the camera for each finger, against m, how well the camera sees that finger. It has one curve per level of camera–EMG disagreement d (blue = they agree, red = they disagree). Each finger's live point is marked; above the green *learns* line (w = 0.8), the camera teaches the EMG.

- **HAND — posture from …, IMUs — forearm & hand orientation, EMG — 8 channels (RMS µV)** (right column, top). The HAND panel shows a close-up of the hand turned to its estimated orientation and five bars with each finger's curl (0–100%). The curl is the EMG's estimate, or, when the hand is snapped to a key posture, that posture's own curls, so the bars match the drawn hand. While the camera sees the hand, the bars are green, orange or red by how well it sees each finger; otherwise they take the colour of the posture source. The panel also shows the active key posture with its probability. An "orientation:" line names the source of the hand rotation (camera, knuckle IMU, wrist IMU or held) and gives the wrist bend, or the progress while the camera is still teaching the sensor mounting. The IMUs panel covers the WRIST and KNUCKLE sensors. For each, it shows a compass of the heading relative to body forward, a tilt bar, the tilt and heading in degrees, calibrated ✓ or uncalibrated, and the raw quaternion. It also gives the wrist bend between the two sensors. The EMG panel has eight bars of muscle activity, each the RMS (root-mean-square) amplitude of one channel, with the scale and the armband's accelerometer ("band accel … g").
- **POSITION — wrist** and **HEADSET / BODY** (right column, bottom). The POSITION panel has a badge naming the source (CAMERA, PREDICTED or HELD). It gives the wrist's position relative to the neck in the body frame (cm right or left, above or below, in front or behind) and its depth from the eyes. While predicting, it adds a line with four items: the reason the camera was lost, the seconds lost, the number of anchors (camera frames just before the loss that the prediction starts from). The fourth is the motion gate, which measures how much the forearm has moved since the loss and scales the learned correction. Last is a 10-s depth trace, pink for the camera and teal for the prediction. The HEADSET / BODY panel shows the head position in metres, how far the head is turned from the body and its pitch, and whether the body heading follows the headset or is locked. It also shows the occlusion gate's verdict, for example "gate: ✕ OCCLUDED — blocked by an object".

What was done

- Added on 2026-09-24 (commit ad1ae80) as an all-in-one showcase: the whole hand, position and posture together, attached to a body seen from the front and from above, with every signal on screen and an on/off switch for optical, IMU, EMG, position and gate. The work was strictly additive. One file changed (629 lines added), and the existing tabs, models and weights were checked to be byte-identical.
- Brings the project's separate results together. Out of view, the wrist **position** comes from the Hybrid Position model (forearm kinematics plus a learned model of elbow motion). The hand **orientation** and wrist bend come from the CyberFinger's wrist module and knuckle sensor, through a mounting calibration that the camera teaches. The **finger posture** comes from the 8-channel EMG, either as a continuous curl estimate or by snapping to a recognised key posture.
- A body model built from the headset. The neck sits at the headset position plus R·(0, −0.10, 0.08) m, where R is the head's rotation. The body heading is a slow average of the head's horizontal direction (time constant 2.5 s) unless ⌖ Set forward locks it. The elbow is placed 0.26 m back from the wrist along the forearm. The legs are drawn a fixed 0.85 m below the hips, because the OpenXR space used by Live Preview is head-local and gives no floor.
- Fills a gap in the product. The CyberFinger manual v2.1 says that out of view the hand keeps turning with the joint IMU while *"its position waits where the headset last saw it"*. The Studio adds the out-of-view position and the finger posture. The v2.1 manual does not yet describe the Studio or the EMG armband, although release v2.1.0 ships them.

- Later additions. Since 2026-10-09 (commit a05aa16), small versions of the Fusion Logic diagrams (the posture transition map and the camera ↔ EMG weight curves) sit under the stages and are redrawn about 20 times a second. The left column now scrolls when the window is too short (commit eb08998): it needs about 960 px and had been cut off at 1280×720.
- On a replay of the recorded session low_new3 through the real Studio update, 0.9 s into a simulated camera loss, the POSITION panel placed the wrist 19 cm right, 13 cm below the neck and 32 cm in front (“lost 0.9 s · 4 anchors · motion gate 99%”). In a render made on 2026-10-09 from key-posture clips, before REST was added, the transition map showed that a fist was usually followed by an open hand (88%).

→ In detail: Section 9.1 — ✦ Fusion Studio

2.25 Key Postures

Figure 2.27 *The Key Postures tab during a 25-second replay of the transition recording of 22 October 2026: recorded raw MindRove EMG and CyberFinger IMU data were fed through the real GUI, with the camera hidden so that the EMG alone decides. The current seven-class recogniser shows OPEN at 87% and nothing has been taught today. The camera panel shows its placeholder, and the dataset and held-out score panels on the right were blank in this capture; their titles are cut off at the window edge. The frame and clip counts on the left are those of the recorded dataset.*

What it is for. The Key Postures tab teaches the system to recognise a few important hand postures (open, fist, point, pinch, victory and rest) from forearm EMG alone. EMG is the electrical activity of the forearm muscles, which the MindRove armband picks up. The tab was needed because the headset camera loses the hand at the hip, at the side and behind the back, and cannot see a closed fist, while the EMG keeps streaming. The ✦ Fusion Studio tab uses this recogniser to rebuild the hidden hand in the right posture. The wearer labels each clip by holding a chosen posture, so no camera is needed for the labels, and clips can be recorded exactly where the camera cannot see.

On the screen

- ⬡ **Key Postures — recognise your important postures from EMG, rebuild them out of view**: the title and a short guide. It says that ● **Record** captures 12 s under the chosen label without the camera, that ⚙ **Train** scores the recogniser on the newest clip of each posture (a clip it never trained on), and that the Fusion Studio uses the result when the camera cannot see the hand. The guide still mentions wrist tilt, but that input was dropped on 2026-10-01 and its inputs are now fixed at zero.
- **POSTURE**: seven radio buttons (open, fist, point, pinch, victory, rest, other), each with its colour dot. Next to each one is the number of frames and clips already recorded for it (here, for example, open 3589 frames in 10 clips). The selected posture becomes the label of the next clip. The guide asks for *other* to be recorded with the hand relaxed and while it moves between postures; *other* never snaps the drawn hand to a posture.

- **ARM POSITION while recording**: radio buttons for front, hip, side, back and moving. This second label is stored with each clip. The dataset grid counts frames per arm position, the held-out report names the arm position of each tested clip, and the label made it possible to test the recogniser on arm positions it was not trained on.
- ● **Record 12 s**, 🗑 **Archive last clip**, ⚙ **Train recogniser**: Record captures EMG, both CyberFinger IMUs (wrist module and knuckle sensor) and, when the camera sees the hand, its 26 joints. During capture the button counts down ('■ recording… N s left'), and pressing it again stops early and saves the clip; a clip with fewer than 60 frames is discarded. Archive moves the newest clip into an archive folder, so the step can be undone. Train fits the v2 recogniser (about one minute), writes the held-out score in the status line under the self-teaching controls and refreshes the report on the right.
- **SELF-TEACHING**: the checkbox **the camera teaches today's EMG while it sees a posture** (on by default) turns on live fine-tuning from the camera's geometric posture labels. ↺ **Forget today's teaching** returns to the stored, trained model. Below this are the status line and **HOW TO RECORD**, a five-step recording procedure.
- **LIVE — what the recogniser sees right now**: on the left, the posture the decision gate has committed, in large letters with its probability (here **OPEN**, 'active · 87 %'). The gate shows a posture only after it has clearly led for a short time. Without a committed posture, this line names the current leader and says it is not confident enough yet. Under it are the self-teaching status (here 'self-teaching ON — nothing taught yet') and sensor checks for EMG, wrist IMU and camera. On the right, one bar per posture shows its live probability. A dashed line marks the 75% a posture must exceed before the gate considers it.
- **RECOGNISED POSTURE — its template hand**: a 26-joint hand in the committed posture, drawn in that posture's colour. For most postures the template is the camera's median shape from the recordings. The fist uses an anatomical model, because the camera cannot see a closed fist. This is the hand the Studio shows when the camera loses sight of it.
- **CAMERA (when it sees the hand)**: the hand as the headset camera tracks it, drawn in green for comparison with the template. When the camera has no hand, as in this capture, the panel shows 'show your hand + armband on'.
- **TIMELINE — recognised posture over the last 30 s**: a strip coloured by the committed posture over time, marked from −30 s to 'now'. In this capture it stays on open.
- **DATASET — frames per posture × arm position** and **LAST TRAINING — held-out score (newest clip of each posture)**: the first is a grid of recorded frames per posture and arm position, shaded greener as the count grows. The second shows the model type and training date, the counts of real and synthetic frames, and the overall held-out accuracy. Under that are one bar per posture (green at 90% or more, orange at 70% or more, red below) with the arm position of the tested clip, a confusion grid (rows = true posture, columns = recognised posture) and the source of each template. Both panels were blank in this capture.

What was done

- **Hold-to-label recording with an honest report.** Each 12-second clip is labelled with a posture and one of five arm positions. The recogniser is always scored on the newest clip of each posture, which it never trained on. The tab was built and checked with simulated sensors on 2026-09-24 and first committed on 2026-09-25.

- **v1 recogniser** (40 standard time-domain EMG features plus a 3-number wrist tilt, with a linear softmax model). In the first real session (25 clips) it scored **76%** on the newest clips. In leave-one-position-out (LOPO) testing, where the model is trained without one arm position and tested on it, it scored 79%, and the hip was hardest at 58%. The template hands were replaced by the camera's median shape of each posture, so a pinch now closes to a 1.5 cm gap between thumb tip and index tip.
- **v2 with camera self-teaching**, which adds channel-covariance features, a 64-unit neural network (MLP) and live fine-tuning from the camera's labels while it sees the hand. A second session about six hours after the first had dropped v1 to 2% on open. On held-out second-session clips, accuracy rose from **45% (v1) to 86%** (offline refit). On frames the camera could not see, measured through the real GUI loop, it rose from 50% to 77%.
- **A faster, steadier decision gate.** A fast path for very confident postures and a shorter 96-ms EMG window cut confident switches from 167 ms to **67 ms median** (100 ms at the 90th percentile). Dropping the heading-dependent tilt input raised LOPO accuracy from 88.7% to 91.4%. Together with a rule that holds the posture while the arm turns, it cut wrong switches while a posture is held from 12.1 to 6.5 per minute.
- **Transition prior.** Which posture tends to follow which was estimated from Meta's public emg2pose data, relabelled with the project's own camera labeller. Most changes pass through open. Two ✦ Fusion Studio switches use it. The optional hidden-Markov filter (**HMM posture smoothing**) cut simulated wrong switches from 19.3 to 5.8 per minute, but it adds delay, so it ships switched off. The **expected next posture (prior)** rule, on by default, cut false flashes of point or victory while a fist opens from **30% to 0%** through the real Studio loop.
- **REST, a sixth posture** (2026-10-23): the relaxed hand, recognised 83% of the time at arm positions it was not trained on. The current seven-class model (16,661 real and 99,966 synthetic rows) scores **75.7%** on 2,412 held-out frames: fist 98.7%, pinch 98.1%, rest 93.5%, point 88.3%, victory 86.5% and open 75.8%. The *other* class scores 0.3%: its held-out clip was read as rest on 363 of 398 frames, because the old *other* recordings look like a relaxed hand.

→ In detail: Section 8.2 — Key Postures

2.26 ◈ Fusion Logic

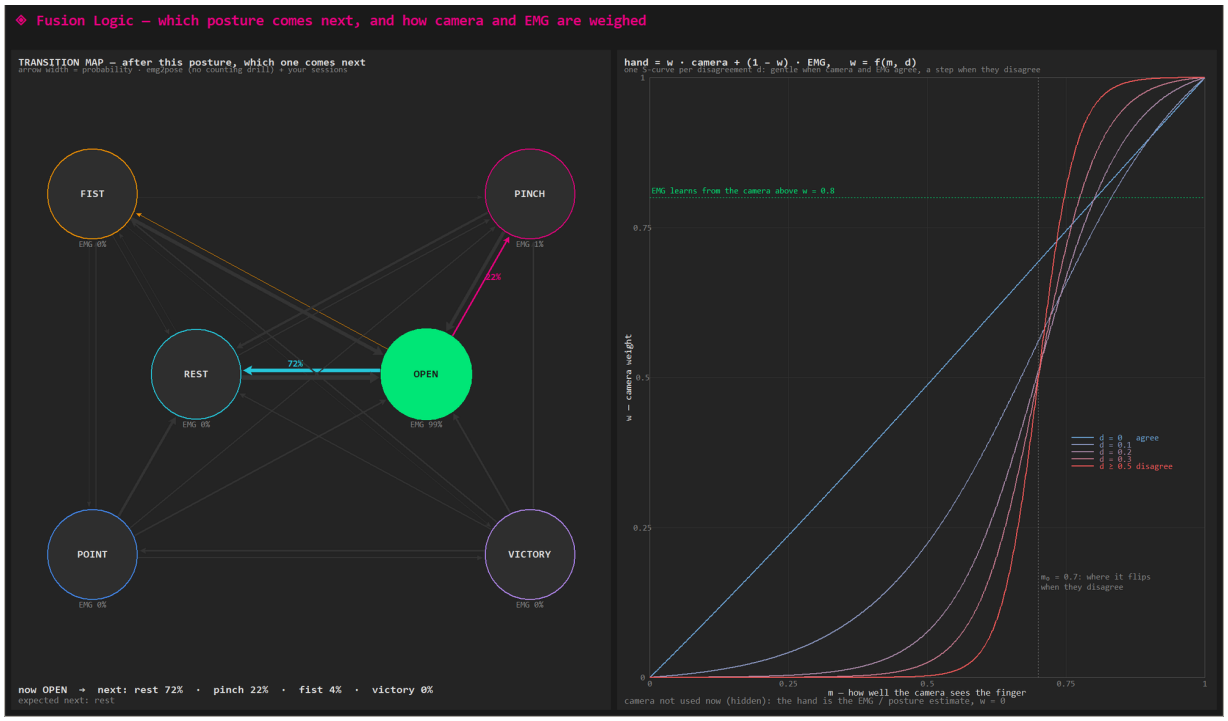


Figure 2.28 *The ◈ Fusion Logic tab about 12 s after it was opened, during a replay of the recorded transition session with the camera hidden; this is not live hardware. On the left, OPEN is the recognised posture (EMG 99%); its most likely successor is rest at 72% (then pinch 22%), and the gate's line reads "expected next: rest". On the right, the five weight curves are drawn without finger dots, because the hidden camera leaves the hand to the EMG / posture estimate (w = 0).*

What it is for. The ◈ Fusion Logic tab draws, live, the two rules that drive the Fusion Studio's decisions, so a reviewer can see why the hidden hand looks the way it does. The first rule is the transition prior: which posture usually comes next. It also shows how the posture gate (the step that decides which posture is shown) uses that prior to hold back shapes that only flash by on the way. The second rule is how much the camera and the EMG (the forearm muscle signals from the armband) are each trusted for every finger. The Studio's output alone does not show why it chose a hand, so this tab makes both rules visible as they act.

On the screen

- ◈ **Fusion Logic — which posture comes next, and how camera and EMG are weighed** — the tab's title. A thin status line under it stays empty unless a drawing error occurs. The tab has no buttons. About 20 times per second it redraws what the Fusion Studio and the posture gate computed in that frame, and while the tab is open the Studio keeps computing in the background.
- **TRANSITION MAP — after this posture, which one comes next** (left panel) — the heading, with the subtitle *arrow width = probability · emg2pose (no counting drill) + your sessions*. The subtitle adds "prior switched OFF in the Studio" when the Studio's **expected next posture (prior)** switch is off.
- **Posture nodes** (FIST, PINCH, REST, OPEN, POINT, VICTORY) — one circle per posture, each outlined in its own colour. OPEN sits right of centre as the hub, and REST sits between FIST and OPEN. The posture the recogniser shows now is filled in its colour (OPEN, green, in the screenshot). Under each node, **EMG nn%** gives the EMG recogniser's live probability for that posture.
- **Transition arrows** — one arrow for each transition with a probability of at least 1.5%. The two directions between a pair of postures are drawn side by side, and an arrow gets thicker as its probability rises. Arrows

leaving the current posture take the colour of their target posture, and those of 5% or more carry their percentage (in the screenshot, 72% towards REST and 22% towards PINCH; the 4% arrow to FIST is orange but unlabelled). The other arrows are dimmed grey. With no posture recognised, all arrows are mid-grey and those of 5% or more are labelled.

- **now … → next: …** (bottom of the left panel) — the current posture and its four most likely successors with their probabilities (in the screenshot, "now OPEN → next: rest 72% · pinch 22% · fist 4% · victory 0%"). When the Studio's prior switch is on, the gate's reasoning follows beneath it: **expected next: …** and, when there are any, **held back on the way there: …**. The latter lists the postures whose finger curls lie between the current and the expected posture; the gate makes them lead for longer before it shows them, so they do not flash by on the way (REST is never held back). When the EMG power drops while open is the expected posture, a green **relaxing → opening** appears at the bottom right. With no posture recognised, the panel says "no key posture recognised right now", or, if the Studio has produced no frame yet, asks you to open the Fusion Studio sources to see it live.
- **hand = w · camera + (1 − w) · EMG, w = f(m, d)** (right panel) — the heading of the camera↔EMG weight plot, with the subtitle *one S-curve per disagreement d: gentle when camera and EMG agree, a step when they disagree*. The subtitle adds "switched OFF in the Studio" when the Studio's **camera ↔ EMG weight w = f(m, d)** switch is off.
- **Axes and S-curves** — horizontal axis **m — how well the camera sees the finger** (0 to 1); vertical axis **w — camera weight** (0 to 1). Five curves, for d = 0, 0.1, 0.2, 0.3 and ≥ 0.5, are computed from the same weight function the Studio uses, shading from blue to red. The legend runs from **d = 0 agree** (a near-straight line, w ≈ m) to **d ≥ 0.5 disagree** (almost a step).
- **Dashed guide lines** — a green horizontal line labelled **EMG learns from the camera above w = 0.8** (the EMG model is taught by the camera only above this weight) and a grey vertical line labelled **m_0 = 0.7: where it flips when they disagree**.
- **Live finger dots and values** — while the camera contributes, the five fingers appear as coloured dots marked T, I, M, R and P at their current (m, w). A readout in the upper left of the plot gives each finger's name with its "m … d … w …" values. In the screenshot the camera is hidden, so no dots are drawn.
- **Camera status line** (bottom of the right panel) — one of three messages: "camera sees the hand: every finger weighed live"; "another hand on top: camera fingers at their lowered m"; or, as in the screenshot, "camera not used now (hidden): the hand is the EMG / posture estimate, w = 0".

What was done

- Built the tab (commit efe2c08, 2026-10-09) to draw live the two rules behind the Fusion Studio's decisions: my supervisor's transition prior and how the gate uses it, and the per-finger camera↔EMG weight. While the tab is open, the Studio keeps computing in the background.
- The transition map combines the public emg2pose dataset (without its counting drill) with my own sessions, and open is the hub. Since 2026-10-23, rest has its own node between fist and open. After rest the hand opens 86%; after a fist it opens 79% or rests 10%; after open it rests 72%. In the earlier 5-posture prior, a fist was followed by open 88% of the time, with pinch, point and victory held back on the way.
- Implemented my supervisor's whiteboard rule for the camera↔EMG weight on 2026-10-08 (commit 18d60db). For each finger, the drawn hand is w·camera + (1−w)·EMG with w = f(m, d), where m is the camera's reliability for that finger (its ray-cast visibility, lowered further by the occlusion gate) and d is the curl difference between the camera and EMG estimates. The S-curve's steepness rises from 1 when the two agree to 30 at d ≥ 0.5, where it becomes a step at m0 = 0.7: if they disagree, the camera is trusted fully above 0.7 and not at all below. The weight is smoothed over 60 ms, and the EMG learns from the camera only where w > 0.8.

- Measured through the real Studio tick on 40 held clips, using EMG models that had not seen the clip's arm position, the rule raised the share of frames showing the right posture from 95.8% to 96.8% and lowered the frames showing no posture from 3.0% to 2.0%. A later anatomy fix moved posture accuracy on the same 40 clips from 96.8% to 96.6%.
- A step at m0 = 0.5 was tested and rejected. With partly hidden fingers, the share of frames with the right posture fell from 93.2% to 92.1%, because a partly hidden camera that disagrees with the EMG is right only about 70% of the time.
- A compact copy of both drawings, using the same drawing code, sits under the Fusion Studio's front and top views (commit a05aa16). It is redrawn about 20 times per second, and the Studio stayed at 60–62 frames per second with it. The drawing code was then added to the generator of the single-window release (commit cf71b91), so SciCortex's Fusion Studio shows it too.

→ In detail: Section 8.3 — ◈ Fusion Logic

2.27 ⚡ Transitions

Figure 2.29 *The ⚡ Transitions tab, captured about 30 s after the tab was opened, during a replay of a recorded transition session. The raw EMG was fed through the application with the camera hidden, and the trained learned detector was flagging at P > 0.93. The camera lane and the camera sees field are empty because the camera was hidden. The statistics count only the 10 flags raised so far in this replayed stream, so they illustrate the display and are not a test result.*

What it is for. The ⚡ Transitions tab watches the forearm EMG (electromyography: the electrical activity of the forearm muscles, picked up by the MindRove armband) for the short burst that starts every posture change. It raises a flag before the fingers move, guesses where the hand is going, and checks each flag against the camera and the EMG gate (the stabilised posture decision of the recogniser from the Key Postures tab). It exists because the EMG gate drives the hand once it leaves the camera's view, and the gate is slow and flickers, so a reliable early flag can make that hidden hand move sooner without adding flicker. The tab also records my own transitions, labelled by the camera, to train and test the detector. The idea came from one of my supervisor's whiteboard sessions.

On the screen

- ● **Record session** and ↺ **Reset**: the record button starts recording the continuous EMG, the EMG gate's posture, the camera's hand and posture label, and the orientations of both CyberFinger IMUs (wrist module and knuckle sensor). While recording it reads ■ **Stop & save (N s)**. Pressing it again saves everything, with the session's flags, to a tx_*.npz file in bridge/transition_logs. ↺ **Reset** clears the timeline and the statistics but keeps the burst patterns learned so far.
- **burst threshold** slider (0.6 to 2.5, default 0.90) and the **veto while the forearm turns > 60 °/s (wrist IMU)** checkbox (on by default). When the learned detector is not in use, the slider sets the level the plain change score must cross to raise a flag. With the veto on, a flag raised while the CyberFinger's wrist IMU shows the forearm turning faster than 60°/s is marked as vetoed and not acted on, because turning the forearm changes the EMG on its own.

- ⚙ **Train from recordings** and **flag with the learned detector (else the burst threshold)**: the button trains the learned detector in the background on every recording in bridge/transition_logs, saves it, and hands it to the live tracker when training finishes. The checkbox (ticked by default) chooses between that detector and the plain threshold. The green line beside them states what the detector was trained on and how it scored on recordings it never saw. Here it reads 6 recordings and 190 transitions, with a flag when the detector's probability P of a change exceeds 0.93: 75% caught at 2.2 false flags per minute, against 63% at 1.8 for the burst threshold. The grey line underneath gives the stream rate: *EMG 500 Hz × 8 ch · a score every 10 ms*.
- **LAST 8 s — the change score crosses the line → a flag**, top lane: a heat map of the change in each of the 8 EMG channels (ch0 to ch7), red where activity rises and blue where it falls (*EMG change per channel — red: rises, blue: falls*). Small changes stay dark, so bursts stand out.
- Flag-signal lane, labelled **learned P(change)** (or **change score** in threshold mode): a white trace of the flag signal, with an orange dashed line at the flag level (*dashed: flag at P > 0.93*). Each flag is a vertical line through the heat map and this lane, coloured by its outcome and labelled with the transition: *open→fist* ✓ when the predicted destination was right, *(said …)* when it was wrong, or *no change* or *veto*. The legend at the bottom of the timeline reads: green changed, grey no change, brown vetoed, white waiting (not yet settled), ▼ change without a flag.
- **EMG gate** and **camera** lanes, then **forearm turn °/s**: coloured bands show which posture the EMG gate and the camera reported over time. A red ▼ marks a posture change with no flag in the second before it. The bottom lane plots the wrist-IMU turn rate against the brown dashed 60°/s veto line. In this screenshot the camera lane is empty because the camera was hidden.
- **NOW** panel: the EMG gate's current posture in large coloured letters (here PINCH), what the camera sees (here —), and a round transition lamp. The lamp is grey with *holding*, lights yellow with *TRANSITION?* for 0.6 s after a flag, and turns brown with *turning* if that flag was vetoed. Under **last flag: victory → ?**, each bar is one possible destination. Its length is the transition prior (how often each posture follows the one the flag started from, counting my own transitions as well) combined with the burst's channel pattern. A white tick on each bar marks the prior alone.
- **STATISTICS (this session)**: the number of flags and how they ended (posture changed, no change, vetoed), false flags per minute while a posture was held, the share of EMG-gate and camera changes that had a flag before them, how much earlier the flag came than the gate and the camera (median), and how often the next posture was guessed right by the prior alone and by prior × burst.
- **LATEST FLAGS — from → where it went · predicted · flag earlier than**: a list of the most recent flags, newest first. Each row gives the time since the tracker started, the starting posture, the posture the hand settled in, the prediction marked ✓ or ✕, and the lead in milliseconds over the gate (and over the camera when it saw the change). Vetoed flags show the turn rate instead. Rows use the same green, grey and brown outcome colours as the timeline.
- **BURSTS — one cluster per transition (first 20 ms) · mean score around flags**: on the left, each real change's burst pattern (the mean channel change over the first 20 ms after its flag), projected onto its two main axes of variation. Each dot is filled with the colour of the destination posture and ringed with the colour of the starting posture, and a list counts each transition type (shown after 3 real changes). On the right, the mean flag signal from −0.3 s to +0.6 s around flags: green for flags followed by a posture change, grey for flags with no change, with the threshold drawn in orange.

What was done

- Tested the idea before building the tab. The test used 249 transitions in the public emg2pose dataset and a causal change score (one that uses only past samples): each channel's log envelope is compared with its

own 300-to-30-ms past, the RMS is taken over the channels, and a new value is computed every 10 ms. At threshold 1.0 it caught **94%** of transitions, about 55 ms before the fingers moved and about 275 ms before the posture could be read from the hand shape. On about 6 minutes of my own MindRove holds with the arm still, it raised 12.8, 5.6 and 2.2 false flags per minute at thresholds 0.8, 0.9 and 1.0 (measured later, with the tab's 50-ms envelope). With the arm moving, most false flags came while the wrist turned faster than 60°/s, which led to the IMU veto.

- Built the tracker (bridge/transition_events.py) in three parts. **ChangeScore** computes the burst score. **DirectionModel** predicts where the hand is going: it multiplies the prior by the nearest burst pattern and learns online. **TransitionTracker** raises flags, applies the veto, settles each flag by the posture the hand settled in, and keeps the statistics. In a replay, settling by the gate's first switch would have matched the true destination only 24 of 35 times, against 35 of 35 for the settled posture. The veto reads the wrist IMU only, because the knuckle IMU turns at 60–105°/s when the hand closes and 4 of the 5 flags it vetoed were real.
- Made six recordings with the tab: about 409 s in total, with 190 camera-seen transitions. In the first recording every transition had a burst, with a median peak of 1.1 against a 99th percentile of 0.87 on holds. That recording set the new defaults: a 50-ms envelope and threshold 0.9. They flagged 20 of 36 changes at ≤ 2 false flags per minute, against 12 of 36 for the original 25-ms envelope at threshold 1.2.
- Replaced the fixed threshold with a learned detector: a logistic regression on 27 EMG-change features, labelled by the camera, with its decision level set for ≤ 2 false flags per minute on holds. It was tested leave-one-recording-out, meaning each recording was scored by a model trained on the other five. It caught **143 of 190 (75%)** at 2.2 false flags per minute, about 57 ms before movement onset. The plain threshold caught 120 of 190 (63%) at 1.8. With the wrist veto on, the figures were 74% against 60–63%. On the recording made with the arm turning strongly, the veto cost catches (5 of 16 with it, 9 of 16 without). The veto stays on but can be switched off.
- Fed the flags into the Fusion Studio's hidden hand through optional switches, which stay off by default until tried live. The Studio was run on the six recordings (190 changes) with the camera hidden. With the soft hand and anticipation from flags, the hidden hand followed the camera's own hand with a median lag of **45 ms instead of 230 ms**. It also flickered less while the forearm turned (4.8 instead of 20.1 switches per minute). Basing the anticipation on my own transitions instead of the general prior raised next-posture accuracy from 44% to 51%.
- One honest negative: on emg2pose, the first 20 ms of the burst predicted the next posture 66% of the time against 54% for the prior alone. On MindRove data, burst-based prediction did not yet clearly beat the prior: 6 (burst) vs 5 (prior) of 20 on recording 1, and 48% vs 48% on the replayed recording 3.

→ In detail: Section 8.4 — ⚡ Transitions

3 System architecture, instrumentation and data pipeline

This section describes the system built during the internship. Three sensors (the Meta Quest 3's optical hand tracking, a MindRove 8-channel forearm EMG armband and the CyberFinger's two inertial sensors) were brought into one Windows program on one clock, and the fused hand was sent back to SciCortex's SteamVR driver. It covers the Gloves, Armband and Capture tabs, the synchronised 175-column clip format, and what was measured on each stream: IMU reports at 99.4–100.0 per second with no lost packets, EMG at 500 Hz, and the headset marking every joint as tracked on 100% of frames even when the hand was hidden. It also covers the contributions to SciCortex's product: the FUSED link to the driver at 120 packets/s, the merge of release v2.1.0, and a driver fix for the SteamVR update of 2026-10-21 that SciCortex committed to its own main branch.

Every later result in this report depends on one piece of infrastructure. It is a program that receives the three sensor streams at the same time, puts them on one clock, displays and records them, and passes the fused hand on to VR applications. In the Mitacs proposal this is sub-objective 1, *Instrumentation & Baselines*, task 1.1, *analysis pipelines of synchronized data*. This section covers the hardware, the software that joins the streams, the logging format, and the work on SciCortex's SteamVR driver and bridge. It ends with the three tabs that form the base of the research GUI: Gloves, Armband and Capture.

The report keeps what SciCortex built separate from what was built during the internship. SciCortex wrote the original CyberFinger Bridge, the Bluetooth link with its report decoder, and the SteamVR driver, including the later version 2. At Bridge v1.4.0 (commit eaeadbe, 2026-07-28), the Bridge was a single window of 2,999 lines with no tabs. The research work branched from the commit just before it (24745fe, 2026-07-20, with the same GUI code) and added a layer on top of it. July and August work was first committed on 2026-09-17 in one large commit (c7704d3: 100 files, +15,961 lines, a 9,408-line GUI with 16 tabs). Dates before that come from the project notes and file timestamps. The commit history holds 56 of my commits on the research branch (21 in September, 35 in October) and 2 more on the release branch.

Table 3.1 Who built which part of the system

Component	Author	Reference
CyberFinger hardware and firmware (incl. 1.3.2 IMU lag fix, 1.4.1)	SciCortex	STEAMVR_PLAN.md; manual v2.1
Bridge GUI v1.4.0, BLE link, report decoder	SciCortex	eaeadbe (2026-07-28)
SteamVR driver v2 and release v2.1.0	SciCortex	93359ca (2026-09-28)
Armband tab (MindRove dashboard embedded)	Internship	by 2026-08-03
OpenXR skeleton backend, ◉ Live Preview, Occlusion Test	Internship	July–August 2026
Capture tab and synchronised clip format	Internship	2026-08-19/20
Firmware-version read, IMU logger, joystick auto-centre, BODY 1 fallback	Internship	6ea858d and others
Merge of release v2.1.0 into the research branch	Internship	266731d (2026-09-29)
FUSED link from Fusion Studio to the driver	Internship	374b22b, e3be82e
Driver fix for the 2026-10-21 SteamVR update	Internship; committed upstream by SciCortex	c35efc5; d469b53

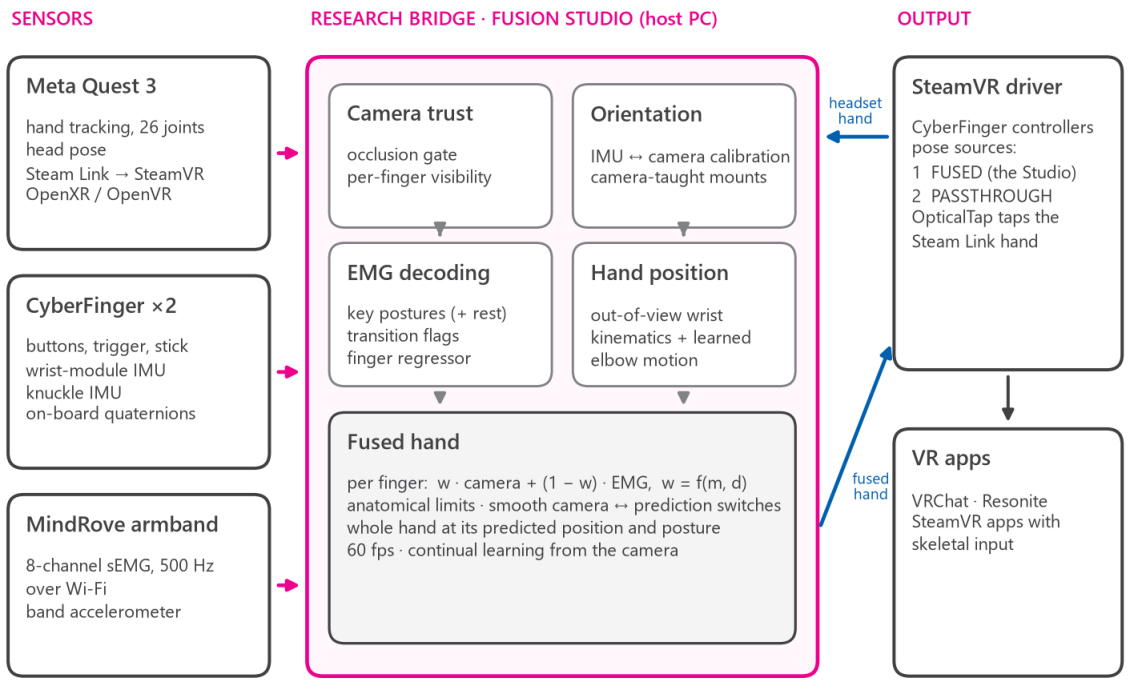


Figure 3.1 *System overview. Three sensor streams feed the research bridge and Fusion Studio on the host PC: Quest 3 hand tracking through Steam Link and SteamVR, two CyberFingers over Bluetooth, and the MindRove armband over Wi-Fi. The fused hand goes to the CyberFinger SteamVR driver as its FUSED pose source. The driver's tapped headset hand (PASSTHROUGH) is the fallback, and it is also the Studio's camera input.*

3.1 Hardware and sensors

The CyberFinger

The CyberFinger is SciCortex's hand-worn VR controller, one per hand (see the product render in the overview). A finger module on the index finger carries a push-stick, a trigger and a row of buttons. A wrist module carries a screen, a black button, a pink power button and the "body" IMU. An inertial measurement unit (IMU) combines an accelerometer and a gyroscope to estimate orientation. Some hardware revisions add buttons C, D and E, a vibration motor, and a second IMU on the back of the hand, the "joint" IMU, which the research GUI also calls the knuckle IMU. The device charges over USB-C.

The proposal described the CyberFinger as a ring on the index finger, with "wrist orientation from a SlimeVR IMU embedded in the CyberFinger". The lab units carry two IMUs instead: one in the wrist module and one on the back of the hand. That difference later made it possible to track the forearm and the hand separately.

Figure 3.2 *The wearable sensor set used throughout the project (2026-08-25). On the index finger is the CyberFinger finger module. On the back of the hand is the pink joint IMU module (ICM-45686). On the pink strap is the wrist module (screen, QMI8658 body IMU). On the forearm is the MindRove 8-channel EMG armband.*

What the CyberFinger IMUs can and cannot measure

I checked the firmware source and wrote up the IMU pipeline in docs/firmware-and-imu.md. Each sensor runs its own VQF filter, which fuses accelerometer and gyroscope data into an orientation, and sends the result as a quaternion (four numbers that encode a 3D rotation). Pitch and roll are referenced to gravity, so the two sensors agree on them. Heading (yaw) cannot be observed without a magnetometer. Each sensor's yaw starts at an arbitrary zero at power-on and drifts on its own. Neither the firmware nor the bridge applies any mounting or cross-sensor calibration; the quaternions are forwarded unchanged. Conventions are [w, x, y, z], Hamilton product, q = earth←sensor. The accelerometer arrives as raw int16 at 2048 LSB/g (±16 g), and the gyroscope is not transmitted.

Table 3.2 The two CyberFinger IMUs (source: docs/firmware-and-imu.md)

Property	BODY (wrist module)	JOINT (back of hand)
Chip	QMI8658, onboard	ICM-45686, I2C 0x68
Sensors	Accel + gyro (6-axis), no magnetometer	Accel + gyro (6-axis), no magnetometer
Orientation	Own VQF filter (6D quaternion)	Own VQF filter (6D quaternion)
Heading reference	None: arbitrary at power-on, drifts	None: drifts independently

The practical consequence runs through the rest of the project. The rotation of the hand relative to the wrist, conj(q_body)⊗q_joint, is reliable in tilt only, and its yaw is corrupted. Every later orientation method therefore had to recalibrate against the camera whenever the hand was visible.

Firmware versions. The bridge decodes a different report layout for each firmware version. The fixed reports are 12, 28, 61 or 79 bytes. From firmware 1.3.0 a variable-length IMU tail is appended, giving a 57-byte report for BODY 1 + JOINT, or 58 bytes with the pink-button extension byte added in 1.3.3. The old 1.0.1 build has no IMU, firmware 1.3.0/1.3.1 added the IMUs, and the bridge gained IMU support in v1.4.0/1.4.1 (both tagged 2026-07-28). To know exactly what each unit runs, I added a read of the Bluetooth Device Information Service (0x180A, Firmware Revision String 0x2A26), so the GUI logs the version at every connect. SteamVR driver 2.1 requires firmware 1.4.1, so both units were flashed on 2026-09-30 to 1.4.1-beta+f48c87f. Before flashing, the units reported 1.1.1-beta+d4f3bcb over Bluetooth. That string does not fit the documented version line, because IMU support starts at 1.3.0 and these units had streamed IMU data all summer. An August note had assumed 1.3.x. The discrepancy remains unexplained.

The flashing produced two practical lessons. Two USB-C cables were faulty: one carried charge only, and the other gave "Device Descriptor Request Failed". Windows also caches Device Information values until the CyberFinger is removed and re-paired. Firmware 1.4.x moved the wrist QMI8658 from the BODY 2 slot to the primary BODY 1 slot (imu_present changed from 6 to 5). I made the research tabs fall back from BODY 2 to BODY 1 (commit 6ea858d). Recordings still write the wrist IMU into the same g2 columns, so every trained model stayed valid. SciCortex's firmware 1.3.2 also fixed an IMU lag. The body IMU had trailed the joint IMU by 31–38 ms because of a ~15 Hz low-pass filter, and after the fix it trailed by 3–5 ms (capture of 2026-09-27). The cost was a small rise in rotation jitter at rest, from 0.03° to 0.04–0.06°.

The MindRove armband

Surface electromyography (sEMG) records the electrical activity of muscles through electrodes on the skin. The muscles that move the fingers sit in the forearm, so forearm EMG carries information about finger posture even when the hand is outside the camera's view. The MindRove armband has 8 EMG channels and a 6-axis IMU (accelerometer and gyroscope, with no orientation output). It samples at 500 Hz with 24-bit resolution and streams over its own Wi-Fi network. Six Transitions recordings of 2026-10-22 confirm fs = 500.0 Hz, with 497.3–499.5 samples/s actually delivered. Two properties of the band shaped the Armband tab: only one program can own its Wi-Fi session at a time, and its network has no internet access. The EMG work started in the first week. My armband repository (first commit 2026-07-07) holds a 57-page developer guide to the MindRove SDK, checked against the SDK source, and the standalone dashboard that later became the Armband tab.

Meta Quest 3 optical hand tracking

The Quest 3 streams to SteamVR on the PC through Steam Link. Steam Link delivers hand poses that are already predicted and noisy, with broadband noise between 8 and 40 Hz. Virtual Desktop's hand poses, by comparison, are clean. The research GUI can read the hand in two ways:

- **OpenVR** (SciCortex's reader): SteamVR skeletal input with 31 bones plus curl and splay values. It connects as a background application, so it runs alongside a VR world and the gamepad emulation. It stays the default.
- **OpenXR** (added in July 2026): the cross-platform XR_EXT_hand_tracking extension with 26 joints. It runs in a separate helper process that sends one JSON datagram per frame to a local UDP port, capped at ~60 Hz.
- A full OpenXR session takes over the headset, which then shows a blank page. When OpenXR was briefly made the default, it silently broke the VR world and gamepad workflow. From then on the rule was that OpenXR runs only on demand: ◉ Live Preview, ▶ Start Occlusion Test, and the Capture tab.

One early study had consequences for the whole project. The OpenXR standard defines per-joint bits that say whether each joint's position is valid and actually tracked, which looked like a free way to detect occlusion. In July I reviewed the specification and wrote a probe script (bridge/openxr_handflag_probe.py) that logs every per-joint flag through a five-phase occlusion sequence. pyopenxr 1.1.5301 exposes the raw flags, although its convenience joint-locate call returns an empty buffer. The research indicated that through SteamVR and Steam Link every joint reads VALID and TRACKED whenever the hand is active. The labelled Capture clips of 2026-08-19 confirmed this.

> **In the labelled clips of 2026-08-19, the headset marked every joint as valid and tracked on 100% of frames in the open, fist, hand-behind-hand, out-of-view and behind-object clips, and on 79% in the wrist-bend clip. The optical stream does not report when it is guessing, so camera trust had to be inferred from hand geometry and learned models.**

3.2 One program, one clock: the host software

The GUI must run under native Windows Python 3.12. The reason is that the Bluetooth link uses WinRT, which WSL2 cannot reach. Development and offline analysis ran in WSL on the same files. The CyberFingers use Bluetooth Low Energy (BLE) and the armband uses Wi-Fi, two independent radios, so both can stream into a single process.

- **CyberFinger path.** A background thread finds the paired CyberFingers, subscribes to notifications on characteristic 0xCF01 of service 0xCF00, and uses 0xCF02 for haptics. SciCortex's decoder parses every firmware layout. On the IMU logs of 2026-08-18 (a 66 s and a 541 s session), each CyberFinger delivered 99.4–100.0 reports/s with 0 sequence gaps in 121,104 packets.
- **EMG path.** A reader thread polls the MindRove SDK. The signal is filtered with a 20 Hz high-pass, a 200 Hz low-pass and a mains notch into a rolling 4 s buffer. On 2026-10-01 (commit 0c446df) polling went from every 20 ms to every 5 ms. The main loop now pulls the newest samples before any tab reads the buffer; before this, tabs could see EMG up to one tick (~16 ms) old.

- **Optical path.** On each tick the program chooses one skeleton source. While “send this hand to SteamVR” is on, the hand tapped inside the driver replaces every other source. Otherwise an on-demand OpenXR source (Occlusion Test or Live Preview) is used, then the default reader. Taking the camera hand from the driver keeps everything in SteamVR's raw space and avoids a feedback loop, because the ordinary readers would read back the Studio's own output.

The main loop. A single GUI tick picks the skeleton source, pulls the newest armband samples, and runs every tab's update in a fixed order. The Capture tab records inside this tick. SciCortex's base GUI ticked every 33 ms (~30 fps), and in practice it consumed only ~22 Hz of the 60 Hz OpenXR stream, which showed as jitter and lag. On 2026-08-26 I changed it to a dynamic ~16 ms period with a 60 Hz target, which cut the resampling latency from ~45 ms to ~16–20 ms. With the full set of tabs running, however, the target is not reached. The Transitions recordings of October append one row per tick and show 36.4–43.3 ticks/s (median spacing 24.0 ms, 90th percentile 31 ms, in tx_20261022_211143).

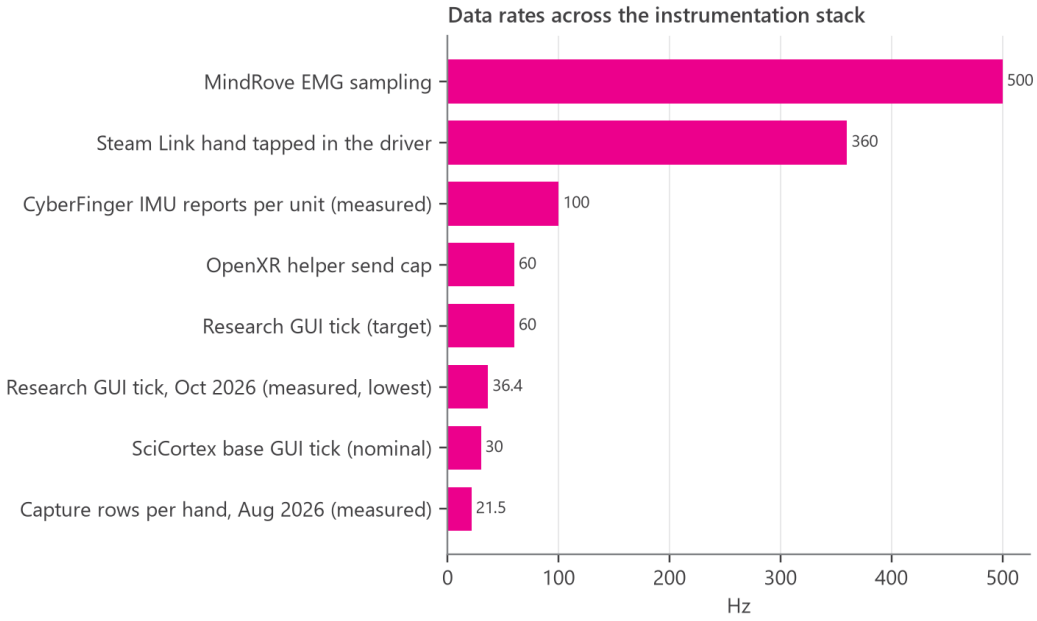


Figure 3.3 *The sensors run at very different rates, and the GUI tick is where they meet on one clock. The August Capture clips were limited by the tick of that time (~21.5 rows/s per hand). The tick target was later raised to 60 Hz, but October sessions with all tabs running measured 36.4–43.3 ticks/s. The EMG rate, OpenXR cap, GUI tick target and SciCortex base tick are nominal values; the IMU rate (99.4–100.0 Hz), the October tick and the Capture rows were measured.*

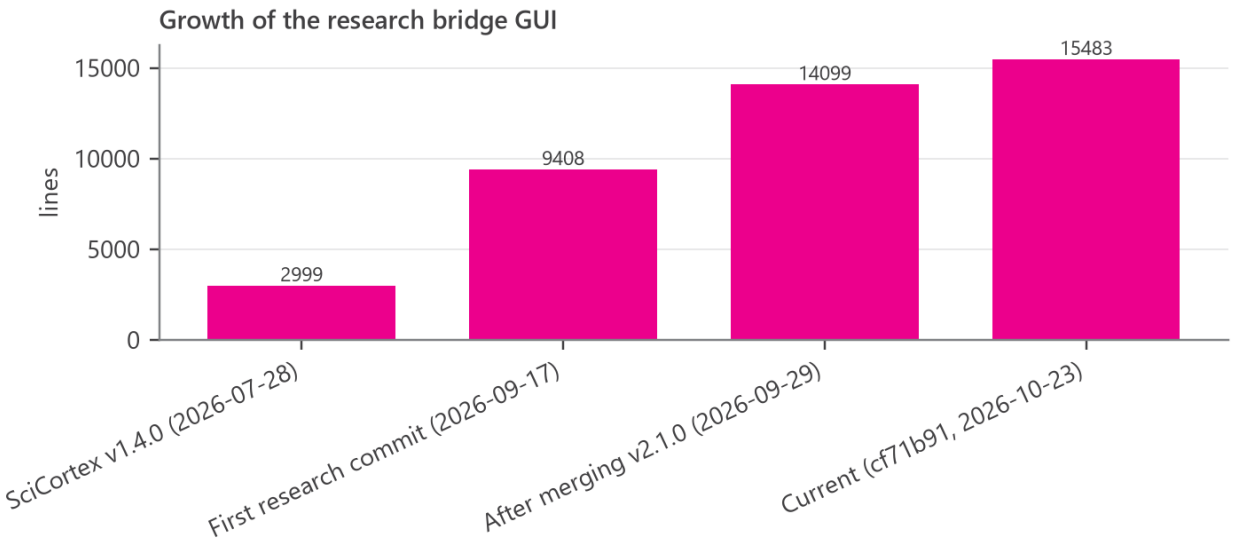


Figure 3.4 *SciCortex's single-window bridge grew into a 27-tab research instrument. At the four points shown it had no tabs, then 16, 25 and 27 tabs.*

3.3 From sensors to VR applications: the SteamVR link

SciCortex's driver v2

SciCortex's driver v2 was released as v2.1.0 (tag 93359ca, 2026-09-28) together with firmware 1.4.1 and Bridge 2.1. It registers two controllers, CYBERFINGER_L and CYBERFINGER_R, with hand-selection priority 1000; Steam Link's hands declare −1. Its load priority of 2000, against 1000 for Virtual Desktop and 110 for Steam Link, means its hooks are installed before Steam Link creates its hands. For each hand the driver uses the first source available: FUSED (the hand sent by Fusion Studio), then PASSTHROUGH (the headset hand captured inside SteamVR by the driver's OpticalTap hooks), then NO_POSE. It falls back from FUSED after 150 ms without packets. A One Euro filter (minimum cutoff 1.0 Hz, beta 15, pose prediction 0.5, rotation prediction 0.0, 5 cm glitch gate) smooths Steam Link's hand.

The driver also contains SciCortex's own IMU fusion (commit e3fcbc0, 2026-09-27, tag v2.0.0). While the hand is visible and turning slowly, it learns the joint IMU's heading and mounting and the streamer's lag, and stores the mounting and lag in %LOCALAPPDATA%\CyberFinger\imu_calibration.txt. It rejects headset orientations more than 25° away from the fused one. In the words of the v2.1 manual, out of view “the hand keeps turning with your wrist, and its position waits where the headset last saw it”.

Table 3.3 Wire protocol v2 between the bridge, Fusion Studio and the SteamVR driver (source: README.md)

Packet	Direction (UDP port)	Size	Content
CFG2	bridge → driver (27015)	34 B	Buttons, stick, trigger; one per BLE report
CFIM	bridge → driver (27015)	96 B	Raw IMU slots; one per BLE report
CFHS	Fusion Studio → driver (27015)	1140 B	Fused hand skeleton (FUSED source)
CFOP	driver → bridge (27016)	2200 B	HMD pose, per-hand mode, Steam Link hand pose and skeleton
CFHP	driver → bridge (27016)	36 B	Haptic requests

The FUSED link. The Studio-side pieces that feed the driver's FUSED input (bridge/steamvr_link.py; commits 374b22b and e3be82e) are described in the Fusion Studio section: the sent hand matches the drawn hand in finger shape to 0.03° (median), and the stream runs at 120 packets per second.

End-to-end bring-up. On 2026-10-01, with both CyberFingers on firmware 1.4.1 and Bridge 2.1.0 installed, the driver reported both hands in PASSTHROUGH mode (mode = 2) with 31 bones at ~360 Hz. During this bring-up I found a packaging bug: the installed Bridge 2.1.0 executable lacks the openvr module, so its Skeleton tab asks the user to "pip install openvr". The research GUI was run from source instead.

Keeping up with SteamVR: the IVRDriverInput_005 break

A SteamVR and Steam Link update on 2026-10-21 (14:30) changed vrserver.exe and driver_vrlink.dll. On 2026-10-22 the hand in VR kept one fixed shape, the fingers no longer bent, and the driver status read "skel=0 bones=0 age=999s". vrserver's run-time type information showed the cause. Steam Link now requests only the newest input interface, IVRDriverInput_005, while OpticalTap hooked only _004 and _003. In the new vrserver, _005 is the latest implementation (11 slots), and _001 to _004 are adapters that forward into it, with slots 0–6 sharing signatures. The fix (commit c35efc5, 2026-10-22, OpticalTap.cpp +34/−13) hooks _005 as well. Because a call through a hooked adapter now also reaches the hooked _005, a per-thread guard reports a call only from the outermost detour. On older SteamVR versions without _005 nothing changes. The driver was built with a portable MSVC 14.44 toolchain (/MT), and driver_tests passed. In the live check, 19 hooks were installed, both VRLINKQ_Hand skeleton sources were found, and the status read "skel=1 bones=31" at 360 Hz.

> **SciCortex committed this fix to its main branch the next day (d469b53, 2026-10-23, "Fix for latest steamvr and skeleton data"). Its OpticalTap.cpp differs from my c35efc5 only in comments.**

Packaging and documentation. The packaging of Fusion Studio as a Windows application is described in the Fusion Studio section, and my review of SciCortex's v2.1 user manual in the final section.

3.4 Merging SciCortex release v2.1.0 into the research branch

When SciCortex released v2.1.0, the research branch had to absorb it without losing either side's work. Git's line-by-line merge interleaved the research Occlusion Test code with SciCortex's new start/stop code. The merge (commit 266731d, 2026-09-29) was therefore done unit by unit, where a unit is a module-level statement, class or method. A 3-way comparison of the parsed code against the common base 24745fe sorted every unit. Afterwards, all 25 tabs and the new controls ran without errors. Replays of Fusion Studio, the wrist-bend clip and the key-posture model gave the same outputs as before the merge. Only timings differed, plus one learning-error value printed as 0.067 instead of 0.066.

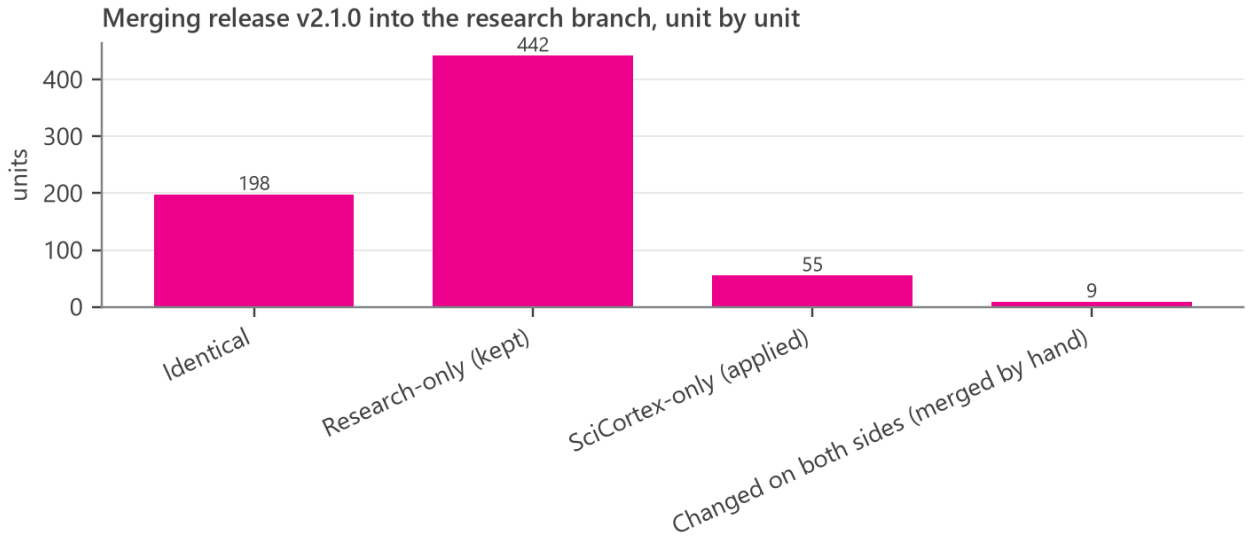


Figure 3.5 *A 3-way comparison at the level of classes and methods replaced git's line merge. After the merge, the replays matched the pre-merge results apart from timing and one rounding difference (0.066 vs 0.067).*

3.5 Gloves: connecting the CyberFingers

Purpose. The Gloves tab is SciCortex's original CyberFinger Bridge window, kept as the first tab. It connects both CyberFingers over BLE and forwards them either to the SteamVR driver (VR Mode) or to a virtual Xbox 360 gamepad (Gamepad Mode, in a Resonite or a VRChat flavour). It shows each CyberFinger's buttons, stick, trigger, battery and IMU orientations. It also hosts the research controls that bring in the headset's optical hand.

How it works. Since the 2026-09-29 merge the tab follows SciCortex's Bridge 2.1 layout. A *Buttons + IMU* sub-tab shows, for each hand, the stick, button lamps, BODY 1 / BODY 2 / JOINT orientation triads with angles, and a haptic strip. A *Skeleton* sub-tab shows both tracked hands and contains the Occlusion Test, and a *Console* sub-tab shows the log. The option rows hold Auto-start, Log IMU (CSV), the right pink button mapping, Forward IMU to SlimeVR and Resync IMU. A research row adds skeleton start/stop, ◉ Live Preview (an on-demand OpenXR source that leaves the default OpenVR backend untouched) and joint logging. The research additions are:

- The firmware version, read over BLE (0x180A / 0x2A26) and logged at every connect.
- A per-packet IMU logger, written from the Bluetooth thread. Each row holds host time, sequence number, hand, imu_present, the body and joint quaternions, and raw acceleration at 2048 LSB/g.

- A joystick auto-centre. The centre is the median of the first 20 packets after every connect, followed by a 0.15 radial deadzone. A log line is written when the offset exceeds 1,500 counts, and the raw counts are shown under each stick.
- A one-way BODY 2 → BODY 1 fallback for firmware 1.4.x (6ea858d).
- ▶ Start Occlusion Test. After a 3 s countdown it runs 5 scripted phases of 8 s each (VISIBLE, SELF OCCLUSION, VISIBLE, HAND BEHIND HAND, VISIBLE) and writes the skeleton and the phase times to CSV.

Results. The IMU stream was measured at 99.4–100.0 reports/s per CyberFinger, with no lost packets in 121,104 (2026-08-18). Before firmware 1.4 both lab units reported imu_present = 6 (BODY 2 + JOINT). After the 2026-09-30 flash the wrist sensor arrives as BODY 1, which the fallback handles. The RIGHT unit's stick rested at about (−16087, −15815), roughly −49% of full scale, while the LEFT read (0, 0). Low battery was ruled out, since both units were at 15–18% but only the RIGHT stick was offset. After the auto-centre fix (2026-08-26) the stick reads centred and the spurious gamepad input stopped. A stick that was actually broken would still need repair. The skeleton log folder (80.1 MB) holds 9 Quest skeleton captures from 2026-08-18 to 2026-08-29, of which 4 are phase-labelled Occlusion Test runs and 2 complete all five phases. It also holds 369 gate-event logs from 2026-08-18 to 2026-10-23.

Status. The tab is in daily use. The path from VR Mode to the driver was verified end to end on 2026-10-01 (both hands PASSTHROUGH, 31 bones, ~360 Hz). The default skeleton backend deliberately remains OpenVR, and OpenXR runs only on demand.

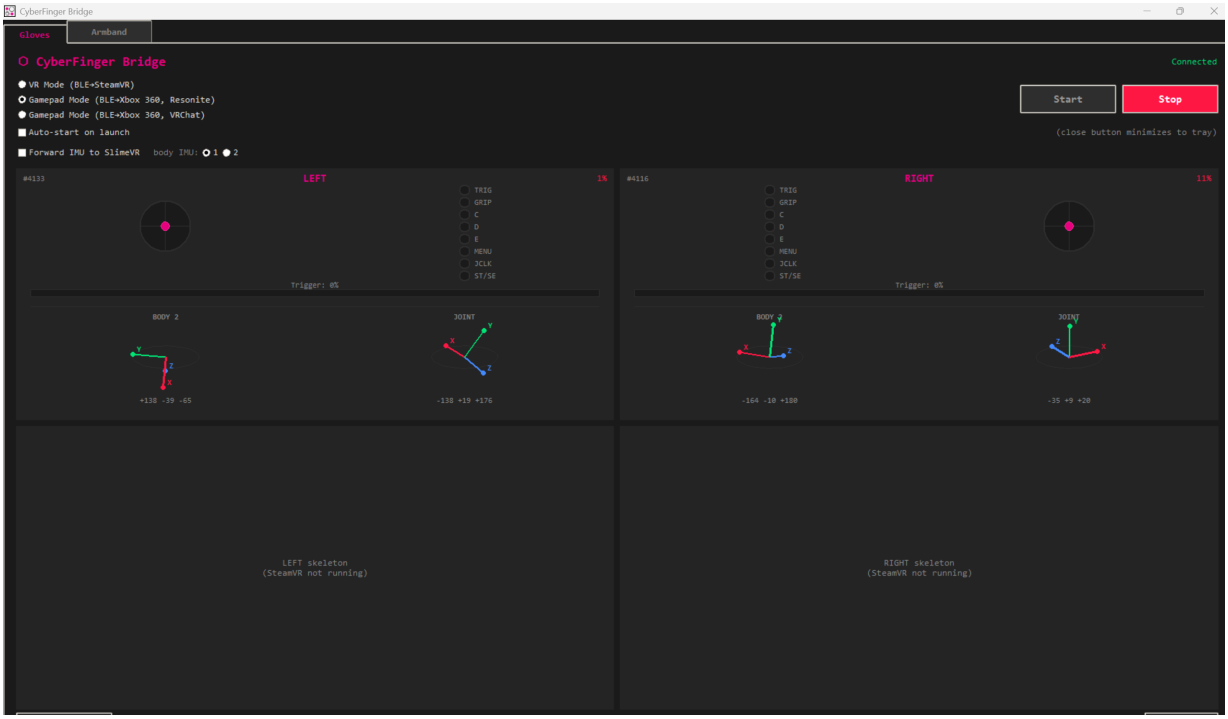


Figure 3.6 *The research bridge on 2026-08-03, after the EMG merge. SciCortex's CyberFinger Bridge window became the Gloves tab, next to the new Armband tab. Both CyberFingers are connected, and each reports a BODY 2 and a JOINT orientation (axis triads with angles), as the lab units did before firmware 1.4.*

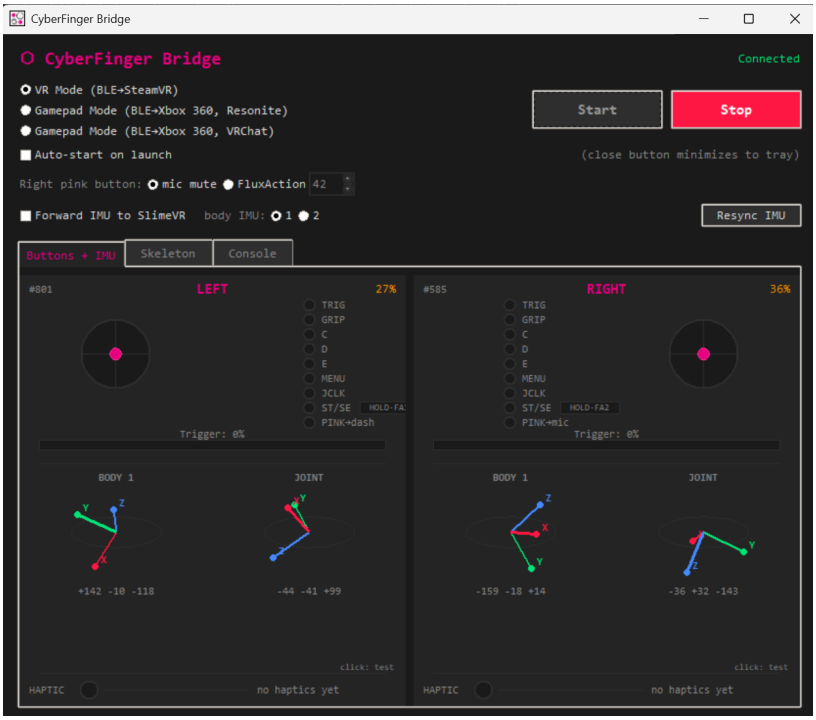


Figure 3.7 *SciCortex Bridge 2.1 layout, as shown in the v2.1 user manual: Buttons + IMU / Skeleton / Console sub-tabs, right pink button row and Resync IMU. With firmware 1.4.x the wrist IMU arrives in the BODY 1 slot. The Gloves tab adopted this layout at the 2026-09-29 merge. The manual's image lacks the SteamVR option that the code offers in the pink button row.*

3.6 Armband: forearm EMG in the same program

Purpose. The Armband tab brings the MindRove armband into the same process as the CyberFingers, so that EMG, IMU and optical data share one clock and one program. It provides a live EMG scope and serves a 3D hand to the browser. The hand is posed by a gesture recogniser and can be turned by the CyberFinger IMUs.

How it works. I embedded my standalone MindRove dashboard rather than launching it beside the bridge or rewriting it, because only one program can own the armband's Wi-Fi session. The dashboard runs in embedded mode, its plot is placed inside the tab, and it is driven by the GUI's tick. Data are read every tick even when the tab is hidden. The plots are redrawn every second tick, and only when the tab is visible; after 5 consecutive drawing failures, redrawing stops while the data and the browser hand keep working. The signal chain runs at 500 Hz through a 20 Hz high-pass, a 200 Hz low-pass and a mains notch into a 4 s rolling buffer. Panels show the 8 EMG lanes, per-channel RMS (root mean square, a measure of signal strength), and the accelerometer and gyroscope. “Show 3D hand in browser” serves the hand from inside the same process on port 8770. A bundled linear model with 9 classes (fist, ok, open, peace, pinch, point, rest, rock, thumbs_up) on 40 features names the gesture. Hand position is deliberately not taken from the IMUs, because it would drift.

Results. The two-tab GUI (Gloves + Armband) was running by 2026-08-03. The browser hand at first failed on the armband's internet-less Wi-Fi, because it loaded its 3D-model loader and hand mesh from the internet, so it fell back to a cylinder. Embedding the loader (GLTFLoader r128) and the mesh in the page fixed this on 2026-08-03; the page grew from 624 KB to 846 KB and is 849,093 bytes today. The armband was later confirmed at 500 Hz in recordings (497.3–499.5 samples/s delivered). Every EMG-based tab, from EMG Hand through Transitions and Fusion Studio, reads its EMG from this embedded dashboard.

Limitations. The bundled gesture model did not hold up on live data. On 2026-08-27 the armband's 9-class model failed on live features, and a custom model that scored 100% in cross-validation on its own samples failed live about 20 minutes later. Both failures came from surface-EMG drift between sessions and electrode placements, the problem the later EMG tabs were built to handle. The embedded dashboard also leaves its mains notch at the 50 Hz default, while mains in Canada is 60 Hz. The effect was not measured, but the notch should be set to 60 Hz. In the WSL test environment the embedded plot occasionally hit a FreeType drawing overflow, which the stop-redrawing fallback handles. The browser hand's IMU axis mapping was never tuned on the hardware and is still the identity.

Figure 3.8 *The Armband tab's browser hand in use on 2026-08-03, with the MindRove armband on the forearm and the CyberFinger on the hand. The label (PEACE) and the curled mesh do not match the open hand in the photo, an early sign that the bundled gesture model was not reliable on live data.*

3.7 Capture: synchronised, labelled clips

Purpose. The Capture tab records short, labelled clips in which the optical hand, the CyberFinger IMUs and the armband EMG are written on one shared timestamp in one file. It was the project's first synchronised dataset, delivering proposal task 1.1. The clips were used first to tune the occlusion gate on real data instead of guesses, and then to calibrate and test the fusion of IMU and optical orientation.

How it works. The tab lists 7 presets, each with Start/Stop and the gate label it should produce. Recording needs OpenXR and starts Live Preview automatically. Inside the main tick, the tab writes one row per hand per tick to bridge/gesture_logs/<gesture>_<hand>_<timestamp>.csv. Each row holds the latest value of every modality at that tick, stamped with the shared wall_time. (The xr_time column is the host clock in nanoseconds, not the OpenXR runtime's time.) The Left/Right/Both selector only names the file; both hands are logged in every file. The synchronised columns were appended at the end of each row, so the older 143-column clips and the offline gate tools, which read columns by name, kept working.

Table 3.4 The seven Capture presets

Preset	Expected gate label	Note
open	TRACKED	
fist	SELF-OCCLUDED	
wrist_bend	SELF-OCCLUDED	
hand_behind_hand	OCCLUDED BY OTHER	
out_of_view	OUT OF VIEW	
behind_object	BEHIND OBJECT	
rigid_hold	—	20 s auto-stop; fingers frozen, forearm swinging, to measure IMU-vs-optical drift

Table 3.5 The synchronised clip format (from 2026-08-20)

Column group	Columns	Content
Header	13	Host, wall and host-ns time; hand; is_active; headset (HMD) pose
Optical joints	130	26 joints × (x, y, z, flags, radius)
CyberFinger	17	Gate label, imu_present, BODY 1 / wrist / JOINT quaternions, BODY acceleration
Armband	15	8 EMG RMS values, armband accel and gyro, emg_present
Total	175	One row per hand per GUI tick

Results. Thirteen clips were recorded between 2026-08-19 and 2026-08-26. The seven from 2026-08-19 are optical-only (143 columns), and the six from 2026-08-20 on are synchronised (175 columns, imu_present = 6 apart from one clip described below). Two of the thirteen are aborted stubs (42 and 34 rows), leaving 11 usable clips. Rows were written at ~21.5 per second per hand (median spacing 45.0–47.1 ms), the GUI tick of that time. The clips gave the following results:

- **Synchronisation.** The cross-correlation of IMU and optical rotation speed peaks at lag 0 (r = 0.645, against 0.34 one frame later), so the streams were aligned to within one frame (<47 ms) and no fixed delay needed subtracting (2026-08-25).
- **Optical flags.** The validity and tracking flags were set on 100% of frames in five of the six scenarios (79% in the wrist-bend clip), confirming that they carry no occlusion information.
- **Gate tuning.** The clips set the gate's finger-curl rule: open → TRACKED on 100% of frames, fist → SELF-OCCLUDED on 96%, wrist bend → SELF-OCCLUDED on 63%.
- **First IMU-to-optical calibration.** The median residual was 10.6–12.6° on the two open-hand clips of 2026-08-20.
- **Dropout horizon.** In the rigid-hold clip the arm swept 179°. The wrist IMU then carried the orientation alone, using the one-sided fuser of that time, and its error stayed between 13 and 19° from 0.5 to 5 s (no growth over time), giving a safe dropout horizon of at least 5 s.

Limitations. The EMG columns hold an RMS over the dashboard's whole 4 s buffer, about 33 ms stale at the tick of that time. That is an activity level, not a signal fast enough to decode fingers. Later recorders stored raw EMG instead, as windows in the Encoder tab and as continuous 500 Hz EMG in the Transitions logs. The clips were recorded at the old ~21.5 Hz tick. Recording needs an exclusive OpenXR session, so it cannot run beside a VR application. One clip (rigid_hold_right_20260825_184146) has no EMG because the armband was

not streaming. Another (rigid_hold_right_20260826_141828) contains rows with imu_present = 0 and was recorded while the knuckle module was temporarily mounted on a finger.

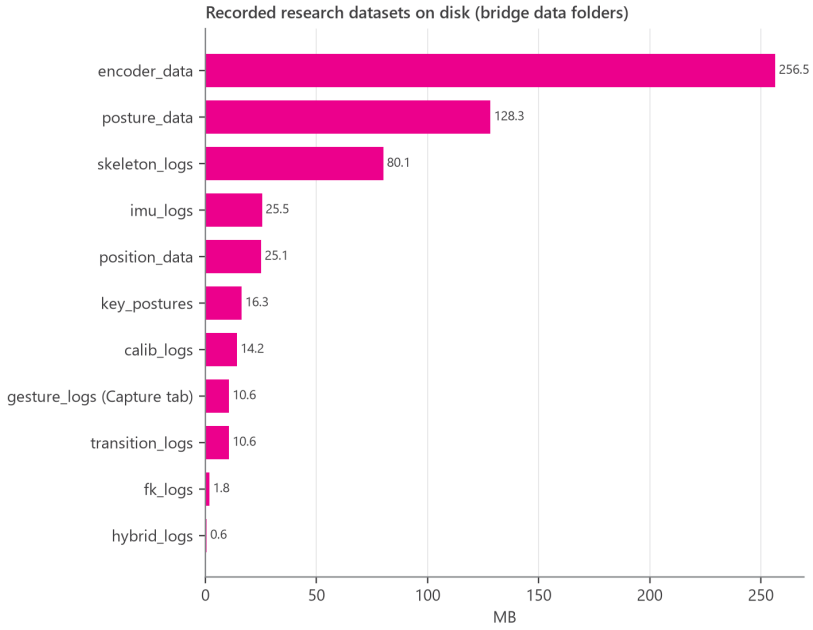


Figure 3.9 *Size of each recording folder on 2026-10-28, 569.8 MB in total, recorded between 2026-08-18 and 2026-10-23. Not all folders combine the three sensors: imu_logs hold IMU data only, and skeleton_logs hold optical captures and gate-event logs. The gesture_logs folder (Capture tab) also contains two diagnostic files from the EMG Hand tab.*

3.8 Limits of the instrument

Several limits of this system shaped the research described in the later sections:

- The optical stream's per-joint flags carry no occlusion information, so whether the camera is guessing must be inferred.
- The two 6-axis IMUs share no heading, and each one's yaw drifts on its own. Only their tilt can be trusted without calibration against the camera.
- The GUI tick reaches 36.4–43.3 Hz with all tabs running, not the 60 Hz target.
- The early Capture clips store an EMG activity level (4 s RMS), not raw EMG.
- OpenXR sessions are exclusive, so the OpenXR-based tools cannot run beside a VR application.
- The embedded EMG notch is set for 50 Hz mains instead of Canada's 60 Hz (effect not measured).
- The Windows exe's live headset path had not been tested at build time.

How the IMU mounting finding changed. The knuckle IMU's mounting became a recurring theme. On 2026-08-25, securing a loosely mounted knuckle clip cut the live mismatch between this IMU and the camera from ~57–76° to 2–5°. On 2026-08-26 the knuckle IMU was measured to report only ~0.64× of the true rotation, leaving a ~10° error floor in fast motion, which I then attributed to the elastic mount and the sensor. A later study (2026-09-25, all 8 position-capture clips) replaced that explanation. The large out-of-view error came from the fusion code's one-sided offset. A camera-taught two-sided mounting calibration held a median error of 8.6, 8.7, 8.9, 9.5 and 10.3° at 1, 3, 7, 15 and 30 s out of view. The IMUs proved to be good absolute orientation sensors, and the main source of error was how their mounting was modelled.

4 Optical confidence, occlusion and online calibration

This section covers how the system decides, for each hand and each finger, whether the Meta Quest 3 camera can be trusted, and how that trust is used to recalibrate the EMG model while the user simply moves. The Quest's own tracking flags turned out to be useless for this, because they stay at "tracked" through every kind of occlusion. The Gate tab therefore classifies why the camera fails using geometry tuned on real clips, and the Occlusion tab ray-casts the visibility of each finger. Live Calib implements the proposal's attention-based online calibration. In recorded sessions it cut in-view finger-curl error against the camera from 0.455–0.547 to 0.056–0.124, and two validated changes raised out-of-view posture identification from 72% to 92%; the shift of the EMG when the arm is lowered remains the main limit.

4.1 Why camera trust matters

Every part of the fusion system depends on one question: at this moment, for this finger, can the headset camera be believed? When it can, the Meta Quest 3's optical hand tracking is the ground truth. It fixes the absolute pose of the hand, and it is the teacher the EMG and IMU models learn from. When it cannot, the system has to rely on the MindRove EMG armband and the CyberFinger IMUs instead, and it must stop the camera from teaching those models wrong labels. The camera fails when the hand is outside its view, behind an object, behind the other hand, or hidden by its own fingers.

The proposal planned this work as two tasks. **Task 1.2** was per-finger confidence, built from the OpenXR per-joint validity flags plus a geometric ray-casting occlusion model after Graf & Barthet (2023); the ray-casting part (T5.2) was scheduled for Internship 2. **Task 3**, the project's central contribution, was an attention-based online calibration that uses the camera as an opportunistic teacher. The work was done in three GUI tabs, each covered below. **Gate** says, for each hand, why the camera is unreliable. **Occlusion** gives the visibility of each finger by ray-casting. **Live Calib** holds the online calibration and depends on both.

For comparison, the CyberFinger product as documented in SciCortex's manual v2.1 handles a hand out of view by turning it with the joint IMU while its position stays where the headset last saw it.

Table 4.1 Timeline of the confidence, occlusion and calibration work (sources: file dates, project notes and commit history)

Date (2026)	Milestone
July	Research shows the OpenXR per-joint flags are whole-hand only on Quest. A flag-probe script is written (26 July), and an OpenXR skeleton backend is added alongside OpenVR
18 Aug	Gate specification, reference gate and synthetic benchmark; first scripted occlusion captures; first live gate-event log
19 Aug	Live Gate rules tuned on labelled clips recorded with the Capture tab
20 Aug	Gate verdict stored on every row of the synced optical + IMU + EMG Capture clips
28 Aug	Shown that the Quest cannot label a closed fist
28–30 Aug	Ray-cast occlusion module (fusion/occlusion.py) and Occlusion tab
30 Aug	Live Calib (fusion/online_calib.py); its bugs diagnosed and fixed on recorded sessions
31 Aug	Per-finger camera/EMG blend in Live Calib; per-finger learning gate in Pose Hand
17 Sep	Pipeline committed to the research repository (commit c7704d3)
1 Oct	Ray-cast made 27 times faster (2172b3e); Fusion Studio cross-checks the camera against the knuckle IMU (f06b11f)
8 Oct	Per-finger camera/EMG weight w = f(m, d) in Fusion Studio (18d60db)
23 Oct	Last of 369 gate-event logs

4.2 Starting point: the headset's own flags are whole-hand only

The OpenXR hand-tracking extension (XR_EXT_hand_tracking) defines POSITION_TRACKED and ORIENTATION_TRACKED bits for every joint. These are exactly the signals a fusion gate needs. Research in July 2026 found three problems with them:

- On Quest, the runtime keeps producing poses for hidden fingers, inferred by its neural network, with the bits still set. The flags change only for the whole hand, when it is acquired or lost.
- The core extension has no per-joint confidence number, only the bits and a joint radius.
- Through SteamVR and Steam Link, the setup used in this project, every joint reads VALID and TRACKED whenever the hand is active, because SteamVR builds the joints from skeletal input that is always complete.

A five-phase probe script (bridge/openxr_handflag_probe.py) was written to log every per-joint flag through an occlusion sequence, but no recorded run of it exists, so the July verdict rests on that research. The empirical confirmation came from the first scripted captures on 18–19 August, recorded through the GUI's on-demand OpenXR source. In the scripted capture of 18 August 17:35 (38.4 s, 3,443 right-hand frames), is_active and the per-joint POSITION_VALID flag stayed at 1.00 in every phase, including self-occlusion and hand-behind-hand. In the labelled clips of 19 August, is_active was 1.00 even with the hand held out to the side or behind an object (0.79 for the wrist-bend clip). The Quest infers a pose rather than admit that it has lost the hand.

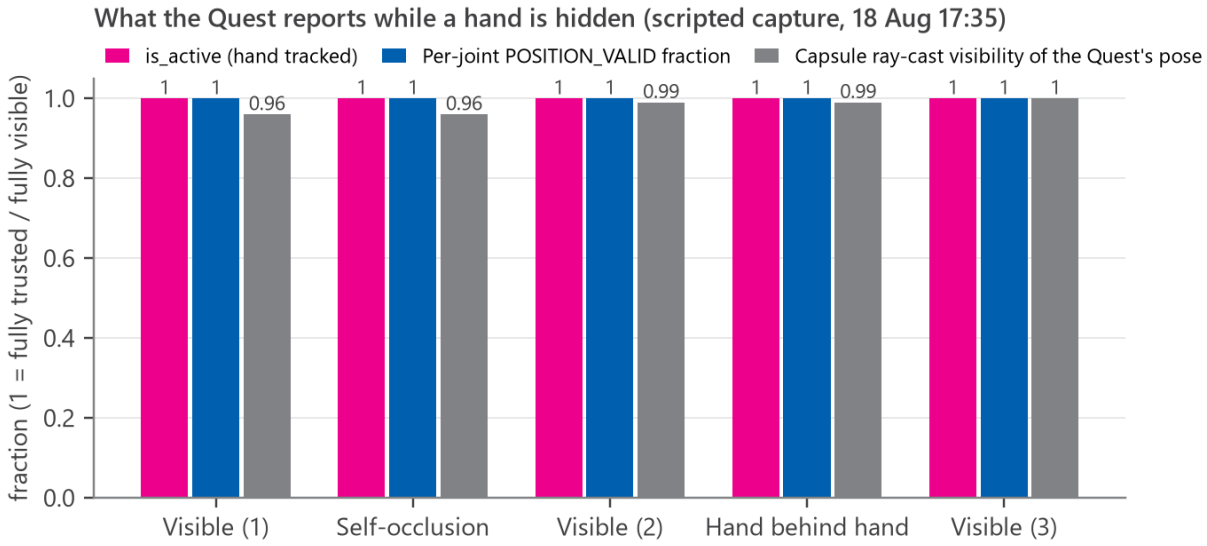


Figure 4.1 *Through every phase of a scripted occlusion test, the Quest reports the hand as tracked and every joint as valid. Its inferred pose is not geometrically occluded either: the Occlusion tab's ray-cast finds the fingers 96–100% visible. Per-finger trust therefore cannot come from the headset's own signals.*

So per-finger reliability had to come from geometry, from how the joints move over time and, later, from agreement with the IMU and EMG. The flags remain useful only as a whole-hand kill switch: is_active did drop to 0 in 9% of the frames of the 18 August 15:54 capture. The finding also settled an engineering question. Better occlusion information was not a reason to move the GUI from OpenVR to OpenXR. OpenVR stayed the default, because it coexists with the VR world and the gamepad, and the new OpenXR backend runs only on demand for the live preview and the scripted occlusion captures.

4.3 Gate: why the camera can or cannot be trusted right now

The Gate tab states, for each hand and in plain language, why the camera can or cannot be trusted at this moment. It gives one of five verdicts: TRACKED, OUT OF VIEW, SELF-OCCLUDED, OCCLUDED BY OTHER or BEHIND OBJECT. The tab displays these names; recordings and code store them as CLEAR, OUT_OF_VIEW, SELF, OTHER and OBJECT, and later sections use those short codes. Every later fusion tab uses this verdict as its switch signal between camera and EMG/IMU, and it also sets how much weight each camera label gets when models are trained. The tab was built in two steps. The first was a reference gate with a formal specification and a synthetic benchmark. The second was the live classifier, whose rules I tuned on real recordings after the reference cues failed on them.

Reference gate and synthetic benchmark

On 18 August I wrote a mathematical specification (docs/gate-spec.md), a reference implementation (fusion/gate.py), a synthetic benchmark, and a driver that runs the gate on recorded captures. Each joint j receives a confidence $c_j = h \cdot g_{fov} \cdot g_{occ} \cdot g_{tmp}$, the product of four cues:

- **h**: the whole-hand OpenXR kill switch (0 when the hand is not active).
- **g_fov**: whether the joint is inside the camera's field of view, modelled as a fixed cone with half-angles of 55° horizontal and 45° vertical and a 6° soft edge.

- **g_occ**: a ray from the camera to the joint, tested against a hand model made of one 1 cm sphere per joint.
- **g_tmp**: whether the motion is physiologically plausible (speed below 2.5 m/s, acceleration below 60 m/s^2) and the joint is not frozen (6 identical frames).

The product is smoothed with a 0.10 s time constant and combined per finger, with the fingertip joints weighted most. A hysteresis ramp between 0.35 and 0.70 then turns it into a blend weight. In this specification, camera joints enter the calibration buffer only above 0.85. One design point in the specification proved important: the occlusion ray should be cast against the fused, last-confident pose, not against the raw camera pose, because the camera pose is exactly what degrades during occlusion.

The benchmark is a 30-second synthetic hand sequence at 60 Hz (46,800 joint samples) in which hidden joints freeze at their last-seen value, as they do on the Quest. A joint counts as reliable when its camera error is below 1.5 cm, which is true of 61% of samples. Each cue was scored by ROC-AUC, the area under the receiver operating characteristic curve, which measures how well a score separates reliable from unreliable joints (0.5 = chance, 1 = perfect).

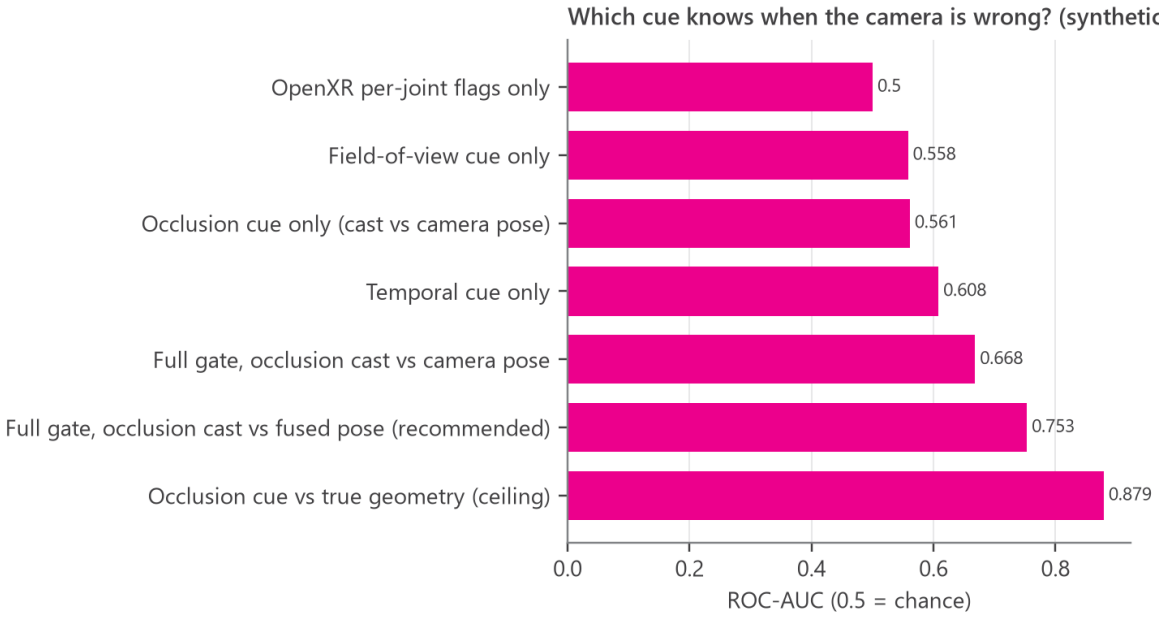


Figure 4.2 *How well each cue predicts that a camera joint is wrong (error above 1.5 cm) in a 30-s synthetic sequence where hidden joints freeze, as on the Quest. The recommended gate reaches 0.753. An occlusion test against the true geometry would reach 0.879, which is the headroom a learned reliability estimator would target. The flag-only bar is a constant score by construction: it encodes the assumption that the flags never change, and the evidence for that assumption comes from the real captures, not from this bar.*

Gating cut the joint error during occlusion from 23.4 cm (camera only) to 16.0 cm. The cost was a slightly larger error while the hand was visible (0.71 cm instead of 0.32 cm).

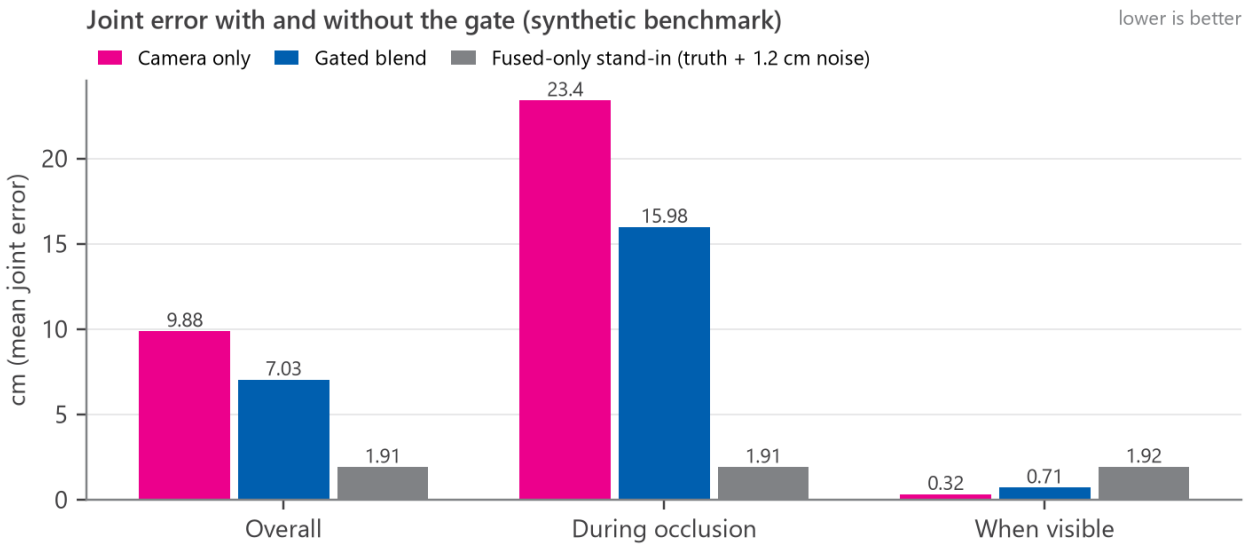


Figure 4.3 *Gating cuts the error during occlusion from 23.4 to 16.0 cm at a small cost while the hand is visible. The fused estimate here is an idealised stand-in, not the real EMG/IMU estimate, so the chart shows how the gate routes between sources, not end-to-end accuracy.*

The benchmark has limits that should be stated plainly:

- The fused estimate is the true pose plus 1.2 cm of noise, far better than the real EMG/IMU estimate. That is why the stand-in alone (1.91 cm) beats the gated blend. The benchmark measures how well the gate discriminates, not end-to-end accuracy.
- The hand is always active in the benchmark (h = 1), so the full gate is effectively a three-cue gate.
- The scores are not calibrated probabilities: the expected calibration error is 0.21.
- The benchmark script no longer runs unchanged. A later extension of the occlusion cue, which also reports which hand blocks the ray, makes it fail with an IndexError. A one-line fix reproduces every documented number exactly.

What the reference gate did on real captures

Run offline on a real capture (18 August 15:54, 3,233 frames, 36.8 s), the reference gate produced plausible per-finger traces, with mean confidence between 0.52 and 0.70 per finger (figure below). The scripted captures, however, were recorded with the headset worn on the forehead so that the hand and the live skeleton could be watched at the same time. On those captures two of the cues failed:

- **Field of view.** The fixed cone judged tracked hands to be outside the view. g_fov was 0.02–0.07 in the VISIBLE phases of the 17:35 capture, so the gate distrusted the camera almost all the time. The Quest tracks far below the gaze, and with a forehead-worn headset, tracked hands appear at up to 90° azimuth. Every fixed azimuth/elevation bound tried false-fired in 35–66% of frames.
- **Occlusion.** Ray-casting against the Quest's pose did not separate the phases: visibility was 0.94 while visible and 0.96 during self-occlusion. The Quest returns a plausible pose that is not geometrically occluded. A depth test fails for the same reason: the fingertips of the Quest's fist are not deeper than the knuckles (depth ratio 0.08 for the fist, against 0.25 for an open hand).

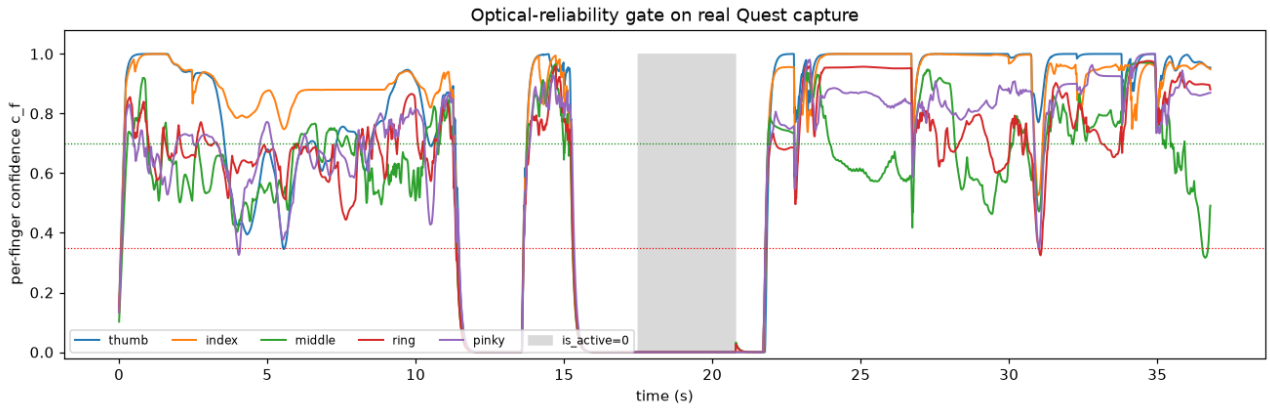


Figure 4.4 *The reference per-finger gate (fusion/gate.py) run offline on a real Quest capture (18 August, 36.8 s). Each line is one finger's confidence, and the dotted lines mark the hysteresis thresholds 0.35 and 0.70. Confidence falls to zero while the Quest reports the hand lost (grey band, is_active = 0). It also falls to zero at about 11.5–13.5 s and 15.5–17.4 s, while the hand was still reported active, most likely because of the fixed field-of-view cone.*

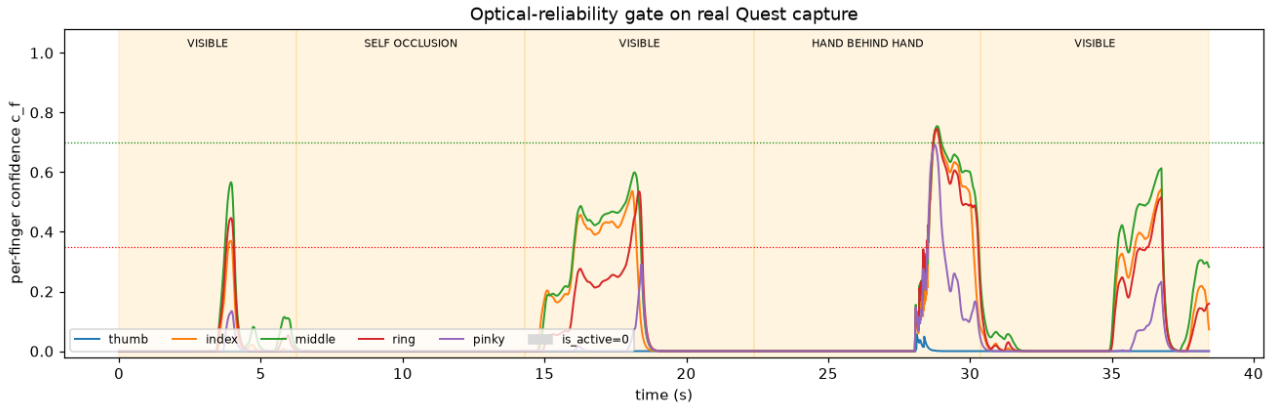


Figure 4.5 *A negative result that shaped the live Gate tab: the same reference gate on a scripted occlusion capture with the forehead-worn headset (18 August 17:35; shaded phases VISIBLE / SELF OCCLUSION / HAND BEHIND HAND). The fixed 55°/45° field-of-view cone treats tracked hands as outside the view, so confidence stays mostly near zero even in the VISIBLE phases, while the Quest's is_active and joint flags stay at 1.0 throughout. Produced with the repository's offline script fusion/run_gate_on_log.py.*

How the live Gate tab decides

The tab shows a "Gate Radar": the headset camera's view of both hands, with each full 26-joint skeleton drawn as bearings, inside a green tracking box. A split LEFT | RIGHT banner gives each hand its verdict in colour: TRACKED in green, OUT OF VIEW in red, SELF-OCCLUDED or OCCLUDED BY OTHER in orange, and BEHIND OBJECT in purple. Each hand also has a confidence bar. The rules were tuned on real clips because the reference cues failed there:

- **OUT OF VIEW**: the wrist is outside the calibrated tracking box, or the hand is lost or frozen at the edge of the box.
- **OCCLUDED BY OTHER**: at least 15% of this hand's joints lie inside the other hand's silhouette as the camera sees it, and are deeper than it. Per-joint sphere ray-casting was too strict here, because it needs the joints to line up within about 1 cm.

- **SELF-OCCLUDED**: at least 30% of the fingers (with five fingers, at least two) are curled, meaning the straight-line knuckle-to-tip distance is less than 0.65 of the finger's length. This test does not depend on the viewing angle. On the tuning clips the mean ratio was 1.0 for an open hand and about 0.47 for a fist.
- **BEHIND OBJECT**: the hand vanished, or froze for at least 5 identical frames, well inside the box (8° margin). The freeze cue works because a truly tracked hand always jitters: runs of 5 or more identical frames occurred in 0–2% of open, fist and hand-behind-hand frames, against 18% out of view and 20% behind an object.

The tracking box is learned from where the Quest actually tracked the hand. The "⊹ Calibrate box" button takes the 98th and 2nd percentiles of the tracked bearings, adds 3°, needs at least 300 samples, and saves the result. For this user the box is ±87° in azimuth and −82° to +18° in elevation. It is asymmetric because the Quest tracks far below the gaze, and it absorbs the forehead tilt of the headset. One limitation: samples are collected only from frames already inside the current box, and the result is clamped (azimuth 20–90°, upper elevation 10–60°), so a single calibration can widen the box by only about 3°.

The confidence bar is a geometric score: 1 minus the larger of the curled-finger fraction and the overlap with the other hand. It is not a probability. Every change of verdict is written with a timestamp to skeleton_logs/gate_events_*.csv. From 20 August, every row of a synced Capture clip also carries the gate verdict (gate_type) on the same clock as the optical, IMU and EMG data. The verdict also sets the weight of each camera label used in training:

Table 4.2 Training-label weights derived from the Gate verdict (ENC_LABEL_CONF in bridge/cyberfinger_gui.py). Later recorders use the mean ray-cast finger confidence instead when it is available.

Gate verdict	Weight of the camera label in training
TRACKED (CLEAR)	1.0
SELF-OCCLUDED	0.5
OCCLUDED BY OTHER	0.4
BEHIND OBJECT	0.3
OUT OF VIEW	0.2

Results on labelled clips

On 19 August I recorded labelled right-hand clips with the Capture tab (open, fist, wrist bend, hand behind hand, out of view, behind an object) and tuned the thresholds on them instead of guessing. Replaying the exact GUI rules on those clips gives the results below. They agree within one percentage point with the figures recorded at tuning time.

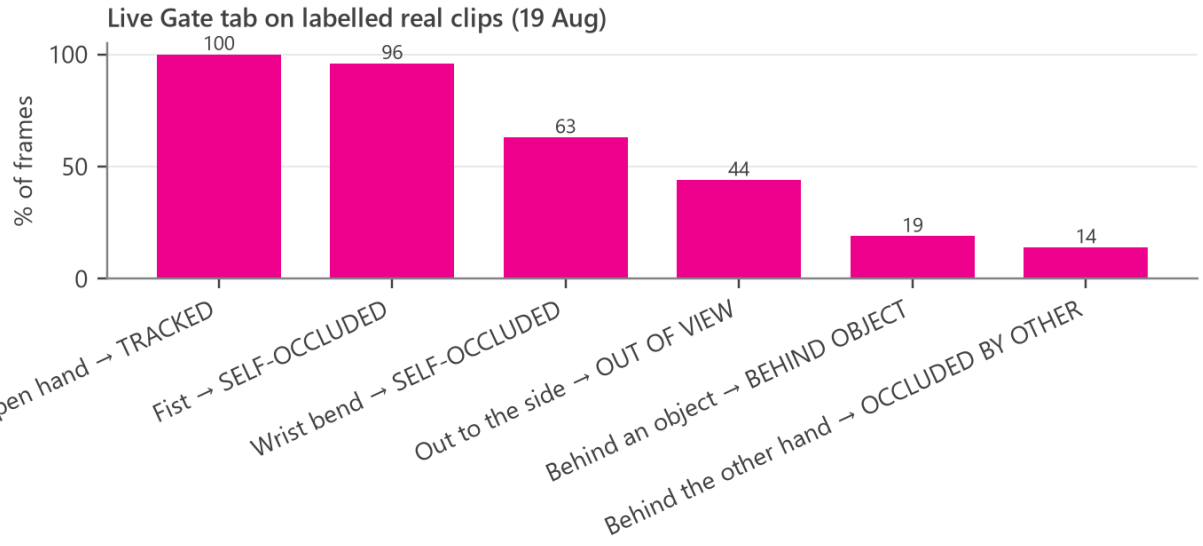


Figure 4.6 *The tuned live classifier is reliable for open hands and fists. Out of view, behind objects and behind the other hand remain hard, because the Quest tracks very wide (±87°) and keeps inferring plausible poses instead of reporting a loss. A second, 21-frame fist clip was only 38% SELF-OCCLUDED, and the wrist-bend frames were also 19% BEHIND OBJECT and 18% TRACKED.*

On real Quest data, geometry rather than the headset's flags carries the gate. The tuned classifier labels 100% of open-hand frames TRACKED and 96% of fist frames SELF-OCCLUDED, while the Quest's own is_active flag stayed at 1.00 in those same clips.

The hard cases have clear causes. Out of view, the Quest tracks so wide that only 44% of the frames left the box or froze. Behind an object, the Quest often keeps updating a plausible pose that looks like a real open hand, so only frozen frames can be detected (19%). The hand-behind-hand clip was staged "behind" relative to the body rather than along the forehead camera's line of sight, so the hands barely overlapped in the camera's view (14%). A fair test of that verdict needs a capture with one hand between the headset and the other hand.

The live tab does not run the four-cue reference gate. The per-hand reference-gate objects are created in the GUI but never updated, and a code comment that says otherwise is out of date. The live verdict comes from the tuned rules above, and the reference gate is used offline. The tab has been in regular use: the gate-event folder holds 369 logs from 25 days between 18 August and 23 October (248 in August, 60 in September, 61 in October).

Beyond geometry: cross-checking the camera against the IMU

Geometry is not the whole answer. The Quest sometimes flips the hand while the gate still reports TRACKED; Fusion Studio therefore also distrusts a camera hand that disagrees with the knuckle IMU (IMU section). The full answer to "when can the camera be trusted" thus comes in two layers: geometry first (this tab and the Occlusion tab), then agreement with the IMU and the EMG (Fusion Studio and Fusion Logic sections).

4.4 Occlusion: which fingers the camera really sees

The Occlusion tab estimates, finger by finger, how well the camera can actually see the hand right now, instead of giving one verdict for the whole hand. It covers proposal Task 1.2 and brings forward the ray-casting task T5.2 from Internship 2. It works as a quality filter, so that the camera teaches only what it really saw.

Why one number per hand was not enough

The Quest cannot label a closed fist (EMG section): every fist frame carries the SELF-OCCLUDED weight 0.5, and a model trained on those labels learns the wrong pose.

The general lesson is that camera label quality sets the ceiling, and one confidence number for the whole hand is too blunt:

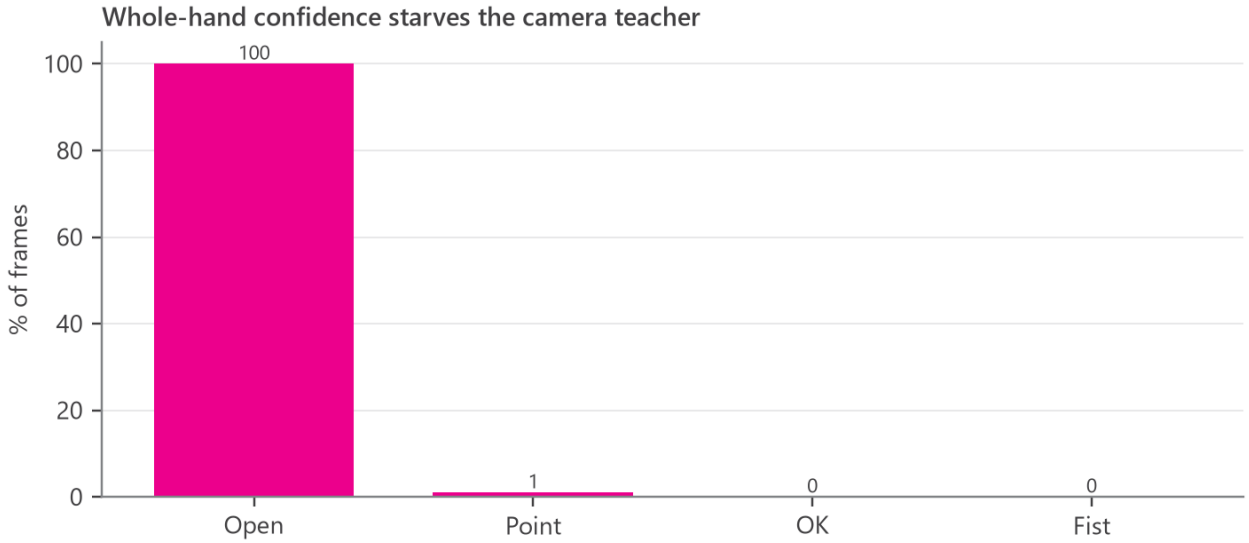


Figure 4.7 *With one confidence number per hand, the camera can teach almost nothing but an open hand. Any posture with two or more curled fingers is labelled SELF-OCCLUDED (label weight 0.5), so it never reaches the 0.9 trust level. Per-finger visibility lets the visible fingers of such a posture still teach.*

How it works

The module fusion/occlusion.py follows Graf & Barthet (2023). A ray is cast from the headset camera to each joint and tested against the hand's own bones:

- Bones are modelled as capsules: 1.0 cm radius for the fingers, 1.8 cm for the palm bones, plus cross-links along the knuckle ridge.
- A joint counts as occluded in proportion to how deeply its ray passes through a bone in front of it. The joint's own bones are never counted.
- Each finger's confidence is a weighted mean of its joints, with the weight rising from 1 at the base to 2 at the fingertip.

- The GUI then multiplies by a per-joint field-of-view test against the calibrated box (10° soft edge). It sets everything to 0 when the gate says OUT OF VIEW and scales by 0.35 for BEHIND OBJECT or OCCLUDED BY OTHER, since the ray-cast cannot see an external blocker. Finally it smooths the result over time (EMA 0.6/0.4).

The tab draws the live hand with each joint coloured from green (seen) to red (hidden), five per-finger bars, and a banner reading "N/5 fingers reliable", where a finger counts as reliable at a confidence of 0.6 or more.

Results

In the module's self-test, a synthetic right hand viewed from the back-of-hand side, all fingertips of the open hand are fully visible (1.00). The fist's fingertips average 0.10, because they sit behind the knuckle ridge.

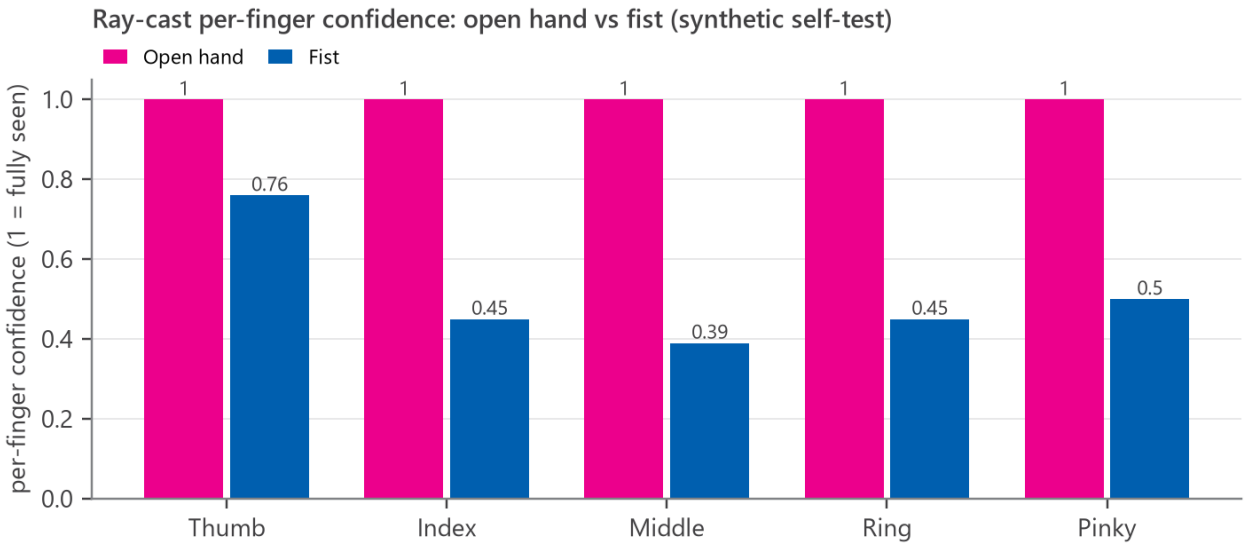


Figure 4.8 *In the Occlusion module's self-test, the curled fingertips of a fist lose most of their visibility (fingertip mean 0.10), while every finger of an open hand stays fully visible.*

- **Per-finger teaching on real data.** In the Pose Hand tab, the visible index finger of a point was learned even though the whole hand had low confidence: the predicted index curl moved from 0.60 to 0.08, close to the camera's 0.16.
- **Speed.** On 1 October the ray-cast was vectorised, cutting its cost from 4.8 to 0.18 ms per frame (27 times faster), with identical results on 3,200 recorded frames. Together with lighter drawing, this took Fusion Studio from 35 to 62 frames per second.
- **Downstream use.** The same ray-cast feeds Live Calib (which pairs enter the buffer and the per-finger blend), Pose Hand (per-finger online learning), Fusion Studio and Fusion Logic. In the last two, each finger's camera/EMG weight is w = f(m, d), where m is this visibility and d is how far the camera and the EMG disagree on that finger's curl. When camera and EMG disagree, the choice flips at m0 = 0.7, and the EMG learns only where w > 0.8. Placing the flip at 0.5 instead cost partly hidden fingers one point of posture accuracy (93.2% → 92.1%).

Limits

On real Quest output, the ray-cast lowers confidence only moderately for self-occluded postures, because the Quest's inferred fist is not geometrically closed. In the recorded Live Calib sessions, mean finger confidence was 0.88–0.89 for SELF-OCCLUDED frames against 0.96–1.00 for TRACKED frames, so many self-occluded frames still pass the 0.6 threshold. (These values come from the ray-cast that Live Calib runs itself, without the field-of-view taper and smoothing.) On the scripted captures the separation is weaker still: 0.96 visible against 0.96 self-occluded on 18 August, and 0.99 against 1.00 on 19 August. An earlier expectation that fists would rarely enter the calibration buffer was overturned by these measurements. Fingers hidden inside a posture, such as the curled fingers of a point, also cannot be taught by the camera at all, and their predicted curl stays at about 0.6 (half-curled).

4.5 Live Calib: online calibration with the camera as teacher

Live Calib implements the proposal's central contribution (Task 3): it corrects the EMG finger encoder for the current session, user and electrode placement while the user simply moves. There are no calibration gestures and no retraining. The motivation is set out in the proposal. Surface EMG drifts between sessions and electrode placements, with 20–30% drops in accuracy between sessions reported in the literature (Buteau et al., 2024). The camera is the absolute reference and corrects the EMG the way GPS corrects inertial dead-reckoning. No weights change and no gradients are computed at runtime. Instead, a rolling buffer of (EMG, camera pose) pairs, admitted only when the camera saw the fingers well, is queried by cross-attention, and the result is a correction specific to this session.

How it works

1. **Admit.** When the mean per-finger ray-cast confidence is 0.6 or more, the current frame is stored as a pair: 40 time-domain EMG features plus the wrist's gravity tilt (which does not depend on heading) as the key, and the camera's five finger curls as the value. The buffer holds up to 800 pairs. Live Calib runs its own ray-cast with the same module and the same gate attenuation.
2. **Look up.** The current EMG (smoothed over about 0.7 s) and wrist tilt form the query. Similarity combines z-scored EMG cosine similarity (temperature 0.12) and wrist-orientation similarity (temperature 0.08). The 96 best matches are averaged, weighted by similarity, to give a calibrated curl for each finger.
3. **Blend by coverage.** A coverage score, the EMG match multiplied by the orientation match and smoothed over time, decides how much to trust the lookup. When coverage is low the output falls back to the base encoder, so the hand never freezes on a stale pose.
4. **Keep it fair and balanced.** Each frame is predicted from past pairs before the current pair is added. When the buffer is full, density-based eviction removes a member of the largest cluster, so every posture stays represented.
5. **Detect electrode shift.** A smoothed residual (0.98/0.02) is kept between the new camera label and the nearest buffered pair. If it exceeds 0.22 with more than 40 pairs in the buffer, the buffer is cleared and rebuilt.
6. **Teach through short occlusions.** When the hand has just become hidden, the last camera label is copied onto the occluded EMG, but only if the label is at most 1 s old, the wrist has not turned (tilt cosine > 0.9), the live estimate agrees (difference < 0.30), and at most once every 0.15 s.

The tab has three panels: **ENCODER alone (base)**, **+ ONLINE CALIBRATION** and **FUSION — camera + EMG + IMU**. FUSION blends per finger. The camera's weight ramps up between finger confidence 0.30 and 0.70,

and each calibrated finger is attached at the camera hand's knuckle, so fingers never detach from the hand (tip offset 0.000 was verified). Fingers are coloured green when the camera drives them and orange when EMG drives them. The banner shows the buffer size, the coverage, the base and calibrated error against the camera, and a drift warning. The buttons are "↺ Reset calibration", "Simulate electrode drift" (adds a fixed per-finger bias of ±0.40–0.45 to the encoder so that recovery can be shown on demand) and "● Record session" (writes calib_logs/*.npz for offline analysis).

Problems found on recorded sessions, and their fixes

On 30–31 August the module was tested against recorded sessions, and four problems were found and fixed. Each fix was verified on the recordings:

1. **Every EMG pattern looked alike.** EMG features are all non-negative, so the raw cosine similarity was about 0.8 for everything. Z-scoring the features restored a real gap: 0.98 for familiar patterns against 0.33 for unfamiliar ones.
2. **"It only learns the last gesture."** A first-in-first-out buffer evicted the earliest posture, and plain softmax attention was dominated by the size of each cluster. Density-based eviction kept fist, point and open balanced (about 270 of 800 pairs each; 5 of 5 postures retained), and top-k attention let match quality decide. The drift residual had been measured against the blended estimate, which caused false buffer wipes; it is now measured against the nearest pair. After a real remapping, the buffer heals itself: the index finger re-learned from 0 to 0.76 over 600 frames.
3. **Out of view, the calibration barely engaged.** In the 22:05 session, wrist-orientation similarity stayed at 0.98–1.0 in every gate state, but the EMG itself shifted. Logged similarity read 1.00 in view and about 0.76 with the other hand in the way or out of view; the in-view value is inflated because each in-view frame was compared with a memory that already held it or the near-identical frames just before it. Recomputed leave-self-out on the 23:17 session (2,554 frames), it is 0.90 in view (CLEAR) against 0.78 out of view (EMG section). This shift is the limb-position effect (Fougner et al., 2011), analysed in the EMG section. Lowering the start of the coverage ramp from similarity 0.70 to 0.58 raised coverage, while random unfamiliar inputs still get 0. Smoothing the coverage cut engage/disengage flicker by 78%.
4. **Fixes ranked best in the literature failed on real data.** Adaptive normalisation, AdaBN and CORAL all made out-of-view posture identification worse (the numbers are in the EMG section). Two simple changes did help, and both were built into Live Calib: smoothing the query EMG over about 0.7 s, and the gated camera-teacher pseudo-labels described above. An ungated version of the pseudo-labels had hurt accuracy.

Results

Table 4.3 Live Calib results on synthetic data (module self-test). MAE = mean absolute error of finger curl (0 = straight, 1 = fully curled). Posture identification on the real occlusion recording is shown in the chart further below; the in-view versus out-of-view comparison on that recording is in the EMG section.

Test	Data	Result
Drift recovery (synthetic)	700 synthetic labelled pairs	Curl MAE 0.203 → 0.063 (69% lower)
Unseen wrist angle (synthetic)	Query at 40° (between seen angles) vs 200° (never seen)	Unsmoothed coverage 0.55 vs 0.00: interpolates, and fully defers to the encoder

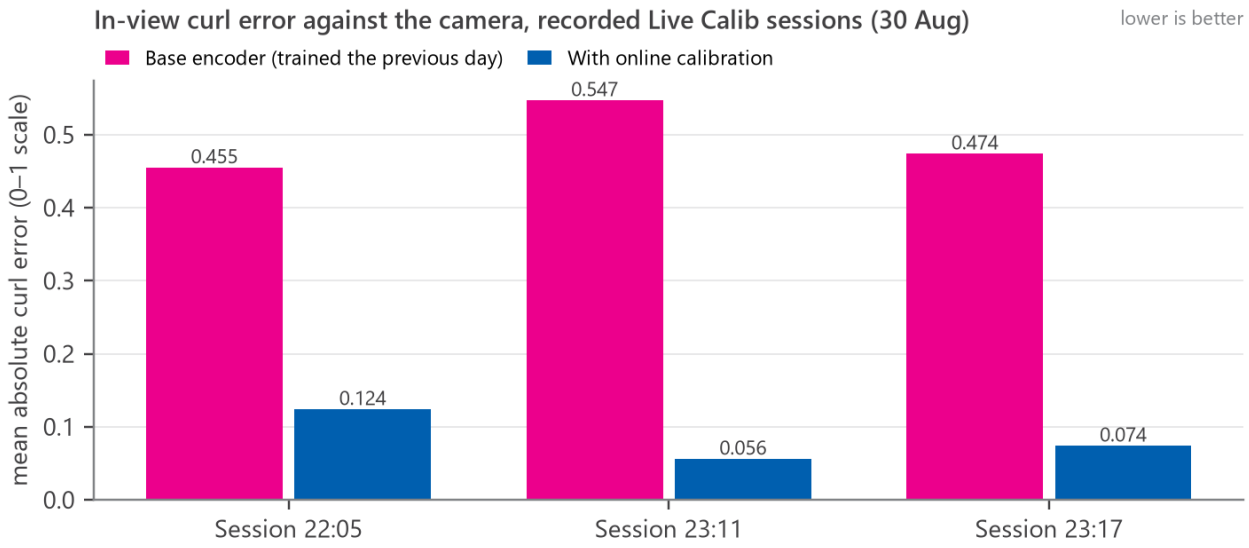


Figure 4.9 *While the camera sees the hand, the calibrated output follows the camera far more closely than the stale encoder (543–1,716 in-view frames per session). Caveat: the buffer already holds the same pose from moments earlier (the 23:17 session began with a full 800-pair buffer), so this shows correction within a session, not generalisation to new sessions.*

With no retraining and no calibration gestures, the online calibration brought the in-view finger-curl error against the camera from 0.455–0.547 (day-old encoder) down to 0.056–0.124 in three recorded sessions.

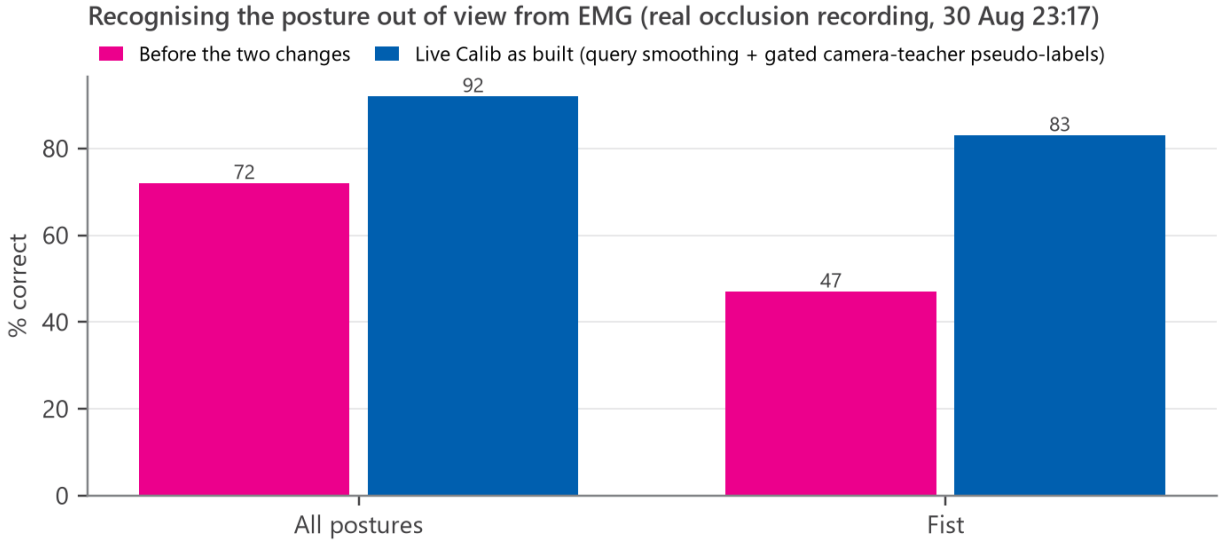


Figure 4.10 *The two cheap changes built into Live Calib raised out-of-view posture identification from 72% to 92% (fist 47% to 83%) on the 2,554-frame recording in which point, fist and open were each held in view and then out of view. The contribution of each change and the normalisation-based fixes that made it worse are in the EMG section; other probes of the same recording reported there started from different baselines and were not reconciled with these figures. These figures are recorded in the project notes and a code comment; the analysis scripts were not found in the repository.*

The out-of-view behaviour depends on the session. The table compares the three recorded sessions. In the later two, recorded with the new coverage ramp already live, engagement out of view was lower than the recomputation on the 22:05 session suggested.

Table 4.4 Calibration engagement (coverage: 0 = use the encoder, 1 = use the calibration) when the camera loses the hand, recorded sessions of 30 August. In view, EMG similarity was 0.96–1.00 and coverage 0.98–1.00 in the 23:11 and 23:17 sessions. In-view similarities are the logged values, which overstate the in-view match (see the EMG section).

Session · state (frames)	EMG similarity	Wrist-orientation similarity	Coverage
22:05 · other hand in the way (845)	0.76	0.98	0.44 → 0.71 (old → new ramp, recomputed)
22:05 · out of view (17)	0.75	0.99	0.30 → 0.69 (old → new ramp, recomputed)
23:11 · out of view (507)	0.77	0.78	0.52 (new ramp, recorded)
23:17 · out of view (817)	0.65	0.94	0.35 (new ramp, recorded)
23:17 · behind an object (21)	0.63	0.97	0.22 (new ramp, recorded)

Limitations

- The calibration can only correct postures and wrist angles the camera has labelled; it does not extrapolate to unseen ones. In the validation recording the fist was buffered over only ±37° of wrist angle, against ±74° for open and ±109° for point.
- The limb-position shift caps engagement out of view. EMG similarity fell to 0.65–0.77, so out of view the output was roughly 35–70% calibrated and the rest came from the encoder, depending on the session.
- The occluded fist's index finger stays soft (about 0.69 against 0.95 for a full fist). Fist and point differ only at the index, and that subtle EMG difference degrades out of view. The velocity decoder in the Tracking Hand tab (covered in another section) was the follow-up.
- Re-wearing the electrodes from one day to the next is a real problem: a model trained the previous day read the next day's fist as open. Live Calib corrects this only within a session, and only for what the camera has seen.
- The design differs from the proposal's wording. Instead of an affine correction of EMG latent vectors, it interpolates camera poses directly, using attention as a differentiable nearest-neighbour lookup.
- No formal Task 3.3 benchmark (convergence rate against window size, cross-session generalisation) was carried out. It remains future work.
- The recorded sessions do not store whether the drift simulation was switched on. Their base errors, with every finger under-curled, look like a stale encoder rather than the simulated ± bias pattern.

4.6 Status against the proposal and next steps

Table 4.5 Confidence, occlusion and calibration work mapped to the Mitacs proposal tasks

Proposal task	Planned for	Status
T1.2 Per-finger confidence	Internship 1	Done: Gate (per hand) and Occlusion (per finger); Quest flags shown to be whole-hand only
T3.1 Rolling observation-pair buffer	Internship 1	Done: 800 pairs, confidence-gated, density eviction
T3.2 Cross-attention calibration	Internship 1	Done, as attention over camera poses rather than an affine correction of EMG latents
T3.3 Convergence and cross-session benchmark	Internship 1	Not done; future work
T5.2 Per-finger occlusion by ray-casting	Internship 2	Brought forward; live since late August
T7.2 Buffer invalidation on large residuals	Internship 2	Brought forward: residual above 0.22 rebuilds the buffer

Recommended next steps, in order of expected value:

- Train a learned reliability head to close the gap between the hand-built gate (AUC 0.75) and the geometric ceiling (0.88), and calibrate its scores (Platt or isotonic scaling). This was planned in the gate specification for Internship 2.
- Bring the camera-against-IMU cross-check, already in Fusion Studio, into the gate itself, since it catches flips and hand swaps that geometry cannot see.
- Run the formal Task 3.3 benchmark: convergence against buffer window size, and accuracy across sessions and electrode re-wearing.
- Record each posture across the full range of wrist angles and with the arm lowered, so the buffer covers what the camera loses. The Posture Clusters analysis of 31 August found that a single fist spreads into three orientation clusters.
- Re-stage hand-behind-hand captures along the camera's line of sight to measure the OCCLUDED BY OTHER verdict fairly.
- Replace confidence-gated switching with transformer fusion trained with masked modalities (Internship 2, Task 6).

5 EMG-based finger and posture decoding

The forearm EMG armband is the only sensor in the kit that still carries finger information when the Quest 3 camera loses the hand. Across eleven tabs, I moved from recognising a few gestures to continuous per-finger decoding that keeps learning from the camera. A raw-EMG network reached 1.44 cm mean joint error in view, per-finger curl correlations reached 0.88–0.92, and the Pose Hand regressor, which is the hidden-finger fallback in the Fusion Studio release, cut its error from 0.167 to 0.089 within about 1.7 s of camera view on a simulated re-donned band. The main scientific finding is a sobering one: out of view, the limb-position effect shifts the EMG pattern, normalisation fixes made recognition worse, and fine single-finger differences remain the limit.

5.1 Why EMG, and how this section is organised

The Quest 3 camera loses the hand whenever it leaves the field of view or the fingers hide one another. The forearm muscles that move the fingers keep firing either way. That makes the MindRove armband (8 surface-EMG channels sampled at 500 Hz, worn on the forearm) the only sensor in the kit that still carries information about finger shape when the camera cannot see the hand. The CyberFinger's two IMUs, in the wrist module and the knuckle module (the 'joint IMU' in SciCortex's manual), tell the system how the hand is oriented but not what the fingers are doing. This section describes how finger shape was decoded from EMG, what worked, what did not, and why. In the Mitacs plan this is Task 2.1 (sEMG encoder for finger regression) and part of Task 2.3 (metrics and evaluation), and it feeds Sub-objective 3 (attention-based online calibration).

Four terms recur below:

- **TD features** are the five classic time-domain features of Hudgins, computed per channel: mean absolute value, RMS, waveform length, zero crossings and slope-sign changes. For 8 channels that gives 40 numbers.
- A finger's **curl** is its bend, from 0 (straight) to 1 (fully bent), derived from the camera skeleton.
- A **TCN** (temporal convolutional network) is a small 1-D convolutional network that reads the raw EMG window directly instead of hand-made features.
- The **Gate** is the occlusion classifier built in the Gate tab. It tags every frame CLEAR, SELF, OTHER, OBJECT or OUT_OF_VIEW, which tells the learning code when the camera's label can be trusted.

Most of this work was done between 25 and 31 August 2026. It was committed to the research branch in a single commit on 17 September (c7704d3, 'Add sensor-fusion hand tracking: EMG + IMU + optical pipeline'). Pose Hand (gravity) followed on 20 September (a118f35). The table below summarises the eleven tabs.

Table 5.1 The EMG tabs at a glance (details and measurement conditions in the subsections below).

Tab	Question it answers	Headline result	Status
EMG Hand	Can EMG pick user-defined gestures and draw them as a full hand?	100% cross-validation, but failed live 20 min later (drift)	Superseded; handoff logic reused
Encoder	Record synchronised EMG, IMU and camera training pairs	15-step protocol; up to 16,963 pairs per session	Data tool
Train	Does EMG predict the hand? Does IMU input help?	1.45 cm (EMG) vs 1.30 cm (EMG + IMU)	Baseline
Fusion Hand	Camera fingers when seen, EMG + IMU when not	TCN 1.44 cm; per-finger correlation 0.88–0.92	Demo
Tracking Hand	Integrate EMG-predicted finger change from the last camera pose	Integration drifted; amplitude openness used out of view	Experimental
emg2pose (ref)	How does Meta's pretrained model compare and transfer?	11.0° on its own data; fine-tune ties from-scratch	Offline reference
Posture Library	Record labelled posture × orientation clips	16 clips, 13,138 frames	Data tool
Posture Clusters	Can EMG separate postures across wrist angles?	Full coverage 99–100%; removing orientation backfires	Analysis tool
Posture Hand	Recognise a held posture, also out of view	89% on an unseen wrist angle	Working
Pose Hand	Continuous curls that keep learning from the camera	Error 0.167 → 0.089 in ~1.7 s of view	In Fusion Studio release (fallback)
Pose Hand (gravity)	Same, with the tilt taken from true gravity	Heading-invariant; 0.284 vs 0.319 across sessions	A/B tab

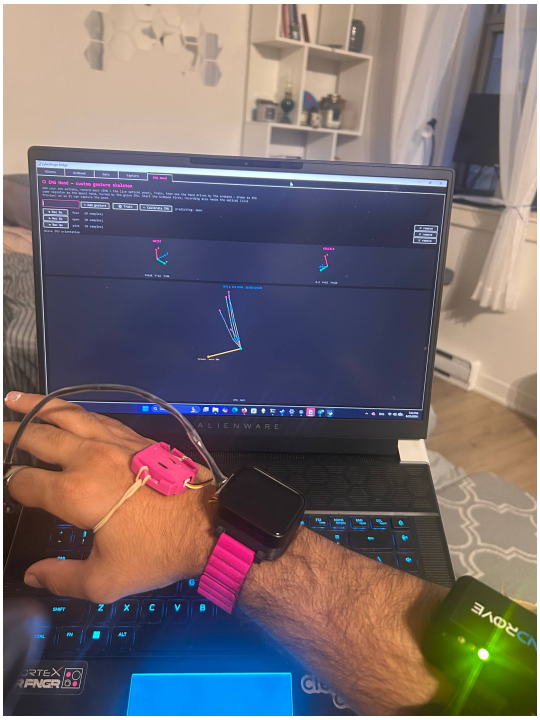


Figure 5.1 *All three sensors worn together on 25 August 2026: the MindRove 8-channel EMG armband on the forearm, the CyberFinger wrist module (pink strap) and knuckle module (pink, on the back of the hand), with the EMG Hand tab running. At this stage the tab turned the hand with the IMUs; the camera/EMG handoff was added the next day.*

5.2 EMG Hand

EMG Hand was the first end-to-end EMG pipeline. It was working by 25 August 2026 and refined on 26–27 August. The EMG output is drawn with the same 26-joint skeleton renderer used for the Quest's optical hand, so the EMG hand and the camera hand can be compared directly. The EMG hand takes over only when the camera loses the real one.

The user defines gestures and records each one while the camera watches the hand. A take lasts 7.5 s. The first 1.5 s are discarded while the contraction settles, and the next 6 s are sampled. Training needs at least two gestures with at least five samples each. A shrinkage linear discriminant analysis (LDA) classifier, with shrinkage 0.6, learns the gestures from the 40 TD features. At run time, EMG decides *which* gesture is held and the tab replays the camera skeleton stored for that gesture, so EMG never invents finger angles. The knuckle IMU turns the hand, and the wrist IMU turns an orange forearm bone, so wrist bend is visible.

The handoff design was settled on 26 August. While the camera sees the hand, the skeleton simply *is* the optical hand (verified identical, 0.0000 mm difference). When the hand is occluded, EMG supplies the finger shape and the IMUs rotate the hand from the last camera orientation, applied relative to that orientation so there is no jump. Cross-fades between sources are adaptive (about 40–200 ms) and preserve bone lengths. A naive blend of joint positions shrank the index finger by 21% at mid-fade; blending bone directions kept the lengths exact (0%).

Keeping the orientation right through an occlusion required aligning the IMU with the camera (detailed in the IMU fusion section). The IMU's rotation axes were about 28° off the camera's. An axis map fitted online cut the error of 1 s of dead reckoning through fast motion from 21° to 13°, and an offline fit cut it to 8°. The

knuckle IMU under-reports rotation (0.64 times the true rotation). Combining both IMUs (knuckle 10°, wrist 11°, combined 10°) did not get below a floor of about 10°, which I attribute to the flexible mounts.

The main lesson came on 27 August. The gesture model scored 100% in cross-validation on its own recordings but failed live about 20 minutes later. A code audit found no bug, and no classifier setting fixed it: shrinkage from 0.2 to 0.99, a nearest-centroid classifier, a magnitude threshold and even the armband's own 9-class model all failed on the same live features. The cause was surface-EMG drift: the pattern across the 8 channels changes between recording and use. The immediate remedy was to record and use a model in the same session. The longer-term answer, learning continuously from the camera, became Pose Hand. Four real-time improvements were kept:

- Predictions with confidence below 0.3 are rejected.
- A 3-frame debounce (about 50 ms) removes 1–2-frame flickers.
- The EMG window was cut from 128 to 64 samples (about 0.13 s at 500 Hz), roughly halving the detection delay.
- The gesture is held while the wrist turns faster than about 1.2° per frame (about 72°/s). This was a first guard against the limb-position effect (described below), in which turning the forearm changes the EMG for the same hand shape.

Figure 5.2 *The EMG Hand tab on 25 August 2026, before any gesture had been recorded (fist, open and pick show 0 samples). Top: wrist and knuckle IMU orientation triads. Bottom: the 'EMG + IMU hand', drawn with the same 26-joint skeleton as the camera hand and turned by the IMUs; the orange bone is the forearm from the wrist IMU. The finger shape shown is the last camera hand.*

EMG Hand handles discrete gestures only. For continuous fingers it was superseded by the Encoder → Train → Fusion Hand chain, which reuses its handoff and IMU-orientation logic.

5.3 Encoder

The Encoder tab records synchronised training pairs, which was the first step from snapping between a few gestures to continuous finger angles (Task 2.1). For every frame it stores:

- the 40 TD features;
- the camera's 26-joint pose, rotated into the hand's own frame so that only finger shape remains;
- the wrist and knuckle IMU quaternions;
- the raw EMG window (8 channels × 256 samples, about 0.5 s) and a timestamp;
- a label confidence from the Gate (CLEAR 1.0, SELF 0.5, OTHER 0.4, OBJECT 0.3, OUT_OF_VIEW 0.2), so frames where the Quest is guessing count less in training;
- from 29 August, the camera's hand rotation.

A 15-step guided protocol covers the wrist orientations the model must handle: four palm-down steps, two palm-up, two sideways, arm raised, arm low, and five fast-movement steps. The first capture on 28 August was unguided (12,172 pairs). The same evening it was followed by two guided sessions: 10 clips with 13,027 pairs, and 15 clips with 16,963 pairs that also stored the raw EMG. The 29 August set (3 clips, 4,113 pairs) re-recorded the protocol's 'curl each finger alone' step, this time palm to camera. It is also the only set that stores the camera rotation, so the later orientation-aligned models were trained on it.

5.4 Train

The Train tab fits a baseline encoder inside the GUI. Its purpose was to show that EMG predicts the hand and to test whether IMU input helps. It is a ridge regression (regularisation $\lambda = 10$) from [40 EMG features + wrist quaternion + knuckle quaternion] to the 26-joint pose (78 numbers). It saves a model file that Fusion Hand reloads. On the 13,027 pairs of the first guided session, the held-out mean per-joint error was 1.45 cm with EMG alone and 1.30 cm with EMG plus both IMUs, so conditioning on hand orientation helps. The absolute numbers are optimistic: the tab splits frames randomly 80/20, so neighbouring frames of the same clip fall on both sides of the split. (The tab labels the metric 'RMSE'; it is actually the mean Euclidean joint distance.)

A fairer offline test on the 15-clip session used a contiguous train/test split, confidence-weighted training, and scoring only on clean camera labels (confidence ≥ 0.9). Ridge regression reached 1.80 cm and a raw-EMG TCN 1.44 cm. Learning directly from the raw signal wins once fast movements are in the data, because the TD features average away the dynamics.

The TCN has four 1-D convolution layers (8 → 32 → 32 → 48 → 48 channels, kernels 7/5/5/3). The first and third layers downsample by 2, and the second and fourth are dilated by 2. Their output is mean-pooled, combined with the IMU quaternions as extra input, and passed through a 128-unit layer. I trained it in PyTorch and exported it to a pure-NumPy forward pass so the Windows GUI needs no PyTorch. The export matches PyTorch to 3×10^{-8} and takes 0.33 ms per frame.

Table 5.2 Continuous finger encoders. Rows 1–2 and rows 3–4 use different data and splits, so compare within each pair; the contiguous split is the fairer test.

Model	Inputs	Data and split	Mean joint error
Ridge (Train tab)	40 TD features	13,027 pairs, random 80/20	1.45 cm
Ridge (Train tab)	40 TD + wrist and knuckle IMU	13,027 pairs, random 80/20	1.30 cm
Ridge (offline)	40 TD + IMU	16,963 pairs, contiguous, clean labels	1.80 cm
Raw-EMG TCN (offline)	Raw EMG + IMU	16,963 pairs, contiguous, clean labels	1.44 cm

5.5 Fusion Hand

Fusion Hand demonstrates Tasks 2.1 and 2.2 working together, end to end. Three panels show the camera-only hand, the EMG + IMU-only hand, and the fusion: camera fingers when the hand is visible, encoder fingers turned by the IMU when it is not. It can run either the raw-EMG TCN above or a per-finger curl TCN, which maps raw EMG (8 × 128 samples) plus camera-aligned wrist orientation to five curls. The curls are smoothed with a One-Euro filter (min cutoff 1.5, beta 0.8), and the bones are rebuilt to reference lengths, which removed a 0.39 cm bone stretch.

The fist the camera cannot see

Building Fusion Hand exposed a limit of the sensor itself: the Quest cannot label a closed fist. In the 200 most-curled frames (up to 66° mean bend), the middle, ring and little fingers are drawn extended and crossing through each other, and every fist frame carries Gate confidence 0.5 (self-occlusion). The later posture data confirm this: all 3,497 fist frames are at 0.5, and none reaches 0.9. A model trained on these labels faithfully reproduces the errors. This is a data limitation, not a model bug.

The fix was an anatomical hand prior. It takes a clean open skeleton from confident frames and a flexion axis per finger, and curls the fingers procedurally, so the rendered hand is always plausible. A 72° thumb-opposition swing wraps the thumb over the fingers. The TCN's overall openness drives the closure, calibrated so an open hand maps to 0.02 and a fist to 0.95. Raw EMG amplitude could not do this job because flexors and extensors both fire (amplitude 54 for open against 52 for fist). An early sign error, in which the fingers curled backwards, was fixed by aligning each finger's axis with the open-to-fist direction found in the data (fingertips move from −1.8 cm to +2.2 cm towards the palm side).

The training recipe mattered as much as the architecture. An 80-epoch recipe kept the fingers responsive, with an open-versus-fist output gap of about 2.4 cm against 2.64 cm in the labels. A 200-epoch recipe with a cosine schedule and weight decay 'froze' at 0.19 cm, predicting the average hand. A perturbation test showed that wrist yaw was not what garbled the fist: a 90° yaw change moved the fist error only from 1.76 to 1.70 cm. I therefore did not pursue swapping the quaternion input for a gravity tilt in this network.

Per-finger control

In the first protocol steps the fingers moved together (finger-to-finger curl correlation 0.78–0.98), so any model could only open and close the whole hand. I added a 'curl each finger alone' step, recorded palm to camera with 100% CLEAR labels and weighted 2.5 times in training. It brought the finger-to-finger correlation down to about zero. The per-finger curl model then reached a mean curl error of 0.099 (previously 0.20), with per-finger correlations of 0.88–0.92. In recordings without such isolation clips, middle, ring and little-finger curls still correlate 0.93–0.99, so these three fingers are not decoded independently in normal use.

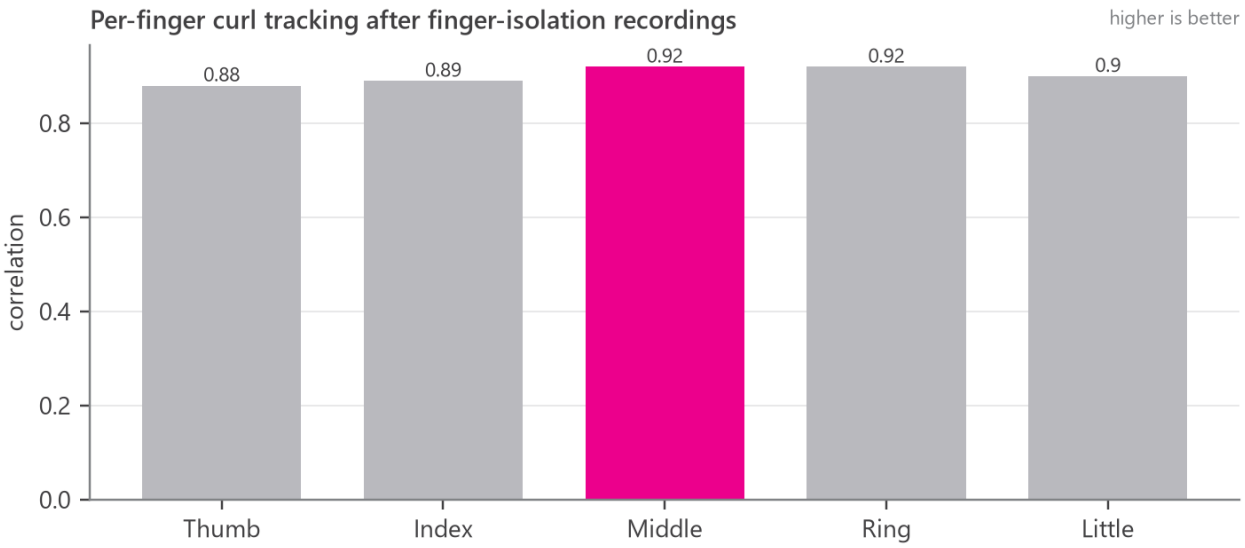


Figure 5.3 *Per-finger TCN (raw EMG + camera-aligned wrist orientation). Mean curl error 0.099, down from 0.20 before the 'curl each finger alone' step.*

Day-to-day drift appeared again. A model trained the day before read that day's fist as open, while a same-session model reached a curl error of 0.147 (open → 0.14, fist → 0.85). One process lesson: the original tcn_model.npz was overwritten during this work and lost. Since then, model files are backed up before retraining.

5.6 Out-of-view EMG: the limb-position effect

The central scientific finding of this section came on 30–31 August, while testing the attention-based online calibration of Sub-objective 3 (the Live Calib tab, described in its own section). Poses recognised in view were lost out of view. Re-analysis of the recorded sessions showed why: the EMG of the same held poses becomes less similar to the stored in-view examples when the hand leaves the camera's view. In the first session, wrist-orientation similarity stayed at 0.98–1.00 throughout. So the wrist angle was not the problem; the EMG itself shifts when the arm is lowered or moved out of view.

The similarity logged live overstated the gap. The logger compared each in-view frame against a memory that already contained that frame, so in-view similarity read 1.000. Recomputed leave-self-out on the 2,554-frame matched session, the values are 0.903 (CLEAR), 0.873 (SELF) and 0.778 (OUT_OF_VIEW). The shift is real but smaller than the raw logs suggested.

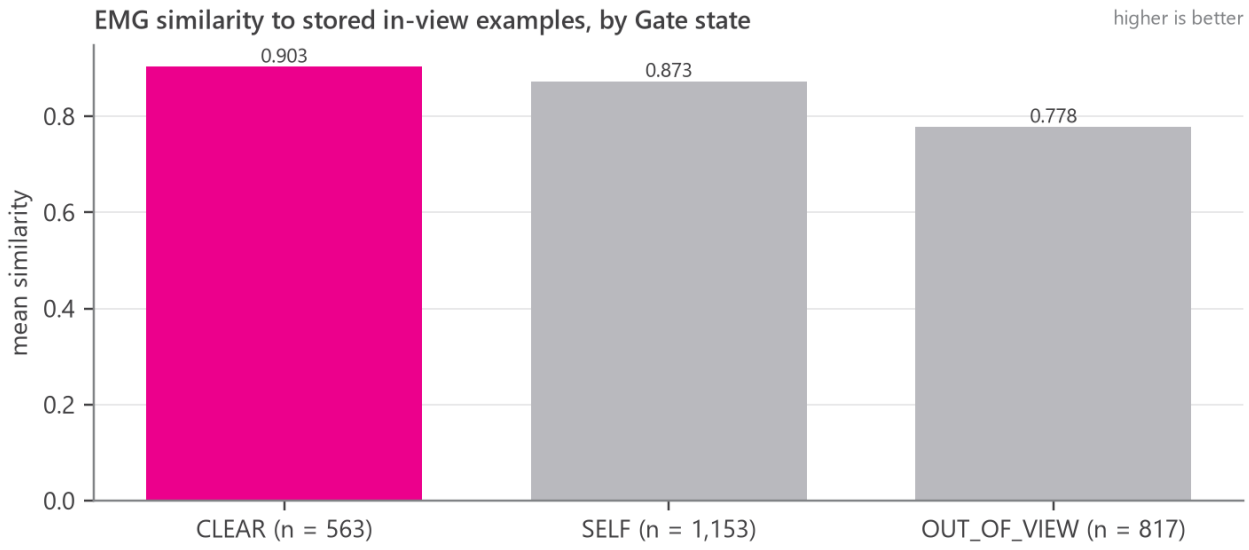


Figure 5.4 *Recomputed leave-self-out on the 2,554-frame matched session of 30 August. Out of view, the same poses give a measurably different EMG pattern.*

This is the limb-position effect described by Fougner et al. (2011). Changing arm posture changes gravitational loading and muscle co-contraction, so the same hand shape produces a different 8-channel pattern. Memory-based methods fail out of view for two reasons. First, the shifted query is normalised with in-view statistics, so it matches the wrong stored neighbour with false confidence. Second, the memory holds only camera-labelled in-view examples, so there is nothing recorded out of view to match against.

The matched recording (2,554 frames, 49 s) held point, fist and open each in view, then out of view. EMG pose identification was 89% in view. Out of view, a single-nearest-neighbour probe reached 75% (open 94%, fist 54%), and the full top-k blend about 90%. The fist was worst partly because the memory covered it most narrowly: fist examples spanned ±37° of wrist rotation, against ±74° for open and ±109° for point.

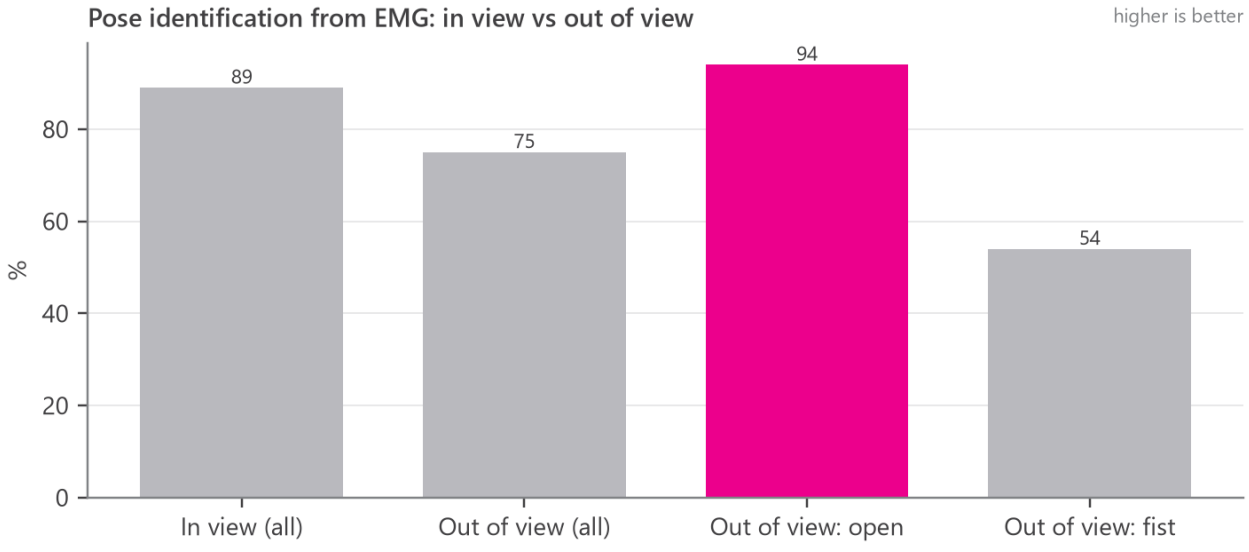


Figure 5.5 *Matched recording (2,554 frames, 49 s). Out-of-view values are from a single-nearest-neighbour probe; the full top-k blend reached about 90%.*

I then tested a literature-ranked list of fixes on the real recording. The measurements overturned the ranking:

Table 5.3 Out-of-view fixes tested on real data. The 72% smoothing baseline comes from a different probe from the 75% quoted above.

Approach	Out-of-view result	Verdict
Adaptive per-channel normalisation, AdaBN, CORAL, causal running normalisation	37–44% pose ID	Worse: the shift is not a clean scale or offset, and short-window normalisation erases the pose signal
Temporal smoothing of the EMG query (~0.7 s)	72% → 84% overall; fist 47% → 67%	Helped; adopted
Camera-teacher pseudo-labels for the first occluded frames	Ungated version increased the error	Adopted only gated: ≤1 s after losing the camera, stable wrist, estimates agreeing within 0.30
All variants, occluded fist	Index-finger curl plateaus at 0.67–0.69 (target 0.95)	Hard ceiling

The remaining limit is physical: fist and point differ only in the index finger, and that subtle EMG difference is swamped by the limb-position shift. I recorded the following candidate fixes for later work, in the literature's ranked order: adaptive normalisation per tilt bin; IMU-tilt bucketing with CORAL alignment; camera-teacher pseudo-labels; supervised affine alignment; covariance (Riemannian) features; a velocity decoder re-anchored by the camera; and an occlusion-invariant contrastive encoder. Covariance features, and recording postures where the camera cannot see, were later taken up by Key Postures. The general lesson: validate on real out-of-view data before shipping.

Out of view, the same hand shape produces a shifted EMG pattern (the limb-position effect). Pose identification fell from 89% in view to 75% out of view (single-nearest-neighbour probe), normalisation-based fixes made it worse (37–44%), and only temporal smoothing (72% → 84%) and gated camera-taught examples helped.

5.7 Tracking Hand

The proposal (Section 2.5) treats sEMG as inherently differential. In that view, the system integrates finger-pose changes from a baseline set by the camera, like GPS-corrected dead reckoning. Tracking Hand (30–31 August) tested this premise directly, following the 'Tracking' formulation of Meta's emg2pose, which predicts the change in pose rather than the pose itself. The hand holds the last camera pose, integrates EMG-predicted changes while hidden, and snaps back to the camera once its confidence reaches 0.6.

A two-head TCN reads a 128-sample raw EMG window plus seven conditioning numbers (camera-aligned wrist quaternion and tilt). It outputs five curls and five curl velocities. It was trained for 120 epochs (Adam, learning rate 2×10^{-3}, loss = position + 0.5 × velocity) on the 4,113 samples of the 29 August set, 3,195 of which have velocity labels, and it runs in NumPy (parity about 10^{-7}). Its position error of 0.105 is a training-set figure, because the training script has no held-out split.

Pure integration drifted (curl error 0.34, openness correlation 0.45). I added a complementary filter, which adds the integrated velocity (gain 1.2) while pulling slowly (0.06 per step) towards the network's absolute-position output, and is switched off while the wrist rotates. It reached 0.09 and 0.86. Both pairs of numbers come from in-view recordings the model was trained on, and this in-view validation did not capture

the out-of-view behaviour. A self-tuning version of the filter was worse both in and out of view, and I reverted it.

A real out-of-view recording (1,315 frames, 32 s) exposed the failure. On its 645 out-of-view frames, the absolute-position output sat near 'fist' (mean 0.85), and the velocity output read all muscle activity as closing (+1.31 per second). What worked was much simpler. The 8-channel log-RMS amplitude pattern is projected onto a fist-minus-open direction learned live from the camera and re-centred adaptively; the hidden hand eases towards this openness (blend 0.18 per update, suppressed while the wrist rotates). This feature separated open from fist in 77% of in-view frames, and the share of out-of-view frames drawn as open rose from 0% to 38%. In the final code, once both an open hand and a fist have been seen, the out-of-view hand is driven by this amplitude openness alone. The velocity integration runs only before that calibration.

With this hardware, this is a clear negative result for the differential premise. Pure integration of EMG-predicted change drifted, and out of view a simple amplitude feature was more robust than the orientation-conditioned network. Two limitations remain. Opening out of view is perceptibly late (the user's verdict: 'works but with delay'). And the user must show both an open hand and a fist to the camera before the feature is calibrated.

5.8 emg2pose (ref)

The emg2pose (ref) tab (31 August) is a reference benchmark. I ran Meta's pretrained vemg2pose Tracking checkpoint offline on the emg2pose mini dataset. It scored 11.0° mean joint-angle error on a 20-s window of one session. The tab replays its 607 frames (30 fps) as ground truth, prediction and overlay, with the per-frame error. The model cannot run on our band: emg2pose uses 16 channels at 2 kHz on the wrist, while ours has 8 channels at 500 Hz on the forearm. The dataset's licence (CC-BY-NC-SA 4.0) allows research use only, so it is not a default for SciCortex's product.

A cross-montage probe asked whether Meta's learned features still help on our hardware. I replaced emg2pose's input layer with an 8-channel one and trained the network on our data with a clip-wise 75/25 split (3,084 training and 1,029 validation frames). With the rest of the network frozen, pretrained features beat random ones by 18.4% in a first run and by 13.9% in a second, so the features do transfer to the forearm. But a warm-started fine-tune tied a network trained from scratch (curl error 0.0926 against 0.0915). I kept the from-scratch decoder: at this data volume, more recordings on our own hardware are the bigger lever. This probe is comparable to, but not the same as, the proposal's plan to pretrain on NinaPro or EMaGer; no NinaPro or EMaGer test was run.

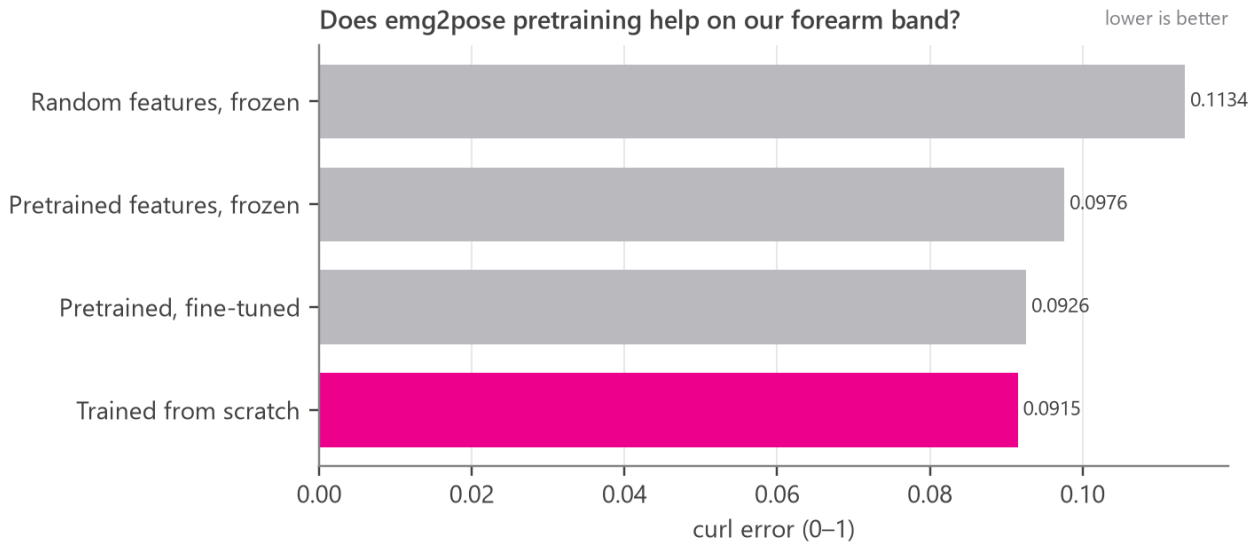


Figure 5.6 *Second run of the transfer probe (3,084 training frames). Pretrained features transfer (+13.9% over random), but fine-tuning only ties training from scratch.*

5.9 Posture Library

From 31 August the work shifted from free finger regression to postures, whose labels are more reliable. Posture Library records labelled clips per posture and wrist orientation, in the same format as the Encoder. On 31 August it collected 16 clips with 13,138 frames: four clips each of fist (3,497 frames), ok (3,245), open (3,076) and point (3,320), held palm up, palm down and sideways, plus a sweep between orientations (for open, a clip recorded live in Posture Hand instead). In this data 'ok' is a thumbs-up shape: thumb out, fingers curled, with canonical curls of 0.21, 1.00, 0.97, 1.00 and 0.96 from thumb to little finger. Only the open clips (and 32 point frames) carry confident camera labels; every fist and ok frame has confidence 0.5. So even the canonical thumbs-up shape is based on frames the camera was guessing.

5.10 Posture Clusters

Posture Clusters asks a direct question: can the EMG features tell postures apart, and how much does wrist orientation get in the way? It plots every clip's features in a rotatable 3-D principal-component (PCA) view. It reports how separable the clusters are by posture and by posture × orientation, and it runs a head-to-head of approaches with one orientation held out.

One posture at three wrist angles formed three separate clusters. A fist held palm up, palm down and sideways (877, 594 and 761 frames) could be classified by its orientation 76% of the time. The cluster centres pointed in nearly opposite directions (cosine −0.83 for up against down, −0.72 for up against side), and all 40 features shifted. Orientation dominates the EMG variance, which visually confirms the limb-position effect.

Yet postures stayed separable across orientations. With logistic regression trained on two angles and tested on the third, fist vs open reached 98% with raw features. 'Orientation-invariant' transforms backfired: projecting onto an invariant subspace gave 92%, and regressing out the wrist tilt only 24%. On the subtle fist vs point pair, which differ only in the index finger, IMU conditioning helped (92% → 95%). Recording all orientations gave 99–100% on every pair. The live tab shows quicker nearest-centroid estimates of the same

comparison, which differ slightly.

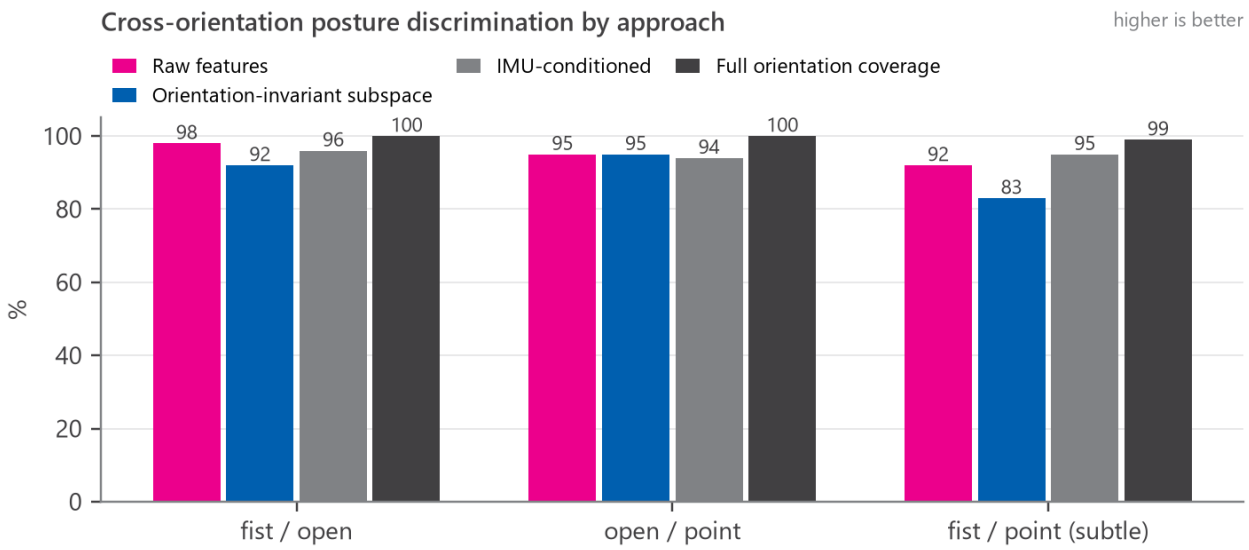


Figure 5.7 *Logistic regression trained on two wrist orientations and tested on the held-out one (chance 50%). Removing orientation backfires, IMU conditioning helps only on the subtle pair, and recording all orientations is best.*

A decision-boundary view projects the features onto three LDA dimensions (keeping 86% of the separation) and draws the deployed linear classifier's boundaries as planes. The largest shared boundary is between fist and ok, the pair later found hardest to separate. The tab's resulting guidance: record full orientation coverage, use IMU conditioning for unseen angles, and never blindly remove orientation.

5.11 Posture Hand

Posture Hand is a separate EMG + IMU posture recogniser. It identifies which posture is held and draws that posture's canonical hand (the median camera curls of its clips), including out of view. It is a class-balanced softmax classifier on 43 inputs (40 TD features + a 3-axis wrist tilt) with its own model file. On 6,104 frames (fist, open and point at three orientations) it scored 99.9% on the training data and 89% with one orientation held out. The current model has four classes: fist, ok, open and point. The table below summarises the experiments behind its design.

Table 5.4 Posture Hand design experiments.

Experiment	Result	Consequence
Class balancing	Point recall 98–100% (point was slow to detect before)	Adopted
Five classifier families (logistic, LDA, QDA, kNN, MLP), 4 classes, unseen wrist angle	All 74–76%	The limit is in the features, not the classifier
Add knuckle-IMU tilt, then wrist-to-knuckle rotation	75% → 63% → 50%; fist vs ok 74% → 53% (EMG alone 76%)	Rejected: the knuckle module sits on the metacarpals and cannot see the thumb
Synthetic EMG augmentation	Clean 95% → 90%; simulated re-donned 46% → 75%	Adopted (synthetic weight 0.4)
Persistence gate (0.15 s hold)	0 switches in 10 s of noisy hold; a real change in 133 ms	Adopted

The augmentation is grounded in what happens physically. It rotates the electrodes by one channel, as when the band is put back on slightly turned; it varies gain and contraction intensity; it adds noise; and it jitters the tilt. It gives up a little clean accuracy for a large gain under a simulated re-donned band. For context, the proposal cites 20–30% inter-session accuracy drops from the literature (Buteau et al., 2024). Here, a simulated re-donning (a one-channel band rotation plus gain and noise) dropped accuracy from 95% to 46%.

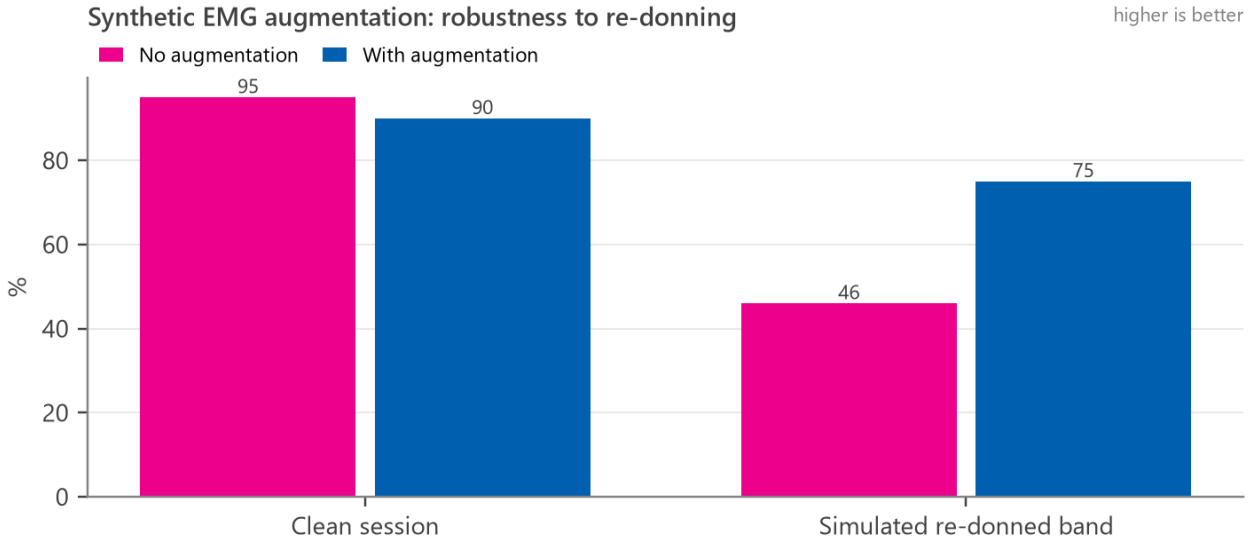


Figure 5.8 *Posture recogniser accuracy. The re-donned test is simulated: a one-channel band rotation plus gain and noise.*

A switch requires a top probability above 0.55 and a margin above 0.15. The 0-switch and 133 ms figures describe a fixed 0.15 s hold. The committed gate is confidence-adaptive: the hold shortens for confident predictions (hold = 0.15 s × (1 − 0.85 × strength)), down to about 23 ms. The tab also offers in-app fine-tuning and an active-learning recorder that logs frames where the camera clearly sees a posture the model got wrong.

5.12 Pose Hand

Posture Hand snaps between fixed shapes. Pose Hand (31 August) is the continuous alternative: it regresses five finger curls, so any in-between hand can be drawn, and it keeps learning from the camera while in use. A weighted ridge regression ($\lambda = 5$) maps the same 43 inputs to five curls, which drive the anatomical hand oriented by the IMU.

Its training labels come free from the camera, but selectively. It uses camera curls only where the camera is confident (3,108 frames: 3,076 open and 32 point), the fist prior (0.90, 0.95, 0.95, 0.95, 0.90) for the occluded fists (3,497 frames), and nothing for the rest (6,533 ok and point frames). On a random 80/20 frame split the per-finger correlation was 0.90–0.91. Because the trusted targets are essentially two shapes, this number mostly measures how well open and fist are separated. A point came out too soft (about 0.57 on every finger, instead of index 0.13 and the others near 0.9), because these data had no finger-isolation clips. The offline model is weak on its own; online learning does most of the work.

Every frame in which the camera sees the hand applies one normalised least-mean-squares (NLMS) update to the weights. On a simulated re-donned session, the curl error fell from 0.167 to 0.089 after about 50 in-view

frames (about 1.7 s), and to 0.075 at convergence. Two problems were found and fixed:

1. **Starved learning.** The whole-hand confidence is high almost only for an open hand (share of confident frames: open 100%, point 1%, ok 0%, fist 0%). I therefore gated learning per finger with a ray-cast visibility estimate, weight = clip((confidence − 0.5) / 0.4), which is the per-finger ray-casting idea of the proposal (Task 5.2). A point's visible index finger is now taught: its predicted curl moved from 0.60 to 0.08, against about 0.16 from the camera.
2. **Catastrophic forgetting.** After a spell of open-hand-only frames, the point's thumb curl drifted from 0.75 to 0.25. A 300-slot rehearsal buffer, which evicts its most redundant samples to keep variety, replays 8 stored samples at every update. With it, the thumb stayed at 0.72.

Learning online from the camera, Pose Hand cut its finger-curl error on a simulated re-donned band from 0.167 to 0.089 within about 1.7 s of camera view, and a rehearsal buffer kept it from forgetting rarely seen postures.

Pose Hand's model file (posture_pose_model.npz) supplies the hidden fingers in Fusion Studio and in SciCortex's Fusion Studio release. In the current release, a recognised Key Posture takes priority and Pose Hand is the fallback. Its 'Save refined' button deliberately writes the online-refined weights back into that shared file.

5.13 Pose Hand (gravity)

The IMU FK Position work established that the CyberFinger IMU's world frame is z-up. Pose Hand's 'gravity tilt', computed as $R(q)^T[0, -1, 0]$, is therefore not gravity: it is a horizontal axis that turns with the IMU's drifting heading. Pose Hand was already in use, so instead of changing it I added Pose Hand (gravity) on 20 September. The new tab has its own trainer, model file and online-learning state, and it computes the tilt from true gravity, $R(q)^T[0, 0, -1]$. A shadow model with the old axis learns from the same frames with the same rule, which gives a live, fair A/B comparison. The change was purely additive: 212 lines added and none removed, with Pose Hand's model file verified byte-identical (commit a118f35).

With the hand unchanged, turning the heading by 30°, 90° or 180° swings the old feature by exactly that angle and the gravity feature by 0.00°. In the real update loop with learning off, a 120° heading turn changed the gravity model's curls by 0.0000, against 0.156 on average (0.40 at most) for the old axis. Across sessions (trained on 31 August, tested on clips from 17 and 19 September), the mean curl error was 0.319 for the old axis, 0.284 for gravity and 0.343 with no tilt. Gravity was better on 4 of 5 clips.

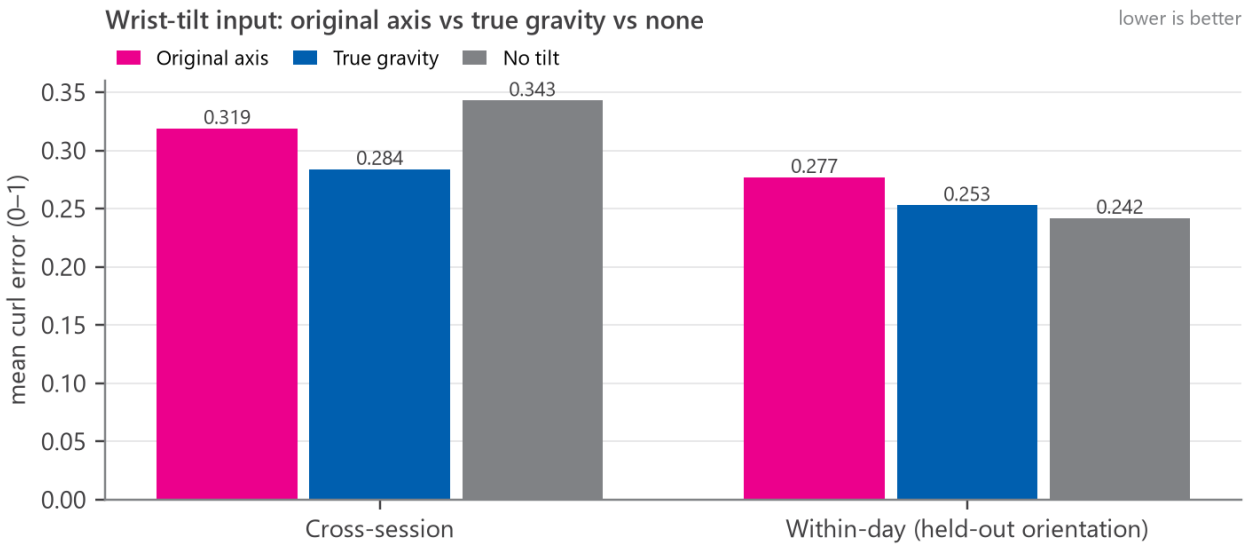


Figure 5.9 *Gravity tilt is best across sessions (better on 4 of 5 clips); within a single day, using no tilt is marginally best.*

The picture is not one-sided. Within a single day (with one orientation held out), using no tilt at all was marginally best: 0.242, against 0.253 for gravity and 0.277 for the old axis. With online learning, both versions re-converge while the camera sees the hand (0.035 old against 0.033 gravity after a 120° heading jump). The difference matters out of view, where nothing can re-teach a heading-dependent feature. The later Key Postures recogniser went further and dropped the tilt input entirely, after measuring that it acted as a shortcut. With one arm position held out, accuracy rose from 88.7% to 91.4% and wrong switches fell from 12.1 to 9.0 per minute; a true-gravity tilt was worse (15 per minute). The Fusion Studio release still uses Pose Hand with the original axis.

5.14 Findings, limitations and next step

- In view, EMG decodes hand shape well: 89% pose identification, 1.44 cm mean joint error with a raw-EMG TCN, and per-finger curl correlations of 0.88–0.92 after finger-isolation recordings.
- Out of view, the limb-position effect shifts the EMG. Normalisation tricks failed on real data, temporal smoothing and gated camera-taught examples helped, and the fist-versus-point index finger remains the ceiling.
- The camera is a free teacher only where it is confident, and it cannot label a fist, so anatomical priors and per-finger visibility gating are essential.
- Session-to-session drift is real: 100% cross-validation turned into live failure within 20 minutes, and a day-old model read a fist as open. Same-session data, online learning from the camera and electrode-rotation augmentation are the answers that worked.
- Data, not model choice, is the bottleneck: five classifier families tie at 74–76%, and emg2pose pretraining only ties training from scratch.
- The knuckle IMU adds orientation but no thumb information; adding it to posture recognition made results worse.
- Integrating EMG-predicted finger change, the proposal's differential premise, drifted; out of view, a simple amplitude feature was more robust.

Limitations

- All recordings come from one wearer in the project's own sessions; there is no cross-user evaluation.
- Several robustness numbers come from simulated conditions (simulated re-donning, synthetic heading turns).
- Some evaluations are optimistic. The Train tab and Pose Hand use random frame splits, and the velocity decoder's figures are in-sample. The scripts behind the TCN comparisons (1.44 cm and the per-finger correlations) are not in the repository, so their splits cannot be re-run.
- The middle, ring and little fingers move together in most recordings, and per-finger results depend on recording the fingers where the camera can see them.
- Tracking Hand's out-of-view opening is perceptibly delayed and needs an open hand and a fist shown to the camera first.

These results shaped the next step, Key Postures (described in its own section). For a held posture, the label is simply whatever the user is holding, so it can be recorded where the camera cannot see. That addresses both the problem that the camera cannot label a fist and the limb-position problem. Key Postures started from the Posture Hand recipe (40 TD features + tilt, the same augmentation, a class-balanced softmax). It later moved to covariance features and dropped the tilt input.

6 IMU–optical orientation fusion and sensor-mount calibration

The CyberFinger has two motion sensors (IMUs), one in the wrist module and one on the back of the hand. They keep reporting how the hand is turned when the headset camera cannot see it, but each reports in its own arbitrary frame. I built the calibration that maps them onto the camera in two stages. The first was a self-calibrating complementary filter that matched the camera to about 11°. The second was a camera-taught sensor-mount calibration: after the camera loses the hand, it keeps the knuckle sensor within a median of about 9° of the camera for 30 s and more, where the earlier form was 50° off after 1 s and over 100° off after 7 s. This gave the Fusion Studio a live wrist bend out of view and a check that rejects the headset's own hand flips. SciCortex's SteamVR driver then adopted the same two-sided calibration model.

6.1 Why the CyberFinger's motion sensors needed calibrating

When the hand leaves the headset camera's view, the only sensors that still know how it is turned are the CyberFinger's two inertial sensors. One is in the wrist module, which is worn like a watch. The other sits in a small clip on the back of the hand. SciCortex's manual calls it the *joint IMU*; this report calls it the *knuckle IMU*. Before these sensors can stand in for the camera, their readings must be expressed in the camera's coordinate frame. This section explains how that was done, what went wrong along the way, and how accurate the final calibration is.

Both sensors are 6-axis: an accelerometer and a gyroscope, with no magnetometer (compass). The CyberFinger firmware fuses each sensor on the device with its own VQF orientation filter and sends only the resulting orientation. Gravity fixes the tilt (pitch and roll), but nothing fixes the compass heading (yaw). Each sensor's heading starts at an arbitrary zero when the CyberFinger is switched on, then drifts slowly and independently of the other sensor. Neither the firmware nor the PC bridge applies any offset or mounting correction, so the orientations arrive unchanged. The raw gyroscope is not transmitted, and the accelerometer arrives as raw 16-bit values at 2048 counts per g (±16 g). I confirmed all of this from SciCortex's public firmware source on 2026-08-18 and documented it in docs/firmware-and-imu.md.

- **IMU** (inertial measurement unit): a chip with an accelerometer and a gyroscope that measures how it is turned.
- **Quaternion**: four numbers that describe a 3-D rotation. Here they are scalar-first [w, x, y, z] with the Hamilton product (⊗).
- **Extrinsic calibration (R_off)**: the rotation that maps an IMU orientation onto the orientation the camera sees.
- **Mounting (M)**: the fixed rotation of a sensor relative to the body segment it is strapped to.
- **CLEAR**: the occlusion gate's verdict (Gate tab) that the camera currently sees the hand properly.
- **Complementary filter**: a simple filter that trusts one signal for fast changes (here the IMU) and another for the slow absolute reference (here the camera).

- **Procrustes fit**: the standard least-squares method for finding the rotation that best aligns two sets of orientations.

Table 6.1 The CyberFinger's two inertial sensors as used in this work (docs/firmware-and-imu.md; hardware map confirmed from photos on 2026-08-25).

Sensor	Firmware slot	Chip	How it was worn	Turns
Wrist IMU	BODY (g2 columns)	QMI8658	CyberFinger wrist module on a solid strap	the forearm
Knuckle (joint) IMU	JOINT (gj columns)	ICM-45686	3D-printed clip on elastic bands, back of the hand	the hand

Figure 6.1 *The CyberFinger's two IMUs on 2026-08-25: the wrist module on its pink strap and the pink knuckle (joint) IMU clip held by an elastic band on the back of the hand, with the finger module on the index finger.*

Orientation work in the Gloves tab

The Gloves tab connects to the CyberFingers. For this work it had to guarantee two things: that every research tab reads the right sensor, and that the exact firmware version is known. The version decides which data exist at all. IMU streaming arrived with firmware v1.3.0/v1.3.1 (v1.3.1 is dated 2026-07-28), and the older 1.0.1-beta SteamVR build sends a 12-byte report with no IMU data. I therefore added a reader (BLEManager._read_firmware_revision) that reads the exact version over Bluetooth from the Device Information Service (0x180A, Firmware Revision String 0x2A26) and logs it on connect.

On 2026-09-30 both of the lab's CyberFingers were flashed to 1.4.1-beta+f48c87f, which SciCortex's SteamVR driver 2.1 requires. Firmware 1.4.x sends the wrist sensor in the BODY 1 slot instead of BODY 2. A one-way BODY 2 → BODY 1 fallback (commit 6ea858d) lets every tab find the sensor, and the recorder still writes it to the same columns, so all models trained on earlier recordings stayed valid. The tab's Resync IMU button came with SciCortex's v2.1 merge and is not part of this work.

6.2 Orientation work in the EMG Hand tab

The EMG Hand tab is where the IMUs were first tied to the camera (Task 2.2 of the proposal), between 2026-08-25 and 2026-08-28. Its EMG side is described in the EMG section. The goal was a hand that does not jump or vanish when the camera loses it. While the camera sees the hand, the skeleton *is* the camera's hand, lightly smoothed. When the hand is hidden, the IMUs take over: they turn the last camera orientation by the rotation they have measured since then. The handoff is therefore smooth (median jump 5°). An orange forearm bone is always drawn from the wrist sensor.

Chips above the skeleton show how well each sensor currently agrees with the camera, as "wrist X° knuckle Y°": green below 25°, orange below 45°, red otherwise. A "hand: knuckle/wrist/auto" button chooses which sensor turns the hidden hand. In auto mode, it switches between sensors with an 8° hysteresis on their match errors; the default is the knuckle sensor.

Why a complementary filter rather than a Kalman filter

The Mitacs proposal planned that "a Kalman filter fuses IMU wrist orientation with the optical signal (T2.2), reducing jitter and enabling short-dropout dead-reckoning". Inspecting the firmware changed that plan. The CyberFinger already integrates the fast motion and locks the tilt to gravity on the device, and it sends only the fused orientation. On the PC, the hand orientation is therefore always q_hand = q_off ⊗ q_imu. The only unknown left is the slowly varying offset q_off. Its roll and pitch are physically constant, and only its heading drifts.

With no raw gyroscope signal left to fuse, the planned Kalman filter reduces to a complementary update of q_off (fusion/orient_fusion.py). On every CLEAR frame, q_off is moved (slerped) toward q_opt ⊗ conj(q_imu) by the fraction $\alpha = 1 - e^{(-dt/\tau)}$, with $\tau \approx 1$ s. The first CLEAR frame sets q_off directly instead of crawling toward it. When the hand is not CLEAR, q_off is frozen and the hand dead-reckons on the IMU. The gate's verdict is the only trust switch. This is because the Quest keeps reporting the hand as active (is_active = 1) during occlusion while it emits a moving, plausible-looking but wrong pose. The change of filter type is an engineering finding about the hardware, not a missing deliverable.

Estimating the IMU-to-camera rotation

Three numpy-only modules in fusion/ do the calibration and its evaluation. They were committed with the research pipeline in c7704d3 (2026-09-17).

- **fusion/extrinsic.py** builds a camera hand frame from the headset's 26 hand joints. The forward axis runs from the wrist to the midpoint of the index and pinky metacarpals, and the across axis runs from index to pinky, signed by handedness. On every CLEAR frame it computes R_off = R_opt · R_imu^T and then takes a robust average: an SVD projection followed by Huber weighting at 15°, so frames with a bent wrist count less. A swing-twist split separates the constant tilt from the drifting heading.
- **fusion/orient_fusion.py** holds OrientationFuser, the live complementary filter described above.
- **fusion/run_fusion_on_log.py** scores a recorded clip three ways: the fit residual, convergence from a deliberately 180°-wrong start, and a dead-reckoning hold-out (fit on the first 5 s, freeze, then score against the still-visible camera).

The choice of camera frame mattered. A "metacarpal-plane" frame on the back of the hand, first intended for the knuckle sensor, turned out to be ill-conditioned. Its plane normal flipped in 68 of 387 frames, and even the wrist sensor fitted it at 77–82° (median). The live GUI therefore fits both sensors to the wrist/palm frame.

Accuracy against the camera

Table 6.2 Wrist-IMU calibration against the camera's wrist/palm frame. Scored with fusion/run_fusion_on_log.py; the numbers were reproduced on 2026-10-28 with the unchanged script.

Clip (right hand)	CLEAR frames	Residual median	Residual 90th pct	Steady error after 180° wrong start
open_right_20260820_183421	339	12.6°	28.1°	9.8°
open_right_20260820_183723	264	10.6°	22.9°	6.4°
rigid_hold_right_20260825_184146	388	15.9°	27.3°	14.5°

On the open-hand clips, the wrist sensor matched the camera to about 11°, and from a start 180° wrong it settled at 6–10°. Most of the residual comes from wrist articulation: the watch rides on the forearm, but the camera frame sits on the palm. Only about 4° of it is heading error. After calibration, the rendered hand matched the camera's hand to about 1.7 cm per joint.

Continuous self-calibration and the ⊹ Calibrate IMU button

A saved R_off does not carry over between sessions, because the heading origin resets at every power-on. Testing a saved R_off against an older clip gave a 76° heading and 29° tilt mismatch. Two clips recorded 3 minutes apart even had extrinsic headings of +74° and +165°. Later measurements put the slow heading drift at about 1°/min, so most of that difference is not drift. A one-sided offset also changes with hand pose, which is explained below. The live design therefore never relies on a fixed offset. Two fusers run on every update cycle: wrist → forearm and knuckle → hand. Each snaps to the camera on the first CLEAR frame and re-snaps whenever the camera finds the hand again.

The "⊹ Calibrate IMU" button resets both fusers and reports both fit residuals. From 2026-08-28 it also saves the result to bridge/imu_calib.npz, which is loaded at start-up, so the IMU-driven hand is oriented even with the camera off. Because the heading resets, the button must be pressed again after the CyberFinger is power-cycled.

What the diagnosis uncovered

The first finding was a rendering bug. The forearm bone had been drawn from the raw wrist quaternion, which was 82° off; with R_off applied it was 14.7° off.

The second finding was a loose mount. The knuckle sensor's tilt wobbled by about 57° even while the hand was held still. I ruled out software causes first, including a brute-force search over all 48 possible quaternion axis orders. That search found a 15° fit to the camera, but the fit failed a held-out consistency check against the wrist sensor at 95°: it was overfitting, not a real convention error. The fix was physical. Once the elastic band of the same clip was tightened, the live per-sensor read-outs fell from about 57–76° to "wrist 2° knuckle 2°" and "wrist 7° knuckle 5°".

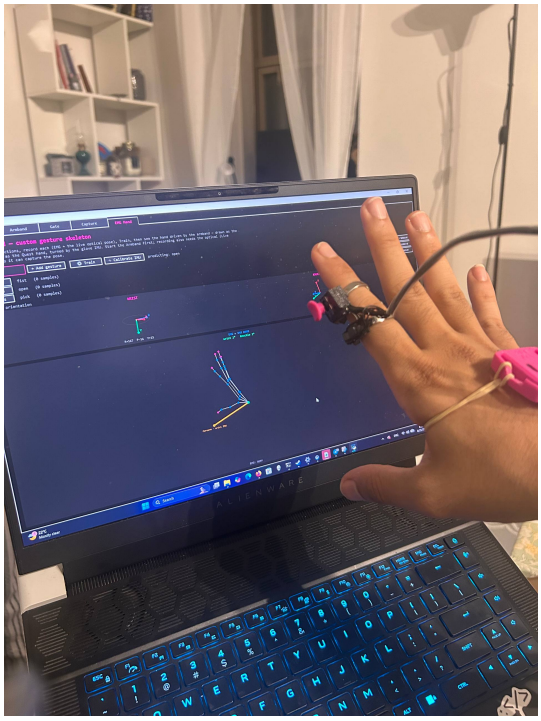


Figure 6.2 *The EMG Hand tab live on 2026-08-25, after the knuckle clip was tightened. It shows the WRIST and KNUCKLE orientation triads, the hand skeleton with the orange forearm bone from the wrist IMU, the "⊹ Calibrate IMU" button and the per-sensor match read-out "wrist 2° knuckle 2°".*

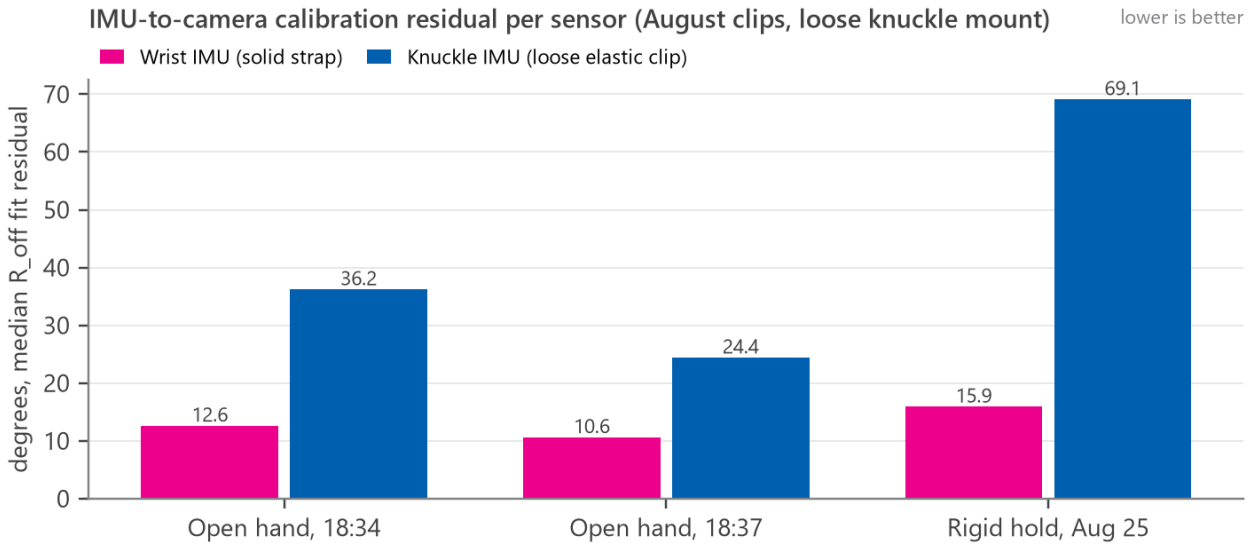


Figure 6.3 *Offline fit residuals on clips recorded before the knuckle clip was tightened: open_right_20260820_183421, open_right_20260820_183723 and rigid_hold_right_20260825_184146, all fitted to the wrist/palm camera frame and recomputed on 2026-10-28. The knuckle sensor did 2–4 times worse than the wrist sensor. The live 2–7° values quoted in the text are instantaneous smoothed read-outs, not median fit residuals, so they are not directly comparable to these bars.*

I also checked timing. Cross-correlating the IMU's and camera's rotation speeds gave the best alignment at zero lag (correlation 0.645, against 0.34 for a one-frame shift). The two streams in the research pipeline were therefore aligned to within one frame (under 47 ms). The large disagreements during fast motion came from

the camera: its frame-to-frame jitter was 5.5° moving against 1.8° still. Since the IMU did not lead the camera in this pipeline and the still-hand jitter was only 0.27° per frame, the IMU was not used to smooth the visible hand. The perceived jitter was fixed instead by raising the GUI's update rate from about 22 Hz to about 60 Hz, and by a speed-gated low-pass on the visible hand that reduced still jitter from 0.27° to 0.22° per frame and leaves motion untouched.

Out-of-view dead-reckoning in the EMG Hand tab

When the hand was hidden, the IMU's rotation axes turned out to be about 28° off the camera's. An online hand-eye rotation W, a slowly forgetting Kabsch fit on angular velocities, maps IMU rotations into camera axes. It cut the error after a 1-s occlusion from 21° to 13°, against 8° when W was fitted offline on all the data. In fast turns, the knuckle sensor reported only 0.64 times the true rotation. Combining the two sensors did not help: knuckle 10°, wrist 11°, combined 10°. At the time, this ~10° floor was attributed to the hardware (flexible mounts and under-rotation). With hindsight, W corrected the sensor's axes on the body side. It was an early, partial form of the mounting correction that the Fusion Studio later made complete.

Negative results kept as lessons

- **Decomposed fusion, reverted.** I tried taking the global orientation from the wrist IMU and the wrist bend by differencing two separately calibrated fusers. The result was a persistent false bend of 30–38°, because the bend carried both sensors' errors. Lesson: do not derive a small relative quantity by differencing two noisy absolute estimates.
- **Axis-order brute force, rejected.** The best of 48 quaternion axis orders fitted the camera at 15° but was 95° off on a held-out check. It was overfitting.
- **Calibrating only on slow frames, reverted.** An intermediate version learned R_off and scored the match only on frames turning slower than 3° per frame (about 75°/s). The read-out became steadier (mean 15.9° → 11.1°, std 5.0° → 3.6°, max 33° → 21°). However, the heading drift then went uncorrected and the hand diverged by about 90°. Calibration was made continuous again (wrist vs camera 15°, 4 cm joint gap), and the committed GUI scores the match on every CLEAR frame.

6.3 Orientation tests in the Capture tab (rigid_hold)

To test IMU dead-reckoning without finger movement contaminating the result, I added a "rigid_hold" row to the Capture tab. The user holds a fixed hand shape and swings the whole forearm, and the recording stops by itself after 20 s with a live countdown. Clips are scored offline: R_off is fitted on the first 5 s, then frozen, and the IMU-driven hand is compared with the still-visible camera.

The clip rigid_hold_right_20260825_184146 was clean: 388 CLEAR frames, fingertip movement below 0.1 cm (standard deviation), and an arm sweep of 179°. The wrist sensor's calibration residual was 15.9°.

Table 6.3 Dead-reckoning with a frozen offset on the rigid_hold clip (fusion/run_fusion_on_log.py, rerun 2026-10-28). The original note summarised this as "flat 13–17°"; the rerun's 5-s value is 19.3°.

Time after freezing R_off	Wrist IMU error vs camera
0.5 s	17.3°
1 s	17.1°
2 s	13.9°
3 s	13.0°
5 s	19.3°

The error did not grow over 5 s. Instead it sat at a static floor of about 13–19°. Smoothing did not reduce it, and only 3.6° of it came from viewing angle. A second clip on 2026-08-26 (flat, then bend, rotate and wiggle; 455 CLEAR frames) showed where the floor comes from. A constant R_off fitted on the flat segment matched the camera to 3° for both sensors there, but through the bends the error rose to 42° (max 69°) for the wrist and 49° (max 180°) for the knuckle. For that clip the knuckle sensor had been moved to a finger temporarily, which was later undone. The lasting lesson was that an IMU can only drive the orientation of the segment it is rigidly attached to.

6.4 Getting the coordinate conventions right

While building the IMU FK Position tab on 2026-09-20, I fixed the remaining coordinate conventions. The CyberFinger's IMU world is z-up, while the headset's world is y-up and looks along −z. A sensor direction v therefore maps into the headset frame as Ryaw(α) · C · R(q) · v, with C = [[1,0,0],[0,0,1],[0,−1,0]]. The evidence was that gravity measured in the sensor frame agrees with both the camera hand frame and, independently, the armband's accelerometer only for the IMU world's z axis (5–12° residual, against 44–50° for x or y). The heading drift was measured at about 1°/min.

This exposed a flaw elsewhere. The "gravity tilt" input of the Posture Hand and Pose Hand tabs had used the IMU world's −y axis, which is horizontal, not gravity. The fix led to the separate Pose Hand (gravity) tab, and later Key Postures dropped the tilt input altogether (commit 5a262b5). Two further findings from this period fed the next step. First, for the wrist sensor a one-sided and a two-sided calibration form (defined below) both matched the camera at about 13–15° while visible. After a camera loss, however, the one-sided form started at 0° and degraded to the two-sided level within 4–8 s. Second, the Learned Position work showed that a held offset drifts from the camera-corrected one by about 6° after 1 s and 13° after 4 s. Evaluations of out-of-view methods must therefore use the held input, not the camera-corrected one.

6.5 Orientation work in ✦ Fusion Studio: camera-taught sensor-mount calibration

The ✦ Fusion Studio tab shows the whole hand at its predicted position and posture when the camera cannot see it. For this section, the job was to keep the hand's orientation and wrist bend right out of view. The proposal's central idea is to use the camera as an opportunistic ground-truth reference that keeps calibrating the other sensors whenever it sees the hand. The mount calibration below applies that idea to the IMUs.

The symptom: a hand folded back onto the forearm

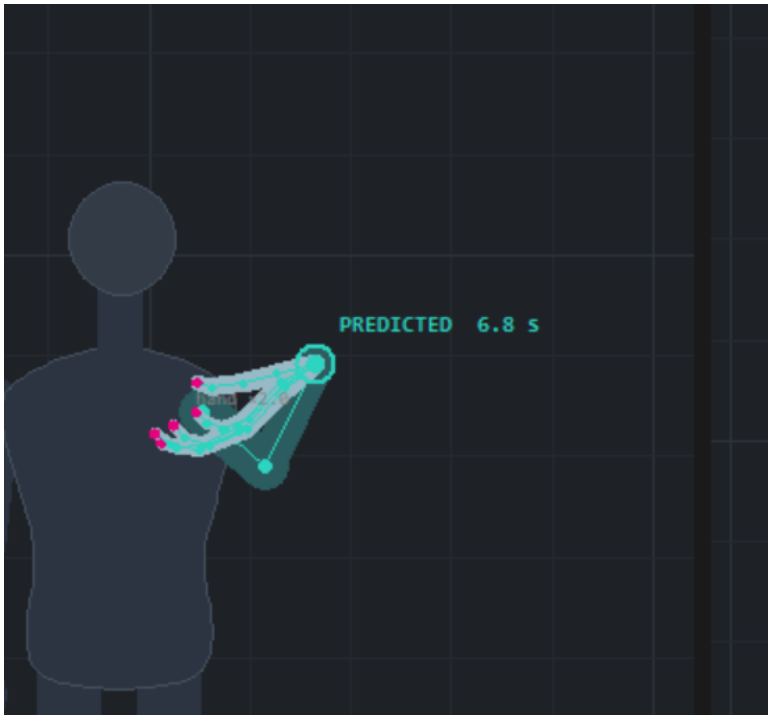


Figure 6.4 *The failure that triggered this work (2026-09-25). Out of the camera's view ("PREDICTED 6.8 s"), the Fusion Studio drew the hand folded back onto the forearm.*

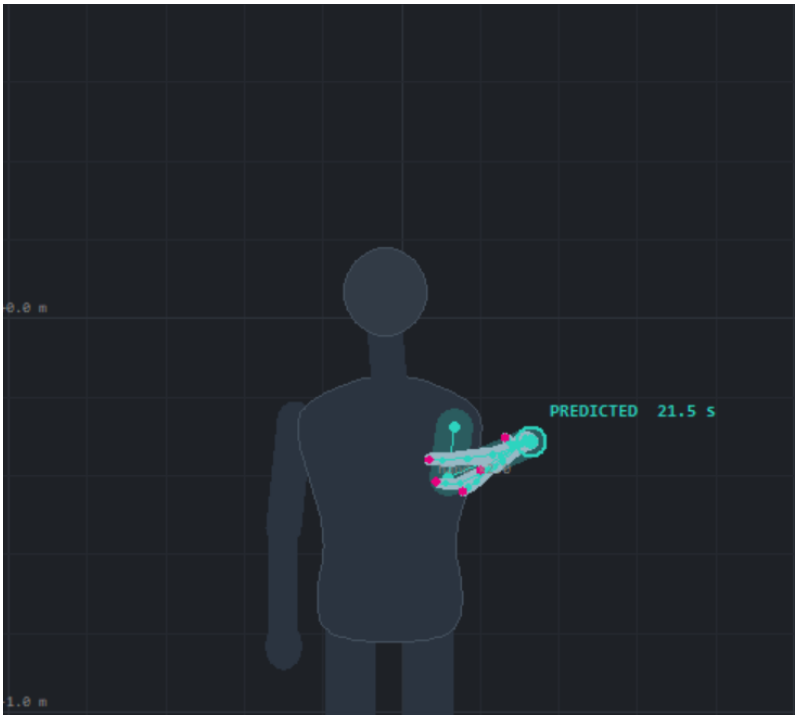


Figure 6.5 *The same failure at "PREDICTED 21.5 s": the fingers point back at the elbow.*

A first diagnosis on three clips held each sensor's one-sided offset from the last camera fix. The knuckle sensor's error in the hand's forward axis reached a median of 31–48° after 3 s and 37–67° after 7–30 s, and

exceeded 90° in 10–24% of cases. The wrist sensor stayed at 5–26°. A first fix used the wrist sensor plus a 70° anatomical limit. It stopped the fold-back but froze the wrist straight, and a rigid hand defeats the purpose of expressive social VR.

A second attempt took the bend from the raw rotation between the two sensors, re-centred at the last camera fix. Pooled over four clips, its hand-forward error was 21.8, 30.9, 39.6, 46.8 and 51.3° at 1, 3, 7, 15 and 30 s, with 19–44% of cases more than 60° off. A rigid hand driven by the wrist sensor alone scored 10.4, 14.1, 17.9, 19.0 and 20.9° at the same times, so this attempt was worse than the rigid hand.

Root cause: the form of the calibration, not the sensor

The fusers all used a **one-sided** offset: R_hand = R_off · R_imu, one correction applied on the world side. This form silently folds the sensor's tilt on the hand into the world-side correction, so it is exact only for the pose at which it was measured. After the camera is lost, the error grows with how far the hand turns away from that pose. A sensor strapped to a body segment needs a **two-sided** model: R_seg(t) = Ryaw(α) · C · R_imu(t) · M. Here M is the fixed rotation of the sensor on the segment, C relabels the IMU's z-up world as the headset's y-up world, and α is the unknown heading (bridge/mount_calib.py, commit f103567).

How the mount calibration works

- While the camera sees the hand, MountCalib stores pairs of (IMU orientation, camera orientation) at 10 Hz in a rolling 90-s window.
- Every 3 s it solves for the heading α on a 6° grid, then refines it within ±5° at 1° steps. Once a solution exists, it searches a warm-started ±10° grid at 2° steps instead. For each candidate heading, M is the Procrustes optimum.
- A solve needs at least 40 pairs (4 s at 10 Hz). Once a solution exists, a new solve also needs at least 12° of spread in the IMU's orientation; otherwise the earlier, better-conditioned solution is kept.
- M is re-solved every 3 s while the hand is visible and held when it is not. Between solves the heading is nudged toward the camera with a 2-s time constant.
- The mount is trusted when a running average of its live residual is below 25°.
- The code is numpy only, and each 3-s solve takes 17 ms or less.
- The Studio runs two instances. The knuckle mount gives the hand frame; the wrist mount gives the forearm frame, whose first axis is the forearm direction. The angle between the two is the live wrist bend.

Result: about 9° for 30 s and more

The module's self-test is causal: the calibration only ever sees data from before the moment being scored. It replays the 8 camera-labelled Position Capture clips recorded between 2026-09-17 and 2026-09-21, about 28 minutes of visible hand in total. Starting at 20 s and every 5 s after that, it simulates a camera loss, but only while the knuckle mount is trusted. It then compares the knuckle-derived hand frame with the camera's frame (wrist to middle-finger knuckle) 1–30 s later. The shortest clip lasts only 10 s, so the losses come from 7 clips. Between 5% and 10% of the frames used were not gate-CLEAR. The one-sided baseline is an offset taken from the single frame at the moment of the loss. The test was rerun on 2026-10-28 and reproduced the original numbers exactly.

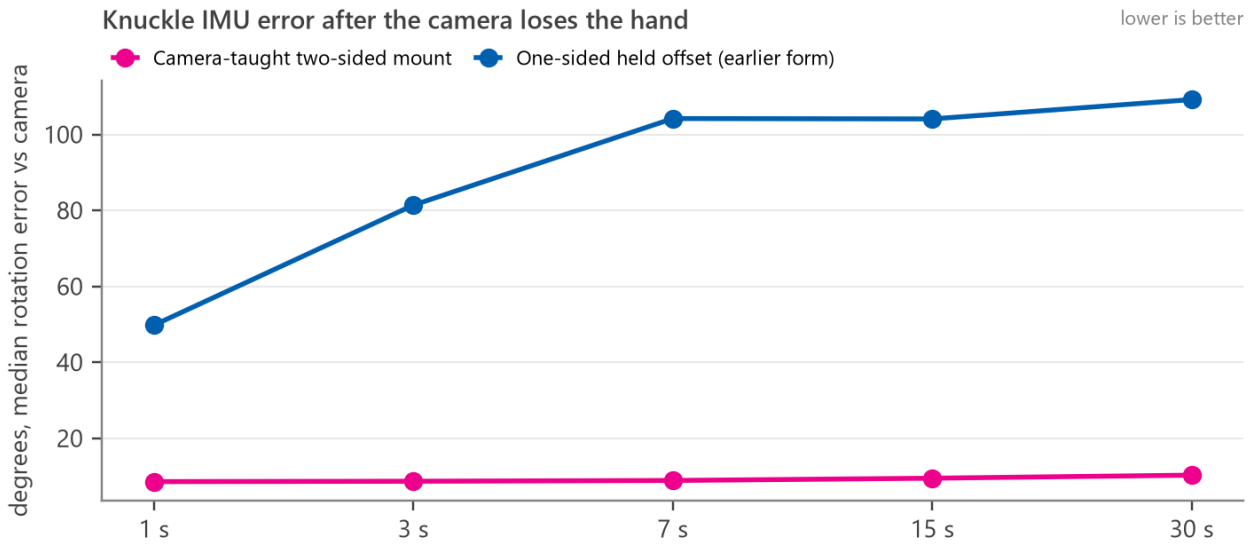


Figure 6.6 *Self-test of bridge/mount_calib.py: 241–281 simulated camera losses from 7 of the 8 recorded clips, simulated only while the mount was trusted. With the sensor's mounting learned from the camera, the median error stays at 8.6–10.3° for 30 s; the 90th percentile is 20–25°. The one-sided form passes 100° after 7 s.*

Table 6.4 Knuckle hand frame against the camera after a simulated camera loss, pooled over the recorded clips (mount_calib.py self-test, rerun 2026-10-28). The share of gross failures, the "folded hand" picture, drops from 44–79% to 1–4%.

After the loss	Two-sided: median (90th pct)	Two-sided: over 60° off	One-sided: median (90th pct)	One-sided: over 60° off
1 s (n = 281)	8.6° (23.2°)	1%	49.8° (135.5°)	44%
3 s (n = 280)	8.7° (22.8°)	4%	81.4° (154.9°)	64%
7 s (n = 274)	8.9° (24.6°)	3%	104.2° (165.2°)	76%
15 s (n = 262)	9.5° (20.3°)	1%	104.1° (164.7°)	77%
30 s (n = 241)	10.3° (21.6°)	2%	109.2° (162.7°)	79%

With its mounting learned from the camera, the knuckle IMU stayed within a median of 8.6–10.3° of the camera from 1 s to 30 s after the camera lost the hand. The earlier one-sided calibration was 49.8° off after 1 s and more than 100° off after 7 s.

Per clip, the knuckle mount fitted the camera to 5–19° and the wrist mount to 8–28°. The conclusion is that the CyberFinger's IMUs are good absolute orientation sensors; the held one-sided form was the problem. This applies to the knuckle figures behind the fold-back diagnosis. Earlier results that used the hand-eye rotation W, or the two-sided test of 2026-09-20, already included a body-side correction.

Live wrist bend in the Studio

A "HAND OUT OF VIEW" switch chooses between "knuckle IMU (live wrist bend)", the default, and "wrist IMU (bend held)". The live bend is used only when the knuckle mount is trusted (residual below 25°) and the knuckle-to-forearm angle is at most 100°; otherwise the bend from the last camera fix is held. An anatomical clamp remains as a safety net: the bend is limited to 80° (70° when there is no wrist mount), and a sensor reading beyond 110° is rejected. The orientation line reports the state in plain words, for example "knuckle IMU, wrist bend 20° live (mount 8°, residual 6°)".

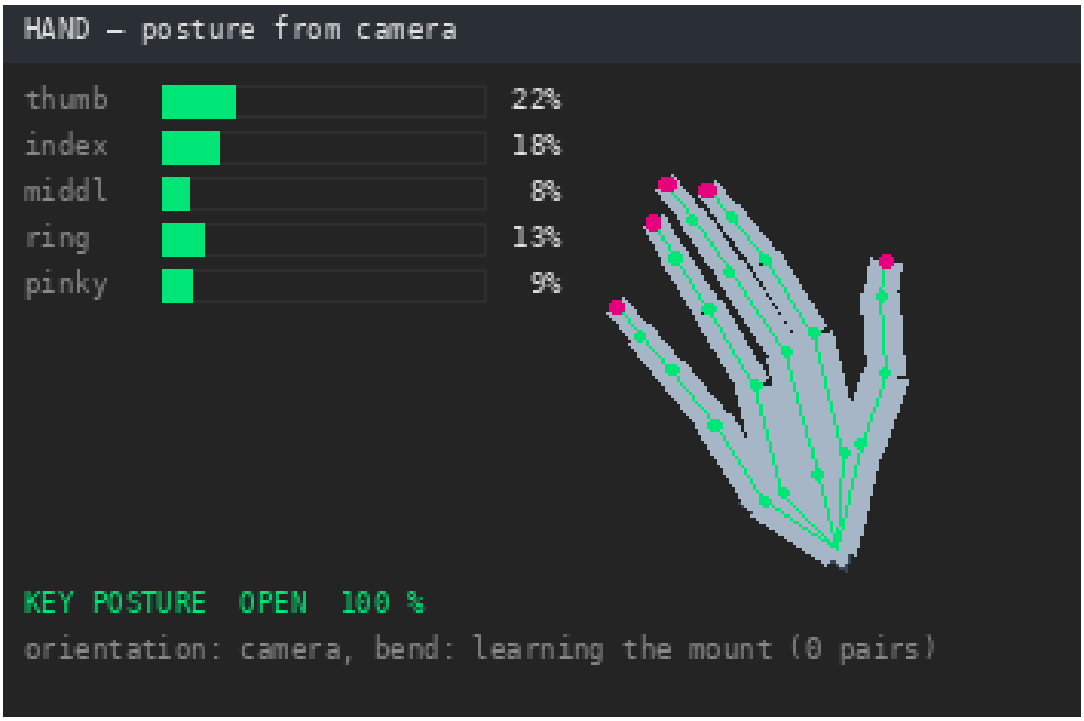


Figure 6.7 *The Fusion Studio's HAND panel at start-up. The orientation line reads "bend: learning the mount (0 pairs)" until the mount calibration has seen enough of the hand.*

To verify the feature in the real program, I replayed the clip low_new3_20260919_145801 through the Studio's actual update function with the camera hidden from a chosen moment.

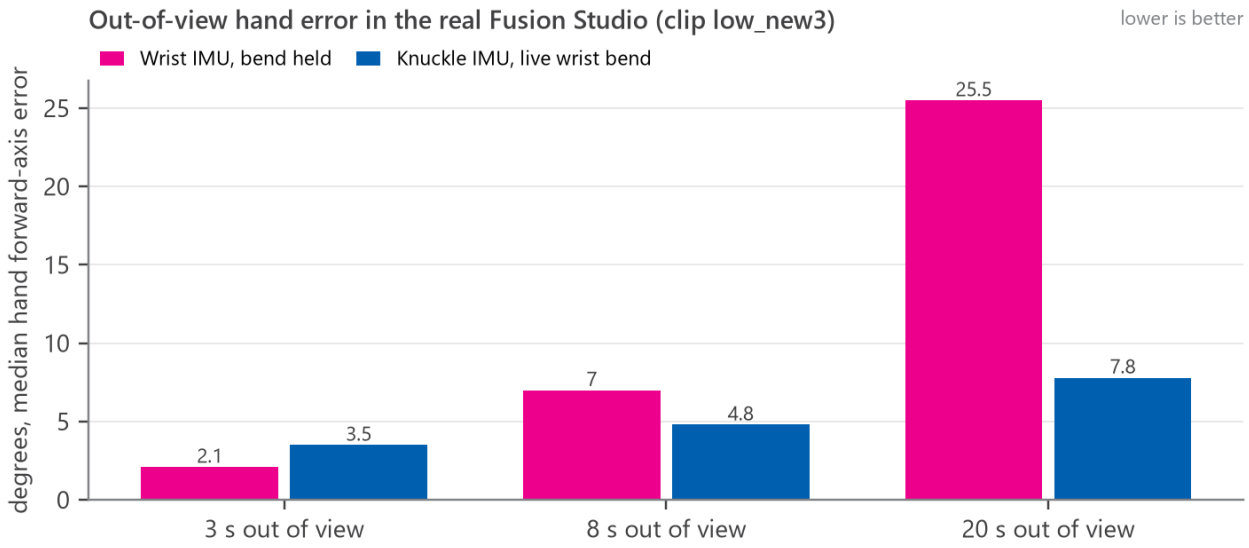


Figure 6.8 *Replay through the real Studio update with the camera hidden. Each bar is one replay span (90, 240 or 600 frames) of one clip, so the sample is small. For a short gap, holding the last seen bend is slightly better. Over 20 s the live bend stays under 8° (90th percentile 15.5°) while the held bend grows to 25.5° (90th percentile 48.0°).*

The full-rotation error at 20 s was 27.1° held against 9.5° live. On a second clip (face_depth), the error at 8 s was 4.1° live against 7.4° held; that result is recorded in my notes but not in a saved log. I also tested robustness by injecting faults. A simulated 150° jump of the knuckle sensor while out of view never folded the hand: the bend was held or limited to 80°. The clamp bounds what is drawn, however, not the error, which still reached a 90th percentile of 85° at 3 s and 94° at 20 s. A jump while the hand was visible marked the mount as not trusted (residual 97°), and the hand fell back to the held bend at 9.9° median error.

I also checked whether the EMG armband could supply the bend instead. A ridge regressor on the armband's 40 time-domain EMG features did no better than predicting the mean, both across clips (leave-one-clip-out) and within a clip. This finding was not saved to a log file. Out of view, the wrist bend therefore has to come from the IMUs.

The calibrated IMU as a check on camera errors

The Quest sometimes flips the hand or swaps it with the other hand while the gate still reports CLEAR or SELF. In recordings, the camera turned the hand by 60–146° within a single frame while the knuckle IMU turned 1–3°. From 2026-10-01 (commit f06b11f), the Studio compares the camera's hand frame with the knuckle IMU seen through its camera-taught mount. Normally they are about 6° apart, and they differ by more than 45° in 0.65% of clear frames. Above 45° (30° while the camera is already distrusted), the camera hand is not drawn, not learned from, and kept out of every calibration. Before the mount is trusted, the check instead compares how much each source has turned since the last trusted frame. After 2.5 s of disagreement the camera is trusted again and the mount's heading is re-fitted to it (MountCalib.reanchor). This prevents a restarted or drifted IMU from locking the camera out.

In replays, injected 150° flips left a drawn error of 4.7°. On clean clips, 1.2–1.5% of CLEAR frames were falsely distrusted, in episodes of at most 0.64 s. After a simulated 70° heading jump of the CyberFinger, the mount returned to 5–8°; before the re-fit was added, it had stayed 30–60° off for more than 20 s. The check has costs. On the glitchy clip low_20260919_141356 it distrusted 4.9% of CLEAR and 30.4% of SELF frames, and after the

simulated heading jump the camera was distrusted for about 4 s before the re-fit. Lower thresholds were rejected because they produced 8–13 false switches per minute. The recordings contain almost no hand-over-hand occlusion, so this feature still needs live testing. SciCortex's driver has its own, independently built camera-versus-IMU gate (src/TrackingTrust.h, 2026-09-27: a 25° gate with an escape after 3 s). The Studio's check was developed separately for the research GUI four days later.

6.6 Uptake in SciCortex's product

The Fusion Studio, including bridge/mount_calib.py and the mount-based live wrist bend, was delivered to SciCortex in pull request #1, merged as 95c7936 on 2026-09-25. Two days later, SciCortex's SteamVR driver gained "Joint IMU heading assistance" (65b19b6, 2026-09-27), and "First driver level IMU fusion" (e3fcbc0) extended it the same day. Both commits are SciCortex's own work. The driver's header src/ImuFusion.h states the model as "q_hand(t) = Y(α) · C · q_imu(t) · M (bridge/mount_calib.py)" and reuses the same settings: 40 pairs, 12° spread, a solve every 3 s and a 25° residual limit. STEAMVR_PLAN.md says the driver "solves the joint IMU's heading and mounting (Studio's mount_calib model)".

The driver adapts the model to its own setting. It uses a 60-s pair window, measures the spread on the optical orientations, and uses the 25° limit to reject a solve rather than to gate trust on a running residual. It also measures the optical lag continuously, saves the mounting and lag per hand, and follows the optical orientation only slowly (0.5-s time constant) while the fast motion comes from the IMU. That is a different fusion law from the Studio's.

Two days after the Fusion Studio was delivered, SciCortex's SteamVR driver adopted its two-sided IMU mounting model. The driver header cites bridge/mount_calib.py and reuses the same solve settings.

Table 6.5 Results of the driver-level IMU fusion as measured and reported by SciCortex in STEAMVR_PLAN.md (2026-09-27 onward). These are SciCortex's measurements, not this project's.

Measure (SciCortex's driver)	SciCortex's reported result
Offset from the headset at rest (offline)	0.6–1.9°
Through simulated occlusions	5–20° (holding the last pose: 13–76°)
Live over Steam Link	follows the IMU (+2 ms); headset trails by ~90 ms
Laser-pointer jitter, hand still	11.3 → 5.0 mm
Laser-pointer jitter, fast motion	64.5 → 21.8 mm
Start-up with a saved mounting	0.5–0.8 s (was 4–5 s offline, 20–60 s live)
Solved mounting between sessions	scatters 14–21°, mostly tilt

Latency differs between the two pipelines, and the figures do not contradict each other. In the research GUI's OpenXR Live Preview stream, the IMU and camera were aligned to within one frame. Over Steam Link, SciCortex's 2026-09-26 capture found the IMUs about three times quieter than the optical rotation and leading it by about 40 ms. Steam Link's lag varies by session (10–100 ms) and drifts within one.

For end users, the v2.1 manual describes the result. A setting text (pages 18–19) says the driver "learns how the CyberFinger sits within a second or two and remembers it". Page 8 says that, out of view, "with the joint

IMU the hand keeps turning with your wrist, and its position waits where the headset last saw it". The single-window release of the Fusion Studio (commit b42142f, 2026-10-23) carries both features: "Out of view the wrist bend follows the camera-taught IMU mounting", already present since the first delivery, and the newly added "A camera hand that turns away from what the knuckle IMU measures is set aside".

6.7 Limitations and next steps

- Every accuracy figure is measured against the headset's own optical hand estimate, which is itself noisy. No independent motion-capture ground truth was used.
- All data come from one user in one lab setup, mostly on the right hand.
- The two-sided mount needs about 4–5 s of visible hand with some rotation before it is trusted, and it is learned per session. SciCortex found the solved mounting scattering 14–21° between sessions, so a saved mounting is only a starting point.
- The median error is about 9°, but the 90th percentile is 20–25°.
- For gaps of a few seconds, holding the last seen bend was as good or better (3 s: 2.1° held vs 3.5° live). The real-program replay covers one clip, and the EMG-cannot-predict-the-bend finding was not logged.
- The anatomical clamp prevents an impossible drawing, not a wrong orientation.
- The camera check was tuned on recordings with little hand-over-hand occlusion. It needs live testing.
- The IMUs give orientation, not position. Out-of-view position is covered in the position sections.
- Next step: Task 5.1 of the second internship unit, "learn a latent representation of inter-subject sEMG and IMU mounting variability", follows directly from this work. A learned mounting prior could make the calibration trusted sooner for a new user.

7 Hand position when the camera cannot see the hand

The CyberFinger's IMUs report orientation only, so when the headset camera loses the hand, the shipped driver keeps turning the hand but freezes its position. Across six tabs I built a camera-labelled dataset of 46,203 frames, a learned estimator, metric body views, a forward-kinematics model and a strict benchmark in which the head is turned away after the hand is lost. The final method, Hybrid Position, computes the forearm exactly from the IMU and uses small networks to learn only how the elbow moves. Its mean wrist error over 0.5–8 s was 14.58 cm, against 22.62 cm for the earlier learned model and 36.95 cm for freezing, and it now supplies the out-of-view hand position in Fusion Studio and in SciCortex's Fusion Studio release. Most of the remaining error comes from the upper arm, which no sensor measures, and from drift in the IMU's heading, so the recommended next step is an upper-arm IMU.

7.1 The problem and the starting point

The Quest 3 tracks a hand only while its cameras can see it. When the hand drops to the hip, reaches behind the body or passes behind an object, the optical tracker loses it, but the avatar's hand still has to be drawn somewhere. SciCortex's CyberFinger manual (v2.1) describes what the shipped driver does: *"Hands out of view: with the joint IMU the hand keeps turning with your wrist, and its position waits where the headset last saw it."* The hand's orientation keeps updating, but its position freezes. The Mitacs proposal asks for hand tracking *"outside the headset's optical field of view"*, with IMU–optical fusion *"enabling short-dropout dead-reckoning"* (task T2.2), backed by benchmarks and metrics (tasks T1.3 and T2.3). This section covers the position half of that goal: where is the wrist when the camera cannot see it?

The hardware set a hard limit from the start. An IMU (inertial measurement unit) combines an accelerometer and a gyroscope. The CyberFinger's wrist module and knuckle sensor fuse these on board (6-axis fusion, with no magnetometer) and stream only **orientation**, as quaternions. In every recorded clip (13 gesture clips and all 8 position clips) the CyberFinger accelerometer columns were zero, because the firmware did not send that report. The gyroscope was never transmitted, and early logging ran at about 21 Hz. Orientation tells how the forearm is turned, not where the wrist is. The only accelerometer available was the one in the MindRove armband.

I built six GUI tabs for this question between 17 and 21 September 2026; the table of the six position tabs below lists them. Each one was additive: it subclassed or reused an existing tab and left every other tab and model file unchanged, so each step could be compared with the one before it on the same data.

Table 7.1 The six position tabs in GUI order, with the date each was built and the commit that added it

Tab	Built	Commit	Role
Pose + Position	17 Sep 2026	cc8f704 (20 Sep)	Holds the last position; first negative results
Position Capture	17 Sep 2026	cc8f704 (20 Sep)	Camera-labelled training data
Learned Position	17–19 Sep 2026	cc8f704 (20 Sep)	Learned IMU-to-position model
Body View	19 Sep 2026	cc8f704 (20 Sep)	Metric FRONT and TOP body views
IMU FK Position	20 Sep 2026	a118f35 (20 Sep)	Forward kinematics (experimental)
Hybrid Position	21 Sep 2026	b98f13d (21 Sep)	Best method; feeds Fusion Studio

7.2 Pose + Position

Purpose. Pose + Position asked the first question: what should the hand's position do when the camera loses it? The tab reuses Pose Hand's panels and posture model unchanged and adds a fourth panel. That panel shows where the wrist is relative to the head, on the same dome radar as the Gloves tab: the centre is straight ahead of the gaze, with rings at 30°, 60° and 90°. Pink marks a camera-seen position, grey a held position and orange an IMU-carried estimate. The occlusion gate overrides stale joints, because the Quest keeps repeating the last pose of a hand it can no longer see.

A fixed-elbow pivot that failed on real data

The first idea treated the elbow as a fixed pivot: wrist ≈ pivot + L × forearm direction. The pivot is learned while the hand is in view, and the wrist IMU then swings the hand around it. The first validation used a simulation that assumed a fixed elbow, so it tested the code, not the idea. I then replayed five real synchronised clips, with the optical wrist as ground truth; the first 35 % of each clip was treated as seen and the rest as lost. On these clips the pivot lost to simply holding the last position (chart below): freeze gave 3.8 cm, the pivot 6.8 cm, and a learned ridge regression from orientation to position 8.8 cm. Even with a perfect orientation sensor the pivot gave 15.2 cm, so the model was the problem, not the IMU. The elbow position implied by the data wandered 47 cm while the wrist moved 13 cm, and forearm orientation explained only 41–58 % of the wrist position, even when fitted in-sample. One caveat applies: these clips held only small in-place motions (11–17 cm of wrist travel), which favours freezing.

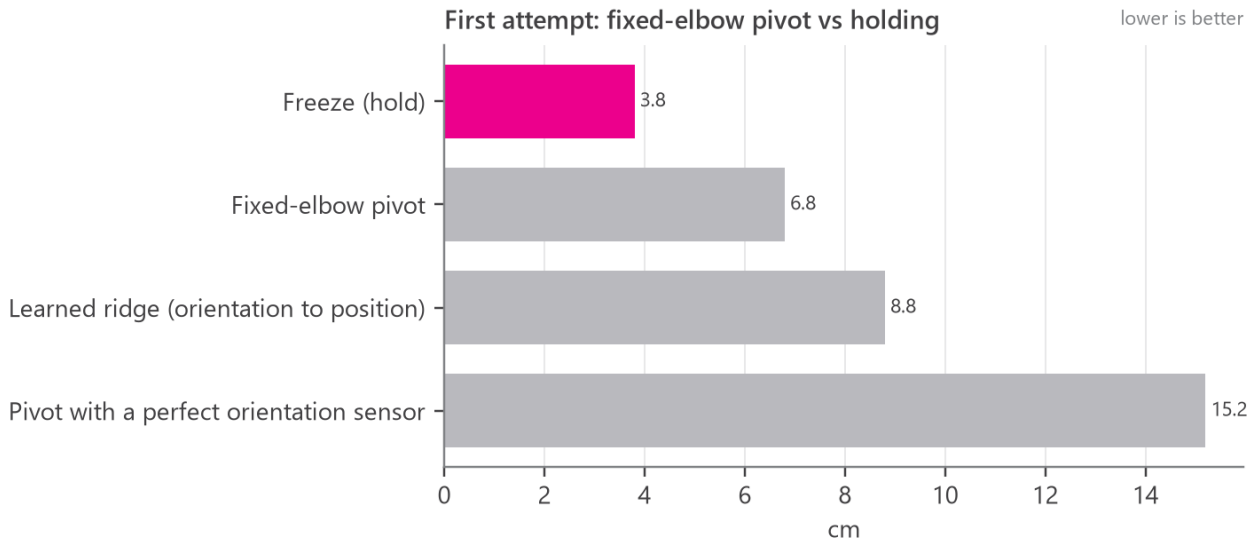


Figure 7.1 *Mean wrist error after a simulated camera loss, replaying 5 real synchronised clips (first 35 % seen, rest treated as lost; only 11–17 cm of wrist travel). The pivot model looked perfect in a simulation that assumed a fixed elbow, but on real data it lost to holding, which is why Pose + Position holds by default.*

Integrating the accelerometer

I also tested dead-reckoning with the armband's accelerometer: integrating acceleration twice, starting from the last camera fix. It failed within a fraction of a second. Mean errors were 10.6 cm after 0.25 s, 38.8 cm after 0.5 s, 150 cm after 1 s and 524 cm after 2 s, against 1.4–3.6 cm for freeze. The cause is gravity leakage: an orientation error of 15–20° leaves about 2.5 m/s^2 of false acceleration, and even a 1° error produces 8.6 cm of drift in one second.

What the literature does

Published IMU-only methods get position in one of two ways. (a) A kinematic chain with an IMU on every segment from a fixed root. Nie et al. (Sci Rep 2021, PMC8602299) used upper-arm and forearm IMUs to estimate reach relative to the shoulder, with a mean signed error of 0.09 ± 1.81 cm against Vicon motion capture. Their setup is not directly comparable: they used 9-axis IMUs with a magnetometer, reported a one-dimensional reach distance rather than a 3-D position, and tested two healthy participants. (b) Learned inertial odometry anchored by the camera. HOOV (Streli et al., CHI 2023) samples raw accelerometer and gyroscope data from a single wrist IMU at 427 Hz. It reports 7.77 cm mean error offline, and a median of 15.11 cm (mean 16.97 cm) in its real-time interactive study. The CyberFinger setup supported neither approach: there was no upper-arm IMU and no raw, high-rate inertial data.

Decision: hold by default

By default, Pose + Position therefore **holds** the last camera-seen position in world coordinates. It re-projects that position every frame, so the dot still moves correctly when the head turns. The held dot is drawn grey with a dashed ring that widens over time to show growing uncertainty. The pivot remains available as the opt-in checkbox "IMU-carry out of view (experimental)", off by default. The lasting lesson was about method: validate on recordings, never on a simulation that builds in the model's own assumption.

7.3 Position Capture

Purpose. Holding is honest but static. To *learn* where the hand is from the IMU posture, a model needs many pairs of IMU orientation and wrist position that cover the whole space around the body, with the camera as the teacher. The existing clips had too little hand travel, so I built Position Capture (17 September) to record exactly this data.

A body frame that lets the user look at the hand

The key design choice is the coordinate frame. Positions are stored in a body (torso) frame whose origin is a neck pivot, 10 cm below and 8 cm behind the headset, so nodding barely moves it. The axes are room-up plus a body heading captured once with the "⌖ Set forward" button, not the live head direction. The user can therefore look down at a hand held by the hip, and the camera labels a posture that is normally out of view, while the stored position stays in the same body frame. I checked this through the tab's real update code: with the hand fixed, turning the head ±60° in yaw and −55° in pitch changed the stored position by 0.03 cm.

A frame counts as camera-labelled ("seen") only if four conditions hold: the hand joints are present, the headset pose is valid, the pose is not frozen (the Quest repeats a pose it cannot see), and the occlusion gate does not report out of view, behind an object or other. While recording, the tab shows:

- Live FRONT and TOP coverage heat-maps with 10 cm cells; a cell counts as full at 20 frames.
- Six bars for the palm's direction.
- A "SLOW DOWN" warning above 1.2 m/s, where motion blur degrades the camera labels.

A clip with fewer than 60 labelled frames is discarded. Each clip is saved as position_data/pos_<label>_<timestamp>.npz and keeps every frame:

- hand joints and headset pose;
- three IMU quaternion slots: BODY1 (empty on the lab CyberFinger), the wrist module and the knuckle;
- the CyberFinger accelerometer columns, EMG and armband acceleration;
- the camera-calibrated forearm orientation, the body-frame wrist position and the palm normal.

The measured lag between the IMU and the camera was at most 10 ms, found by cross-correlating their angular speeds.

What the data revealed

The first real clip (114 s, 93 % labelled) exposed a bias in the data. Waist, hip and lap coverage was 0 %: the lowest point reached was chest height, about 37 cm below the neck. There were no far reaches (more than 45 cm forward), and the user turned their body about 25 % of the time. Because the camera labels only what the head points at, the training data was skewed toward the gaze direction. This finding shaped three later choices: new recordings aimed at low and far postures, the heading-free frames used later, and the "look-away" test of the position lab.

Table 7.2 The Position Capture dataset: one user, right hand, 46,203 camera-labelled frames in total

Clip	Recorded	Duration	Labelled	Notes
sweep (test)	17 Sep	10.0 s	69.6 %	Pipeline test
sweep	17 Sep	114.3 s	92.8 %	No low or far data
low	19 Sep	195.1 s	92.4 %	Low postures
low_new	19 Sep	384.5 s	90.3 %	First hip/lap data (down to −0.83 m); fast
low_new2	19 Sep	297.9 s	95.4 %	Logged at 18 Hz
low_new3	19 Sep	401.2 s	92.9 %	Slow, clean; main held-out clip
face_depth	21 Sep	95.8 s	89.0 %	Forearm upright, push toward/away
face_move	21 Sep	89.7 s	94.4 %	Hand near the face, elbow moving

The eight clips hold 46,203 camera-labelled frames in total. They are the training and evaluation data for everything that follows, and the training set of the deployed Hybrid Position weights.

7.4 Learned Position

Purpose. Learned Position shows, live, a learned estimate of the wrist position computed from the forearm IMU's orientation and anchored to the last camera fix. While the camera still sees the hand, the model's own guess is drawn as a hollow blue ring next to the camera's pink dot, so its error is always visible. The tab subclasses Pose + Position, and a "⚙ Train position model" button retrains the model inside the application.

The model

The model takes the forearm orientation, expressed in a frame that is centred at the neck, aligned with gravity and turned with the head's heading. It outputs the wrist's offset from the neck. It uses 500 random ReLU features followed by ridge regression, a regularised linear fit ($\lambda = 30$ was selected). Two data augmentations encode physical facts. Random yaw rotations (6 copies over ±180°) stop the model from using gaze direction as a shortcut. Random forearm twists stop it from reading palm-up versus palm-down in place as a movement. The trainer and the GUI share one code file (position_model.py), so they cannot drift apart. An early probe on the first clip alone (same session, so optimistic) already clearly beat freezing: 14.1/14.7/14.8/13.5 cm against 23.6/29.5/37.3/45.5 cm at 0.5/1/2/4 s. Adding the knuckle orientation made the model worse (17.4 vs 14.9 cm), so it uses the forearm only.

Anchoring to the last camera fix

A model trained across sessions carries an offset of its own, so the estimate is anchored to the last camera fix. The first version faded the anchor out over time ($\exp(-h/\tau)$, $\tau \approx 0.5$ s). I replaced it with a **motion-based** fade: est = model(now) + $\exp(-|\text{model(now)} - \text{model(then)}| / 0.5\text{ m}) \times (\text{fix} - \text{model(then)})$. If the arm is still, the model's output does not change and the answer stays exactly at the camera fix. Once the arm moves, the model is trusted progressively more. With the arm held still for 4.5 s out of view, the anchor weight stayed at 1.000 and drift was 0.00 cm.

An evaluation trap

Live and out of view, the forearm calibration (the offset between the IMU and the camera's forearm frame) is held at its last value. In the recorded clips, however, the stored forearm orientation is camera-corrected on every clear frame. Evaluating with the stored values was therefore optimistic. On one test segment it gave 14.3/15.5/15.3/15.1 cm at 0.5/1/2/4 s, against 14.7/16.7/17.2/18.4 cm with the held calibration the live tab actually sees; the held input drifts from the stored one by about 6° after 1 s and 13° after 4 s. Every later evaluation used held-calibration inputs.

Results

On the whole held-out clip low_new3, a model trained on the other five clips (28,240 camera-labelled frames) gave 11.1/15.4/18.2/21.1 cm at 0.5/1/2/4 s, against 13.1/20.5/26.9/36.8 cm for freeze, with head motion as recorded. The shipped model was then retrained on all six clips available at the time (40,448 frames). A leave-one-clip-out test then compared the two anchor rules: each of the five long clips was predicted by a model that never saw it (table below). The motion anchor was never worse than freeze; the timer anchor was up to 5.1 cm worse. Replaying the unseen low_new3 through the tab's real update code gave 10.6/16.9/20.7/22.6 cm for the motion anchor, 16.8/23.3/28.1/29.1 cm for the timer anchor and 12.9/20.3/27.4/37.5 cm for freeze. On the stricter look-away benchmark described later, Learned Position scored 22.62 cm.

Table 7.3 Learned Position: leave-one-clip-out mean 3-D wrist error over the five long clips (head as recorded, held calibration)

After loss	Freeze	Timer anchor	Motion anchor
0.5 s	24.5 cm	20.0 cm	18.1 cm
1 s	27.5 cm	22.1 cm	18.3 cm
2 s	35.7 cm	23.7 cm	20.5 cm
4 s	43.3 cm	24.3 cm	22.5 cm
8 s	50.0 cm	24.3 cm	23.8 cm

Limits

- **Floor across sessions:** with posture-only input, the error did not fall below about 20–26 cm, and more data stopped helping. With no camera fix at all, the held-out posture-only error was 27.7 cm.
- **Depth is weak:** on pure forward/back reaches (28–46 cm) the model moved only 61–71 % of the way and added 16–18 cm of sideways or vertical drift. A forward reach comes mostly from the shoulder and elbow while the forearm keeps pointing forward, so the forearm's orientation barely changes.
- **Twist:** before augmentation, a ±90° forearm twist in place shifted the guess by about 37 cm (42 cm on real in-place rotations). With twist augmentation, the synthetic ±90° shift fell to about 11 cm, and the errors on real rotations fell from 42 and 31 cm to 15 and 19 cm. The accuracy cost was at most about 1 cm at 2–4 s.
- **Armband accelerometer as an extra input:** it lowered the error from 20.3 to 19.6 cm, from 24.0 to 20.0 cm and from 27.3 to 23.7 cm, almost entirely in the vertical axis. It was not added to the tab.

7.5 Body View

Purpose. The dome radar shows only the *direction* of the hand from the eyes; it cannot show whether the hand is 20 cm or 70 cm away. Body View (19 September) adds reach. Below a short row of the three hand panels, it shows metric FRONT (right × up) and TOP (right × forward) views of the body, in metres.

Both views use Position Capture's body frame (neck origin, "⌖ Set forward" heading), so turning the head does not swing the hand across the display. They show a 20 cm grid, rings at 25, 50 and 75 cm around the neck, a body outline and a schematic shoulder-to-wrist arm line. A 3-second fading trail is tinted by the position's source. The Learned Position model's guess appears as a hollow blue "ghost" with an error ring drawn to true scale. The tab reads the Learned Position model without changing it and keeps its own state. Without a headset pose, it falls back to a head frame.

Verification: when the low_new3 clip was replayed through the tab's real update code, the camera frames landed on the clip's stored body-frame positions to 0.0000 cm. A smoke test of all display states in the real application raised no exceptions. Body View's layout became the base of the Hybrid Position tab (see the Hybrid Position FRONT and TOP views below).

7.6 IMU FK Position

Purpose. IMU FK Position (20 September, experimental) stops asking a model for XYZ. It computes the wrist by forward kinematics (FK), chaining segment directions and lengths out from the shoulder: wrist = shoulder + L_u × (upper-arm direction) + L_f × (forearm direction). No acceleration is integrated, so the error does not keep growing with time. The tab tests whether FK recovers reach and depth better than a learned model, and it prepares the pipeline for an upper-arm sensor.

Measuring the frame conventions

FK only works if each sensor's axes are mapped correctly, so I measured the conventions on recordings rather than assuming them. The CyberFinger IMU world is z-up, with an arbitrary heading for each sensor. Gravity matched only the IMU-world z axis (residual 5–12°, against 44–50° for x and y), and the heading drifted by about 1° per minute in those recordings. The OpenXR world is y-up, and the headset looks along −z (the camera-seen wrist was on the −z side in 100 % of frames). One side finding mattered elsewhere: the "gravity tilt" input of the Pose Hand and Posture Hand models was a horizontal axis in the z-up IMU world, not gravity. This led to the separate Pose Hand (gravity) tab; the original models were left untouched.

Geometry and the upper-arm source

The default geometry came from a sphere fit on the clips. The shoulder is placed relative to the neck at 16.5 cm right, 19.0 cm down and 5.0 cm forward; the upper arm (L_u) is 30 cm and the forearm (L_f) 26 cm. A free fit of L_u gave an unphysical 43 cm. The lengths are editable, and a "⚙ Fit shoulder from camera" button refits them. The upper-arm direction can come from one of four sources:

- solved from the camera and then held (for validation);
- the BODY1 IMU slot, which is empty on the lab CyberFinger (imu_present = 6) but already logged;
- the armband worn on the upper arm;
- a hanging-arm assumption.

A BLIND TEST mode makes FK ignore the camera while the camera keeps scoring it, and every run can be logged to CSV.

Results

On the recorded clips, with a perfect upper-arm direction taken from the camera, FK reached a median wrist error of 5.2–5.6 cm (3.9–4.8 cm in depth). With 10° or 20° of upper-arm error it reached 7.5 or 12 cm. The learned model scored 20–26 cm on the same data. Without new hardware (upper arm held), the real update code gave 8.6/14.6/21.1/20.5 cm at 0.5/1/2/4 s, against 11.0/21.4/33.1/31.8 cm for freeze.

The first live blind log (55 s, 48 s of it blind, pure forward/back reaches, no upper-arm sensor) is summarised in the first live blind log table below. FK followed depth well (correlation 0.97, 61 % of each pure depth move), whereas the learned model moved the wrong way (−20 %). Freeze nevertheless had the lowest 3-D error. The reaches were purely forward/back (sideways spread 2 cm), so freeze erred only in depth. FK's extra error had two causes:

- **Upper arm set too long.** The length configured for that session (32 cm) was too long; the camera implied 25.9 cm just before the blind span. Recomputing with 25.9 cm gives 11.4 cm, level with freeze, and the depth bias falls from +6.1 to +0.3 cm.
- **Sideways arc.** FK slides sideways on an arc around the held elbow: the sideways error correlated +0.93 with the reach, and the lateral bias grew from +5 to +10 cm over the 48 s.

Because the camera-implied upper-arm length varies with posture, it should be measured on each user.

Table 7.4 IMU FK Position, first live blind log: 48 s of pure forward/back reaches without an upper-arm sensor

Method	Median 3-D error	Depth error	Depth moves followed
FK, elbow held	14.3 cm	6.1 cm	61 %
Learned model	31.7 cm	28.0 cm	−20 %
Freeze	11.3 cm	9.6 cm	none (holds still)

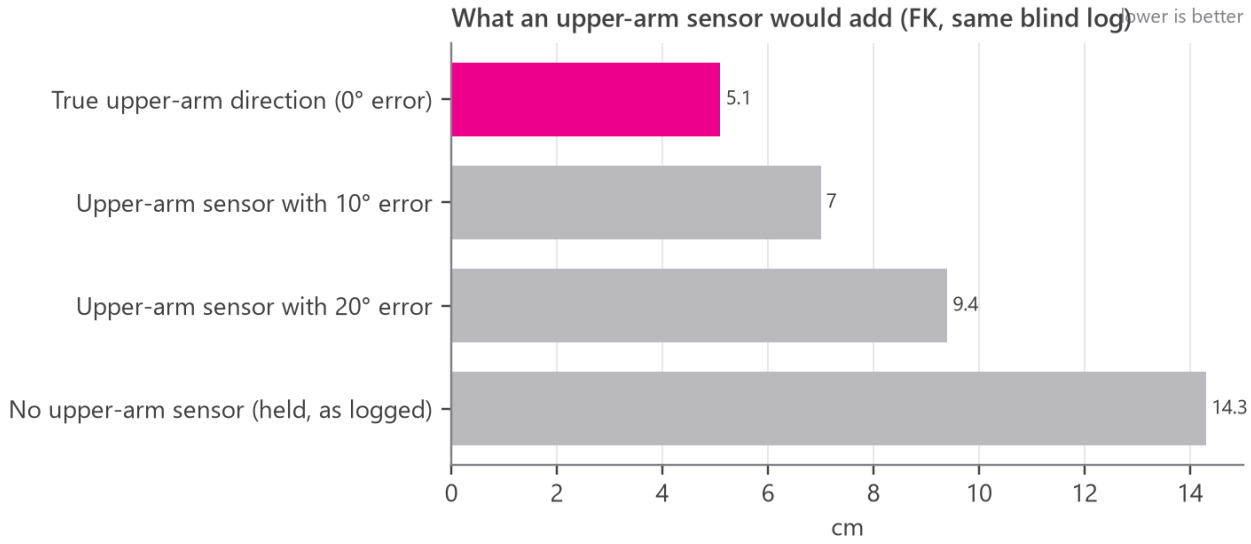


Figure 7.2 *FK wrist error on the 48-s blind log when the upper-arm direction comes from a simulated sensor with increasing error, compared with holding it. The true direction was taken from the camera, so 5.1 cm is a best case. During the log, the real upper arm swung 25° (median) and 37° (90th percentile) away from the held direction.*

A sensor-free "synergy" model, which predicts the upper-arm swing from the forearm, reached about the learned model's level: 17.3/18.5/21.2/23.9 cm, against 16.7/19.4/24.9/31.0 cm for FK-held and 22.9/24.6/31.0/37.8 cm for freeze at 0.5/1/2/4 s (leave-one-clip-out). This is the same information limit seen before. Two results carried forward:

- **Held-elbow fix.** Holding the camera-implied elbow vector, instead of the typed upper-arm length, cut the error at the moment of loss from 4.7–7.8 cm to 0.6 cm. At 0.5 s it fell from 7.1 to 4.3 cm and at 1 s from 9.3 to 7.0 cm; from 2 s on it was unchanged.
- **Simulated BODY1 IMU.** A simulated upper-arm IMU (random mounting and heading, 3° noise, two-pose calibration) run through the real FK update code gave 4.0 cm (2.6 cm in depth). This proves the pipeline and the axis conventions, not real-world accuracy. With real hardware I expect an error nearer the 10–20° cases above, about 7–12 cm.

IMU FK Position remains experimental until a real upper-arm sensor is available. Its forearm term, with the elbow held, became the backbone of Hybrid Position.

7.7 The position lab benchmark

By 20 September there were several candidate methods, each measured under a slightly different protocol. To compare them fairly, I set up a separate benchmark outside the repository, the "position lab", with one fixed protocol and one score:

- **Leave-one-clip-out** over the five long clips: each test clip is predicted by models trained on the other four plus the 10-s clip.
- **Anchors.** After a 30-s warm-up, every 12th camera-seen frame is an anchor where the camera is "lost". The estimator sees camera data only up to the anchor, and the calibration is held, as in the live fuser.
- **Score.** The error is the mean 3-D wrist error at 0.5, 1, 2, 4 and 8 s after the loss; the score is the mean over these five horizons.

- **Look-away** (the official score). In the recordings the head always pointed at the hand, which is not true when the hand is really out of view. After each anchor the head is therefore turned 40–100° in yaw (to either side) and −20° to +35° in pitch, ramped over 0.4 s, about a fixed neck. A method that secretly relies on gaze collapses under this test.

I scored three baselines under this protocol: freeze at 36.95 cm, FK with the elbow held at 26.53 cm, and the deployed Learned Position method at 22.62 cm (22.15 cm with the head as recorded, so it leaned slightly on gaze). I then explored seven separate ideas, named anchors, fusion, temporal, online, u_multi, displacement and geo_hybrid in the lab. All seven beat the deployed method, with scores from 18.51 down to 15.33 cm (see the method-score chart below); the temporal (sequence) idea found no useful signal. Combining the best parts gave 14.27 cm with a per-session nearest-neighbour memory, and 14.58 cm without it (the "lite" variant). The lite variant scored 11.3/12.7/14.6/16.5/17.8 cm at 0.5–8 s. It recovered 74 % of each reach, against 58 % for Learned Position, with 10.7 cm sideways error against 16.2 cm, and it was at least 5.2 cm better than freeze in every clip and horizon.

An oracle given the true upper-arm direction, which is what a perfect upper-arm sensor would provide, scored 8.13 cm on the same episodes. The gap between 14.58 and 8.13 cm therefore comes from the upper arm, which no sensor measures.

Honesty notes

- Three settings were tuned on the test folds (about 15 configurations). The true gain of the combined method over geo_hybrid is therefore expected to be only about 0.7–1.0 cm. The 8.0 cm gain of the shipped lite variant over the deployed Learned Position method is not in doubt.
- In a sensitivity check, changing the tuned gate, multi-anchor and memory settings by ±30 % moved the combined method's score by at most 0.06 cm.
- The planned adversarial audits were stopped partway. Instead, I re-ran the lite result and reproduced it exactly (14.58 cm), and read the estimator code to confirm that it never uses head orientation. For the combined method, the look-away and as-recorded scores are identical (14.27 cm). No separate as-recorded run of the lite variant was saved.
- The partial leakage audit found no leakage: the as-recorded run, 180° head turns and a test that corrupted inputs the method does not use all gave exactly 14.27 cm. Shuffling the history before the loss raised the score to 30.02 cm, and dropping the history gave 15.24 cm.
- The audit also showed that the method relies on the headset-tracked neck position after the loss: holding the neck fixed at the anchor raised the score to 16.59 cm.

7.8 Hybrid Position

Purpose. Hybrid Position (21 September) puts the best method from the position lab into the GUI. It combines the strengths of the earlier approaches: the IMU gives the forearm's motion exactly through kinematics (as in IMU FK Position), and learning is used only for what the sensors cannot see, namely how the elbow moved since the camera last saw the hand. The tab switches automatically between the camera and the prediction.

How it works

- **Forearm by kinematics.** The base estimate is the camera wrist at the moment of loss plus L_f × (forearm direction now − forearm direction then), with L_f = 26 cm. This is FK with the elbow held where the camera last implied it.
- **Elbow motion by small networks.** Twelve small multilayer perceptrons (MLPs; 33 or 27 inputs, two hidden layers of 128 units, 3 outputs) predict only the change of the elbow offset since the loss.
- **Network families.** One family works in the body frame and is trained with ±30° yaw augmentation; the other works in a frame turned toward the hand at the moment of loss, so it needs no body heading. Each family comes with and without the armband accelerometer, with three random seeds each. Forearm-twist noise ($\sigma = 30°$) is injected in training, so twisting the wrist is not read as moving.
- **Several anchors.** The camera also saw the hand 0.25, 0.5, 1 and 2 s before it was lost. Each of these frames gives its own estimate, and the estimates are averaged with weights $(h/(h+\delta))^4$. At short horizons the last fix dominates; at 4–8 s the average helps.
- **Still-arm gate.** Everything learned is multiplied by $g = 1 - \exp(-G/0.2)$, where G is the forearm axis's path, counting only motion faster than 0.1 rad/s, since the loss. If the forearm does not move, the output is exactly the last fix, carried along with the neck.
- **Safety limits.** The learned part ramps in over the first 0.25 s. The elapsed time fed to the networks is capped at 10 s, their trained range. Head orientation is never read; only the neck position derived from the headset is used.
- **Not shipped.** The per-session nearest-neighbour memory (worth 0.3 cm) was left out: it was the ingredient trusted least across users and the only one that needs a growing per-session state.

Inference is pure numpy, so the GUI needs no deep-learning library. Retraining (train_hybrid_position.py, PyTorch, about 30 s on the GPU) imports the same feature code, so training and live use cannot diverge. The deployed weights were trained on all eight Position Capture clips (46,203 camera-labelled frames, 85,200 training pairs) on 21 September; the previous six-clip weights were kept as a backup. When the armband is off, the tracker falls back to the networks without accelerometer input.

> **On the look-away benchmark, Hybrid Position's mean wrist error over 0.5–8 s was 14.58 cm, against 22.62 cm for Learned Position and 36.95 cm for freezing (the shipped behaviour). That is 8.0 cm better than the previous best, and at least 5.2 cm better than freeze in every clip and horizon.**

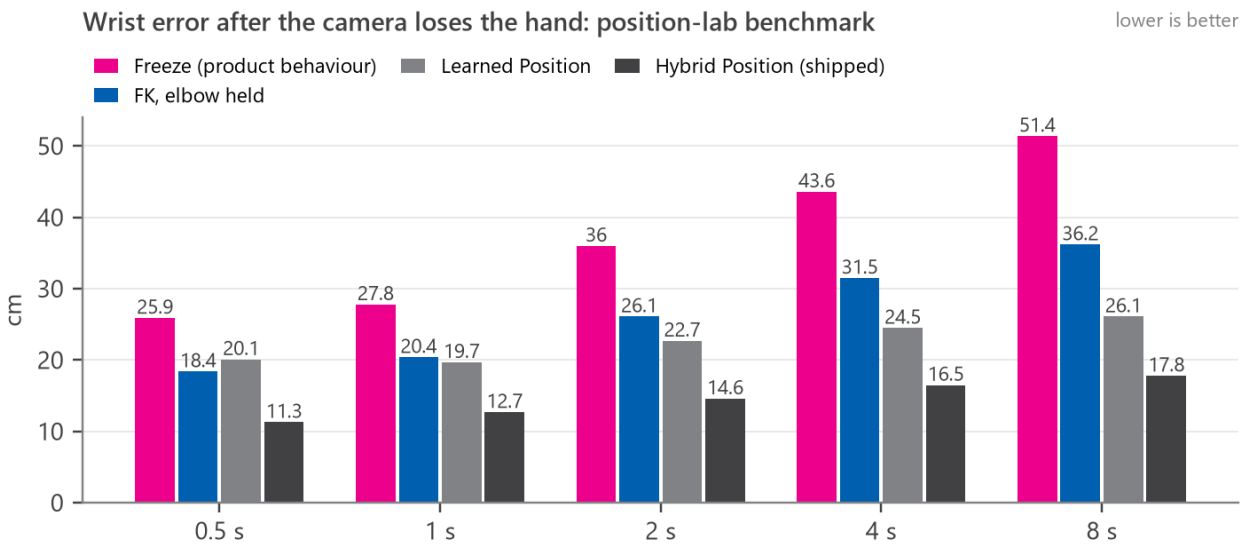


Figure 7.3 *Mean 3-D wrist error against time since the camera lost the hand: leave-one-clip-out over 5 recorded sessions, with the head turned away after the loss (look-away protocol). Hybrid Position cuts the error by about 55–65 % compared with freezing, and the gap grows with time (from 14.6 cm at 0.5 s to 33.6 cm at 8 s).*

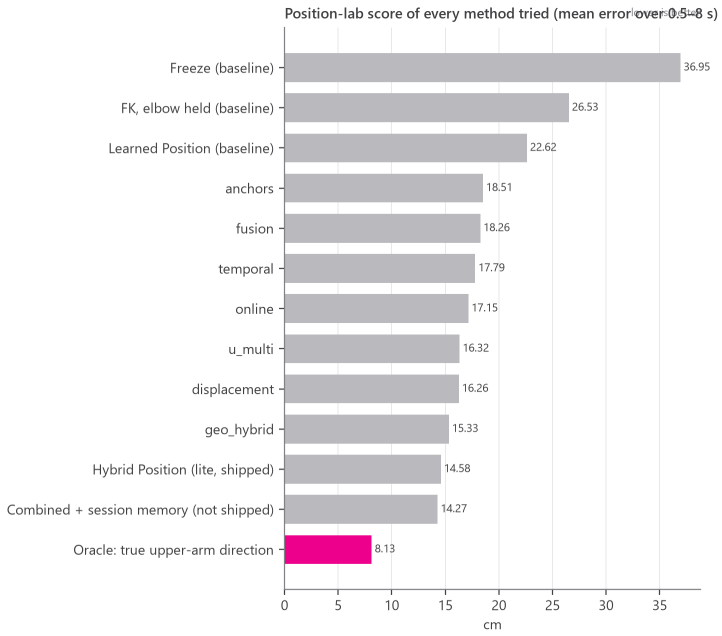


Figure 7.4 *Leave-one-clip-out, look-away protocol; score = mean of the errors at 0.5, 1, 2, 4 and 8 s. All seven explored ideas beat the deployed learned model. The oracle shows what a perfect upper-arm sensor would provide.*

Verification in the real GUI code

The GUI's streaming tracker matched the lab estimator exactly: the maximum difference was 0.0000 cm over 1,600 episodes from two clips, at 3,372–3,593 frames per second. Replaying the unseen clip low_new3 through the tab's own update function in blind mode, with hybrid weights that never saw it, reproduced the lab numbers (see the low_new3 replay table below). With the arm still for 6 s and the sensors frozen, the gate

stayed at 0 % and drift was 0.000 cm.

Table 7.5 low_new3 replayed through the Hybrid Position tab's real update code in blind mode: mean 3-D wrist error. The hybrid weights never saw this clip (lab: 8.6/12.2/14.5/16.1/17.9 cm). The Learned Position column used the shipped model, which was trained on this clip, so it is in-sample and optimistic

After loss	Hybrid Position	Learned Position	FK, elbow held	Freeze
0.5 s	8.3 cm	11.7 cm	8.3 cm	12.6 cm
1 s	12.4 cm	18.2 cm	15.3 cm	20.8 cm
2 s	14.1 cm	18.7 cm	19.7 cm	27.3 cm
4 s	16.3 cm	21.0 cm	26.7 cm	38.5 cm
8 s	17.6 cm	26.1 cm	36.8 cm	51.3 cm

Live blind tests

I logged three live sessions with BLIND TEST on and scored them afterwards against the camera (see the live blind-test chart below).

- **Blind started at the mouse.** Blind mode was switched on with the hand (wearing the CyberFinger) resting on the mouse, and the arm was then lifted unseen. The hybrid still had the lowest error (18.8 cm, against 27.8, 29.0 and 27.3 cm) but followed only 16 % of the depth. The most likely cause is the poor anchor: FK-held sat 22 cm too low throughout, with a single anchor. A "⏱ start BLIND in 5 s" button was added so that the hand can settle in view first.
- **Proper camera fix.** A 39-s log with five anchors, every scored frame more than 8 s after the loss. The hybrid scored 10.8 cm, against 22.5 cm for Learned Position and 14.3 cm for freeze. FK with the elbow held was slightly better on this log (9.6 cm). The hybrid followed 71 % of the depth of each reach (correlation 0.83), against 42 % (0.52) for Learned Position.
- **After about 2.5–3 minutes without the camera** (152–181 s blind): freeze won (30.8 cm against 32.1 cm for the hybrid), and left and right were reversed.

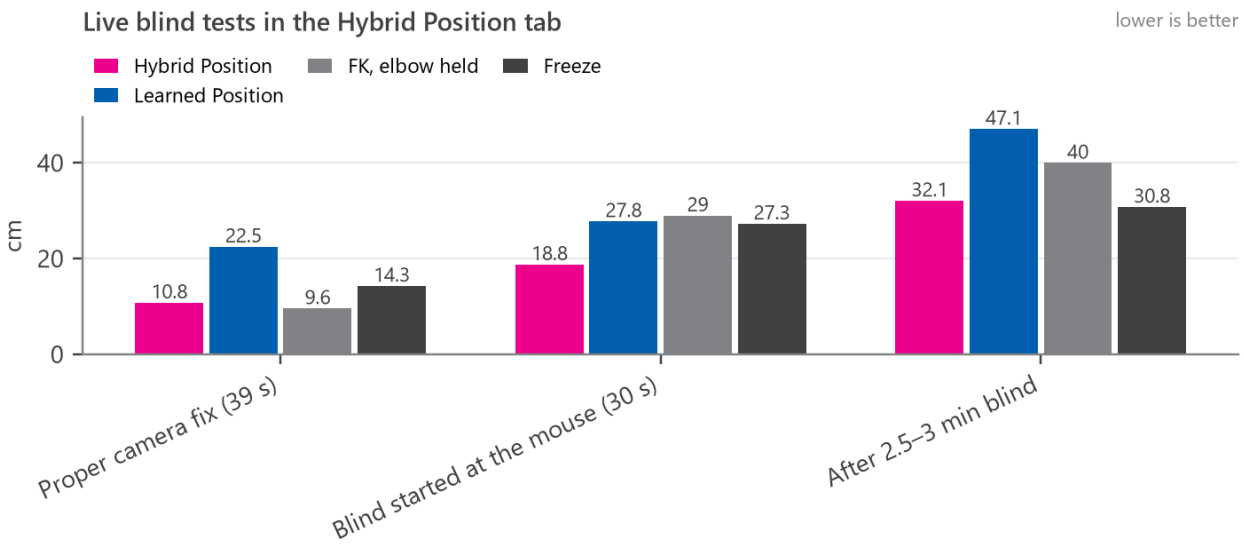


Figure 7.5 *Mean 3-D error over each logged blind span, scored by the camera. From a good camera fix the hybrid follows depth, although FK with the elbow held was slightly lower on that log. A poor anchor (most likely blind mode started with the hand at the mouse) hurts every method. After minutes without the camera, IMU heading drift makes freeze the best.*

The third result is caused by heading drift. Without a magnetometer, the wrist IMU's heading (its rotation about the vertical) is not tied to anything once the camera is gone. The measured drift depends on the motion: about 1° per minute in the recordings used to check the frame conventions, but up to 92° over 5 minutes in a fast clip. On lab clips the hybrid still followed the hand 30–60 s after the loss (18.4 cm), while FK-held collapsed. A working rule from these tests is that the horizontal direction can be trusted for about one minute after the last camera fix. The camera re-anchors it whenever the hand is seen again.

Hand near the face

Two clips recorded on 21 September with the forearm upright and the hand near the face were tested with weights trained only on the six older clips. In face_move, where the elbow slides, the hybrid was clearly best: 8.5 cm mean over 0.5–8 s, following 86 % of forward reaches, against 13.6 cm for Learned Position, 19.2 cm for FK-held and 26.8 cm for freeze. In face_depth the forearm itself carries the reach toward and away from the face, and plain FK with the elbow held was slightly better (7.9 cm against 9.7 cm; Learned Position 23.3 cm, freeze 15.8 cm). The deployed weights were then retrained on all eight clips.

AUTO switching, smoothing and the depth strip

In AUTO mode the tab switches to the prediction when the occlusion gate reports the hand out of view, behind an object, behind the other hand, or a frozen pose. It hands the dot back to the camera after three trusted frames in a row, gliding over 0.25 s. In a replay of 18 simulated out-of-view spans on low_new3, the prediction took over at the first frame and the camera returned after 2–3 frames. Errors were 8.3/12.8/16.9 cm for losses of 1/3/6 s, against 12.4/20.7/36.1 cm for freeze.

I considered a temporal (sequence) model to smooth the motion, but measured first. On 98 unseen 12-s blind spans, the raw hybrid output was already smoother than the camera's own wrist track (second-difference jitter 0.57 against 0.90 cm). Every low-pass filter cost accuracy through lag: exponential smoothing with time constants of 0.1/0.2/0.4 s raised the error from 14.48 cm to 14.8/15.5/16.8 cm. The visible jumpiness came

from the tab redrawing at only 13–15 Hz, and caching the static drawing layers cut its redraw time from 21 to 10 ms. An optional One-Euro filter, an adaptive low-pass that smooths more when the signal moves slowly, is applied to the elbow part only and is on by default. It nearly halves the jitter (0.57 to 0.31 cm) for a 0.16 cm accuracy cost (14.48 to 14.64 cm).

A depth strip under the two body views plots the last 12 s of depth (from the eyes, or optionally from the body) for the camera, the hybrid and Learned Position, and shades the time spent without the camera. Every run can be logged to CSV and analysed with analyze_hybrid_log.py, alongside the Learned Position, FK-held and freeze estimates.

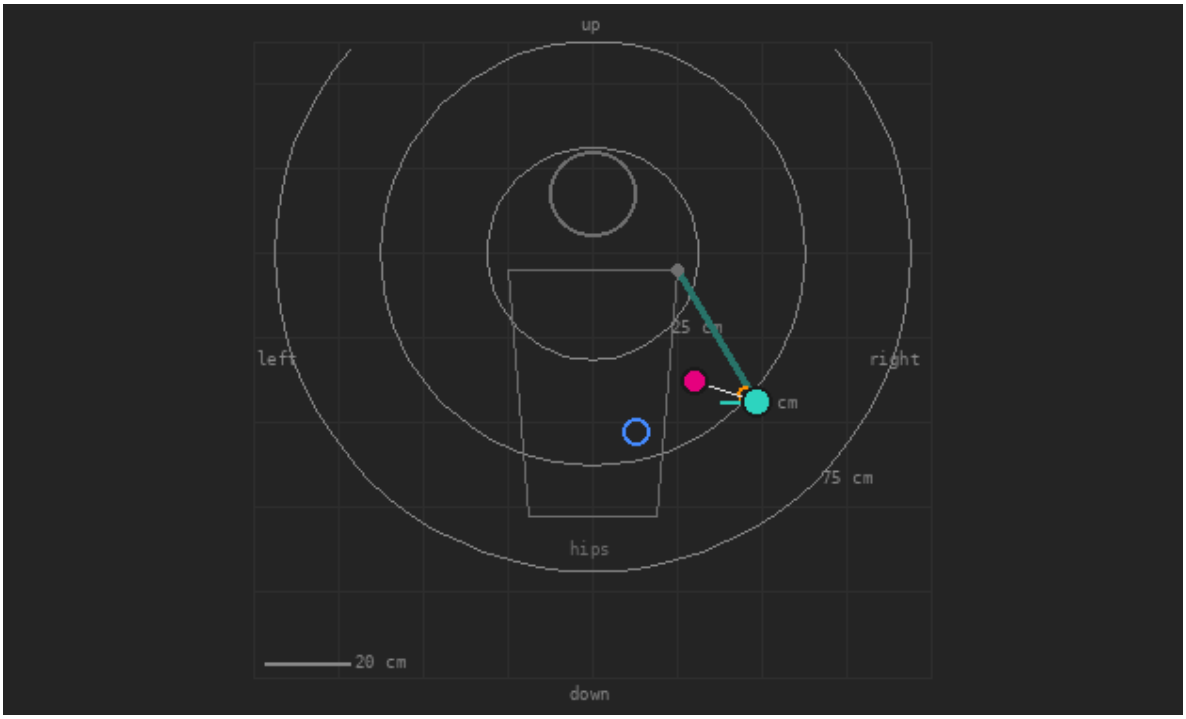


Figure 7.6 *Hybrid Position tab, FRONT body view (left/right × up/down; neck origin; rings at 25/50/75 cm; 20 cm scale bar). Pink = camera wrist, teal = hybrid estimate with its arm line, hollow blue = the Learned Position estimate for comparison, small orange = FK with the elbow held. Rendered from the tab's drawing code during interface testing; the state shown is illustrative, not a measurement.*

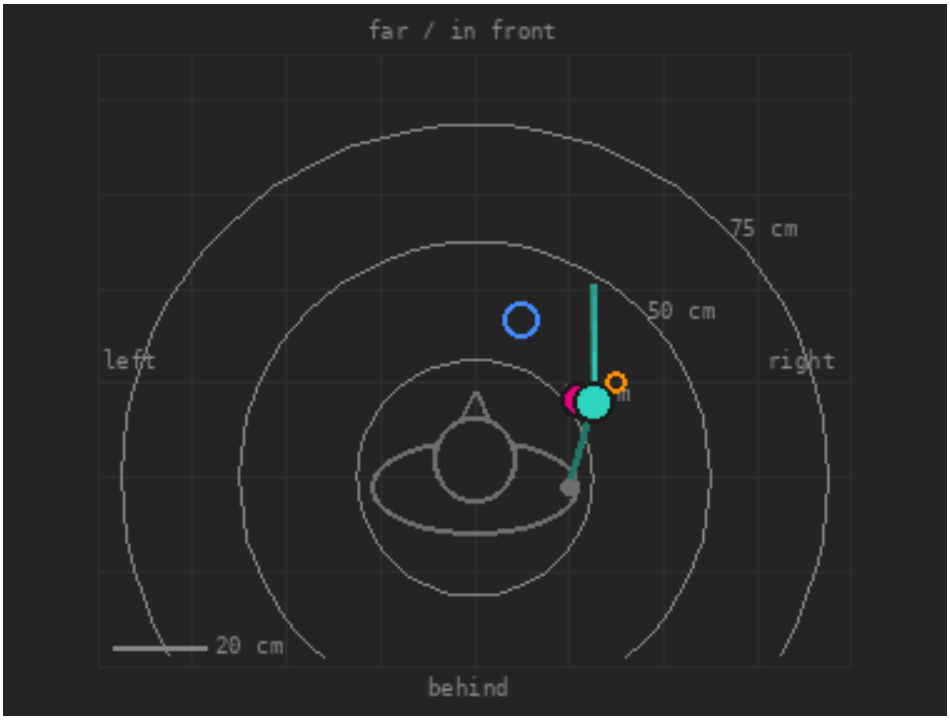


Figure 7.7 *Hybrid Position tab, TOP body view (left/right × near/far, with "far / in front" at the top). It shows reach (depth), which the earlier dome view could not. Same colour code as the FRONT view; interface-test render, illustrative state.*

Robustness and caveats

These were measured on the combined lab estimator, which shares the hybrid core and gate, unless stated otherwise.

- **Twist:** a pure ±45° or ±90° forearm twist after the loss moves the estimate 2.5 or 4.5 cm, against about 11 cm for Learned Position.
- **Heading errors:** a wrong body heading ("⌖ Set forward") of ±30°, 60° or 180° raised the error from 14.08 cm to 14.2–14.3, 14.6 or 16.0 cm. An IMU heading jump of 5° or 15° after the fix moves the estimate 2.1 or 6.3 cm.
- **Armband:** without the armband accelerometer, the geo_hybrid candidate scored 16.79 instead of 15.33 cm, so the band adds about 1.5 cm. The lite variant was not scored without it.
- **Still arm:** zero drift was shown with frozen sensors only. With 0.5° or 1.0° rms of orientation jitter, the output drifted 0.8–9.9 cm or 2.4–12.9 cm over 0.5–8 s.
- **Nearly still arm, real data:** where the wrist stayed within 3 cm, the estimate erred 2.43 cm against 1.80 cm for freeze (worse in 50 % of cases). Where the wrist ended within 3 cm of its start, it erred 4.56 against 1.98 cm (worse in 68 %; 9.5 against 2.3 cm at 2 s).
- **Neck pivot:** the offset used in the GUI and the bench, (0, −0.10, +0.08) m, is not the best-fit pivot, about (0.01, −0.16, +0.16) m. All methods share this imperfection, and it is worth fixing.

7.9 Where the work went, limitations and next step

Hybrid Position became Fusion Studio's out-of-view position source (commit ad1ae80, 24 September). Fusion Studio uses the camera when it can see the hand, otherwise the Hybrid Position model, otherwise the held position. SciCortex's Fusion Studio release ships hybrid_position.py and its weights byte-identical to the research branch. For the product, this replaces a frozen hand position (36.95 cm on the benchmark) with an estimate that follows the arm (14.58 cm). The Fusion Studio section shows its POSITION panel during such a loss ("lost 0.9 s · 4 anchors · motion gate 99%").

Limitations

- One user, one CyberFinger, right hand only; the left arm is mirrored and untested. The lab clips come from two days (17 and 19 September); the deployed weights add 21 September.
- Without a magnetometer, heading drift limits the horizontal direction to about one minute after the last camera fix; after 2.5–3 minutes, freeze was better.
- Pure translation of the elbow or shoulder (for example, forearm upright and hand pushed toward or away) is only weakly observable from orientation.
- Some lab settings were tuned on the test folds; the shipped gain over Learned Position (8.0 cm) is far larger than this tuning effect.
- Exact zero drift holds only for frozen sensors; on real, nearly still arms the estimate is slightly worse than freeze.
- The method relies on the headset-tracked neck position, and the neck-pivot offset is not yet the best fit.

Most of the remaining error comes from the upper arm, which no sensor measures. With the true upper-arm direction, the same framework reaches 8.13 cm, and FK with a perfect upper-arm direction reaches 5.2–5.6 cm on the recordings.

Recommended next step

The most valuable next step is an upper-arm IMU. The BODY1 slot is already logged by Position Capture and is empty on the lab CyberFinger, so recording one needs no pipeline change, and IMU FK Position already accepts it as an upper-arm source. A cheap first test is to wear the MindRove band on the upper arm. HOOV-style learned inertial odometry would also need firmware that streams raw 6-axis data at 200 Hz or more.

8 Key postures, transitions and anticipation

I built an EMG recogniser for six key hand postures (open, fist, point, pinch, victory and rest) that keeps working where the headset camera cannot see the hand. I also built the decision logic that turns its output into a stable, fast hidden hand in the Fusion Studio. Camera self-teaching raised second-session recognition from 45% to 86% in an offline test, and holding the posture while the arm turns cut wrong switches from 12.1 to 6.5 per minute. An expected-next-posture rule, built on a transition prior proposed by my supervisor, removed the brief wrong postures shown while a fist opens (30% to 0%). Following my supervisor's EMG-burst idea, a learned detector flags about three in four posture changes roughly 57 ms before the fingers move. With a soft, anticipating hand (optional, off by default), the hidden hand's lag behind the camera's own hand fell from 230 ms to 45 ms.

8.1 Why recognise a few key postures

Earlier work used a continuous regressor that maps EMG to finger curls and learns from the camera. It ran into two hard limits. First, the Quest 3 camera cannot label a closed fist: the curled fingers hide each other, and every fist frame came with a tracking confidence of 0.50. Second, the same intended gesture produces a different 8-channel EMG pattern when the arm is lowered or moved. This is the *limb-position effect* described by Fougner et al. (2011), and it is strongest where the camera does not look: at the hip, at the side and behind the back.

On 2026-09-24 I changed approach for a small set of fixed postures (open, fist, point, pinch and victory, plus a catch-all *other*). For a fixed posture the label is simply the posture being held, so training data can be recorded with no camera at all, in the places where the hand is actually used: in front, at the hip, at the side, behind the back and while moving. When the camera loses the hand, the ✦ Fusion Studio snaps the hidden hand to the shape of the recognised posture. Three tabs came out of this line of work. **Key Postures** records the postures and recognises them. ◈ **Fusion Logic** shows live the rules the Studio uses to decide. ⚡ **Transitions** detects the moment a posture starts to change.

Terms used in this section:

- **EMG**: the electrical activity of the forearm muscles, measured by the MindRove 8-channel armband.
- **IMU**: an orientation sensor on the CyberFinger. The *wrist IMU* sits in the wrist module and the *knuckle IMU* sits on the hand.
- **Gate**: the rule that turns the recogniser's per-frame probabilities into one stable posture on screen.
- **Wrong switch**: the displayed posture changes although the hand keeps holding the same posture. Reported per minute.
- **LOPO**: leave-one-arm-position-out testing, which trains on four arm positions and tests on the fifth. **LORO**: leave-one-recording-out testing.

- **Real tick**: recorded sensor data replayed through the GUI's own update loop, so the test runs the shipped code rather than a re-implementation.

8.2 Key Postures

The Key Postures tab recognises a few important hand postures from forearm EMG, reliably and out of the camera's view, so that the ✦ Fusion Studio can rebuild the hidden hand in exactly that posture. The tab was built and checked with simulated sensors on 2026-09-24 and first committed on 2026-09-25 (commit 00085f0).

Recording and the tab layout

Recording is hold-to-label. The wearer picks a posture and an arm position (front, hip, side, back or moving) with radio buttons, holds the posture and presses "● Record 12 s". Each 12-second clip is stored with both labels. "Archive last clip" removes a bad clip and "⚙ Train recogniser" retrains the model. The tab shows live probability bars, the template hand next to the camera hand, a 30-second timeline, a dataset table (frames per posture and arm position), and an honest held-out report: the model is scored on the newest clip of each posture, and a confusion grid shows which postures are mistaken for which.

First recogniser (v1)

Version 1 used 40 standard time-domain EMG features (the Hudgins set: mean absolute value, RMS, waveform length, zero crossings and slope-sign changes on each of the 8 channels) plus a 3-number wrist tilt. The model was a class-balanced softmax regression. It was trained on the real clips plus physically motivated synthetic copies: the electrode band rolled by ±1 channel, per-channel gain changes, different contraction intensities, noise, and tilt jitter. The gate made a posture active once it led with $p > 0.75$ and a margin of 0.25 for 0.15 s, and released it after its probability stayed below 0.5. The *other* class never snaps the hand.

The first real session (2026-09-24, 19:21–19:32) produced 25 clips: 5 postures × 5 arm positions, about 400 frames each at about 33 Hz. On the newest clip of each posture (the moving clips), accuracy was **76%** (fist 96, victory 97, pinch 72, point 64, open 51). With the gate, a wrong posture was shown on only 0–8% of frames. LOPO accuracy was 79% pooled: front 94, side 85, back 84, moving 76 and hip 58. The hip was clearly the hardest position. Variants with a gravity-based tilt, normalised amplitude or a 64-unit neural network did not beat v1, so the conclusion at that point was that more data was needed, not a different model.

On screen, the first pinch did not look like a pinch. The templates only curled the fingers, which swung the thumb toward the palm. The templates were therefore replaced by the camera's median 26-joint shape of each posture. In the new templates the thumb-tip to index-tip distance is 1.5 cm for pinch, against 10.8 cm for open and 8.0 cm for point. The fist keeps an anatomical prior, because the camera cannot see a fist.

v2: covariance features, a small neural network and camera self-teaching

A second session exposed a cross-session collapse. Its clips were recorded on 2026-09-25 between 01:41 and 01:47, about six hours after the first session. Version 1, trained on the first session, scored only 2% on open and 30% on point in the second (pinch 76%). The second-session open had 2.3× the EMG level of the first and a pinch-like channel pattern. The cause was not band rotation and not signal quality: the same posture was simply held with a different contraction.

Version 2 (commit 4b800e6) added channel-covariance features computed from the raw EMG window: the log standard deviation of each channel and the 28 correlations between channel pairs. Together with the 40 time-domain features and the tilt, the model takes 79 inputs. Since the tilt input was dropped on 2026-10-01, its 3 columns are fixed at zero, so 76 inputs carry information. A 64-unit MLP (a small neural network) replaced the linear model.

The key addition was **camera self-teaching**. While the camera sees the hand, a geometric labeller (cam_label) names the posture from the 26 tracked joints, using finger curls and the thumb–index gap. It agrees with the recorded labels on 98–100% of clearly seen frames and never outputs *other*. An online teacher then fine-tunes a live copy of the model with 3 Adam optimisation steps per taught frame. Half of each batch is replayed from a class-balanced set of 12,000 rows stored with the model, and the other half comes from the current session, weighted ×3. A frame is taught only after the posture has been held for 0.4 s with camera confidence ≥ 0.9. The stored model itself is never changed, and "↺ Forget today's teaching" returns to it.

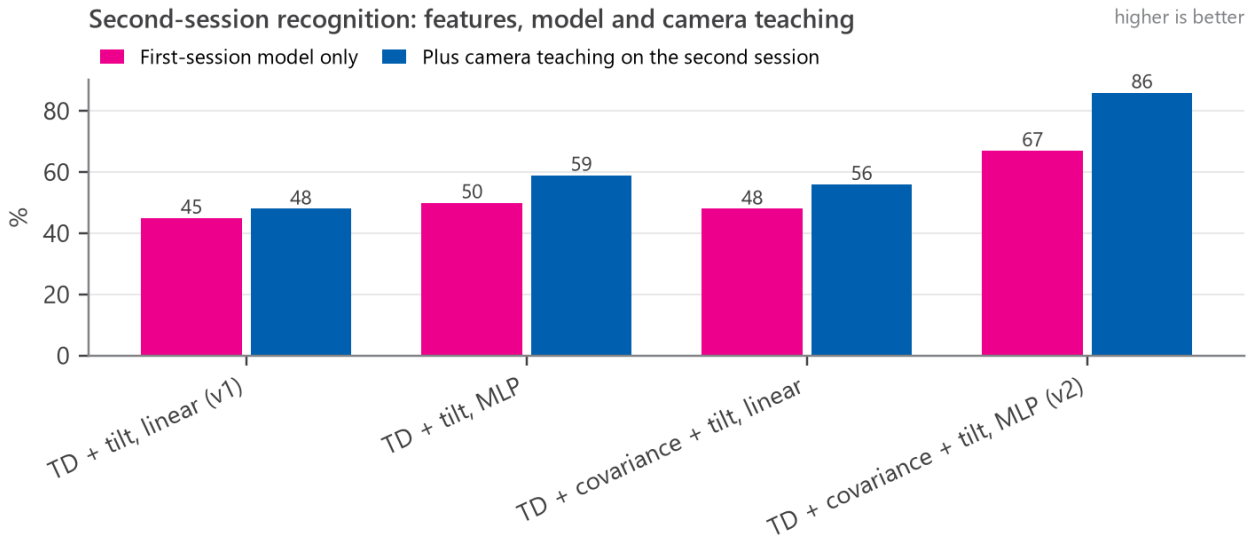


Figure 8.1 *Accuracy on held-out second-session clips (open, point, pinch; all frames). The first-session model was refitted offline on the camera-labelled frames of the other second-session clips. Only covariance features, the MLP and teaching together reach 86%. The live, incremental teacher in the GUI reached 76%. Source: gui_backups/kp_adapt.out.*

Through the real tick, scored only on frames the camera did not see, teaching raised accuracy from 50% to 77% (point 30 → 92%, pinch 99 → 93%, open 1 → 16%). Open stayed low only because the session's first open clip came before the camera had ever seen an open. The later open clip scored 94%. The operating rule that follows is simple: show each posture to the camera once per session. Teaching raised the median tick cost from 10 to 17 ms, and the shorter feature window introduced next made the tick 5 ms cheaper again.

Camera self-teaching made the recogniser survive a new session. On held-out second-session clips, accuracy rose from 45% (v1) to 86% (v2 with teaching, offline refit). On frames the camera could not see, through the real GUI loop, it rose from 50% to 77%.

A faster decision gate

Most of the delay came from the gate's fixed 150-ms hold: a switch took 167 ms median and 233 ms at the 90th percentile, followed in size by the 128-ms EMG window. Two changes fixed this (commit 6adfc7b). A fast path lets a leader with $p \geq 0.97$ commit after 40 ms, and the v2 window was cut from 64 to 48 samples (128 → 96 ms) with no loss of cross-session accuracy (77% for both windows over two seeds). The release time also went from 0.30 to 0.20 s. Confident switches now take **67 ms median and 100 ms at the 90th percentile**. The trade-off was recorded: on an unsure model (70% top-1), the faster gate raises wrong switches from 26 to about 80 per minute. A shorter hold alone (0.05 s) gave the same speed with four times the flicker and was rejected.

Holding the posture while the arm turns

In use, the displayed posture sometimes switched suddenly when I rotated my arm or wrist. Measured on the 42 held clips, accuracy was 90% when the wrist turned slower than 30°/s and 79% above 180°/s, and turning doubled the wrong-switch rate. The tilt input turned out to be part of the problem. It was a horizontal axis in the IMU's z-up world frame, so it followed heading drift and the direction the wearer faced, and the model had learned it as a shortcut: swapping only the tilt changed 18% of the answers. Removing the tilt (commit 5a262b5) raised LOPO accuracy from 88.7% to 91.4%, cut wrong switches from 12.1 to 9.0 per minute, and raised taught second-session accuracy from 67–72% to 76–80% (open 13% → 44–50%). A tilt computed from true gravity was worse, at 15 wrong switches per minute.

The gate then gained a turn rule. The first, strict version (hold above 60°/s for 0.4 s) reached 6.2 wrong switches per minute, and turn-related errors fell from 3.5 to 0.7 per minute. Real Studio sessions, however, move the arm at a median of 25–175°/s, so the strict rule was active most of the time: real switches with an active arm took 141 ms median and 660 ms at the 90th percentile. The retuned rule (commit 13f299f) starts holding above 90°/s and grows the hold to 0.25 s at 180°/s, with a 0.10-s settle time. It reaches 6.5 wrong switches per minute, while real switches take 39/143 ms (median/90th percentile) with a calm arm and 48/237 ms with an active arm, against 39/128 ms without the rule. The lesson: tune gate latency on live motion traces, not on still posture clips.

Figure 8.2 *Dropping the heading-dependent tilt shortcut and holding the posture while the arm turns. The retuned rule gives up 0.3 per minute to keep real switches fast with an active arm (48/237 ms instead of 141/660 ms). Sources: commits 5a262b5, 13f299f; bridge/key_postures.py.*

About 70% of the remaining wrong switches happen while the arm is still. They are ordinary EMG confusions, with open the weakest posture, so they call for better evidence rather than more gating. Allowing the fast path at $p \geq 0.99$ while turning destroyed the protection (8.4 per minute). The gate takes the larger of the wrist and knuckle turn rates, and using both IMUs gave about the same result as the wrist alone (6.2 vs 6.4 per minute).

The Studio's drawn hand also became faster at blending into a new posture (commits 5704ffb, 0c446df). The time for the hand to reach 50%/90% of the new shape after a switch went from 150/317 ms to 83/150 ms and then to 67/100 ms, as the 90% blend time was cut from 0.30 to 0.12 and then 0.08 s. Two further fixes helped: fresher EMG in the poll loop, and a MindRove reader that polls every 5 ms instead of every 20 ms. A looser gate would have saved only 16–34 ms, but it raised wrong switches from 9.0 to 14.6–16.7 per minute and flickers through a third posture from 12% to 21–23% of changes, so it was rejected.

The transition prior: a Markov chain over postures

In a whiteboard meeting in the week before 2026-10-08, my supervisor proposed that postures follow a Markov chain, because “fist can't go straight to pinch”. To estimate which posture follows which, I used Meta's public emg2pose mini dataset (one user, 15 recording stages × 2 hands, 2 kHz EMG with motion capture; licence CC-BY-NC-SA 4.0, research use). I relabelled it with our own camera labeller through a numpy port of the UmeTrack forward kinematics, which matches Meta's implementation to within 0.00005 mm. The relabelled data held 411 transitions, and **open is the hub**: most changes pass through it. My own 23 camera-seen transitions at the time agreed.

A hidden-Markov forward filter (PostureHMM) used this prior (commit 3fc8f01). On simulated sequences built from real EMG, it cut wrong switches from 19.3 to 5.8 per minute, at the cost of 50 ms more median delay. An honest control showed that a uniform prior did almost as well (5.3 per minute), so most of the gain comes from accumulating evidence over time. The prior's own share was smaller: implausible fist↔pinch switches fell from 0.9 to 0.6 per minute, and the slow tail shortened (90th percentile 315 vs 400 ms at equal median delay).

Through the real tick the filter added 33 ms (133 vs 100 ms), so its switch, "HMM posture smoothing", ships OFF by default.

Table 8.1 Every decision-rule family tried sits on the same speed–stability trade-off (simulation built from real EMG, 2026-10-08). Source: project notes, key-postures-tab.

Decision rule	Delay, median / 90th pct	Wrong switches per minute
Gate	33 / 67 ms	14
HMM with transition prior	83 / 117 ms	6.6
Soft blend through the HMM	33 / 67 ms	13.8
Raw soft blend	0 / 25 ms	43

The deeper finding was that the limit lies in the evidence, not in the decision rule. In LOPO testing, open↔pinch are confused 11.4% of the time and fist↔point 5.8%, while all other pairs stay at or below 1.7%. A pair-aware gate and a gate driven by EMG change were also tried and failed: the change detector fired on only 2% of the synthetic crossfades. The general prior has limits of its own. Point and victory are rare in emg2pose (0.9% and 0.3% of frames), and a rule that slowed prior-unlikely switches was rejected because it made open→point take 396 ms. Real transitions last 200–700 ms and the fingers move in sequence. A learned in-between path would be 12–35% closer to the real hand (open→fist: 8.0 → 5.2 mm), but it was not implemented because it needs longer blends, which cost latency.

Opening a fist without flashes

Opening a fist passes through shapes that read as point or victory, and in simulation the gate flashed one of them on 40% of fist→open changes. The prior was also biased: its fist→point probability (38%) came entirely from emg2pose's scripted counting drill (all 20 of its fist→point transitions), while everywhere else a fist opened 23 times out of 24. I rebuilt the prior without that stage (314 transitions from 14 stages). After a fist, the hand now opens 88% of the time, and after a pinch 98%. Two gate rules use it (commit f2a1534):

- **Expected next posture**: a posture that lies between the current one and the expected next one, finger by finger, must hold for 0.25 s and gets no fast path.
- **Relaxation cue**: when EMG power falls below 0.6× its maximum over the last 0.5 s and open is expected, in-between evidence counts toward open, which may then commit at $p \geq 0.8$.

Through the real Studio tick, on realistic changes rebuilt from the clips, fist→open flashes fell from **30% to 0%**, and the new posture appeared after 235 instead of 260 ms. Other changes were unaffected, and wrong switches while holding went from 9.0 to 8.9 per minute. The cost: a genuine fist→point change (2–3% of fist changes) now takes 327 instead of 194 ms and usually shows open first. The Studio switch "expected next posture (prior)" is ON by default.

REST: a sixth posture

On 2026-10-22 my supervisor asked for "something between fist and open, we call rest": the relaxed hand. None of the existing data held a rest. On the position recordings, the camera called the relaxed hand open for 1,151 s and rest-like for only 18 s. Rest was therefore recorded like the other postures: on 2026-10-23 I recorded rest clips at all five arm positions, and the moving clip was later archived. Rest has the quietest EMG of all the postures, 0.25–0.41× that of open.

- **Recognition** (LOPO): rest 83%, with 15% read as *other*, because the two September *other* clips look like a relaxed hand. At the hip, rest scores 46%. Open drops from 85% to 82%, and the other postures barely move (fist 98, point 93 → 92, pinch 87 → 86, victory 96 → 98%).
- **Camera rule**: four fingers loosely and evenly half-curled, with the thumb at least 3.5 cm from the index. It labels 100% of the rest clips' seen frames (99% according to the code comment) and 0% of the other postures' clips.
- **6×6 prior** (emg2pose relabelled plus my position recordings, 1,161 changes): after rest the hand opens 86% of the time; after a fist it opens 79% or rests 10%; after open it rests 72%.
- **Gate**: rest is never treated as an in-between flash, and a relaxing hand's in-between evidence goes to open or rest, whichever leads. Fist→rest changes that showed open first fell from 30/30 to 7/30, and rest now appears after 226 instead of 268 ms. A model without rest decides exactly as before (gate identical on 52,000 frames).

Through the real Studio tick, at an arm position held out from training, rest was named 100% of the time with the gate and 99% with the soft hand (described below), about 250 ms after the change (commit 8261d44). I trained the 7-class model with ⚙ Train on 2026-10-23 at 11:18, using 16,661 real and 99,966 synthetic rows. On the newest held-out clip of each posture it scores 75.7% over 2,412 frames. The held-out rest clip is the back clip, and the others are moving clips.

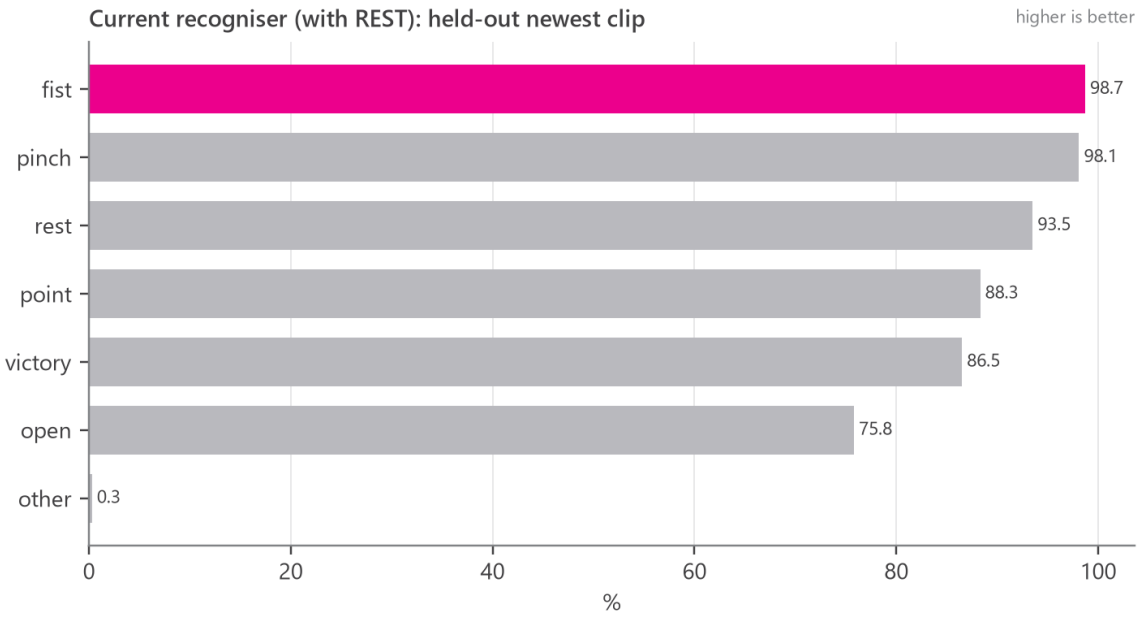


Figure 8.3 *Model trained 2026-10-23 11:18; 75.7% overall on 2,412 frames. The held-out other clip was read as rest on 363 of 398 frames: the old other recordings look like a relaxed hand. Open lost 60 frames to pinch, and victory lost 53 to open. Source: bridge/key_postures_model2.npz.*

Table 8.2 Key Postures milestones. Sources: commits 00085f0, 4b800e6, 6adfc7b, 5a262b5, 13f299f, 3fc8f01, f2a1534, 8261d44; project notes.

Date	Change	Measured effect
2026-09-24	v1: time-domain features + tilt, softmax	Newest clips 76%; LOPO 79%
2026-09-25	v2: covariance features, MLP, camera self-teaching	Second session 45% → 86% (offline); hidden frames 50% → 77% (real tick)
2026-09-25	Fast path, 96-ms window	Confident switches 167 → 67 ms median
2026-10-01	Tilt dropped; hold while the arm turns	LOPO 88.7 → 91.4%; wrong switches 12.1 → 6.5/min
2026-10-08	Transition prior, optional HMM	Simulated wrong switches 19.3 → 5.8/min, +50 ms (off by default)
2026-10-09	Expected-next-posture rule	Fist→open flashes 30% → 0%
2026-10-23	REST posture	Rest 83% at unseen arm positions; current model 75.7% held-out

8.3 ◈ Fusion Logic

The ◈ Fusion Logic tab (commit efe2c08, 2026-10-09) draws live the two rules behind the Fusion Studio's decisions, so that a reviewer can see why the hidden hand looks the way it does. The first rule is which posture usually comes next (my supervisor's transition prior) and how the gate uses it. The second is how the camera and EMG are weighed for each finger. While the tab is open, the Studio keeps computing in the background.

Left: the transition map. Each posture is a node, with open as the hub and, since 2026-10-23, rest placed between fist and open. Arrow width shows the transition probability (emg2pose without the counting drill, plus my own sessions). The recognised posture is lit, its outgoing arrows are coloured, and its likely successors are listed (for example, “now FIST → next: open 88%”). Under each node is the live EMG probability. A text line spells out the gate's reasoning: the expected next posture, the in-between postures being held back, and “relaxing → opening”.

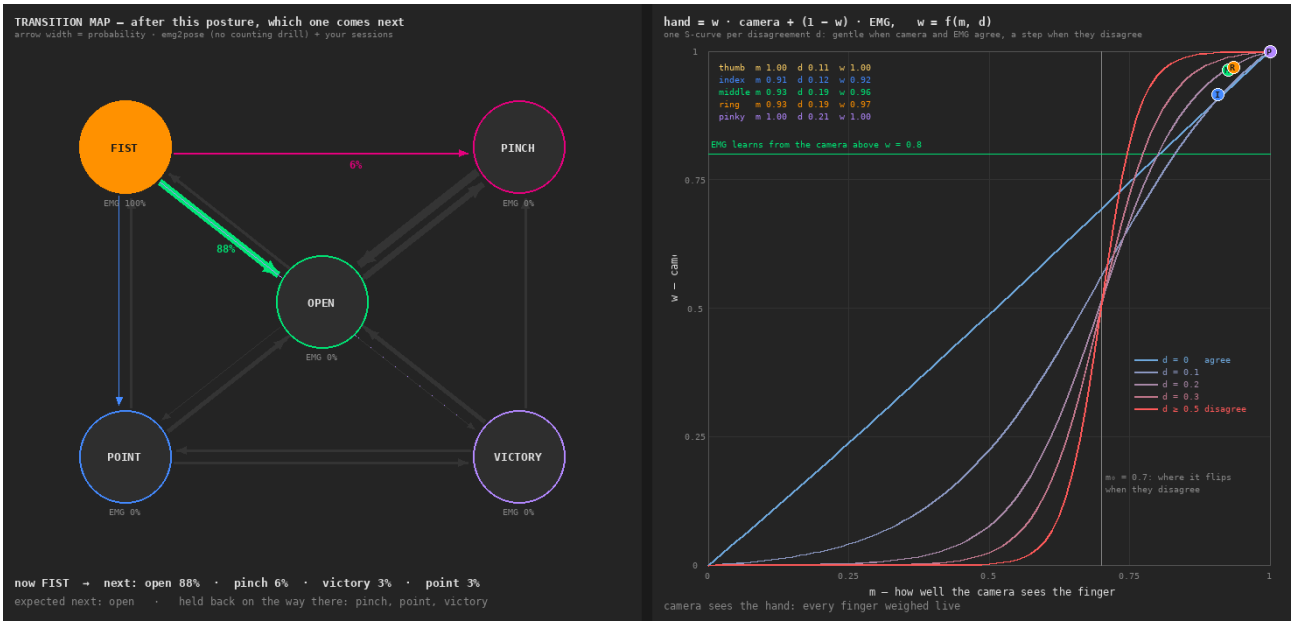


Figure 8.4 ◈ *Fusion Logic with the hand in a fist (2026-10-09, 5-posture prior). Left: the transition map, with open as the hub; next is open 88%, and pinch, point and victory are held back on the way there. Right: the camera↔EMG weight w = f(m, d), with one S-curve per disagreement level d, the flip at m0 = 0.7, the learning line at w = 0.8, and each finger's live (m, d, w).*

Right: the camera↔EMG weight, a rule my supervisor drew on the whiteboard and I implemented on 2026-10-08 (commit 18d60db). For each finger, the drawn hand is w·camera + (1−w)·EMG, where w depends on two numbers. The first, m, is the camera's reliability for that finger: its ray-cast visibility, lowered further by the occlusion gate. The second, d, is the difference in curl between the camera and EMG estimates. The S-curve's steepness rises from 1 when the sources agree (w ≈ m) to 30 when d ≥ 0.5, where the curve becomes a step at m0 = 0.7: if they disagree, the camera is trusted fully above 0.7 and not at all below. The weight is smoothed over 60 ms, and the EMG model learns from the camera only where w > 0.8. The tab draws one curve for each of d = 0, 0.1, 0.2, 0.3 and ≥ 0.5, the learning line, and each finger as a live dot.

Measured through the real Studio tick on 40 held clips, with EMG models that had not seen the clip's arm position, the rule raised the share of frames showing the right posture from 95.8% to 96.8%, and lowered frames showing no posture from 3.0% to 2.0%. A step at m0 = 0.5 was tested and rejected: partly hidden fingers fell from 93.2% to 92.1%, because a partly hidden camera that disagrees with the EMG is right only about 70% of the time. A later anatomy fix moved posture accuracy on the same 40 clips from 96.8% to 96.6%.

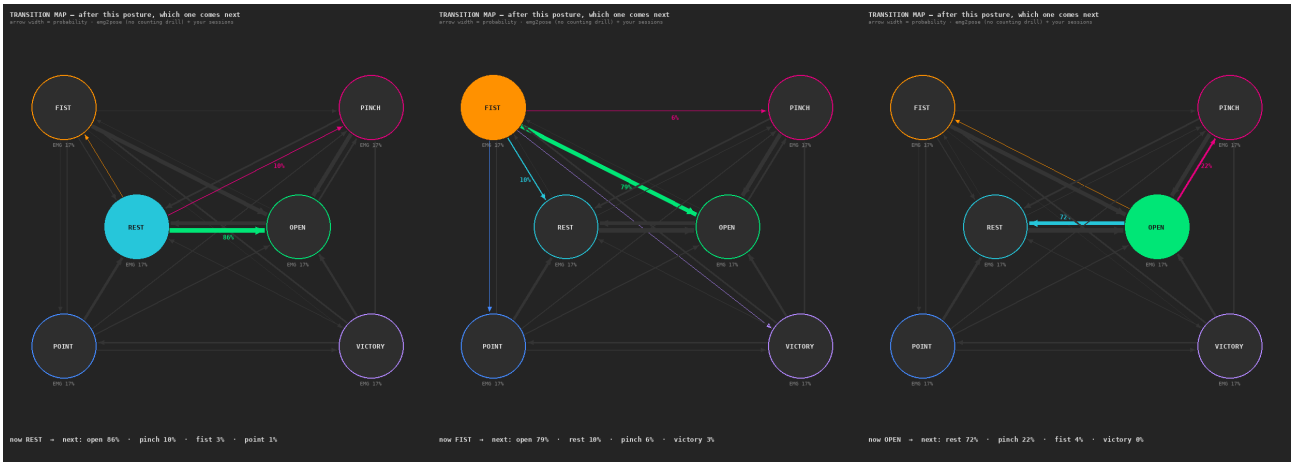


Figure 8.5 *Transition map with REST (2026-10-23), from the real app, with rest, fist and open each as the current posture. After rest the hand opens 86%; after a fist it opens 79% or rests 10%; after open it rests 72%. The uniform EMG 17% labels appear because no live EMG was streaming.*

A compact copy of both drawings sits under the Fusion Studio's front and top views (commit a05aa16) and shares the same drawing code. It is redrawn about 20 times per second, and the Studio stayed at 60–62 frames per second with it. The drawing code was later added to the generator of the single-window release (commit cf71b91), so SciCortex's Fusion Studio shows it too. The compact strip under the Studio's views is shown in the Fusion Studio section.

8.4 ⚡ Transitions

The ⚡ Transitions tab (commits f3f0cef to 05e4a81, 2026-10-22) implements another of my supervisor's whiteboard ideas. A posture change starts with a specific EMG burst, so the burst can serve as a flag that a transition is happening before the fingers move, show where the hand is going, and make the hidden hand faster without more flicker. The tab also records my own transitions, labelled by the camera, to train and test the detector.

The evidence first

Before building the tab, I checked the idea on 249 transitions in emg2pose. A causal change score compares each channel's log envelope with its own 300-to-30-ms past, takes the RMS over channels, and is computed every 10 ms. It peaks at movement onset. At threshold 1.0 it catches **94%** of transitions, about 55 ms before the fingers move and about 275 ms before the posture can be read from the hand shape. The first 20 ms of the burst predict the next posture 66% of the time, against 54% for the prior alone (from a fist: 36% vs 4%; from open: 44% vs 25%). Longer windows added nothing.

False flags had to be measured on my own MindRove data. On 8 minutes of held postures with the arm still, the score raised 12.8, 5.6 and 2.2 false flags per minute at thresholds 0.8, 0.9 and 1.0. With the arm moving, false flags came mostly while the wrist turned faster than 60°/s, so flags raised during fast turns are vetoed using the IMU.

The tracker and the tab

- **ChangeScore** computes the burst score from the EMG.
- **DirectionModel** predicts where the hand is going: the prior multiplied by the nearest burst pattern, learned online.
- **TransitionTracker** raises flags, applies the turn veto, settles each flag by the posture the hand settled in, and keeps the statistics. Settling by the gate's first switch would have been wrong: in a replay it matched the true destination only 24 of 35 times, against 35 of 35 for the settled posture.

The left side of the tab shows the last 8 seconds: a heat map of per-channel EMG change, the score with its threshold and flags, lanes for the EMG-gate posture and the camera posture, and the forearm turn rate. The right side shows the posture now with a transition lamp, the predicted next posture, session statistics, the latest flags, burst clusters (one cluster per transition type), and the mean score around flags. The controls are a threshold slider, a "veto while the forearm turns" checkbox, "↺ Reset", "● Record session" (which writes a tx_*.npz file), "⚙ Train from recordings", and a switch between the learned detector and the plain threshold.

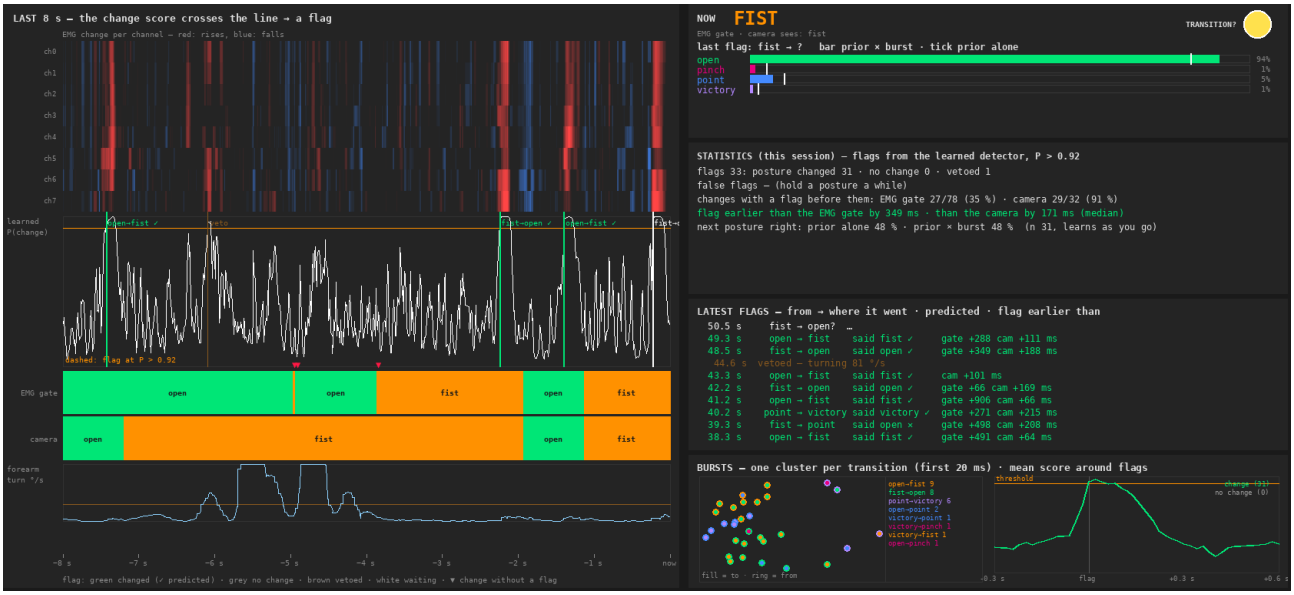


Figure 8.6 ⚡ *Transitions replaying recording tx_20261022_212403 through the real app with the learned detector (P > 0.92): per-channel EMG change, flags, EMG-gate and camera lanes, forearm turn rate, statistics and burst clusters. The detector had also been trained on this recording, so the panel's numbers (29/32 camera changes flagged) are an illustration, not a test.*

Six real recordings and the learned detector

On 2026-10-22, between 21:11 and 21:46, I made six recordings with the tab: about 409 s in total, at 36–43 updates per second, containing 190 scored camera-seen transitions (36, 53, 33, 16, 32 and 20 per recording). A change is scored as a transition when the camera saw both postures held for at least 0.2 s, no more than 1 s apart. The live tracker is less strict (any new camera label held for 0.15 s), so in the first recording it counted 39 camera changes where the scoring gave 36. The first recording confirmed the idea. Every transition had a burst, with a median peak of 1.1 at 6 ms after onset, while the 99th percentile on holds was 0.87. The quietest changes were pinch↔open and victory→point. The defaults changed accordingly: with a 50-ms envelope and threshold 0.9, 20 of 36 changes were flagged at ≤ 2 false flags per minute, against 12 of 36 for the original 25-ms envelope at threshold 1.2. Per-channel normalisation and an adaptive threshold both did worse.

The veto now reads the wrist IMU only. The knuckle IMU turns at 60–105°/s when the hand closes, and 4 of the 5 flags it vetoed were real. The recording also showed how much the EMG gate itself flickered on transition-dense data: 110 switches against the tracker's 39 camera changes, with 72% frame agreement with the camera. A naive gate that only switched on flags did worse, at 67%.

A learned detector (commit 249e77d) then replaced the fixed threshold. It is a logistic regression evaluated every 10 ms on 27 features: the per-channel change of 50-ms and 25-ms envelopes, the change over the last 40 ms, and the score's maximum and minimum. The camera supplies the labels. Frames from 100 ms before to 150 ms after a camera-seen onset count as “starting”, and stable holds count as “holding”. The decision threshold is chosen for ≤ 2 false flags per minute on holds.

Table 8.3 Transition flags on recordings the detector did not train on. Sources: commits 249e77d, 0fa2092, 1b5e8e6; bridge/transition_model.npz (stored threshold 0.926).

Test	Learned detector	Burst-score threshold
LORO, 3 recordings (122 changes)	75–76% at 2.5 false/min	63% at 1.3 false/min
Unseen recording 5 (arm still, 32 changes)	26/32 (81%)	20/32
Unseen recording 6 (forearm median 19°/s, 20 changes)	14/20 with wrist veto (15 without)	9/20
LORO, all 6 (190 changes), no veto	143/190 (75%) at 2.2 false/min, ~57 ms before onset	120/190 (63%) at 1.8 false/min, ~80 ms before onset
LORO, all 6, with wrist veto	74%	60–63%

Recording 4 was made with the arm moving strongly: the forearm turned at a median of 100°/s and was above 60°/s for 62% of the time. Nine of its 16 transitions began mid-turn, and the veto blocked them: 5 of 16 were caught with the veto against 9 of 16 without. Feeding the forearm turn rate to the detector as an extra input did worse (65% over four recordings, 71% over six). The veto stays ON but can be switched off.

A learned EMG detector flags about three in four posture changes on recordings it never saw (143 of 190) at about 2 false flags per minute, about 57 ms before the fingers start to move.

Flags in the Studio and the out-of-view latency budget

The flag-aware gate (Studio switch “⚡ transition flags”, OFF by default, commit 240ff7c) lets a clear leader commit after 50 ms within 0.6 s of a flag. With no flag in the last 0.6 s, a switch needs 0.25 s and gets no fast path. In a replay of the 190 changes, the new posture appeared 197 instead of 252 ms after onset, wrong switches fell from 21.4 to 18.9 per minute, and agreement with the camera was unchanged (83.8% vs 83.9%). The fast part alone reached 163 ms, but at 28.2 wrong switches per minute. The replayed recogniser matches the live gate on only 76% of ticks, so these replay numbers support relative comparisons only.

To see where the remaining delay came from, I replayed the six recordings through the real Studio tick with the camera hidden and compared the drawn hand with the camera's own hand. The drawn hand lagged the camera hand by a median 230 ms (380 ms in the arm-turning session). After movement onset, the camera hand was 50%/90% of the way to the new posture at 79/116 ms, the gate committed at 298 ms, and the drawn hand reached 90% at 328 ms. In a separate replay analysis, the recogniser's top guess settled on the new posture at 125 ms, while the camera named it at 103 ms. The EMG itself is therefore only about 20 ms slower than the camera. The bottleneck is the gate's stability hold. It exists because 27% of in-movement ticks

show a third posture: with no hold, the delay would drop to 79 ms, but wrong switches would rise to 49 per minute instead of 21.

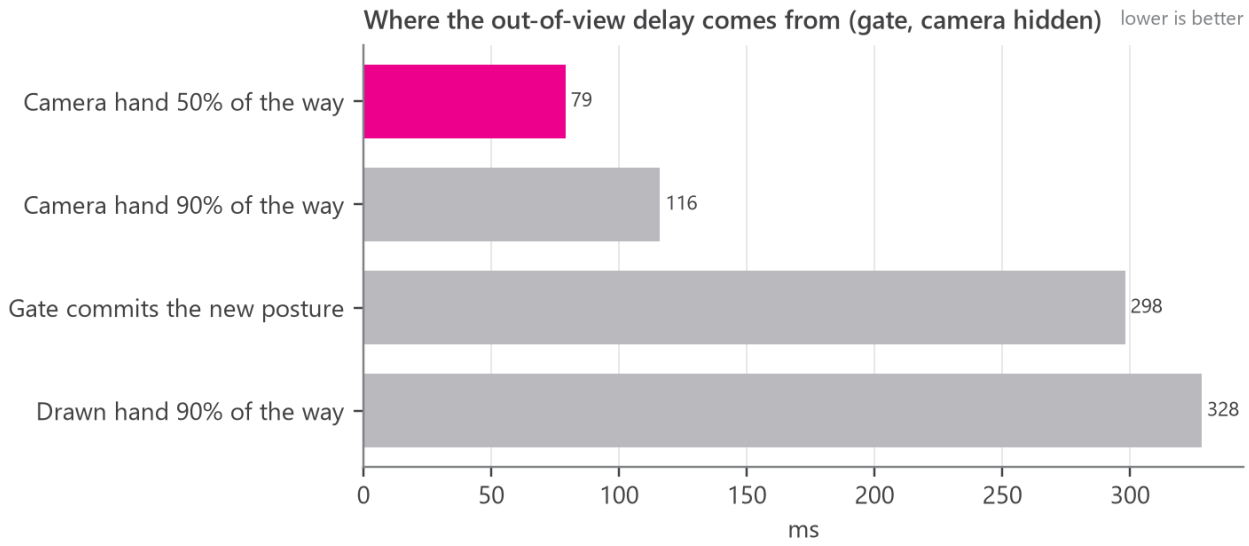


Figure 8.7 *190 camera-seen changes from 6 recordings through the real Studio tick with the camera hidden. The recogniser's top guess settles at about 125 ms (replay analysis), so the gate's stability hold, not the EMG, is the bottleneck. Source: poslab/burst/studio_latency_an.py (re-run), project notes.*

My supervisor also suggested a latch: hold the posture until a flag releases it. A hard latch never showed 34 of the 190 changes (66% agreement). A soft latch reached 10.1 wrong switches per minute, but with 287 ms delay and 79% agreement. A latch only pays off once the release also knows where the hand is going.

The soft hand and anticipation

The "◍ soft hand" switch (OFF by default, commit 82c8ba6) replaces the gate's hard snap with a sticky belief over postures. The belief leaves the current posture at a rate of 0.1 per second, rising to 8 per second for 0.6 s after a flag. Its evidence is the recogniser's probabilities raised to the power 0.35, and the named posture changes when another posture passes 0.9. The hidden hand is drawn as the belief-weighted mix of the posture shapes, so it starts to move before a decision is final.

Offline the soft hand promised a 15-ms lag, but the real Studio tick gave 85 ms. A recogniser taught up front is very confident on held postures, so the sticky belief waits for more evidence. Two beliefs, a quick one for drawing and a sticky one for naming (commit 57310c4), cut the lag to 60 ms and the finger error from 0.135 to 0.102. Live, however, I then saw flicker when turning the hand. The quick belief had bypassed the turning rule, and flicker (clear posture switches outside real posture changes) reached 26.6 per minute overall and 64.6 while turning, against 19.8 and 20.1 for the gate. The fix (commit e901006) returned to one sticky belief, damped while the wrist turns faster than 60°/s. Flicker fell to 14.8 per minute and to 4.8 while turning, at a lag of 100 ms, with 13.5 named wrong switches per minute and 87.6% agreement.

Anticipation (commit 777cf49) won the speed back. For 0.6 s after a flag, provided the forearm is still, the belief is pulled toward the prior's row for the posture where the flag fired, and smoothing drops to 10 ms. The lag fell to **45 ms**, the drawn hand reached 50%/90% of the new shape at 151/186 ms (camera 79/116 ms), and the finger error around changes went from 0.114 to 0.106. Flicker rose slightly to 16.5 per minute overall and stayed at 4.8 while turning.

Table 8.4 Real Studio tick with the camera hidden, 6 recordings, 190 changes (medians; 183–189 changes measured per hand). Times are measured from movement onset; lag is the cross-correlation of the mean finger curl. The one-belief row comes from its commit only (82c8ba6). Sources: commits 82c8ba6, 57310c4, e901006, 777cf49; re-runs of studio_latency_an.py and studio_flicker.py.

Out-of-view hand	Lag behind camera hand	Drawn hand 50% / 90%	Flicker per min (all / turning)
Camera hand (reference)	—	79 / 116 ms	—
Gate + snap (default)	230 ms	311 / 328 ms	19.8 / 20.1
Soft hand, one belief	85 ms	— / 215 ms	not measured
Soft hand, two beliefs	60 ms	134 / 183 ms	26.6 / 64.6
Soft hand, damped while turning	100 ms	189 / 234 ms	14.8 / 4.8
+ anticipation from flags (current)	45 ms	151 / 186 ms	16.5 / 4.8

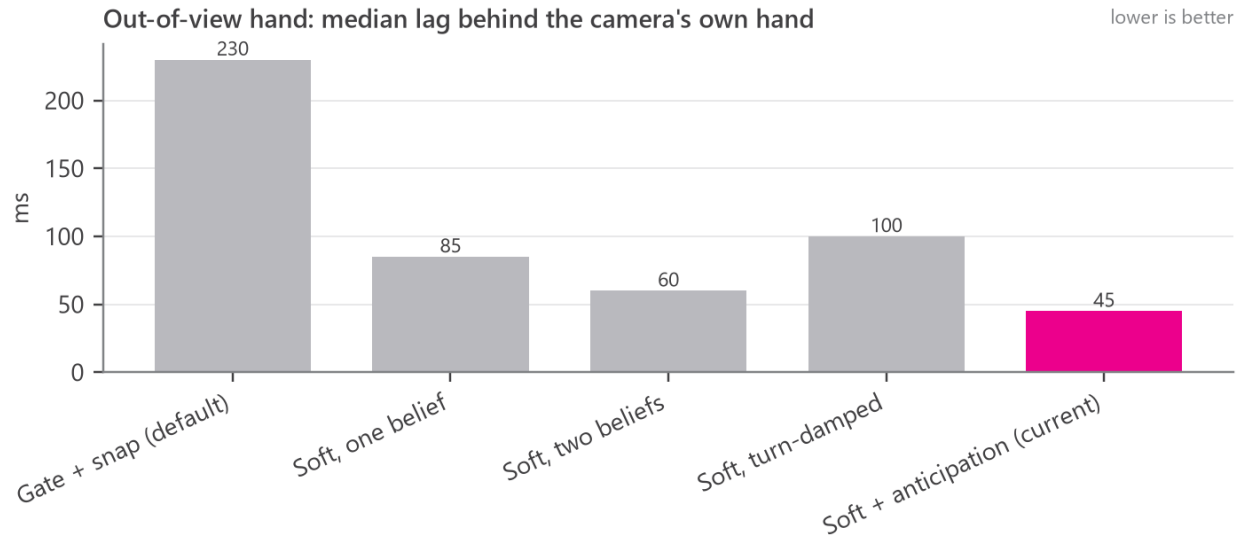


Figure 8.8 *The turn damping, needed against the flicker seen live, cost speed; anticipation from transition flags won it back. Real Studio tick, camera hidden, 6 recordings.*

With the optional soft hand and anticipation from transition flags (both off by default until tried live), the hidden hand followed the camera's own hand with a median lag of 45 ms instead of 230 ms, and flickered less while the forearm turned (4.8 instead of 20.1 switches per minute). Measured through the real Studio loop on 190 posture changes.

Finally, the anticipation can use my own habits instead of the general prior (commit 05e4a81, switch "↳ anticipate from MY transitions", ON under the soft hand). My 190 camera-seen changes are dominated by open→fist (30), pinch→open (27), point→victory (24), fist→open (23) and open→pinch (23). Point→victory and victory→point are much more common here than in the general prior. Scoring each recording with counts taken from the other recordings, the anticipated next posture was right 51% of the time, against 44% with the general prior, and flicker fell from 18.6 to 16.9 per minute. One honest negative: on MindRove data, burst-based prediction of the next posture did not yet beat the prior (6 vs 5 of 20 on recording 1; 48% vs 48% on the replayed recording 3).

8.5 Delivered to SciCortex

Release commit b42142f (2026-10-23, branch fusion-studio-update) is the second Fusion Studio submission. It was rebased onto SciCortex's main branch, followed by the Windows executable build (commit 2ceb348), and submitted to SciCortex as pull request #7 (from AMshoka:fusion-studio-update into SciCortex:main; open at the time of writing). It brings into the single-window Fusion Studio:

- holding the posture while the forearm turns;
- the expected next posture and the Fusion Logic drawings;
- optional transition flags (transition_events.py and the trained transition_model.npz);
- the optional soft hand with anticipation;
- REST as a sixth posture, with the 7-class model trained on 2026-10-23.

The release has no Transitions tab: the tracker runs inside the Studio. Default switches are: turn hold ON, expected next posture ON, ⚡ flags OFF, soft hand OFF, own-transition anticipation ON, HMM OFF. The posture and transition logic is covered by unit tests: 21 in tests/test_posture_hmm.py and 9 in tests/test_transition_events.py.

8.6 Limitations and open work

- **One user.** All data comes from me: posture clips from three sessions (Sep 24, Sep 25, Oct 23; the first two are about six hours apart) and six transition recordings from one evening (about 6.8 minutes), with only one strongly moving-arm session.
- **Open is the weakest posture** across sessions (open↔pinch confusion 11.4%) and has to be shown to the camera once per session. The hip is the weakest arm position.
- **Speed versus stability.** Every decision rule tried lies on the same trade-off. The real lever is better evidence, in particular separating open from pinch and fist from point.
- **Flag coverage.** The detector catches about 75% of changes at about 2 false flags per minute. Quiet changes such as pinch↔open are hard, and the veto blocks changes that start mid-turn.
- **Arm turning.** In the arm-turning recording the soft hand still lags about 240 ms (gate: 380 ms).
- **Replay is not live.** The replayed recogniser matches only 76% of live ticks, and recordings tick at 36–43 Hz while the live Studio ticks faster, so live behaviour may be quicker and twitchier.
- **Defaults.** The soft hand and the ⚡ flags remain OFF by default, pending live trials.
- **Licence.** The general prior is partly derived from emg2pose (CC-BY-NC-SA, research use only). The licence-free path is the wearer's own transition counts.
- **Not done.** The in-between-path model was not implemented. No Transitions session including rest has been recorded yet, so the wearer's own counts cannot learn fist→rest.
- **The other class is ill-defined.** The old other clips look like a relaxed hand; re-recording them would sharpen the line between rest and other. EMG also cannot predict the wrist bend.

9 Fusion Studio: the integrated system

The ✦ Fusion Studio tab puts the project's results together in one view and one working system. It shows the user's body placed by the headset, the tracked arm and the whole 26-joint hand. When the headset camera loses the hand, its position, orientation and finger posture keep coming from the CyberFinger IMUs, the forearm EMG and the learned position model. Measured by replaying recorded sessions through its real update loop, it runs at 62 fps, keeps the hidden hand's orientation within a median 7.8° of the camera over 20 s, against 25.5° when the last seen wrist bend is held, and in a fast session it cut position leaps over 20 cm caused by camera dropouts from 25 per minute to none. It was delivered to SciCortex as a single-window app, shipped in releases v2.0.0 and v2.1.0, can drive SteamVR through the CyberFinger driver, and an update with a Windows .exe is open as pull request #7.

9.1 ✦ Fusion Studio

The ✦ Fusion Studio tab is where the project's separate results come together. Each earlier tab answered one question: when the camera's view of the hand can be trusted (the occlusion gate), how the CyberFinger IMUs line up with the camera, how forearm EMG maps to finger curl, where the wrist is when the camera cannot see it, and which key posture the hand is holding. The Studio uses all of these answers at once to draw one picture. It shows the user's body, placed and turned by the headset, the tracked arm from shoulder to elbow to wrist, and the whole 26-joint hand at its estimated position, orientation and finger posture.

Its job is to keep the hand moving when the Meta Quest 3 camera loses it. Out of view, three sources take over. The wrist **position** comes from the Hybrid Position model (forearm kinematics plus a learned model of elbow motion). The hand **orientation** and wrist bend come from the CyberFinger IMUs (the wrist module and the knuckle sensor), through a mounting calibration that the camera teaches. The **finger posture** comes from the MindRove 8-channel forearm EMG, either as a continuous finger-curl estimate or by snapping to a recognised key posture. Every source has its own on/off switch, so each sensor's contribution can be shown on its own.

This fills a gap in the product. SciCortex's CyberFinger manual v2.1 describes the driver's out-of-view behaviour like this: *"with the joint IMU the hand keeps turning with your wrist, and its position waits where the headset last saw it"*. The driver's own IMU fusion turns the hand. It came from SciCortex's commits e3fcbc0 "First driver level IMU fusion" and 65b19b6 "Joint IMU heading assistance", and has its own mounting calibration. The Studio adds the out-of-view position and the finger posture. The v2.1 manual does not yet describe the Fusion Studio or the EMG armband, although release v2.1.0 ships them.

The tab was added on 2026-09-24 (commit ad1ae80) as an all-in-one showcase. The brief was the whole hand, position and posture together, attached to a body seen from the front and from above. The body moves with the headset, every signal is on screen, and optical, IMU, EMG, position and gate each have an on/off switch. The work was strictly additive: one file changed (629 lines added), and the existing tabs, models

and weights were checked to be byte-identical. It was the last tab when it was added. Key Postures, ◈ Fusion Logic and ⚡ Transitions have been added after it since then.

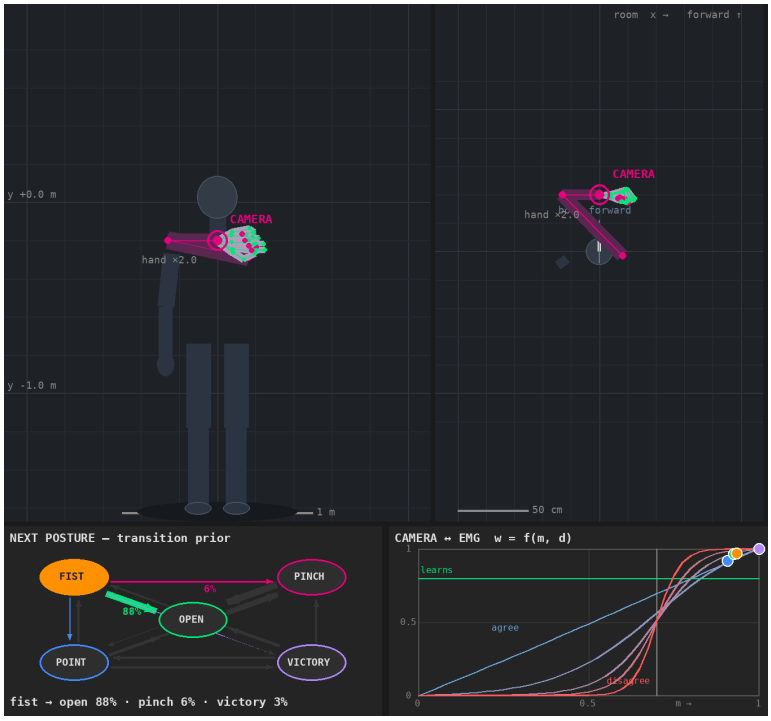


Figure 9.1 *The Fusion Studio while the camera sees a fist. Left: the FRONT mirror view, with the arm and the 26-joint hand magnified 2× about the wrist. Right: the TOP room map. Under the views are the posture transition map (from a fist the hand usually opens, 88 %) and the camera ↔ EMG weight curves w = f(m, d), with the learning line at w = 0.8 and each finger's live (m, w) point. Rendered on 2026-10-09 (before REST was added) from the Studio's canvas while replaying key-posture clips recorded on 2026-09-24. In this render the headset pose is a fixed stand-in and the hand is placed at a fixed offset, so here the body is not placed by a real headset.*

What the screen shows

- **Header.** The title, a live frame-rate counter (green at 50 fps or more), and the switch "⇢ send this hand to SteamVR" with its status line. Below them is a row of status chips: OPTICAL, WRIST IMU, KNUCKLE IMU, EMG, POSITION MODEL, GATE, HEADSET, LEARNING and KEY POSTURE. Each chip reads live, calibrated, off, missing or distrusted.
- **Left column.** The source switches, the view and fusion switches, the out-of-view and learning controls, a hand-size slider and ⌖ Set forward, which locks the body heading (see the table below). The column scrolls when the window is too short. It needs about 960 px and had been cut off at 1280×720 (commit eb08998).
- **Centre.** A FRONT stage, a mirror view drawn at real-world height, and a TOP stage, a room map with a 20 cm room-fixed grid and the camera's field-of-view wedge. Since 2026-10-09 a strip under them shows the Fusion Logic diagrams in small: the posture transition map and the camera ↔ EMG weight curves, redrawn about 20 times a second (commit a05aa16).
- **Right column.** Five signal panels. HAND has a close-up skeleton, five curl bars coloured by camera visibility, the key posture and the orientation source. IMUs has compass, tilt, quaternion and wrist bend for each sensor. EMG has 8 RMS bars and the armband's accelerometer. POSITION has the wrist in the body frame, depth from the eyes, the source and a 10-s depth trace of camera vs prediction. HEADSET / BODY has the head pose, head yaw relative to the body and the gate verdict.

Table 9.1 The Fusion Studio's switches and their defaults (repo bridge/cyberfinger_gui.py, FusionStudioTab)

Group	Controls	Default
Sources	Optical (camera), Wrist IMU, Knuckle IMU, EMG (armband), Position model, Occlusion gate	all on
View	body outline, camera field of view, hand trail (3 s), labels, body heading follows the headset	on
Fusion	smooth the prediction; snap to KEY POSTURES when hidden; hold the posture while the arm turns; expected next posture (prior); ↳ anticipate from MY transitions; camera ↔ EMG weight w = f(m, d); keep the hand anatomical; smooth camera ↔ prediction switches; distrust the camera when the IMU disagrees	on
Fusion, awaiting live trials	⚡ transition flags; ◍ soft hand + flag HMM; HMM posture smoothing (+33 ms)	off
Other	HAND OUT OF VIEW (knuckle live / wrist held); LEARNING with ↺ Reset and ⇊ Save refined; hand size 1–3×; ⌖ Set forward; ⇢ send this hand to SteamVR	knuckle; on; 2×; –; off

Body model. The neck sits at the headset position plus R·(0, −0.10, 0.08) m, where R is the head's rotation. The body heading is a slow average (time constant 2.5 s) of the head's horizontal forward direction, unless ⌖ Set forward locks it. The legs are drawn a fixed 0.85 m below the hips. This is because the OpenXR space used by Live Preview is head-local (height is about zero at the eyes), so it gives no floor to stand on. The elbow is placed 0.26 m back from the wrist along the forearm axis. The untracked arm hangs at the side.

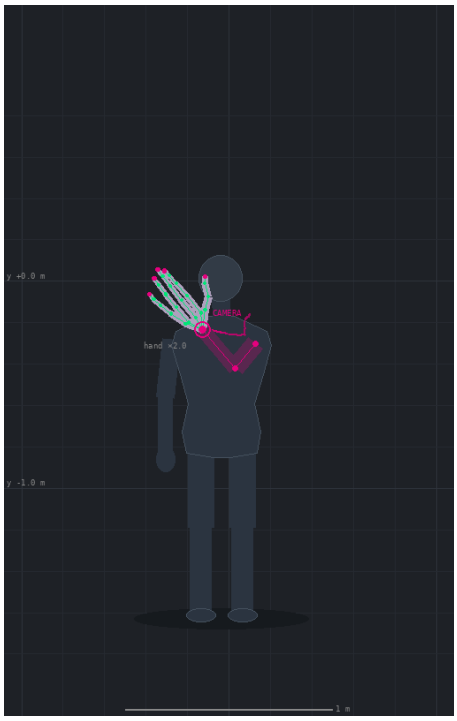


Figure 9.2 *FRONT view while the camera sees the hand (magenta = CAMERA). It shows the body silhouette with its ground shadow, the tracked arm, and the hand shell with the skeleton inside. Replay of the recorded session low_new3 (camera, headset, both IMUs, EMG) through the real Studio update.*

Figure 9.3 *TOP view 0.9 s into a simulated loss of the camera (the FRONT view of the same moment is shown in the overview). The yellow wedge is the camera's field of view, and the forearm and predicted hand (teal = PREDICTED) are drawn relative to the body heading; the trail shows the path since the last camera position. The loss is simulated on low_new3 by forcing the gate to out-of-view and freezing the camera frame, so the predicted hand still lies inside the wedge.*

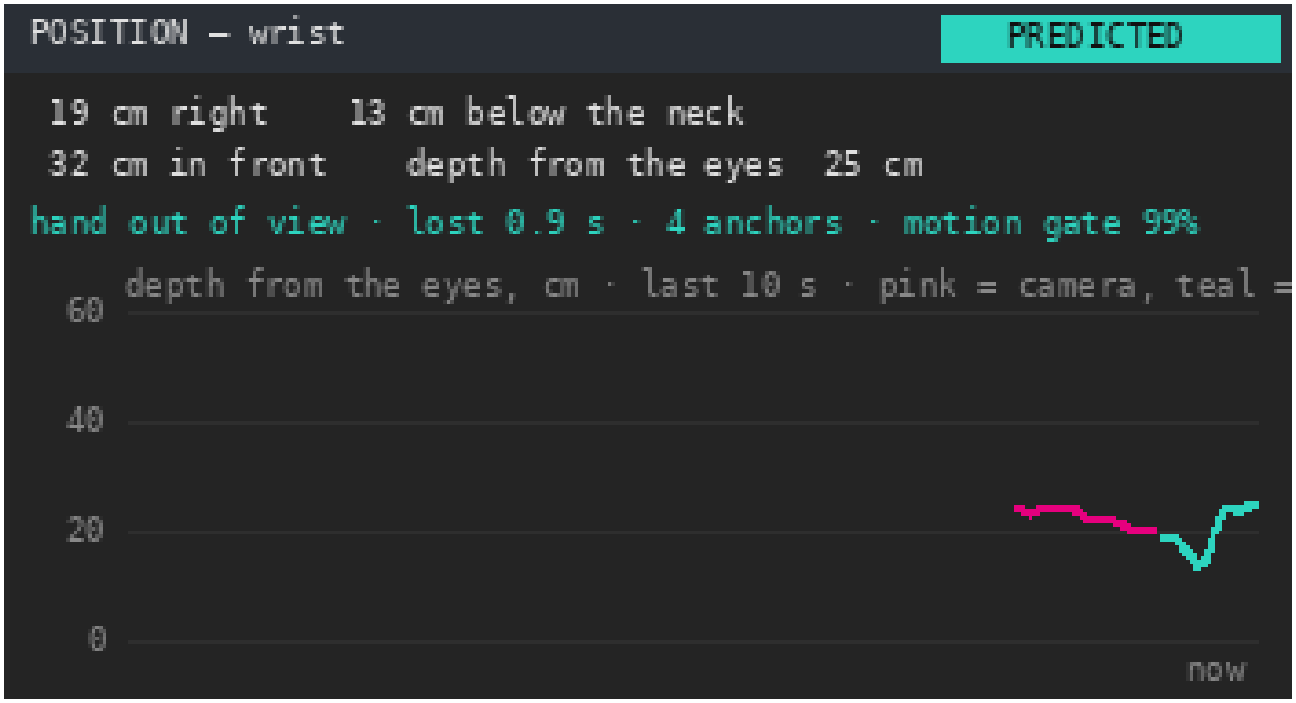


Figure 9.4 *POSITION panel during the same simulated loss: the wrist is 19 cm right, 13 cm below the neck and 32 cm in front, source PREDICTED ("lost 0.9 s · 4 anchors · motion gate 99 %"). The 10-s depth trace is pink for the camera and teal for the prediction.*

9.2 How one frame is computed

All of this runs in one update method, _st_tick, which is called on every frame of the GUI's poll loop. In order:

1. Read the camera hand and head pose, the wrist-module and knuckle IMUs, the EMG features and the occlusion gate's verdict.
2. Cross-check the camera against the knuckle IMU, and set aside a camera hand that disagrees with it.
3. Update the shared IMU fusers and the EMG estimates.
4. Position: the camera; otherwise the Studio's own Hybrid Position tracker; otherwise hold the last value.
5. While the camera sees the hand, teach the two IMU mounting calibrations (wrist module and knuckle).
6. Orientation: the camera; otherwise the knuckle IMU through its taught mounting when that is trusted (fit residual under 25°); otherwise the wrist mounting times the held wrist bend; otherwise the older fuser chain. An anatomical wrist limit acts as a safety net.
7. Fingers: camera fingers where they are seen, weighted finger by finger against the EMG with w = f(m, d). Hidden fingers, or a hidden hand, come from the key-posture shape (or the soft-hand belief), otherwise the EMG regressor, otherwise the held shape. The EMG regressor is refined from the camera only while the camera sees the hand, and only where w is above 0.8.
8. Make the hand anatomical, using the user's bone lengths and joint limits.
9. Smooth the drawn hand through source changes (the transition smoother).
10. Draw both stages every frame, and the signal panels in turns.
11. If switched on, send the drawn hand to the SteamVR driver.

Most steps reuse a module built and tested in an earlier tab. Several pieces were built for the Studio itself: the camera-taught IMU mounting (mount_calib.py), the transition smoother, the camera cross-check, the per-finger weight and the anatomical rebuild (finger_fusion.py), and the SteamVR link (steamvr_link.py).

9.3 Development and measured results

The Studio grew in steps between 24 September and 23 October 2026. Each step was measured by replaying recorded sessions (camera, headset, both IMUs and EMG) through the real update method. Usually the camera hand was hidden from the method, so that the hidden camera hand could serve as ground truth.

First version, continual learning and the mesh look

The first version (ad1ae80, 2026-09-24) was checked by replaying low_new3 through the real update. There were no exceptions, update plus drawing took about 21 ms per frame (median), every switch was turned off and on again, and the out-of-view stretch took the predicted path.

A LEARNING switch (on by default) then gave the Studio the same continual-learning rule as the Pose Hand tab, on the shared model, with a per-finger gate, a replay buffer and refinement. It runs only while the camera sees at least one finger. Tests confirmed it: 60 camera frames gave 60 adaptation steps, the switch off gave none, out of view gave none (status “paused”), and reset returned the count to zero.

The mesh look (commit 00085f0) replaced the stick figure with a filled, smoothed body silhouette with a ground shadow and a hand shell drawn around the skeleton. The palm is pushed 1.1 cm outwards, and the finger capsules taper from about 2 cm to 1.2 cm. Update plus drawing fell to about 15 ms. A true SMPL body mesh (a standard parametric human-body model) was not possible: there were no model files, and the Tk canvas is 2-D.

Hand orientation out of view: a camera-taught IMU mounting

Out of view, the hand's orientation and wrist bend come from the camera-taught two-sided IMU mounting (bridge/mount_calib.py; see the IMU section). In the Studio's real update loop with the camera hidden, it kept the hand within a median 7.8° of the camera over 20 s, against 25.5° with the wrist bend held (low_new3), and 4.1° against 7.4° over 8 s (face_depth). EMG cannot predict the wrist bend.

Out of view, the knuckle IMU through the camera-taught mounting kept the hand's orientation within a median 7.8° of the hidden camera hand over 20 s, against 25.5° when the wrist bend was held at its last camera value (real Studio update, recorded session low_new3).

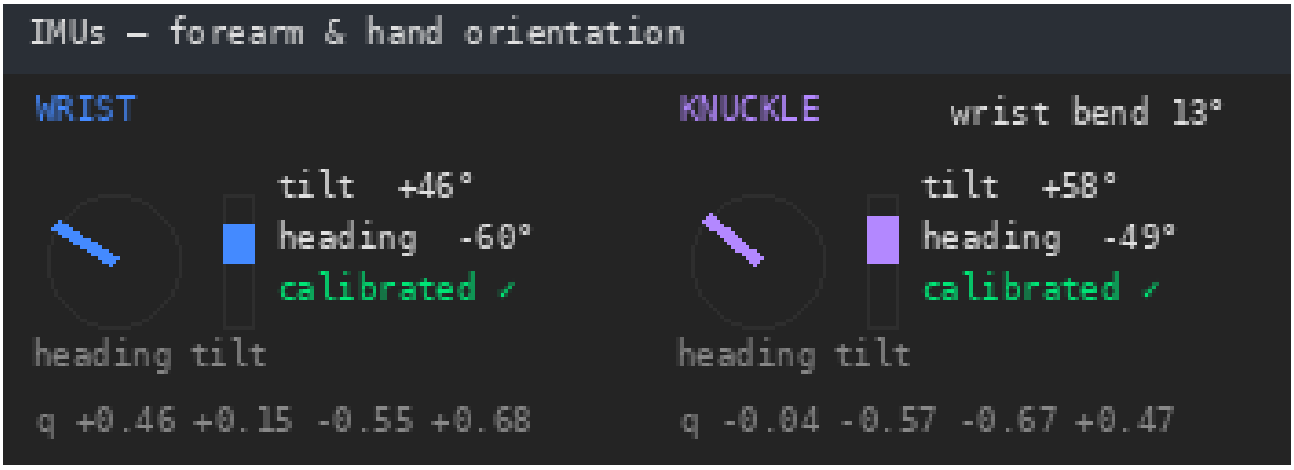


Figure 9.5 *IMUs panel: the wrist and knuckle CyberFinger sensors, with compass heading, tilt, quaternion, calibration state and the live wrist bend (13°).*

Smooth switching between camera and prediction

At first, every channel switched source in a single frame. The recordings showed 14–49 camera ↔ lost switches per minute, with 15–39 % of the losses shorter than 100 ms. Quest glitches went straight through: single-frame leaps of 45–62 cm or 70–100° while the gate still read CLEAR.

The fix (commits e12d9b2 and 35ab838, 2026-10-01) has three parts. First, a switch is drawn only after an 80 ms dwell. Second, it uses *inertialization*, a technique from game animation: when the source changes, the drawn hand keeps its offset from the new source and lets it decay along a critically damped curve, so the hand glides instead of jumping. The decay reaches 90 % in 0.35 s for position and 0.30 s for rotation and shape. Third, a glitch guard rejects leaps faster than a hand can move: over 8 cm at over 3 m/s, over 45° at over 900°/s, or a shape change over 2.5 cm at over 0.6 m/s. The first version froze the hand during the dwell. It stood still in 89 % of frames right after a loss and in 100 % right after a return. The second version coasts with the hand's momentum (friction 0.15 s), and no standing-still frames remain.

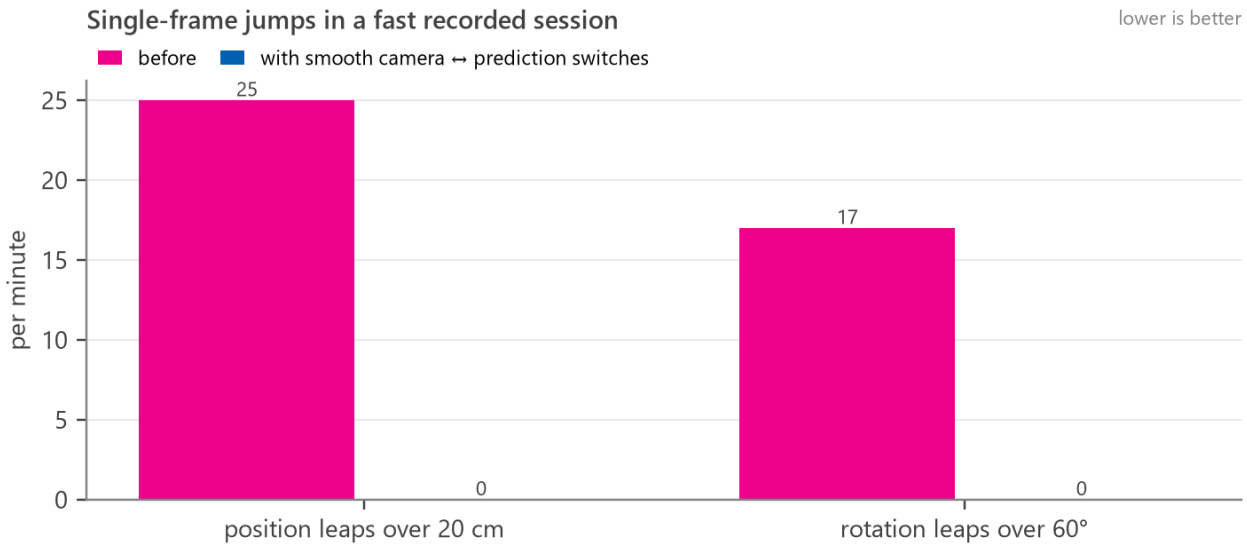


Figure 9.6 *Replay of the fast recorded session (commit e12d9b2). The jump when the camera returns also fell from 26 to 3 cm, and the finger-shape jump at a loss from 0.54 to 0.08 cm.*

The hidden finger-shape error fell from 1.8 cm to 0.2–0.35 cm, and the position error was unchanged. The cost is a glide lag of 2.9 cm after the camera returns (3.6 cm in the first version), and the smoother itself costs 0.07 ms per frame. Two problems remain that smoothing cannot fix. Right after a loss, the drawn hand moves at about 0.5–0.7× the real speed for about 0.4 s, because the position model under-predicts fast motion. Key-posture flips while the hand is hidden (2.3 per minute) come from the recogniser.

Sixty frames per second

Profiling (commit 2172b3e) found two costs. The first was the per-finger visibility ray test, 600 segment tests in a Python loop. Rewritten as vectorised array code, it gave identical results on 3,200 recorded frames and ran 27× faster (4.8 → 0.18 ms), which also helps every other tab that uses it. The second was redrawing about 400 canvas items every frame. Now the stages are drawn every frame and the signal panels in turns. In the real program loop with a recorded session streamed at 60 Hz, the Studio went from 35 to 62 fps, which is about the ceiling of the research GUI's poll loop. Later features kept it at 60–62 fps.

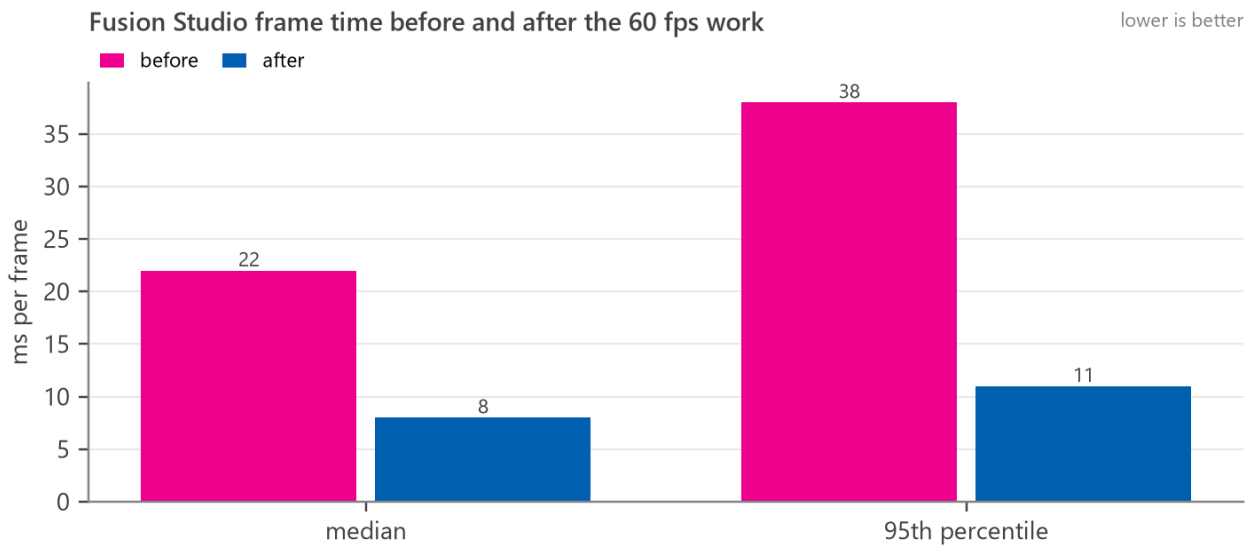


Figure 9.7 *Real program loop with a recorded session streamed at 60 Hz (commit 2172b3e). The frame rate rose from 35 to 62 fps.*

Setting the camera aside when it disagrees with the IMU

The Quest sometimes flips the hand, or swaps it with the other hand, while the gate still says CLEAR or SELF. On the recordings the camera turned the hand 60–146° in one frame while the knuckle IMU turned 1–3° (commit f06b11f, 2026-10-01). Two tests now set such a camera hand aside. The calibrated test compares the camera with the knuckle IMU through its mounting. Normally the two are about 6° apart and differ by over 45° in only 0.65 % of clear frames, so over 45° (30° once already distrusted) counts as disagreement. The relative test compares the camera's turn since the last trusted frame with the IMU's, with a limit of 40° (25° once distrusted). After 2.5 s of disagreement the camera is trusted again and the IMU's heading is re-fitted, so a restarted IMU cannot lock the camera out.

- Injected 150° flips: the drawn error was 4.7° instead of 150°.
- Session low_new3: the largest phantom turn fell from 105° to 60°, and phantom turns over 60° from 0.6 to 0 per minute.
- False distrust: 1.2–1.5 % of CLEAR frames, in episodes of 0.64 s or less.
- A simulated 70° jump in the IMU's heading: the mounting recovered to 5–8°, where before it stayed 30–60° off for 20 s or more.
- Limit: on a glitchy clip (low_141356), phantom turns over 45° fell only from 5.5 to 4.6 per minute. Lower thresholds caught more but added 8–13 false switches per minute and were rejected. The recordings hold at most 311 frames of one hand over the other, so this needs live testing.

Key postures in the Studio

Several switches expose results described in the Key Postures section. In the Studio's real update loop: a key-posture change is drawn at 50 % and 90 % of the new posture 67 and 100 ms after the switch (it was 150 and 317 ms); the turn hold cuts wrong switches while holding a posture from 9.0 to 6.5 per minute; the expected-next-posture rule, a prior proposed in supervision, removes the point and victory flashes when a fist opens (30 % → 0 % of fist → open changes); the per-finger weight w = f(m, d), also proposed in supervision,

raises frames with the right posture from 95.8 % to 96.8 %; and the optional soft hand with anticipation (off by default) cuts the out-of-view lag behind the camera's own hand from 230 to 45 ms.

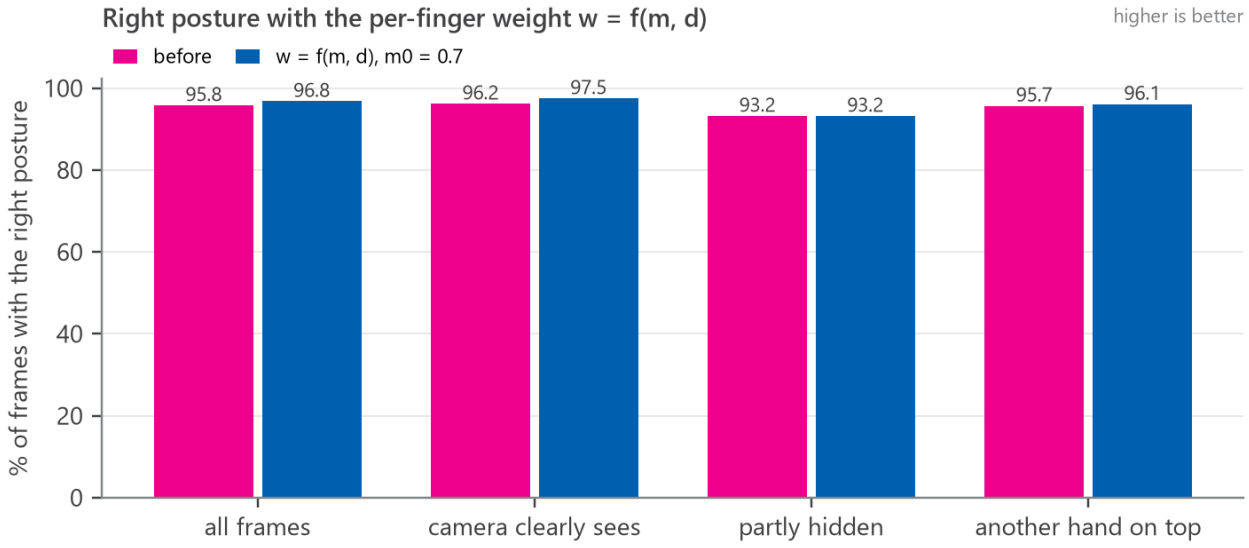


Figure 9.8 *Real Studio update on 40 held-posture clips (14,282 frames), with EMG models that never saw that arm position and another hand in the way for 1 s every 6 s. For each finger, w weighs the camera against the EMG from the camera's reliability for that finger (m) and how differently the two curl it (d); the choice flips at m0 = 0.7, which scored best (96.6 % overall at 0.5 and at 0.6). Hands that are no posture at all fell from 3.0 to 2.0 %.*

An anatomical hand

Mixing hands by averaging joint positions could stretch fingers or bend them backwards. On two replayed sessions, anomalous hands made up 0.4 % of the camera alone, 7.7 % of camera plus filled-in fingers, 12 % of the stored session shape, and 87 % of the EMG-anchored hand. The function finger_fusion.anatomize() (commit 5027e7c, 2026-10-09) now rebuilds every finger from its root with the user's own bone lengths and joint limits. Flexion is limited to −100…+30° at the knuckle (MCP), −120…+10° at the middle joint (PIP) and −100…+20° at the fingertip joint (DIP), and sideways bend to 35°, 15° and 15°. Under occlusion, the last camera shape is kept only if the camera named a posture with a confidence of 0.75 or more, and it fades into the posture's template shape with a time constant of 0.8 s.

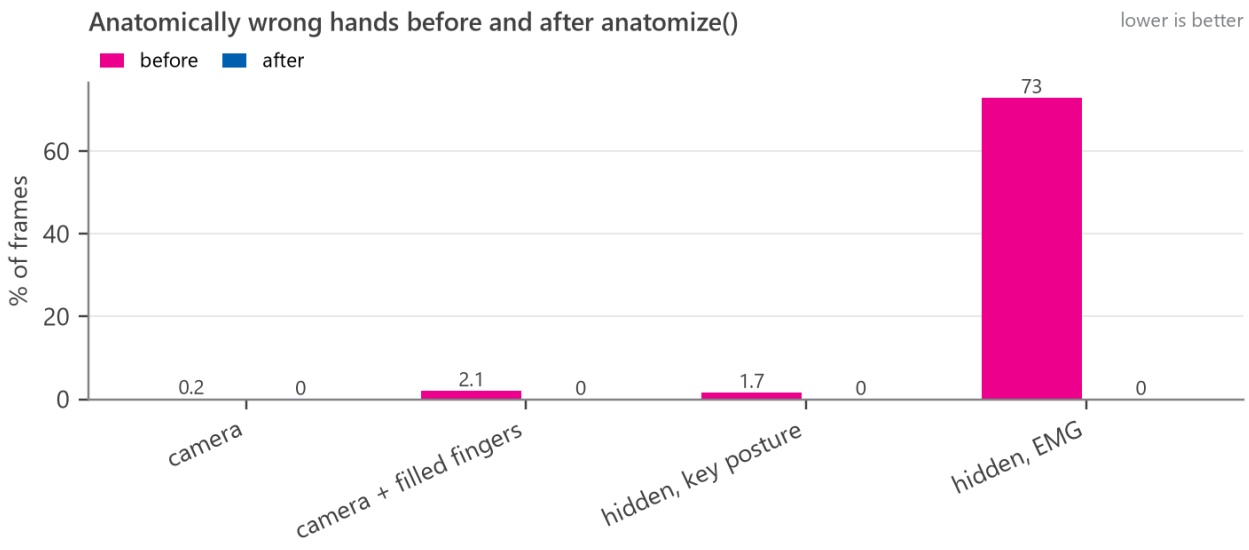


Figure 9.9 *A real session replayed through the real Studio update (commit 5027e7c).*

With the other hand on top, which leaves a stale camera frame (68 posture-change pairs, 11,220 frames), wrong hands while covered fell from 58–60 % to 0.1–0.7 %. The Studio still follows the EMG's new posture in 33 ms. Posture accuracy on the 40 clips moved from 96.8 to 96.6 %, and a clear camera hand moves by at most 2 mm.

9.4 Sending the fused hand to SteamVR

SciCortex's driver chooses a pose source for each hand: FUSED (a hand from the Fusion Studio) first, then PASSTHROUGH (the headset's own hand tracking), then NO_POSE. SciCortex had built the FUSED receiver into the driver (STEAMVR_PLAN.md), but nothing fed it. The driver's own mini-fusion, which turns the hand with the joint IMU and freezes its position out of view, was a fallback until the bridge-side approach was ready.

On 2026-10-01 (commit 374b22b) the Studio began sending its drawn hand to the CyberFinger driver as CFHS packets (about 1.15 kB each, UDP port 27015). The driver shows this hand instead of the headset hand. When packets stop, it returns to the headset hand within 150 ms (blending over 100 ms, per SciCortex's plan). The Studio's camera input becomes the headset hand that the driver taps (CFOP packets, UDP 27016), and a loop guard stops Live Preview and the skeleton reader from reading back the Studio's own output. The module bridge/steamvr_link.py ports the driver's SkeletonSynth (Valve's simulated hand) to Python. The port matches the compiled C++ to within 2e-8. It then retargets each finger bone of SteamVR's 31-bone hand skeleton onto the Studio's joints.

To check it end to end, a fake driver received the packets while a recording streamed through the real GUI. Every packet, rebuilt the way SteamVR renders it, landed on the drawn hand: wrist and palm exact, finger segments within 0.005°. It ran at about 56 packets per second, kept sending with the tab hidden, and sent nothing once switched off. Velocity prediction at a 33 ms look-ahead halved the position error (5.0 → 2.4 mm) and cut the rotation error by 40 % (0.80 → 0.48°).

The first live use was in the social-VR application Resonite on 2026-10-02. There the fused hand looked faster and less natural than in the Studio. The cause was that the link skipped the One Euro filter, an adaptive

smoothing filter that smooths heavily when the hand is slow and lightly when it is fast. The driver applies this filter to the headset hand in PASSTHROUGH, so without it SteamVR extrapolated noisy, already-predicted poses. Through a simulated 90 Hz display, the palm jittered 10.2° RMS. The fix (commit e3be82e) ports the driver's PoseFilter, which matches the compiled C++ to machine precision, glitch gate included. It uses slightly lighter settings (minimum cutoff 1.5 Hz and beta 25, against the driver's 1 Hz and 15). A sender thread streams 120 packets per second, interpolating the two latest Studio frames. As in PASSTHROUGH, the fingers are not filtered. The sent finger shapes match the drawn hand (0.03° median), and the Studio stays at 61–62 fps. In that live session the driver's log showed no FUSED dropouts.

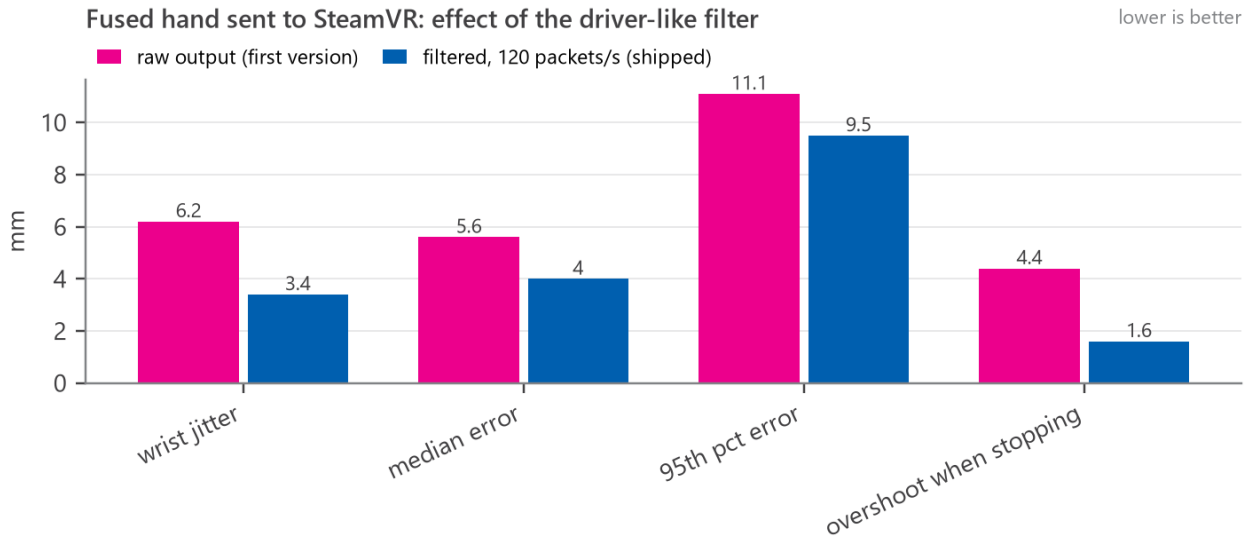


Figure 9.10 *Simulated 90 Hz SteamVR display; errors are against the real hand (commit e3be82e). The delay rose from 15 to 30 ms. The driver's own filter settings would give 1.06° of palm jitter, but with 55 ms of delay.*

Filtered like the driver and streamed at 120 packets per second, the fused hand's palm jitter in a simulated 90 Hz SteamVR display fell from 10.2° to 1.45° RMS, for 30 ms of delay.

The link has its own unit tests: 6 in tests/test_steamvr_link.py, including golden bones from the compiled driver code, and 9 in tests/test_finger_fusion.py. Only the Studio's hand (the right, EMG-instrumented hand) is fused into SteamVR. The other hand stays on the headset's tracking.

9.5 Delivery to SciCortex

The single-window app and SciCortex's releases

The product version of the Studio is not a copy of the research code. It is generated from it. The script bridge/make_fusion_studio.py (commit 02c678a) builds bridge/fusion_studio.py by collecting the Studio's methods and everything they call from the research GUI (an AST closure) and adding a new top bar. The bar has ▶ Start CyberFinger (BLE), ▶ Start EMG (device or simulator), ◉ Start preview (OpenXR) and ▲ Console. The first delivery (commit 490a312, 2026-09-25) had 23 files and 13,496 added lines, with a 4,945-line fusion_studio.py. A smoke test through the real poll loop pressed every button, streamed the EMG simulator and taught the key-posture model on 167 replayed frames, with no exceptions.

SciCortex merged it the same day as pull request #1 (merge 95c7936). The Studio is in SciCortex's tag v2.0.0 (e3fcbc0, 2026-09-27, "First driver level IMU fusion") and in release v2.1.0 (tag 93359ca, 2026-09-28). Both include fusion_studio.py, key_postures.py, key_postures_model2.npz and mount_calib.py. By v2.1.0 SciCortex had edited 10 of the delivered files (+157 / −107 lines). They moved BLE parsing into shared modules, added haptics, the pink button and the driver link, and named the device "CyberFinger" throughout. On 2026-09-29, v2.1.0 was merged back into the research branch unit by unit (merge 266731d): 55 SciCortex-only units applied, 442 research-only units kept, and 9 units changed on both sides merged by hand.

Second submission and the Windows app

The second submission brought the released Studio up to date with the research branch. It could not overwrite SciCortex's hand-maintained copy, so a three-way merge was done function by function. The base was release 490a312, which the generator reproduces byte-exact. "Ours" was the generator run on the current research GUI, and "theirs" was SciCortex's file. Company-only units kept SciCortex's version, and research units took the new one. The generator was fixed to emit every class the Studio uses (commit cf71b91).

The update went out as commit b42142f (14 files, +3,316 / −103). It brings the smooth camera ↔ prediction switches, the per-finger weight w = f(m, d), the anatomical hand, the camera ↔ IMU cross-check, the live wrist bend out of view, 60 fps, the SteamVR output, the turn hold and expected-next-posture rules, the optional transition flags and soft hand, and REST, with tests for the new modules. SciCortex's CyberFinger, driver-link, haptics and pink-button code was left unchanged. The commit was rebased onto SciCortex's main branch at d469b53, which already had SciCortex's fix for the latest SteamVR. The product's fusion_studio.py grew to 5,789 lines. All 10 test files passed. A second smoke test through the real poll loop covered the EMG simulator, the transition detector, Start CyberFinger, the SteamVR switch, a hidden-camera replay including rest, the soft hand with flags, and quitting, with no exceptions.

The Windows app followed as commit 2ceb348 (8 files, +353 / −5), so that the Studio runs without a Python installation. A PyInstaller recipe (fusion_studio.spec and build_fusion_studio.bat) builds CyberFingerFusionStudio.exe as a folder, runs the exe's self-test and zips the folder. A one-file build was rejected because it would unpack about 200 MB on every start and lose the refined posture model that the Studio saves next to itself. Docker was rejected because a container cannot reach Bluetooth LE, the OpenXR runtime and GPU, or the driver's localhost UDP ports. The same exe also runs as the headset's OpenXR helper process (--openxr-provider), since a packaged app has no separate Python to start the helper with, and --selftest checks every component and model without hardware. requirements-fusion-studio.txt lists what the Studio needs, and a lock file pins the tested versions. pyopenxr is held at 1.1.5301, because the API-layer DLL of 1.1.6302 does not load. The built folder holds 1,382 files (224 MB) and the zip is 77,997,806 bytes (78 MB). The pull request carries the build recipe, not the exe itself.

The exe's self-test on 2026-10-23 at 12:16:58 reported "RESULT: healthy", with all 20 of its checks passed. It covered the seven-class key-posture model (fist, open, other, pinch, point, rest, victory), the EMG simulator streaming 1,410 samples and the learned transition detector. The OpenXR helper answered after 8.0 s with "OpenXR runtime not available": no headset session existed at build time, so the exe's live headset path is still untested. Both commits were pushed to the branch fusion-studio-update of my fork and opened as pull request #7 into SciCortex/CyberFinger_SteamVR. On 2026-10-23 it was open, not a draft, with 20 files changed (+3,669 / −108) and no conflicts.

Table 9.2 Fusion Studio deliveries to SciCortex

Date	Delivery	Size	Status
2026-09-25	Single-window Fusion Studio (490a312)	23 files, +13,496 lines	merged as PR #1 (95c7936) the same day
2026-09-27/28	SciCortex tags v2.0.0 (e3fcbc0) and v2.1.0 (93359ca)	Studio, key-posture model, mount calibration	released
2026-10-23	Update (b42142f) and Windows app (2ceb348)	PR #7: 20 files, +3,669 / −108	open, no conflicts

9.6 Status and limitations

The ✦ Fusion Studio is working and released. The tab runs in the research GUI, and the single-window app has been in SciCortex's product since tag v2.0.0. The update open as pull request #7 adds the SteamVR output, smooth switching, w = f(m, d), the anatomical hand, transition flags, the soft hand, REST and the Windows .exe. The transition flags, the soft hand and the HMM smoothing ship switched off until they have been tried live. The known limits are:

- Only the right (EMG) hand is fused into SteamVR. The other hand stays on the headset's tracking, and the Studio's left-hand path has not been tested live.
- The packaged exe's live headset path is untested (no OpenXR session at build time).
- Live Preview's OpenXR space is head-local, with no floor, so the legs are drawn at a fixed length.
- Right after a loss, the drawn hand moves at about 0.5–0.7× the real speed for about 0.4 s, because the position model under-predicts fast motion.
- The camera ↔ IMU cross-check had little real hand-over-hand data (at most 311 frames). On a glitchy clip it reduced phantom turns only from 5.5 to 4.6 per minute, and soft weighting in the 20–45° zone was not tried.
- Key-posture flips while hidden (2.3 per minute) come from the recogniser. Smoothing does not remove them.
- The flag numbers come from an offline gate replay whose recogniser matched the live one on 76 % of ticks. The soft-hand numbers used a recogniser pre-taught on each session.
- The camera cannot label a closed fist, and recordings of real transitions with camera ground truth only exist since 2026-10-22.
- EMG cannot predict the wrist bend, and the body is a drawn silhouette, not an SMPL mesh.
- The figures are re-drawn from the Studio's canvas during replays by an image renderer with square line caps, so they look blockier than the live window. The loss shown in the out-of-view figures is simulated.

10 Contributions, deliverables and next steps

Internship Unit 1 ran from 30 June to 29 October 2026 and addressed all four of its objectives in the Mitacs proposal. Ten of the twelve sub-tasks were completed, several of them with a simpler or better-supported method than planned. Two were partial: there was no participant study, and the buffer window-size study was not done. The work reached SciCortex's product in four ways. The single-window Fusion Studio was merged as PR #1 and shipped in release v2.1.0. A new link feeds the fused hand into the CyberFinger SteamVR driver. A driver fix restored finger tracking after the 21 October SteamVR update. A Studio update with a Windows app was submitted as PR #7. The section then covers the limitations found, an IU2 plan built on the IU1 baselines, and where the code and data are kept. It closes with a conclusion, acknowledgements, a glossary and the references.

10.1 Overview of what was delivered

The four months of IU1 produced three kinds of output. The first is a research instrument: the bridge GUI, bridge/cyberfinger_gui.py (15,483 lines, 27 tabs). It records the Meta Quest 3 hand tracking, the CyberFinger IMUs and the MindRove forearm EMG on a single clock, and every model and test runs inside it. The second is a set of contributions that reached SciCortex's product: the Fusion Studio application, the link that feeds its hand into the CyberFinger SteamVR driver, a driver fix, and a submitted update with a Windows app. The third is a body of measured results, positive and negative. Most of them are backed by the script and log that produced them, which are kept in an experiment lab outside the repository (~/poslab). The exceptions are listed under Data and code availability. The system overview in the architecture section shows how the sensors, the Studio and the driver connect.

The Fusion Studio became part of SciCortex's product. SciCortex merged it as PR #1 on 25 September 2026 and shipped it in release v2.1.0 on 28 September. SciCortex's own SteamVR plan names it as the source of each hand's 6-DoF pose and of its 31-bone skeleton.

10.2 IU1 objectives: planned versus delivered

The plan comes from §2.4–2.5 of the Mitacs proposal. Sub-objectives 1–4 belong to the first intern. In the proposal's timeline table, a page break puts an "Internship 2" heading above task 3.2, but tasks 3 and 4 are marked for intern 1. The proposal also uses the number 4.1 twice, so the demo task is called 4.2 here.

Table 10.1 IU1 sub-tasks from proposal §2.4–2.5 compared with what was delivered (sources: project notes, commits, the fusion/ modules and their replays: online_calib self-test, three calib_session recordings of 30 August, t22 rerun).

Task	Planned	Delivered	Status
1.1 Synchronised data	Timestamped buffers on the host PC	One file per clip: optical joints, Gate verdict, IMU and EMG on one wall_time (175 columns); IMU↔camera lag ≤ 10 ms	Done
1.2 Per-finger confidence	OpenXR per-joint flags	Flags shown to be unusable; ray-cast per-joint visibility plus the Gate (5 verdicts)	Done; method changed, T5.2 pulled forward
1.3 Benchmarks	Performance baselines	Position lab with a look-away protocol (freeze 36.95, FK 26.53, learned 22.62 cm); held-out posture reports; latency replays	Done
2.1 sEMG encoder	TCN from NinaPro/EMaGer weights	Ridge 1.30 cm; raw-EMG TCN 1.44 vs 1.80 cm; per-finger curl r = 0.88–0.92; emg2pose tested	Done; pretraining not adopted
2.2 IMU–optical fusion	Kalman filter	Complementary filter on the sensor-to-hand offset (median 10.6–12.6°); camera-taught mounting 8.6–10.3° over 1–30 s	Done; simpler filter
2.3 Metrics	Evaluation protocol	Every tab measured: held-out clips, leave-one-position-out, look-away, real-tick replays	Done
3.1 Rolling buffer	Observation buffer	~800 pairs, confidence ≥ 0.6, density-based eviction, camera pseudo-labels	Done
3.2 Cross-attention calibration	Affine correction of EMG latents	Top-k attention lookup (k = 96), no runtime weight updates; online NLMS learning; camera self-teaching	Done; non-parametric
3.3 Convergence, cross-session	Window size; cross-session tests	Live Calib curl error: synthetic drift 0.203 → 0.063; three recorded sessions 0.455–0.547 → 0.056–0.124 within a session; Key Postures hidden frames next day 50 → 77% (camera self-teaching)	Partial; window study moves to T7.3
4.1 Gated fusion	Confidence-gated switching	Fusion Studio: per-finger w = f(m, d), anatomical hand, out-of-view position, FUSED output	Done, beyond plan
4.2 Demo and participants	Demo; evaluation with participants	Studio, live Resonite session, Windows app; single-user evaluation	Partial; no participant study
4.3 Code, docs, report	Codebase, documentation, final report	Private backup repository; three documents; PR #1 and PR #7; this report	Done

Where the method changed, and why

- **1.2 Per-finger confidence.** The OpenXR per-joint flags turned out to be useless as a per-finger signal. On the Quest they flip for the whole hand at once, and through SteamVR every joint always reads valid and tracked. I replaced them with a geometric per-joint ray-cast visibility test (fusion/occlusion.py) and a Gate with five verdicts: CLEAR plus four occlusion types (OUT_OF_VIEW, SELF, OTHER, OBJECT). On real clips the Gate labelled open hands CLEAR 100% of the time, fists SELF 96% and wrist bends SELF 63%. This pulled the IU2 ray-casting task (T5.2) forward.

- **2.1 Pretrained encoder.** The NinaPro and EMaGer weights named in the proposal were never tested. Meta's pretrained emg2pose model was used instead. I first reproduced its published accuracy (angle error 11.0° on its mini dataset). Its frozen features transferred to this hardware, cutting curl error by 13.9–18.4% compared with random features in two runs. However, warm-starting from it tied training from scratch (curl error 0.0926 vs 0.0915), so pretraining was not adopted.
- **2.2 Kalman filter.** The CyberFinger already fuses its gyroscope and accelerometer on the device with a VQF filter, so on the host only the slow sensor-to-hand offset drifts. A complementary filter on that offset was enough. On two open-hand clips the offset fit left a median residual of 10.6–12.6°. Once the offset was frozen, the error stayed between 13 and 19° from 0.5 to 5 s while the hand was held rigid.
- **3.2 Calibration.** The proposal planned a learned affine map of EMG features. The delivered method is an attention lookup instead: the current EMG feature is the query, the buffered features are the keys and the camera's finger curls are the values. It never changes model weights at runtime.
- **4.2 Participants.** The proposal (§3.2) declares no human participants. All evaluations used one user, the intern.

Ten of the twelve IU1 sub-tasks were completed and two were partial (the participant study and the window-size study). The work also went beyond the plan: it estimates the hand's position out of view, at 14.58 cm mean error against 36.95 cm for freezing the hand where it was last seen.

Results beyond the IU1 plan

Several capabilities were not in the IU1 plan but came out of the work. Out-of-view hand position comes from Hybrid Position, which combines the forearm direction measured by the IMU (forward kinematics) with networks trained for elbow motion. A live wrist bend out of view comes from the camera-taught IMU mounting calibration. EMG key postures, including a relaxed REST posture, are recognised and keep learning from the camera. EMG transition flags fire before the fingers move. The fused hand is sent to SteamVR. The transition prior, the per-finger weight w = f(m, d), REST and the transition flags followed whiteboard discussions with my supervisor in October 2026. The headline numbers for these results, each measured against its own baseline, are collected in the overview at the start of this report.

Figure 10.1 *The October fusion logic as drawn in the Fusion Studio, here while the camera sees an open hand. The same strip with a fist is in the Fusion Studio section. Rendered on 9 October 2026 in the research GUI from a replayed key-posture clip of 24 September, before REST was added. Top: the FRONT and TOP views, with the hand drawn at twice its size. Bottom left: the posture transition prior, with open as the hub (after open: pinch 81%, fist 16%, victory 2%). With REST added, the hand rests 72% of the time after open. Bottom right: the per-finger camera↔EMG weight w = f(m, d), where m is the camera's reliability for that finger and d is how far the camera and the EMG disagree. Above w = 0.8, the camera teaches the EMG model.*

10.3 Contributions to SciCortex's product

I never pushed directly to the company repository. All research code went to SciCortex as pull requests from my fork (AMshoka/CyberFinger_SteamVR), and SciCortex decided what to merge. The table below lists the contributions. The Fusion Studio release and the manual review are described here. The other contributions get a few sentences each, and their technical details are in the architecture and Fusion Studio sections.

Table 10.2 Contributions to SciCortex's product (sources: git_log_full.txt, git_log_release_branch.txt, project notes).

Contribution	Reference	Date (2026)	Status
Fusion Studio single-window app	490a312 → PR #1 (95c7936)	25 Sep	Merged; in main from 27 Sep; shipped in v2.1.0 (28 Sep)
FUSED link to the SteamVR driver	374b22b, e3be82e	1–2 Oct	Research branch; submitted in PR #7
Driver fix for IVRDriverInput_005	c35efc5	22 Oct	SciCortex committed the same fix (d469b53, 23 Oct)
Studio update and Windows app	PR #7 (b42142f, 2ceb348)	23 Oct	Open as of 23 Oct
v2.1.0 merged into the research branch	266731d	29 Sep	Done
Review of the CyberFinger manual v2.1	Project notes	29 Sep	7 issues stand; not yet sent

Fusion Studio release: PR #1 and v2.1.0

Purpose. SciCortex needed an application that fuses the headset camera, the CyberFinger IMUs and the forearm EMG, but without the 27-tab research bridge. **How.** bridge/make_fusion_studio.py reads the structure of the research GUI's code (an AST-based extractor). It copies the ✦ Fusion Studio tab, plus only the methods that tab actually calls, into a single window: bridge/fusion_studio.py, 4,945 lines. A top bar starts the CyberFinger, the EMG armband (or its simulator) and the headset preview. The release was one commit, 490a312, on branch fusion-studio of my fork. It has 23 files and +13,496 lines, of which about 5,900 are existing modules copied in. It includes the models, the fusion modules and the FUSION_STUDIO.md guide.

Result. On 25 September SciCortex merged it as PR #1 into its fusion-studio branch (95c7936). It reached main on 27 September through f69220c, "Merge pull request #2 from SciCortex/steamvr-driver-v2". SciCortex then built on it with 5f1a46c "Working nice for VirtualDesktop and SteamLink", 65b19b6 "Joint IMU heading assistance", bddc301 "UI fixes, docs, controller-cf swap fixes" and 82f59ce "Clarifying glove->cyberfinger". In parallel, e3fcbc0 "First driver level IMU fusion" added a simple fusion inside the driver itself: the hand turns with the joint IMU, but its position stays frozen out of view. My supervisor described it as a fallback "while we wait for your approach on the bridge". Release v2.1.0 (tag on 93359ca, 28 September 2026) ships bridge/fusion_studio.py, hybrid_position.py with its weights, key_postures.py with its model, mount_calib.py and fusion/extrinsic.py, gate.py, occlusion.py and orient_fusion.py. SciCortex now maintains its own copy of the Studio (4,995 lines in v2.1.0, a difference of +139/−89 lines from my release).

SciCortex's driver plan (STEAMVR_PLAN.md, §1 Goal) names the Studio as the source of each hand's 6-DoF pose: "camera while the hand is in view, CyberFinger IMUs + position model when it is not". It also names the Studio as the source of the 31-bone skeleton: "camera fingers where seen, EMG regressor or recognised key posture elsewhere". The CyberFinger manual v2.1 describes the product's own behaviour out of view: the hand "keeps turning with your wrist, and its position waits where the headset last saw it". That frozen position is the gap the Studio fills, as the out-of-view render in the overview shows.

FUSED link to the SteamVR driver (steamvr_link.py)

The link that feeds the Studio's hand to the driver's FUSED input (374b22b on 1 October, e3be82e on 2 October) closed the gap that SciCortex's own plan lists as G2: "Nothing carries the fused pose or skeleton to the driver". The link streams 120 packets per second. On a simulated 90 Hz display it cut palm jitter from 10.2° to 1.45° RMS, at the cost of a delay that rose from 15 to 30 ms (Fusion Studio section). It is part of PR #7.

SteamVR IVRDriverInput_005 fix

After the SteamVR / Steam Link update of 21 October 2026, Steam Link requested only the new IVRDriverInput_005 interface. As a result, the driver's optical tap saw no finger skeletons and users got a fixed, synthesised hand. My fix (c35efc5, src/OpticalTap.cpp, 22 October) also hooks _005 and was verified live with 31 bones at 360 Hz. SciCortex committed the same fix to main the next day as d469b53, "Fix for latest steamvr and skeleton data". The architecture section describes the fix in detail.

PR #7: Fusion Studio update and Windows app

PR #7 (b42142f and 2ceb348 on branch fusion-studio-update; 20 files, +3,669/−108 lines) brings SciCortex's Studio up to date with the October research. It was merged unit by unit, so that SciCortex's own changes stayed as written. It also adds CyberFingerFusionStudio.exe, a Windows app that runs without installing Python. The pull request was open as of 23 October 2026. The built exe passed its self-test, but its

live-headset path is untested. The build choices and tests are described in the Fusion Studio section.

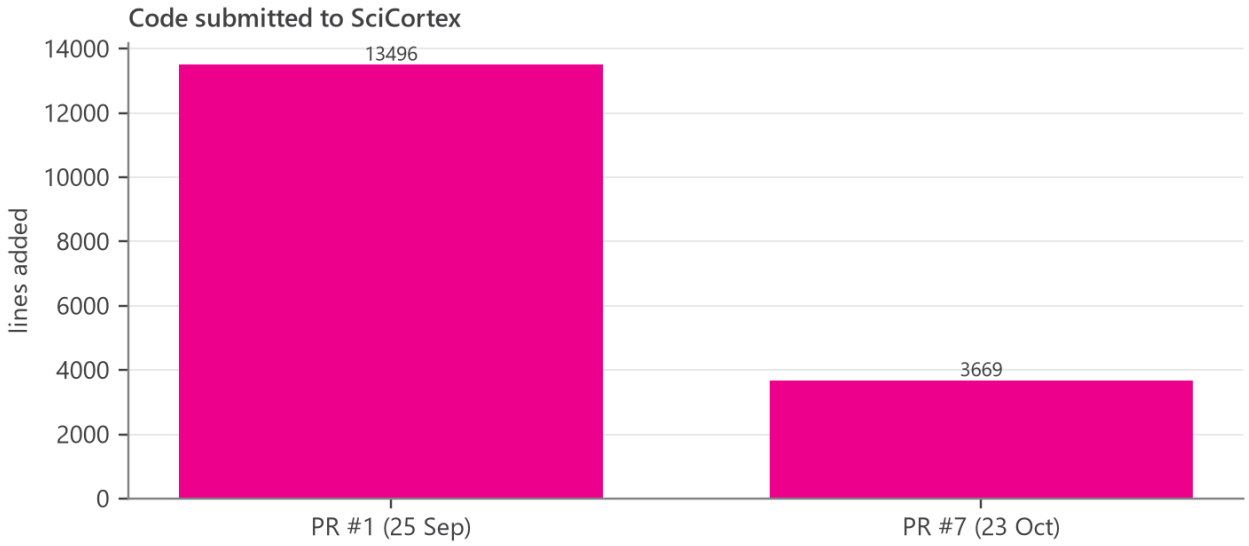


Figure 10.2 *Lines added by each pull request. PR #1 (490a312, 23 files) delivered the single-window Fusion Studio. About 5,900 of its lines are existing modules copied in, so the bars measure size, not new code. It was merged into SciCortex's fusion-studio branch on 25 September and reached main on 27 September. PR #7 (b42142f + 2ceb348, 20 files, 108 lines removed) updates the Studio and adds the Windows app.*

Merging v2.1.0 into the research branch

On 29 September I merged release v2.1.0 into the research branch (266731d). This kept the research GUI on SciCortex's current product: driver 2.1, firmware 1.4.1 and the renamed CyberFinger interface. Git's own line merge had mixed together code from the two sides, so the merge was done on whole code units (classes, methods and statements) with a tool built for the purpose. Afterwards, replays gave the same results as before the merge. The unit counts and checks are in the architecture section.

Review of the CyberFinger manual v2.1

I read the 23-page manual for release v2.1.0 (SteamVR driver 2.1, firmware 1.4.1, CyberFinger Bridge 2.1) in full on 29 September and checked it against the v2.1.0 code. That review found eight issues. One no longer applies: the claim that "every CyberFinger has sensor 1" was false for the lab units on their old firmware, but is true on 1.4.1. Seven still stand:

1. The page-6 screenshot shows only "mic mute / FluxAction" for the pink button, but the code also offers SteamVR, which is the default.
2. The screenshots on pages 16–20 show old labels ("glove's IMU", "Glove button") and Debug captures ON, which is off by default.
3. The right pink button's default FluxAction 42 collides with MoreFluxActions' MuteToggleAction 42.
4. The resync time is given as ½ s, ~1 s and "a second or two" in different places.
5. Page 2 promises out-of-view turning without saying that it needs the joint IMU.
6. "Becomes A held" never explains that A is the black button.
7. Page 5 names Gale/BepisLoader as prerequisites, but CyberFingerMod and ProximityGrab are ResoniteModLoader mods.

The manual never mentions the Fusion Studio or the EMG armband, although v2.1.0 ships them. A related finding: the installed CyberFinger Bridge 2.1.0 executable lacked the openvr module (its Skeleton tab asked for "pip install openvr"), which is a packaging bug. I therefore ran the research GUI from source. No record was found that the review or the bug report has been sent to SciCortex. Both are on the wrap-up list below.

10.4 Engineering practice

- **Version control.** I made 55 non-merge commits on the research branch fusion-hand-tracking between 17 September and 23 October 2026, plus the v2.1.0 merge. Eight of them are backups of logs and data, and one is a log update. The first, c7704d3 "Add sensor-fusion hand tracking: EMG + IMU + optical pipeline", consolidated the July–September work. Together the commits add 37,234 lines and delete 578 in .py, .cpp, .md, .bat and .spec files, touching 39 Python files. These counts include the generated single-window app and the modules copied into it.
- **Additive-only rule.** No new tab was allowed to change an existing one. Each new tab came with a check that existing tabs and model files were unchanged, and commit messages record that the model files stayed byte-identical. A copy of the GUI was taken before each patch (26 backups).
- **Tests.** I wrote 4 test files with 45 test functions: test_finger_fusion 9, test_posture_hmm 21, test_steamvr_link 6, test_transition_events 9. The repository has 10 Python test files in all. key_postures.py, mount_calib.py and fk_position.py also carry built-in self-tests.
- **Parity and replay.** The numpy runtime matches PyTorch to 3e-8 (TCN, 0.33 ms per frame), so Windows needs no PyTorch. The C++ ports match the compiled driver (2e-8, filter 1e-16). Results were measured by replaying recorded sessions through the program's real update loop ("tick"), not a simplified copy.
- **Performance.** Vectorising the occlusion test made it 27× faster (4.8 → 0.18 ms), with identical output on 3,200 recorded frames. The Fusion Studio went from 35 to 62 fps (median frame time 22 → 8 ms; 95th percentile 38 → 11 ms).
- **Documentation.** docs/firmware-and-imu.md (79 lines), docs/gate-spec.md (69 lines) and bridge/FUSION_STUDIO.md (63 lines).

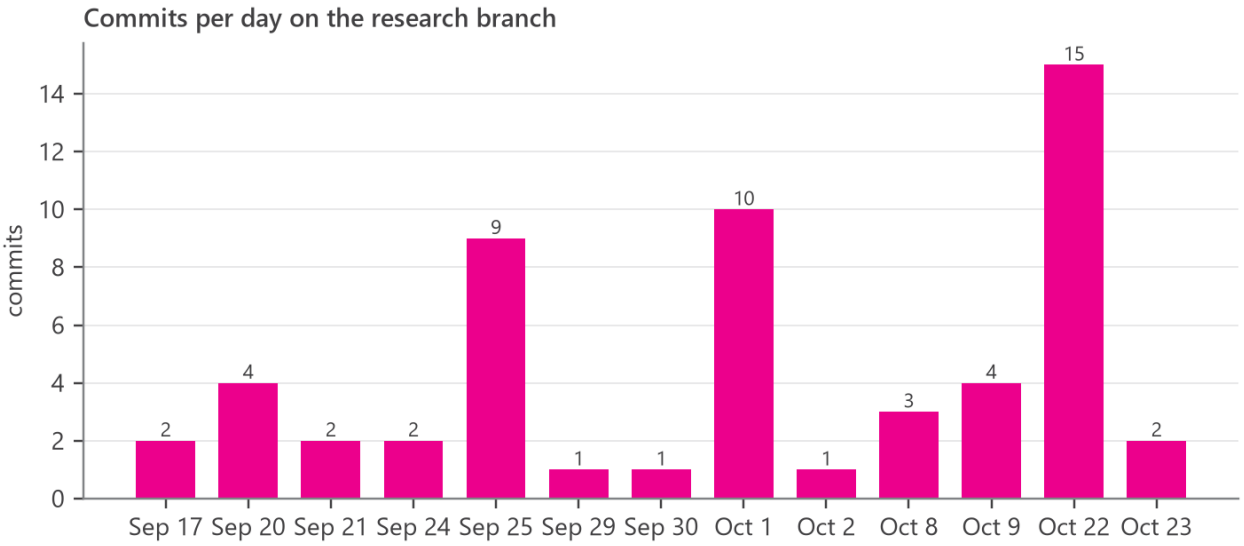


Figure 10.3 *Commits by the intern from the first research commit (c7704d3, 17 Sep) onward; that commit consolidated the July–September work. The peaks are the release to SciCortex (25 Sep), the Studio overhaul and SteamVR link (1 Oct), and the driver fix plus the Transitions work (22 Oct). The 29 Sep bar is the v2.1.0 merge. Eight commits (17–25 Sep) are log and data backups, and 490a312 (25 Sep) was made on the release branch. Source: git_log_full.txt.*

10.5 Data and code availability

The code, recordings and evaluation outputs behind this report are kept in the following places.

- **Code.** The research branch fusion-hand-tracking (head cf71b91, 23 October 2026) is mirrored to the private repository AMshoka/CyberFinger_SteamVR-fusion. The last pushed commit is 8261d44, so the local head is one commit ahead.
- **Product submissions.** AMshoka/CyberFinger_SteamVR, branches fusion-studio (490a312, merged as PR #1) and fusion-studio-update (2ceb348, PR #7).
- **Recordings.** The bridge/ data folders, 569.8 MB, recorded from 18 August to 23 October 2026 with one participant (the intern).
- **Evaluation scripts and outputs.** The experiment lab ~/poslab (about 2.2 GB), which is not yet under version control.
- **Not kept.** The scripts behind three results were not kept: the out-of-view posture identification (72/84/92%), the TCN's 1.44 cm error, and the per-finger curl correlations.

10.6 Limitations and lessons learned

The negative results below are part of the findings. Each one changed a design decision, and several of them shape the IU2 plan.

Sensors and the camera

- The Quest's per-joint flags cannot gate fingers. During occlusion it keeps the hand active and guesses where the hidden fingers are.

- The Quest cannot label a closed fist: every fist frame had a confidence of exactly 0.50, and the curled fingers came out garbled.
- The CyberFinger's 6-axis IMUs have no magnetometer, so their horizontal heading can be trusted for only about 1 minute after the last camera fix. Up to 92° of heading drift was seen in 5 minutes.
- A floppy knuckle mount looked like sensor failure (57–76° error). Securing the mount brought both sensors to about 2°.

EMG across days and arm positions

- EMG drifts from day to day. The first posture recogniser fell to 2% on open and 30% on point in the next day's session.
- Arm position changes the EMG (the limb-position effect). Pose identification was 89% in view but 75% out of view (single nearest neighbour). The fixes ranked best in the literature (adaptive normalisation, AdaBN, CORAL) made it worse, at 37–44%. Temporal smoothing (72 → 84%) and camera pseudo-labels helped.
- Features designed to ignore orientation backfired: subspace removal gave 92% and regressing out tilt 24%, against 98% for raw features. Adding knuckle-IMU features lowered posture accuracy from 75% to 63% and then 50%.
- Five classifiers (logistic regression, LDA, QDA, kNN, MLP) all reached 74–76% across arm angles, so the limit is in the features, not the classifier.
- Turning the forearm lowers posture recognition from 90% (below 30°/s) to 79% (above 180°/s).
- EMG cannot predict the wrist bend: a ridge model on 40 time-domain features did no better than predicting the mean.

IMU, position and transitions

- Taking the wrist bend as the difference of two separately calibrated IMU estimates produced a false 30–38° bend, so that design was reverted.
- Position from forearm orientation alone stalls near 20 cm because the upper arm is not measured. Forward kinematics with a perfect upper-arm direction reaches 5.2–5.6 cm; a real upper-arm sensor is expected to give 7–12 cm.
- The learned transition detector catches ~74% of transitions (6 recordings, 190 transitions, leave-one-recording-out). Quiet transitions such as open↔pinch remain hard.

Scope of the data and tests

- One user and few sessions; Hybrid Position was scored on 5 long clips from two recording days.
- Only one Transitions recording has strong forearm motion (median 100°/s), and the recordings hold at most 311 frames with one hand hidden behind the other.
- Only the right (EMG) hand is fused into SteamVR; the left-hand path and the Windows app's headset path are untested live.
- Process lessons: validate on real recordings before shipping (blind tuning made results worse twice); score flicker alongside lag; whenever the camera sees the hand, it can teach the other sensors for free.

10.7 Next steps: plan for Internship Unit 2

IU2 runs from 1 November 2026 to 27 February 2027 (intern to be confirmed, PhD student). The plan follows proposal tasks 5–8 and starts from the IU1 results and baselines above.

Table 10.3 IU2 plan mapped to the proposal's tasks 5–8 (sources: proposal §2.4; project notes; fusion/online_calib.py).

Proposal task	Starting point from IU1	IU2 plan
5.1 Inter-subject and mount variability	Next-day collapse (open 2%); camera self-teaching 50 → 77%; simulated re-donning with augmentation 46 → 75%; per-session mounting ~9°	Multi-day recordings; learn a session/user embedding from camera-labelled windows; multi-user only after an ethics check
5.2 Ray-cast occlusion	Delivered in IU1 (27× faster after vectorising)	Learned reliability estimator to replace hand-set thresholds; record hand-over-hand occlusion (≤ 311 frames so far)
5.3 Cross-session benchmarks	Look-away protocol; leave-one-position-out posture reports	Extend both to multi-day and multi-user data
6.1–6.3 Transformer, masked modality	Synchronised recordings with natural masks (camera present or absent); soft hand cut the lag from 230 to 45 ms	Train and compare against the IU1 baselines (Hybrid 14.58 cm; posture and transition metrics)
7.1–7.3 Calibration refinement	Limb position 89 → 75%; turning 90 → 79%; residual-triggered buffer rebuild already prototyped	Sensitivity study; buffer-invalidation rule; observation-window study left over from 3.3
8.1–8.3 Integration	Right hand in SteamVR; Windows app built; PR #7 open	Fuse both hands; live-headset test of the app; add an upper-arm IMU; Transitions session with REST; evaluation, demo, documentation

- **Ethics.** The proposal declares no human participants (§3.2), so this must be checked with McGill and Mitacs before recording other users.
- **Buffer invalidation (7.2).** A prototype already exists: fusion/online_calib.py rebuilds its buffer when the running average of the residual against the nearest buffered pair exceeds 0.22 while more than 40 pairs are held.
- **Upper-arm sensor.** On firmware 1.4.1 the BODY 1 slot carries the wrist sensor, so an upper-arm IMU needs its own slot in the firmware and the bridge. The IMU FK Position tab's upper-arm option now reads the wrist sensor and must be pointed at the new slot.

Partner deliverables and publication

The proposal (§2.6) lists five kinds of deliverable: reports for the partner, presentations to SciCortex staff, patent applications, other IP claims, and peer-reviewed publication. This report covers the first. The presentations, any IP claims (SciCortex's decision) and a publication remain open. Three topics are suggested as candidates: the camera-taught IMU mounting calibration, out-of-view wrist position from forward kinematics plus learned elbow motion, and EMG transition flags. Offline, on the emg2pose data, the flags caught 94% of 249 transitions about 55 ms before the fingers moved.

Conclusion

The first internship unit showed that a hand the headset cannot see can still be tracked usefully. Ten of the twelve planned sub-tasks were completed and two were partly done, and the work went beyond the plan by estimating the hand's position out of view. Three ideas carried the results:

- **Camera trust from geometry and cross-checks.** The Quest's own flags stay set while the hand is hidden, so trust has to come from hand geometry and from agreement with the IMU and EMG.
- **The camera as a free teacher.** Whenever the camera sees the hand, it recalibrates the other sensors: the EMG lookup in Live Calib, the IMU mounting, key-posture self-teaching and online learning in Pose Hand.
- **Physics first, learning for the rest.** Hybrid Position computes the forearm by kinematics and learns only the elbow motion that no sensor measures. The anatomical hand keeps every drawn pose plausible.

For SciCortex, the out-of-view hand has moved from a frozen position to an estimated hand: position from Hybrid Position, orientation and wrist bend from the IMUs, and finger posture from the EMG. Fusion Studio is part of release v2.1.0. Its two-sided IMU mounting model now runs inside SciCortex's SteamVR driver. The fused hand can drive SteamVR through the driver's FUSED source, and the driver again receives finger data after the 21 October SteamVR update. Some items are still pending: the October update and the Windows app (pull request #7) were awaiting review on 23 October, the app's live-headset path is untested, and the soft hand and transition flags ship switched off until they are tried live.

The negative results are part of the findings, and several of them shaped the design:

- The Quest cannot label a closed fist.
- Integrating accelerometer data for position diverges, with 150 cm of error after 1 s.
- Normalisation and alignment methods (including CORAL) made out-of-view EMG recognition worse, at 37–44%.
- Pretraining on emg2pose only tied training from scratch.
- EMG cannot predict the wrist bend.

The remaining limits are also clear. All data come from one user. The upper arm is unobserved: an oracle with the true upper-arm direction reaches 8.13 cm, against 14.58 cm. The IMU heading drifts once the camera is gone. EMG patterns shift with arm position and from day to day.

The second unit (1 November 2026 to 27 February 2027) can start from these baselines:

- **Upper-arm IMU.** Forward kinematics with a simulated upper-arm sensor gave 5.1–9.4 cm.
- **More users and sessions.** Multi-day recordings, and multi-user recordings once an ethics check is done.
- **Learned camera reliability.** A learned estimator could close the gap between the hand-built gate (AUC 0.753) and the geometric ceiling (0.879).
- **Planned studies.** The transformer fusion with masked sensors, and the formal convergence and window-size study.
- **Product follow-up.** Fusing both hands into SteamVR, and live trials of the Windows app and of the features that ship switched off.

Acknowledgements

I thank Mitacs for funding this work through the Mitacs Accelerate programme. I thank SciCortex Technologies Corporation for partnering on and co-funding the project. My thanks go to Ania Muller, CEO of SciCortex Technologies, and to Dr. Eilif Muller, who guided the project directly, for their trust and for opening the CyberFinger product to my contributions. I am grateful to my academic supervisor, Majid Mohajerani (McGill University, Centre de recherche Douglas, Department of Psychiatry), for his supervision throughout the internship. I also thank Mila for hosting the internship.

Appendix A — Index of the GUI tabs

Every tab of the research GUI (*bridge/cyberfinger_gui.py*): its illustrated entry in Chapter 2 and the section that describes the work in detail.

GUI tab	Tab tour (screenshot)	Detailed section
Gloves	2.1 · p. 27	3.5 · p. 117
Armband	2.2 · p. 31	3.6 · p. 119
Gate	2.3 · p. 34	4.3 · p. 125
Capture	2.4 · p. 37	3.7 · p. 120
EMG Hand	2.5 · p. 40	5.2 · p. 140
Encoder	2.6 · p. 43	5.3 · p. 142
Train	2.7 · p. 46	5.4 · p. 142
Fusion Hand	2.8 · p. 48	5.5 · p. 143
Occlusion	2.9 · p. 51	4.4 · p. 131
Live Calib	2.10 · p. 54	4.5 · p. 133
Tracking Hand	2.11 · p. 57	5.7 · p. 146
emg2pose (ref)	2.12 · p. 60	5.8 · p. 147
Posture Library	2.13 · p. 62	5.9 · p. 148
Posture Clusters	2.14 · p. 65	5.10 · p. 148
Posture Hand	2.15 · p. 68	5.11 · p. 149
Pose Hand	2.16 · p. 71	5.12 · p. 150
Pose + Position	2.17 · p. 74	7.2 · p. 169
Position Capture	2.18 · p. 77	7.3 · p. 171
Learned Position	2.19 · p. 80	7.4 · p. 172
Body View	2.20 · p. 83	7.5 · p. 174
IMU FK Position	2.21 · p. 86	7.6 · p. 174
Pose Hand (gravity)	2.22 · p. 89	5.13 · p. 151
Hybrid Position	2.23 · p. 92	7.8 · p. 177
✦ Fusion Studio	2.24 · p. 95	9.1 · p. 200
Key Postures	2.25 · p. 100	8.2 · p. 186
◈ Fusion Logic	2.26 · p. 103	8.3 · p. 192
⚡ Transitions	2.27 · p. 106	8.4 · p. 194

Appendix B — Glossary

Table B.1 Abbreviations and terms used in this report.

Term	Meaning
6-DoF	Six degrees of freedom: position (3) plus orientation (3)
AdaBN	Adaptive batch normalisation: re-estimating normalisation statistics on new data
AUC	Area under the ROC curve; 1.0 = perfect ranking, 0.5 = chance
BLE	Bluetooth Low Energy, the CyberFinger's radio link
CORAL	Correlation alignment: matching new features' statistics to those of the training features
ECE	Expected calibration error: average gap between a model's stated confidence and its accuracy
EMA	Exponential moving average, a simple smoothing filter
EMG, sEMG	(Surface) electromyography: muscle electrical activity recorded on the skin
FK	Forward kinematics: wrist position from segment lengths and segment orientations
FUSED	The SteamVR driver's pose source fed by the Fusion Studio
Gate	Labels each camera frame CLEAR, OUT_OF_VIEW, SELF, OTHER or OBJECT
HMM	Hidden Markov model
IMU	Inertial measurement unit (gyroscope and accelerometer)
IU1, IU2	Mitacs internship units 1 and 2
kNN, LDA, QDA	k-nearest neighbours; linear and quadratic discriminant analysis (classifiers)
LOCO	Leave-one-clip-out: test on each clip with a model trained on the others
LOPO	Leave-one-position-out: the same, holding out one arm position
LORO	Leave-one-recording-out: the same, holding out one recording
MLP	Multilayer perceptron, a small fully connected neural network
NLMS	Normalised least mean squares: updates linear weights after every sample
One Euro filter	Low-pass filter that smooths more when the signal moves slowly
OpenXR	Open standard interface through which the headset's hand tracking is read
PR	Pull request: a proposed change to a Git repository
Ridge	Linear regression with a penalty on large weights
RMS	Root mean square
TCN	Temporal convolutional network: convolutions over time
Tick	One pass of the program's real update loop; replays run through it
VQF	Versatile quaternion-based filter fusing gyroscope and accelerometer on the CyberFinger
w = f(m, d)	Per-finger camera weight from camera reliability m and camera–EMG disagreement d

Appendix C — References

- Buteau, É., Gagné, G., Bonilla, W., Boukadoum, M., Fortier, P., & Gosselin, B. (2024). TinyML for Real-Time Embedded HD-EMG Hand Gesture Recognition with On-Device Fine-Tuning. *2024 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 1–6. https://doi.org/10.1109/EMBC53108.2024.10781755
- Casiez, G., Roussel, N., & Vogel, D. (2012). 1€ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '12)*, 2527–2530. https://doi.org/10.1145/2207676.2208639
- Fougner, A., Scheme, E., Chan, A. D. C., Englehart, K., & Stavdahl, Ø. (2011). Resolving the Limb Position Effect in Myoelectric Pattern Recognition. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 19(6), 644–651. https://doi.org/10.1109/TNSRE.2011.2163529
- Graf, M., & Barthet, M. (2023). *Combining Vision and EMG-Based Hand Tracking for Extended Reality Musical Instruments* (arXiv:2307.10203). arXiv. https://doi.org/10.48550/arXiv.2307.10203
- Hudgins, B., Parker, P., & Scott, R. N. (1993). A New Strategy for Multifunction Myoelectric Control. *IEEE Transactions on Biomedical Engineering*, 40(1), 82–94. https://doi.org/10.1109/10.204774
- Laidig, D., & Seel, T. (2023). VQF: Highly Accurate IMU Orientation Estimation with Bias Estimation and Magnetic Disturbance Rejection. *Information Fusion*, 91, 187–204. https://doi.org/10.1016/j.inffus.2022.10.014
- Nie, J. Z., Nie, J. W., Hung, N. T., Cotton, R. J., & Slutzky, M. W. (2021). Portable, open-source solutions for estimating wrist position during reaching in people with stroke. *Scientific Reports*, 11, 22491. https://doi.org/10.1038/s41598-021-01805-2
- Salter, S., Warren, R., Schlager, C., Spurr, A., Han, S., Bhasin, R., Cai, Y., Walkington, P., Bolarinwa, A., Wang, R., Danielson, N., Merel, J., Pnevmatikakis, E., & Marshall, J. (2024). emg2pose: A Large and Diverse Benchmark for Surface Electromyographic Hand Pose Estimation. *NeurIPS 2024 Datasets and Benchmarks Track*. arXiv:2412.02725. https://doi.org/10.48550/arXiv.2412.02725
- Streli, P., Armani, R., Cheng, Y. F., & Holz, C. (2023). HOOV: Hand Out-Of-View Tracking for Proprioceptive Interaction using Inertial Sensing. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. https://doi.org/10.1145/3544548.3581468
- Zanghieri, M., Rapa, P. M., Orlandi, M., Buteau, É., Chamberland, F., Gosselin, B., Benini, L., & Benatti, S. (2024). Wearable High-Density sEMG Processing with Class Activation Maps with an Embedded Temporal Convolutional Network. *2024 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 1–5. https://doi.org/10.1109/BioCAS61083.2024.10798295

A keepsake

A keepsake of the internship: an imagined future for hands, people and robots at SciCortex. An illustration, not a photograph.